Amazon Interview Prep

Meta Answer Book — Days 1–13

153 problems, each answered in five parts

Understand · Brute force · Optimal · Commented Python · Dry run

127 LeetCode problems · 26 reported InterviewDB questions

Herndon, VA · 4 rounds × 60 min · LiveCode

Contents

META ANSWER FILE — Amazon prep, Day 1 → Day 13

Index

DAY 1 — Mon Aug 17 · Arrays, Hashing & Two Pointers

1. Two Sum
49. Group Anagrams
128. Longest Consecutive Sequence
238. Product of Array Except Self
560. Subarray Sum Equals K ★
41. First Missing Positive ★★
15. 3Sum ★
11. Container With Most Water
42. Trapping Rain Water ★★
Common Manager — ✅ Verified (InterviewDB)
Friends Purchases — ✅ Verified (InterviewDB)

DAY 2 — Tue Aug 18 · Sliding Window & Prefix Sum

3. Longest Substring Without Repeating Characters ★★★
424. Longest Repeating Character Replacement
209. Minimum Size Subarray Sum
438. Find All Anagrams in a String
567. Permutation in String
76. Minimum Window Substring ★★★
992. Subarrays with K Different Integers ★★
1658. Minimum Operations to Reduce X to Zero ★
239. Sliding Window Maximum ★★
974. Subarray Sums Divisible by K
525. Contiguous Array
Destination Cache — 🔴 LRU shape with TTL (InterviewDB)
Top Customers — 🟡 Heap aggregation (InterviewDB)

DAY 3 — Wed Aug 19 · Stacks, Monotonic Stacks & Linked Lists

20. Valid Parentheses
155. Min Stack
150. Evaluate Reverse Polish Notation
394. Decode String
739. Daily Temperatures
735. Asteroid Collision
84. Largest Rectangle in Histogram ★★
206. Reverse Linked List
21. Merge Two Sorted Lists
142. Linked List Cycle II ★
19. Remove Nth Node From End of List
143. Reorder List
138. Copy List with Random Pointer ★★
148. Sort List ★
VM Load — ✅ Verified pattern (InterviewDB)
Block Words — 🟡 Trie / Aho–Corasick (InterviewDB)

DAY 4 — Thu Aug 20 · Trees

104. Maximum Depth of Binary Tree
110. Balanced Binary Tree
543. Diameter of Binary Tree ★
102. Binary Tree Level Order Traversal
199. Binary Tree Right Side View
662. Maximum Width of Binary Tree ★
124. Binary Tree Maximum Path Sum ★★
98. Validate Binary Search Tree
230. Kth Smallest Element in a BST
105. Construct Binary Tree from Preorder and Inorder Traversal
236. Lowest Common Ancestor of a Binary Tree ★★
297. Serialize and Deserialize Binary Tree ★
863. All Nodes Distance K in Binary Tree ★
437. Path Sum III ★
Tree Kyoukai — 🟡 Boundary of Binary Tree (LeetCode 545)
Employee Directory — 🟡 Design (InterviewDB)

DAY 5 — Fri Aug 21 · Grids & Backtracking

200. Number of Islands ★★
695. Max Area of Island
130. Surrounded Regions
417. Pacific Atlantic Water Flow ★
994. Rotting Oranges ★★
79. Word Search
78. Subsets
46. Permutations
39. Combination Sum
22. Generate Parentheses ★
212. Word Search II ★★
Gold Thief — 🟡 Path with Maximum Gold (LeetCode 1219)
XO Game — 🟡 Design Tic-Tac-Toe (LeetCode 348)

DAY 6 — Sat Aug 22 · Graphs

207. Course Schedule ★
210. Course Schedule II
133. Clone Graph
547. Number of Provinces
721. Accounts Merge
127. Word Ladder ★★
399. Evaluate Division ★
684. Redundant Connection
802. Find Eventual Safe States ★
310. Minimum Height Trees ★
752. Open the Lock
787. Cheapest Flights Within K Stops ★
542. 01 Matrix
Conversion Rate — ✅ Verified (LeetCode 399 shape)
Friendship Path — 🟡 BFS (InterviewDB)

DAY 7 — Sun Aug 23 · ★★ MOCK LOOP #1

DAY 8 — Mon Aug 24 · Heaps & Top-K

215. Kth Largest Element in an Array ★
347. Top K Frequent Elements
692. Top K Frequent Words ★
23. Merge k Sorted Lists ★★
973. K Closest Points to Origin ★
295. Find Median from Data Stream ★★
767. Reorganize String ★
621. Task Scheduler ★
502. IPO ★
1834. Single-Threaded CPU ★
378. Kth Smallest Element in a Sorted Matrix
Bid Fulfillment — ✅ Verified (InterviewDB)
Minimum Possible Cost — ✅ Verified (InterviewDB)

DAY 9 — Tue Aug 25 · Binary Search (incl. binary search on the answer)

704. Binary Search
34. Find First and Last Position of Element in Sorted Array
33. Search in Rotated Sorted Array ★★
153. Find Minimum in Rotated Sorted Array
162. Find Peak Element
540. Single Element in a Sorted Array ★
74. Search a 2D Matrix
240. Search a 2D Matrix II
875. Koko Eating Bananas ★
1011. Capacity To Ship Packages Within D Days ★
410. Split Array Largest Sum ★
1552. Magnetic Force Between Two Balls ★
528. Random Pick with Weight ★
Inconvenience — ✅ Verified (InterviewDB)
Optimal Packing — 🟡 Minimum Space Wasted From Packaging (LeetCode 1889)
Travel Time — ✅ Verified (InterviewDB, Chinese source)

DAY 10 — Wed Aug 26 · Intervals, Greedy & DP Part 1

56. Merge Intervals ★★
57. Insert Interval
435. Non-overlapping Intervals
253. Meeting Rooms II ★★
1288. Remove Covered Intervals
759. Employee Free Time ★
55. Jump Game
45. Jump Game II
134. Gas Station ★
70. Climbing Stairs
198. House Robber
213. House Robber II
322. Coin Change ★★
518. Coin Change II
Minimum Shipment — ✅ Verified (InterviewDB)
Minimize Absolute Differences — ✅ Verified (InterviewDB)
Permutation Sorter — ✅ Verified (InterviewDB)

DAY 11 — Thu Aug 27 · DP Part 2 & Design-in-Code

139. Word Break
53. Maximum Subarray
152. Maximum Product Subarray
918. Maximum Sum Circular Subarray ★
300. Longest Increasing Subsequence ★★
1143. Longest Common Subsequence ★
72. Edit Distance
416. Partition Equal Subset Sum
494. Target Sum ★
62. Unique Paths
64. Minimum Path Sum
146. LRU Cache ★★

211. Design Add and Search Words Data Structure
380. Insert Delete GetRandom O(1) ★
981. Time Based Key-Value Store
588. Design In-Memory File System ★
Binary String Errors — ✅ Verified (InterviewDB)
Warehouse Stock — ✅ Verified (InterviewDB)
Config Reconstruction — ✅ Verified (InterviewDB)

DAY 12 — Fri Aug 28 · ★★ MOCK LOOP #2 + Weak-Spot Repair

Smart Processor — ✅ Verified (InterviewDB)
Sign In Pages — ✅ Verified (InterviewDB)
Faulty Server Detection — ✅ Verified (InterviewDB)
Count Discount Pairs — ✅ Verified (InterviewDB)
The rest of Day 12

DAY 13 — Sat Aug 29 · ★★★ MOCK LOOP #3 — Dress Rehearsal

META ANSWER FILE — Amazon prep, Day 1 → Day 13

Every problem from `Day01…Day13` in one place, each answered in the same five-part shape:

1. **Understand the problem** — restatement, constraints, edge cases
2. **Brute-force approach** — idea · data structures · algorithm · complexity · why it's not ideal
3. **Optimal approach** — idea · data structures · algorithm · complexity
4. **Python code** — full LeetCode-compatible, heavily commented
5. **Step-by-step dry run** — small example, traced by hand

> **How to use this file:** not while solving. Solve blank-page first (25-min timer, narrated). Open this after the timer, or after your code runs. The *Understand* and *Brute-force* sections are the parts you're actually being scored on in the room — the optimal code is the easy half.

Index

Day	Focus	Problems
1	Arrays, Hashing & Two Pointers	1, 49, 128, 238, 560, 41, 15, 11, 42 · Common Manager, Friends Purchases
2	Sliding Window & Prefix Sum	3, 424, 209, 438, 567, 76, 992, 1658, 239, 974, 525 · Destination Cache, Top Customers
3	Stacks, Monotonic Stacks & Linked Lists	20, 155, 150, 394, 739, 735, 84, 206, 21, 142, 19, 143, 138, 148 · VM Load, Block Words
4	Trees	104, 110, 543, 102, 199, 662, 124, 98, 230, 105, 236, 297, 863, 437 · Tree Kyoukai, Employee Directory
5	Grids & Backtracking	200, 695, 130, 417, 994, 79, 78, 46, 39, 22, 212 · Gold Thief, XO Game
6	Graphs	207, 210, 133, 547, 721, 127, 399, 684, 802, 310, 752, 787, 542 · Conversion Rate, Friendship Path
7	Mock Loop #1	protocol only
8	Heaps & Top-K	215, 347, 692, 23, 973, 295, 767, 621, 502, 1834, 378 · Bid Fulfillment, Minimum Possible Cost
9	Binary Search	704, 34, 33, 153, 162, 540, 74, 240, 875, 1011, 410, 1552, 528 · Inconvenience, Optimal Packing, Travel Time
10	Intervals, Greedy & DP-1	56, 57, 435, 253, 1288, 759, 55, 45, 134, 70, 198, 213, 322, 518 · Minimum Shipment, Minimize Absolute Differences, Permutation Sorter
11	DP-2 & Design-in-Code	139, 53, 152, 918, 300, 1143, 72, 416, 494, 62, 64, 146, 211, 380, 981, 588 · Binary String Errors, Warehouse Stock, Config Reconstruction
12	Mock Loop #2 + repair	Smart Processor, Sign In Pages, Faulty Server Detection, Count Discount Pairs
13	Mock Loop #3	protocol only

DAY 1 — Mon Aug 17 · Arrays, Hashing & Two Pointers

1. Two Sum

UNDERSTAND THE PROBLEM

Given an unsorted integer array `nums` and an integer `target`, return the **indices** of the two elements that add up to `target`. Exactly one valid answer exists, and you may not use the same element twice.

- Input can contain negatives and duplicates.
- Return indices, **not** values — that rules out "sort then two-pointer" unless you carry original indices along.
- Edge cases: `[3,3], target=6` (duplicate values, distinct indices); `n == 2`; negatives like `[-3,4,3,90], target=0`.
- Clarify out loud: *"Is exactly one answer guaranteed? Can I return in any order? Are the values 32-bit?"*

BRUTE-FORCE APPROACH

Idea. Try every unordered pair `(i, j)` with `i < j` and check whether the two values sum to the target.

Data structures. None — just two index variables.

Algorithm.

1. For `i` from `0` to `n-1`.
2. For `j` from `i+1` to `n-1`.
3. If `nums[i] + nums[j] == target`, return `[i, j]`.
4. Return `[]` if the loops finish.

Complexity. Time `O(n²)` — roughly `n²/2` pair checks. Space `O(1)`.

Why it's not ideal. For each `i` we rescan the whole suffix looking for the single value `target - nums[i]`. That's a *membership question*, and a linear scan is the wrong tool for a membership question. With `n = 10⁴` this is 50 million comparisons; the interviewer will ask for better within thirty seconds.

OPTIMAL APPROACH

Idea. Rephrase the problem: while walking left to right, at index `i` I need to know whether the *complement* `target - nums[i]` has appeared **earlier**. "Has it appeared earlier, and at what index" is exactly what a hash map answers in `O(1)`.

Data structures. One dict `seen: value -> index`, holding only elements strictly to the left of the cursor.

Algorithm.

1. `seen = {}`.
2. For each `(i, x)` in `enumerate(nums)`: a. Compute `need = target - x`. b. If `need` is in `seen`, return `[seen[need], i]`. c. Otherwise store `seen[x] = i`.
3. Return `[]`.

The ordering in step 2 is the whole correctness argument: **look up before you insert.** If you insert first, then `x` can match itself when `target == 2*x`.

Complexity. Time `O(n)` — one pass, each dict op amortised `O(1)`. Space `O(n)` — the map can hold every element in the worst case.

PYTHON CODE

```python
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        # seen maps a value we've already passed -> the index it lived at.
        # Invariant: at the top of iteration i, seen contains exactly nums[0..i-1].
        seen = {}

        for i, x in enumerate(nums):
            need = target - x          # the partner that would complete the pair

            # Look up BEFORE inserting x. This is what stops an element from
            # pairing with itself when target == 2 * x (e.g. nums=[3,3], target=6).
            if need in seen:
                return [seen[need], i]  # earlier index first, current index second

            # Not found -> record x so that a LATER element can find it.
            # If x repeats, overwriting the index is harmless: any later match
            # is still a valid answer, and the problem guarantees uniqueness.
            seen[x] = i

        return []                      # unreachable given the problem guarantee
```

DRY RUN

`nums = [2, 7, 11, 15]`, `target = 9`

i	x	need = 9-x	`need in seen`?	action	seen after
0	2	7	no (`seen` empty)	store	`{2:0}`
1	7	2	**yes → index 0**	return `[0, 1]`	—

Answer: `[0, 1]`. Check: `nums[0] + nums[1] = 2 + 7 = 9`. ✅

Self-pair guard, `nums = [3, 3]`, `target = 6`:

i	x	need	in seen?	action	seen after
0	3	3	no — we look up *before* inserting	store	`{3:0}`
1	3	3	yes → 0	return `[0, 1]`	—

Had we inserted first, index 0 would have matched itself and returned `[0, 0]`.

49. Group Anagrams

UNDERSTAND THE PROBLEM

Given a list of strings, group them so each group contains exactly the strings that are anagrams of one another. Return the groups in any order, and the strings within a group in any order.

- Two strings are anagrams iff they have identical character multisets.
- Edge cases: empty string `""` (it's an anagram of itself, forms its own group); a single string; all strings distinct (n groups of 1); all identical.
- Clarify: *"Lowercase English letters only, or full Unicode? That decides whether I can use a fixed 26-slot key."*

BRUTE-FORCE APPROACH

Idea. Maintain a list of groups. For each new string, compare it against a representative from each existing group; if it's an anagram, join that group, otherwise start a new one.

Data structures. A list of lists; a per-comparison sorted copy or counter.

Algorithm.

1. `groups = []`.
2. For each string `s`: for each existing group `g`, test `is_anagram(s, g[0])` by sorting both (or counting).
3. On a hit, append and break; if no hit, append `[s]` as a new group.

Complexity. Time `O(n² · k log k)` where `k` is the max string length — `n` strings × up to `n` group representatives × a `k log k` sort each. Space `O(n·k)`.

Why it's not ideal. We are repeatedly asking "which bucket does this belong to?" by *linear search over buckets*. Any time the answer to "which bucket" is a deterministic function of the item, you should compute the bucket key directly and let a hash map do the lookup. That collapses the outer `n` factor entirely.

OPTIMAL APPROACH

Idea. Compute a **canonical form** for each string — a value that is identical for anagrams and different for non-anagrams — and use it as a dict key. Two natural canonical forms: - the sorted string, `"eat" -> "aet"` — costs `O(k log k)`; - a 26-length count vector as a tuple, `"eat" -> (1,0,...,1,...,1,...)` — costs `O(k)` and is strictly better when the alphabet is fixed.

Data structures. `defaultdict(list)` mapping canonical key → list of original strings. Tuple keys (lists are unhashable).

Algorithm.

1. `groups = defaultdict(list)`.
2. For each `s`: build a 26-int count array, convert to a tuple, append `s` under that key.
3. Return `list(groups.values())`.

Complexity. Time `O(n · k)` — every character of every string is touched once, and hashing a 26-tuple is `O(26) = O(1)`. Space `O(n · k)` for the stored strings plus `O(n)` keys.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        # key -> list of original strings sharing that canonical form
        groups = defaultdict(list)

        for s in strs:
            # Canonical form = character frequency vector over 'a'..'z'.
            # Anagrams have identical vectors; non-anagrams cannot.
            counts = [0] * 26
            for ch in s:
                counts[ord(ch) - ord('a')] += 1

            # tuple(...) because a list is mutable and therefore unhashable.
            # Cost of hashing is O(26), i.e. constant - cheaper than an O(k log k) sort.
            groups[tuple(counts)].append(s)

        # dict values are the groups; order is insertion order, which the problem allows
        return list(groups.values())


# --- Variant when the alphabet is NOT restricted to lowercase a-z ---------
class SolutionUnicode:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        groups = defaultdict(list)
        for s in strs:
            # Sorted string is alphabet-agnostic. O(k log k) per string.
            groups[''.join(sorted(s))].append(s)
        return list(groups.values())
```

DRY RUN

`strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`

Write keys compactly as the non-zero counts only:

s	count vector (non-zero)	key	groups after
`eat`	a:1 e:1 t:1	K1	`{K1: ["eat"]}`
`tea`	a:1 e:1 t:1 → same as K1	K1	`{K1: ["eat","tea"]}`
`tan`	a:1 n:1 t:1	K2	`{K1:[...], K2:["tan"]}`
`ate`	a:1 e:1 t:1 → K1	K1	`{K1: ["eat","tea","ate"], K2:["tan"]}`
`nat`	a:1 n:1 t:1 → K2	K2	`K2: ["tan","nat"]`
`bat`	a:1 b:1 t:1	K3	`K3: ["bat"]`

Result: `[["eat","tea","ate"], ["tan","nat"], ["bat"]]` . ✅

Note `"eat"` and `"tan"` both contain an `a` and a `t` — but the count vector differs in the `e` / `n` slots, so the keys differ. Position-independence is exactly why a *count* is the right canonical form and something like "sum of char codes" is not (it collides: `"ad"` and `"bc"`).

---

128. Longest Consecutive Sequence

UNDERSTAND THE PROBLEM

Given an unsorted array, return the length of the longest run of **consecutive integers** present in the array. The elements need not be adjacent in the array. Must run in `O(n)`.

- Duplicates don't extend a run: `[1,1,2]` has longest run `2` (`1,2`).
- Negatives allowed. Empty array → `0`.
- The `O(n)` requirement explicitly forbids sorting — say that out loud, it's the constraint the whole problem hangs on.

BRUTE-FORCE APPROACH

Idea (a). Sort, then scan linearly counting runs, skipping duplicates. **Idea (b).** For each element `x`, walk `x+1, x+2, …` checking membership by linear scan.

Data structures. (a) the sorted array; (b) nothing.

Algorithm (a).

1. Sort `nums`.
2. Walk with a `current` counter; if `nums[i] == nums[i-1]` skip; if `nums[i] == nums[i-1] + 1` extend; else reset to 1.
3. Track the max.

Complexity. (a) Time `O(n log n)`, space `O(1)` extra (or `O(n)` if you can't mutate). (b) Time `O(n²)` (or `O(n³)` with linear membership), space `O(1)`.

Why it's not ideal. Sorting imposes a *total order* when the problem only needs *membership* ("is `x+1` present?"). Paying `log n` for global ordering you never use is the classic over-solve. And the naive membership walk in (b) revisits the same run once per element of that run, which is quadratic on input like `[1,2,3,…,n]`.

OPTIMAL APPROACH

Idea. Two moves combined: 1. Put everything in a **set** so membership is `O(1)`. 2. Only start counting a run from its **head** — that is, from an `x` for which `x-1` is *not* in the set. Every run is then walked exactly once, total.

Without step 2 you'd walk the run from every one of its members, which is quadratic. With it, the total work across the whole loop is bounded by the total length of all runs, which is `≤ n`.

Data structures. A `set` of the values.

Algorithm.

1. `s = set(nums)`; `best = 0`.
2. For each `x` in `s`: if `x - 1 in s`, `continue` (x is mid-run, not a head).
3. Otherwise walk `length = 1`, then `while x + length in s: length += 1`.
4. `best = max(best, length)`.
5. Return `best`.

Complexity. Time `O(n)` amortised — the outer loop is `n` iterations, and the inner `while` executes at most once per element across the *entire* run of the algorithm. Space `O(n)` for the set.

PYTHON CODE

```python
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        # Set gives O(1) membership AND deduplicates for free.
        s = set(nums)
        best = 0

        for x in s:
            # THE trick: only extend from a sequence HEAD.
            # If x-1 exists, x sits in the middle of some run, and that run
            # will be (or already was) counted from its own head. Skipping here
            # is what turns O(n^2) into O(n): the inner while below runs a total
            # of <= n times across ALL iterations of this for loop.
            if x - 1 in s:
                continue

            # x is a head. Walk forward as far as the set allows.
            length = 1
            while x + length in s:
                length += 1

            best = max(best, length)

        return best
```

DRY RUN

`nums = [100, 4, 200, 1, 3, 2]` → `s = {1, 2, 3, 4, 100, 200}`

Iterating the set (order is arbitrary; using ascending for readability):

x	`x-1 in s`?	head?	walk	length	best
1	`0 in s`? no	**yes**	2✓ 3✓ 4✓ 5✗	4	4
2	`1 in s`? yes	no — skip	—	—	4
3	`2 in s`? yes	no — skip	—	—	4
4	`3 in s`? yes	no — skip	—	—	4
100	`99 in s`? no	**yes**	101✗	1	4
200	`199 in s`? no	**yes**	201✗	1	4

Answer: `4` (the run `1,2,3,4`). ✅

Count the inner-`while` steps: 3 (for x=1) + 0 + 0 = 3 across the whole run — linear, not quadratic. Delete the `continue` and x=2 would walk 3 steps, x=3 two steps, x=4 one step: that's the quadratic blow-up.

---

238. Product of Array Except Self

UNDERSTAND THE PROBLEM

Given `nums`, return `answer` where `answer[i]` is the product of every element *except* `nums[i]`. **Division is forbidden**, and the algorithm must run in `O(n)`. Follow-up: `O(1)` extra space (the output array doesn't count).

- Zeros are the reason division is banned — `total / nums[i]` explodes on a zero and gives wrong results when there is exactly one zero.
- Edge cases: one zero (every position except that one becomes 0); two or more zeros (everything is 0); negatives (sign handled automatically).
- Guaranteed to fit in 32-bit per the problem statement.

BRUTE-FORCE APPROACH

Idea. For each index `i`, loop over the array multiplying every element other than `nums[i]`.

Data structures. Output list only.

Algorithm.

1. `res = [1] * n`.
2. For `i` in range(n): for `j` in range(n): if `j != i`: `res[i] *= nums[j]`.

Complexity. Time `O(n²)`. Space `O(1)` extra.

Why it's not ideal. Every position recomputes products over ranges that heavily overlap with its neighbours' ranges. `answer[3]` and `answer[4]` share almost the entire array. Recomputing shared work is exactly what prefix aggregates exist to eliminate.

(The *other* tempting brute force — compute `total = prod(nums)` then `answer[i] = total // nums[i]` — is `O(n)` but is banned, and it is wrong on zeros anyway.)

OPTIMAL APPROACH

Idea. Split the product into two halves that don't include `i`:

```
answer[i] = (nums[0] · … · nums[i-1]) × (nums[i+1] · … · nums[n-1])
              ^--- prefix product ---^     ^--- suffix product ---^
```

Compute prefixes in a left-to-right pass, then multiply in the suffixes in a right-to-left pass. Both running products are single scalars, so no auxiliary arrays are needed — that's what gets you to `O(1)` extra space.

Data structures. The output array; two scalar accumulators `prefix` and `suffix`.

Algorithm.

1. `res = [1] * n`.
2. `prefix = 1`; for `i` in `0..n-1`: `res[i] = prefix`, then `prefix *= nums[i]`. — after this, `res[i]` holds the product of everything strictly left of `i`.
3. `suffix = 1`; for `i` in `n-1..0`: `res[i] *= suffix`, then `suffix *= nums[i]`. — this folds in the product of everything strictly right of `i`.
4. Return `res`.

Note the **write-then-update** order in both loops: you assign the accumulator's current value *before* absorbing `nums[i]`, which is precisely how `nums[i]` gets excluded.

Complexity. Time `O(n)` — two passes. Space `O(1)` extra beyond the required output.

PYTHON CODE

```python
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        res = [1] * n

        # PASS 1 (left -> right): res[i] = product of nums[0..i-1]
        prefix = 1
        for i in range(n):
            res[i] = prefix        # write BEFORE absorbing nums[i] -> excludes nums[i]
            prefix *= nums[i]

        # PASS 2 (right -> left): fold in product of nums[i+1..n-1]
        suffix = 1
        for i in range(n - 1, -1, -1):
            res[i] *= suffix       # again: multiply BEFORE absorbing nums[i]
            suffix *= nums[i]

        # res[i] == (product of everything left) * (product of everything right)
        # Zeros need no special case: a zero simply never enters its own product.
        return res
```

DRY RUN

`nums = [1, 2, 3, 4]`

Pass 1 (`prefix` starts at 1):

i	write `res[i] = prefix`	then `prefix *= nums[i]`
0	res[0] = 1	prefix = 1·1 = 1
1	res[1] = 1	prefix = 1·2 = 2
2	res[2] = 2	prefix = 2·3 = 6
3	res[3] = 6	prefix = 6·4 = 24

`res = [1, 1, 2, 6]` — each entry is the product of everything to its left. ✅

Pass 2 (`suffix` starts at 1):

i	`res[i] *= suffix`	res[i]	then `suffix *= nums[i]`
3	6 · 1	**6**	suffix = 1·4 = 4
2	2 · 4	**8**	suffix = 4·3 = 12
1	1 · 12	**12**	suffix = 12·2 = 24
0	1 · 24	**24**	suffix = 24·1 = 24

Answer: `[24, 12, 8, 6]`. Check `res[2] = 1·2·4 = 8`. ✅

Zero case, `nums = [0, 4, 3]`: pass 1 gives `res = [1, 0, 0]`; pass 2 gives `res[2] = 0·1 = 0`, `res[1] = 0·3 = 0`, `res[0] = 1·12 = 12` → `[12, 0, 0]`. Correct with no branching.

560. Subarray Sum Equals K ★

UNDERSTAND THE PROBLEM

Given an integer array (which **may contain negatives**) and an integer `k`, return the **number** of contiguous subarrays whose sum equals `k`.

- Count, not the subarrays themselves. Overlapping subarrays each count.
- Negatives are the pivotal detail: the sliding-window technique requires monotone growth of the running sum, which negatives destroy. Say this out loud — it's the single sentence that proves you understand why the hash-map solution is needed.
- Edge cases: `k = 0`; all zeros (`[0,0,0], k=0` → 6); single element; empty (→ 0).

BRUTE-FORCE APPROACH

Idea. Enumerate every `(start, end)` pair and sum the slice.

Data structures. None.

Algorithm.

1. `count = 0`.
2. For `i` in `0..n-1`: `running = 0`; for `j` in `i..n-1`: `running += nums[j]`; if `running == k`: `count += 1`.

Complexity. Time `O(n²)` with the running-sum trick (`O(n³)` if you naively re-sum each slice). Space `O(1)`.

Why it's not ideal. Quadratic on `n = 2·10⁴` is 400 million operations — over time limit. Structurally, the inner loop keeps re-deriving prefix sums that the outer loop already computed. There is a `O(n)` reformulation because the condition `sum(i..j) == k` can be rewritten as a *lookup*.

OPTIMAL APPROACH

Idea. Let `P[j] = nums[0] + … + nums[j]` be the running prefix sum. Then

```
sum(i..j) = P[j] - P[i-1]
sum(i..j) == k   ⇔   P[i-1] == P[j] - k
```

So while sweeping `j`, the number of subarrays ending at `j` with sum `k` is exactly **how many earlier prefix values equal `P[j] - k`**. Keep a frequency map of prefix values seen so far.

The seed `count[0] = 1` represents the *empty prefix* (`P[-1] = 0`) and is what allows subarrays that start at index 0 to be counted. Forgetting it is the classic bug.

Data structures. `defaultdict(int)` mapping prefix-sum value → how many times it has occurred so far.

Algorithm.

1. `count = {0: 1}`, `running = 0`, `total = 0`.
2. For each `x`: `running += x`; `total += count[running - k]`; `count[running] += 1`.
3. Return `total`.

Look up **before** inserting so a subarray is never matched against its own endpoint (this matters when `k == 0`).

Complexity. Time `O(n)` — one pass, `O(1)` amortised map ops. Space `O(n)` — up to `n+1` distinct prefix values.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def subarraySum(self, nums: list[int], k: int) -> int:
        # freq[p] = how many prefixes seen SO FAR have running sum p.
        freq = defaultdict(int)
        freq[0] = 1          # the empty prefix: lets subarrays starting at index 0 count

        running = 0          # P[j]
        total = 0

        for x in nums:
            running += x

            # Every earlier prefix equal to (running - k) marks a valid start point,
            # because running - P[i-1] == k. Count them all at once.
            total += freq[running - k]

            # Record this prefix only AFTER the lookup, so the current subarray
            # can never pair with itself (critical when k == 0).
            freq[running] += 1

        return total
```

DRY RUN

`nums = [1, 2, 3]`, `k = 3`

x	running	`running - k`	`freq[running-k]`	total	freq after
—	0	—	—	0	`{0:1}`
1	1	−2	0	0	`{0:1, 1:1}`
2	3	0	**1** ← the empty prefix	1	`{0:1, 1:1, 3:1}`
3	6	3	**1** ← prefix after `[1,2]`	2	`{0:1,1:1,3:1,6:1}`

Answer: `2` — the subarrays `[1,2]` and `[3]`. ✅

Notice the two hits came from different places: the `[1,2]` hit used `freq[0]`, which only exists because of the seed. The `[3]` hit used the prefix recorded two steps earlier.

Negatives case, `nums = [1, -1, 0]`, `k = 0`:

x	running	need	hits	total	freq after
—	0			0	`{0:1}`
1	1	1	0	0	`{0:1,1:1}`
−1	0	0	1	1	`{0:2,1:1}`
0	0	0	2	**3**	`{0:3,1:1}`

Answer `3`: `[1,-1]`, `[0]`, `[1,-1,0]`. A sliding window would have found none of this correctly — proof that negatives rule it out.

---

41. First Missing Positive ★★

UNDERSTAND THE PROBLEM

Given an unsorted array, return the smallest **positive** integer (≥ 1) that does not appear. Must be `O(n)` time **and** `O(1)` extra space.

- Key observation to state first: with `n` elements, the answer must lie in `[1, n+1]`. If all of `1..n` are present the answer is `n+1`; otherwise it's the smallest gap. Anything outside that range is irrelevant.
- Array may contain negatives, zeros, duplicates, and values far beyond `n`.
- The `O(1)` space rule is what forces the trick: **use the array itself as the hash table.**
- Mutating the input is normally allowed here; ask.

BRUTE-FORCE APPROACH

Idea. Put everything in a set, then test `1, 2, 3, …` until one is missing.

Data structures. A `set`.

Algorithm.

1. `s = set(nums)`.
2. For `i` from 1 upward: if `i not in s`, return `i`.

Complexity. Time `O(n)`. Space `O(n)` — and that's the violation.

(The sorting variant — sort then scan for the first gap — is `O(n log n)` time, also failing the requirement.)

Why it's not ideal. It's linear in time but the set is `O(n)` auxiliary memory, which the problem explicitly forbids. The insight it *does* give you: what we need is a structure answering "is value `v` present?" with `v` restricted to `1..n` — and an array of length `n` indexed `0..n-1` is already exactly that shape. We just have to place each value at its own index.

OPTIMAL APPROACH

Idea (cyclic sort / index-as-hash). Rearrange the array in place so that value `v` sits at index `v-1`, for every `v` in `[1, n]`. Then scan: the first index `i` where `nums[i] != i+1` gives the answer `i+1`. If no such index, the answer is `n+1`.

Data structures. None — in-place swapping on the input array.

Algorithm.

1. For `i` in `0..n-1`:
2. While `1 ≤ nums[i] ≤ n` **and** `nums[nums[i]-1] != nums[i]`: swap `nums[i]` with `nums[nums[i]-1]`.
3. For `i` in `0..n-1`: if `nums[i] != i+1`, return `i+1`.
4. Return `n+1`.

The loop condition `nums[nums[i]-1] != nums[i]` — "the destination slot doesn't already hold this value" — is the termination guard. Writing `nums[i] != i+1` instead causes an **infinite loop** on duplicates like `[1, 1]`.

Why it's linear despite the nested `while`. Every swap places at least one value into its final correct slot, and a value never leaves a correct slot. So there are at most `n` swaps across the entire outer loop → `O(n)` total.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def firstMissingPositive(self, nums: list[int]) -> int:
        n = len(nums)

        # PHASE 1 — place each value v in [1, n] at index v-1 ("cyclic sort").
        for i in range(n):
            # Keep swapping whatever lands in slot i until it's either
            #   - out of the useful range [1, n], or
            #   - already sitting where it belongs, or
            #   - a duplicate of the value already at its destination.
            while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                target = nums[i] - 1          # where nums[i] wants to live
                # Cache `target` first: after the swap, nums[i] changes, so
                # recomputing the index inline would read the wrong value.
                nums[i], nums[target] = nums[target], nums[i]
            # Amortised O(1): every swap finalises one element permanently,
            # so the total swap count across the whole loop is <= n.

        # PHASE 2 — the first slot not holding its own index+1 is the gap.
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1

        # All of 1..n present -> answer is n+1.
        return n + 1
```

DRY RUN

`nums = [3, 4, -1, 1]`, `n = 4`

Phase 1:

i	array before	condition	action	array after
0	`[3,4,-1,1]`	`nums[0]=3` in range; `nums[2]=-1 ≠ 3`	swap idx 0 ↔ 2	`[-1,4,3,1]`
0	`[-1,4,3,1]`	`nums[0]=-1` out of range	stop	`[-1,4,3,1]`
1	`[-1,4,3,1]`	`nums[1]=4` in range; `nums[3]=1 ≠ 4`	swap idx 1 ↔ 3	`[-1,1,3,4]`
1	`[-1,1,3,4]`	`nums[1]=1` in range; `nums[0]=-1 ≠ 1`	swap idx 1 ↔ 0	`[1,-1,3,4]`
1	`[1,-1,3,4]`	`nums[1]=-1` out of range	stop	`[1,-1,3,4]`
2	`[1,-1,3,4]`	`nums[2]=3`; `nums[2]=3 == 3` → already home	stop	unchanged
3	`[1,-1,3,4]`	`nums[3]=4`; `nums[3]=4 == 4` → home	stop	unchanged

Phase 2: `nums = [1, -1, 3, 4]`

i	nums[i]	expected `i+1`	match?
0	1	1	✅
1	−1	2	❌ → **return 2**

Answer: `2`. ✅

Duplicate guard, `nums = [1, 1]`: at `i=1`, `nums[1]=1`, destination index `0` already holds `1`, so `nums[0] == nums[1]` and the `while` exits immediately. Phase 2 finds `nums[1]=1 != 2` → returns `2`. With the wrong condition `nums[i] != i+1`, this would swap `1` and `1` forever.

15. 3Sum ★

UNDERSTAND THE PROBLEM

Given an integer array, return **all unique triplets** `[a, b, c]` with `a + b + c == 0`. The result must contain no duplicate triplets (as multisets), though the array itself may contain duplicate values.

- "Unique triplets" is the entire difficulty. `[-1,0,1]` must appear once even if the array admits it via two different index sets.
- Edge cases: fewer than 3 elements → `[]`; all zeros `[0,0,0,0]` → `[[0,0,0]]` exactly once; no valid triplet.
- Indices must be distinct; values may repeat.

BRUTE-FORCE APPROACH

Idea. Three nested loops over `i < j < k`, collect the triplets whose sum is zero, deduplicate at the end by sorting each triplet and putting it in a set.

Data structures. A `set` of sorted tuples.

Algorithm.

1. For `i < j < k`: if `nums[i]+nums[j]+nums[k] == 0`, add `tuple(sorted(...))` to a set.
2. Return the set as a list of lists.

Complexity. Time `O(n³)`. Space `O(#triplets)`.

Why it's not ideal. `n = 3000` makes `n³ = 2.7·10¹⁰` — hopeless. Also, deduplicating *after the fact* by hashing sorted tuples is a workaround, not a solution; the good algorithm makes duplicates structurally impossible instead. The reduction to see: fix one element and the remaining task is "find two numbers summing to a target", which on a **sorted** array is `O(n)` with two pointers.

OPTIMAL APPROACH

Idea. Sort first. Sorting buys two things at once: 1. Two-pointer convergence becomes valid — if the current sum is too small, the only way to increase it is to move the left pointer right. 2. Duplicates become adjacent, so skipping them is a local `while`.

Then fix `i` and run a two-pointer sweep over `[i+1, n-1]` looking for the pair summing to `-nums[i]`.

Dedupe at **both** levels: - **outer:** if `nums[i] == nums[i-1]`, skip — that anchor value was already fully explored. - **inner:** after recording a hit, advance `lo` past equal values and `hi` back past equal values.

Data structures. Sorted input array; two integer pointers.

Algorithm.

1. `nums.sort()`; `res = []`.
2. For `i` in `0..n-3`:
3. If `nums[i] > 0`: break (everything to the right is ≥ 0, sum can't be 0).
4. If `i > 0 and nums[i] == nums[i-1]`: continue.
5. `lo, hi = i+1, n-1`.
6. While `lo < hi`:
   - `s = nums[i] + nums[lo] + nums[hi]`.
   - If `s < 0`: `lo += 1`. If `s > 0`: `hi -= 1`.
   - Else record, then `lo += 1`, `hi -= 1`, and skip duplicates on both sides.
7. Return `res`.

Complexity. Time `O(n²)` — `O(n log n)` sort plus `n` anchors each doing an `O(n)` sweep. Space `O(1)` extra (excluding output and sort's internal cost).

PYTHON CODE

```python
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()            # sorting enables two pointers AND makes duplicates adjacent
        n = len(nums)
        res = []

        for i in range(n - 2):
            # Prune: sorted, so if the smallest of the three is > 0 the sum is > 0.
            if nums[i] > 0:
                break

            # OUTER dedupe: this anchor value was fully explored on the previous i.
            # Note `i > 0` guard — we must still process the very first element.
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            lo, hi = i + 1, n - 1
            while lo < hi:
                s = nums[i] + nums[lo] + nums[hi]

                if s < 0:
                    lo += 1        # need a bigger sum -> take a bigger small value
                elif s > 0:
                    hi -= 1        # need a smaller sum -> take a smaller large value
                else:
                    res.append([nums[i], nums[lo], nums[hi]])

                    # INNER dedupe: move both pointers off their current values entirely,
                    # otherwise [-2,0,0,2,2] emits [-2,0,2] twice.
                    lo += 1
                    hi -= 1
                    while lo < hi and nums[lo] == nums[lo - 1]:
                        lo += 1
                    while lo < hi and nums[hi] == nums[hi + 1]:
                        hi -= 1

        return res
```

DRY RUN

`nums = [-1, 0, 1, 2, -1, -4]` → sorted: `[-4, -1, -1, 0, 1, 2]`

i = 0, `nums[0] = -4`, need pair summing to `4`. `lo=1, hi=5`:

lo	hi	values	sum	action
1	5	−4, −1, 2	−3	< 0 → lo++
2	5	−4, −1, 2	−3	< 0 → lo++
3	5	−4, 0, 2	−2	< 0 → lo++
4	5	−4, 1, 2	−1	< 0 → lo++
5	5	—	—	`lo < hi` fails, exit

i = 1, `nums[1] = -1` (i>0 but `nums[0] = -4 ≠ -1`, so no skip). `lo=2, hi=5`:

lo	hi	values	sum	action
2	5	−1, −1, 2	**0**	record `[-1,-1,2]`; lo→3, hi→4; dedupe: `nums[3]=0 ≠ nums[2]=-1` ok, `nums[4]=1 ≠ nums[5]=2` ok
3	4	−1, 0, 1	**0**	record `[-1,0,1]`; lo→4, hi→3
4	3	—	—	exit

i = 2, `nums[2] = -1` — equals `nums[1]`, **skip** (outer dedupe). This is what prevents `[-1,0,1]` appearing twice.

i = 3, `nums[3] = 0` — `lo=4, hi=5`: `0+1+2 = 3 > 0` → `hi--` → `lo==hi`, exit.

Answer: `[[-1,-1,2], [-1,0,1]]`. ✅

11. Container With Most Water

UNDERSTAND THE PROBLEM

Given `height[i]` describing vertical lines at x-coordinates `0..n-1`, pick two lines so the container they form with the x-axis holds the most water. Return that area. `area = (j − i) × min(height[i], height[j])`.

- Water is bounded by the **shorter** of the two lines — that's the crux.
- The lines in between are ignored (unlike problem 42, which is about the space *between* the bars).
- Edge cases: `n = 2` (only one choice); all equal heights; a zero height.

BRUTE-FORCE APPROACH

Idea. Try every pair.

Data structures. None.

Algorithm. For `i < j`: `area = (j-i) * min(height[i], height[j])`; track max.

Complexity. Time `O(n²)`. Space `O(1)`.

Why it's not ideal. `n = 10⁵` → $5 \cdot 10^9$ pairs. But more importantly, most of those pairs are provably worse than pairs we've already seen, and there's an exchange argument that lets us discard `n` candidates in a single pass.

OPTIMAL APPROACH

Idea (two pointers from the ends, move the limiting side). Start with the widest possible container: `lo = 0`, `hi = n-1`. Its area is `(hi-lo) * min(h[lo], h[hi])`.

Now the key argument: **the shorter line cannot do better with any other partner.** Suppose `h[lo] ≤ h[hi]`. Any container using `lo` and some `j < hi` has width strictly smaller *and* height still capped at `h[lo]`. So it's `≤` the current area. Therefore `lo` is exhausted — discard it and move `lo` right. Symmetrically for `hi`.

Each step retires one line, so the whole array is processed in one linear sweep.

Data structures. Two integer pointers, one running max.

Algorithm.

1. `lo, hi, best = 0, n-1, 0`.
2. While `lo < hi`:
3. `best = max(best, (hi - lo) * min(h[lo], h[hi]))`.
4. If `h[lo] < h[hi]`: `lo += 1`, else `hi -= 1`.
5. Return `best`.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def maxArea(self, height: list[int]) -> int:
        lo, hi = 0, len(height) - 1
        best = 0

        while lo < hi:
            # Water level is set by the SHORTER wall; width is the index gap.
            h = min(height[lo], height[hi])
            best = max(best, (hi - lo) * h)

            # Move the limiting (shorter) side inward.
            # Justification: keeping the shorter wall and shrinking the width can
            # only lower the area — its height is already capped at h and any new
            # partner is closer. So that wall can be permanently discarded.
            if height[lo] < height[hi]:
                lo += 1
            else:
                hi -= 1
            # Ties: moving either side is fine — both walls are equally limiting.

        return best
```

DRY RUN

`height = [1, 8, 6, 2, 5, 4, 8, 3, 7]`, `n = 9`

lo	hi	h[lo]	h[hi]	width	min	area	best	move
0	8	1	7	8	1	8	8	lo++ (1 < 7)
1	8	8	7	7	7	**49**	49	hi−− (8 ≥ 7)
1	7	8	3	6	3	18	49	hi−−
1	6	8	8	5	8	40	49	hi−− (tie)
1	5	8	4	4	4	16	49	hi−−
1	4	8	5	3	5	15	49	hi−−
1	3	8	2	2	2	4	49	hi−−
1	2	8	6	1	6	6	49	hi−−
1	1	—	—	—	—	—	49	exit

Answer: `49` — lines at indices 1 and 8, `min(8,7) × 7`. ✅

Note step 1: we moved `lo` off the height-1 wall immediately, correctly discarding all 8 containers that used it, without ever evaluating them.

---

42. Trapping Rain Water ★★

UNDERSTAND THE PROBLEM

Given bar heights of unit width, compute how much rainwater is trapped between them after it rains.

- Water above position `i` is `min(maxLeft[i], maxRight[i]) − height[i]`, clamped at `≥ 0` — where `maxLeft` / `maxRight` are the tallest bars at or before/after `i`.
- Water only sits where there's a taller bar on **both** sides.
- Edge cases: monotonic array (traps nothing); empty; all equal.
- Different from problem 11: here every bar in between matters.

BRUTE-FORCE APPROACH

Idea. For each index, scan left for the max and right for the max, then add the trapped amount.

Data structures. None.

Algorithm.

1. `total = 0`.
2. For each `i`: `L = max(height[0..i])`, `R = max(height[i..n−1])`, `total += min(L, R) − height[i]`.

Complexity. Time `O(n²)`. Space `O(1)`.

Why it's not ideal. Each index rescans the entire array for two values that are highly redundant across neighbours: `maxLeft[i]` is just `max(maxLeft[i−1], height[i])`. That single recurrence turns the whole thing linear.

OPTIMAL APPROACH — LEVEL 1: PREFIX/SUFFIX ARRAYS, `O(N)` TIME / `O(N)` SPACE

Idea. Precompute `maxLeft[]` in a forward pass and `maxRight[]` in a backward pass, then one more pass to sum the water.

Algorithm.

1. `maxLeft[0] = height[0]`; `maxLeft[i] = max(maxLeft[i−1], height[i])`.
2. `maxRight[n−1] = height[n−1]`; `maxRight[i] = max(maxRight[i+1], height[i])`.
3. `total = Σ min(maxLeft[i], maxRight[i]) − height[i]`.

Complexity. Time `O(n)`, space `O(n)`.

OPTIMAL APPROACH — LEVEL 2: TWO POINTERS, `O(N)` TIME / `O(1)` SPACE

Idea. We never actually need both maxima at full precision — we only need `min(maxLeft, maxRight)`. Maintain `lo` / `hi` pointers with running `leftMax` / `rightMax`. If `leftMax < rightMax`, then for position `lo` the binding constraint is *definitely* `leftMax`, regardless of what lies between — because there is already some bar at or beyond `hi` at least as tall as `rightMax > leftMax`. So we can settle position `lo` immediately and advance. Symmetric on the other side.

Data structures. Four scalars: `lo, hi, leftMax, rightMax`.

Algorithm.

1. `lo, hi = 0, n−1`; `leftMax = rightMax = 0`; `total = 0`.
2. While `lo < hi`:
3. If `height[lo] < height[hi]`:
   - `leftMax = max(leftMax, height[lo])`; `total += leftMax − height[lo]`; `lo += 1`.
4. Else:
   - `rightMax = max(rightMax, height[hi])`; `total += rightMax − height[hi]`; `hi −= 1`.
5. Return `total`.

Complexity. Time `O(n)`, space `O(1)`.

A monotonic-stack solution also exists (it accumulates water in horizontal layers) — mention it as the third way, but the two-pointer version is the one to write.

PYTHON CODE

```python
class Solution:
    def trap(self, height: list[int]) -> int:
        """Two-pointer, O(n) time / O(1) space."""
        if not height:
            return 0

        lo, hi = 0, len(height) - 1
        left_max = right_max = 0
        total = 0

        while lo < hi:
            # Whichever side is CURRENTLY shorter is the side whose water level
            # is already determined. If height[lo] < height[hi] then some bar at
            # or beyond hi is taller than height[lo], so the right-hand bound for
            # position lo is guaranteed >= left_max — meaning min(L, R) == left_max.
            if height[lo] < height[hi]:
                left_max = max(left_max, height[lo])
                total += left_max - height[lo]   # never negative: left_max >= height[lo]
                lo += 1
            else:
                right_max = max(right_max, height[hi])
                total += right_max - height[hi]
                hi -= 1

        return total


class SolutionPrefixArrays:
    """Level-1 version — O(n) time / O(n) space. Easier to explain; write this first
    in an interview, then offer the O(1)-space upgrade."""
    def trap(self, height: list[int]) -> int:
        n = len(height)
        if n < 3:
            return 0

        max_left = [0] * n
        max_left[0] = height[0]
        for i in range(1, n):
            max_left[i] = max(max_left[i - 1], height[i])

        max_right = [0] * n
        max_right[n - 1] = height[n - 1]
        for i in range(n - 2, -1, -1):
            max_right[i] = max(max_right[i + 1], height[i])

        return sum(min(max_left[i], max_right[i]) - height[i] for i in range(n))
```

DRY RUN

`height = [0,1,0,2,1,0,1,3,2,1,2,1]`, `n = 12`, expected answer `6`.

Two-pointer trace (`lm` = left_max, `rm` = right_max):

lo	hi	h[lo]	h[hi]	branch	lm	rm	added	total
0	11	0	1	left (0<1)	0	0	0−0=0	0
1	11	1	1	right (1≥1)	0	1	1−1=0	0
1	10	1	2	left (1<2)	1	1	1−1=0	0
2	10	0	2	left	1	1	1−0=**1**	1
3	10	2	2	right (2≥2)	1	2	2−2=0	1
3	9	2	1	right	1	2	2−1=**1**	2
3	8	2	2	right	1	2	2−2=0	2
3	7	2	3	left (2<3)	2	2	2−2=0	2
4	7	1	3	left	2	2	2−1=**1**	3
5	7	0	3	left	2	2	2−0=**2**	5
6	7	1	3	left	2	2	2−1=**1**	6
7	7	—	—	—	—	—	—	**6**

Answer: `6`. ✅

Sanity-check index 5 by hand: `maxLeft[5] = max(0,1,0,2,1,0) = 2`, `maxRight[5] = max(0,1,3,2,1,2,1) = 3`, water `= min(2,3) − 0 = 2`. Matches the table.

Common Manager — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

An org tree: each manager has zero or more direct reports (an n-ary tree, one root = the CEO). Given two employees, return their **lowest common manager** — the deepest node whose subtree contains both. Follow-up: generalise to `k` employees.

- Clarify: *"Is an employee their own manager if the other is beneath them?"* Standard answer: yes, a node counts as an ancestor of itself — same convention as LeetCode 236. Ask, and state your assumption.
- Both employees are guaranteed present in the tree (ask; if not, you must return `None` and detect partial matches).
- Edge cases: the two employees are the same node; one is an ancestor of the other; the root is the answer.

BRUTE-FORCE APPROACH

Idea. For every node in the tree, test whether its subtree contains both targets. Among all such nodes, return the deepest one.

Data structures. A recursive `contains(node, target)` helper.

Algorithm.

1. For each node (via any traversal): if `contains(node, a)` and `contains(node, b)`, it's a candidate.
2. Return the deepest candidate.

Complexity. Time `O(n²)` — `n` nodes × an `O(n)` subtree scan each. Space `O(h)` recursion.

Why it's not ideal. `contains()` re-walks overlapping subtrees over and over: the root's check traverses everything, then each child repeats most of it. A single post-order pass can compute "does my subtree contain a? does it contain b?" for **all** nodes simultaneously by combining children's answers — the classic bottom-up tree DP.

A second, decent approach worth mentioning: collect each employee's ancestor chain to the root (if nodes have parent pointers, or via one DFS storing paths), then walk both chains from the root and take the last common node. `O(n)` time, `O(h)` space. If parent pointers exist, this is arguably the cleanest solution — say so.

OPTIMAL APPROACH

Idea (single post-order DFS). Define `dfs(node)` returning the LCA found within `node`'s subtree, or `None`. - If `node` is `None`: return `None`. - If `node` is `a` or `b`: return `node` — this node itself is a witness (and, given the "self-ancestor" convention, is the answer if the other target is beneath it). - Otherwise recurse on all children. Count how many children returned non-`None`. - **≥ 2 non-null children** → the two targets live in different subtrees, so `node` is the split point → return `node`. - **exactly 1** → propagate that child's result upward. - **0** → return `None`.

Data structures. Recursion stack only.

Algorithm for k employees. Same shape, but return `(node_or_None, count_found)`: sum the counts from children plus 1 if this node is itself a target; the first node whose count reaches `k` (checked on the way up) is the answer.

Complexity. Time `O(n)` — every node visited once. Space `O(h)` for the recursion, `h` = tree height (`O(n)` worst case on a chain of managers).

PYTHON CODE

```python
from typing import Optional, Iterable


class Employee:
    def __init__(self, name: str, reports: Optional[list["Employee"]] = None):
        self.name = name
        self.reports = reports or []       # direct reports (n-ary children)


# ---------- Part 1: two employees ----------------------------------------
def lowest_common_manager(root: Optional[Employee],
                          a: Employee,
                          b: Employee) -> Optional[Employee]:
    """Deepest node whose subtree contains both a and b.
    Convention: a node is an ancestor of itself."""
```

```python
    if root is None:
        return None

    # Hit either target -> report it upward. We do NOT recurse further:
    # if the other target is below us, WE are the lowest common manager,
    # and the parent's "one non-null child" rule propagates us correctly.
    if root is a or root is b:
        return root

    found = None       # the single non-null result seen so far
    hits = 0           # how many children reported a hit

    for child in root.reports:
        result = lowest_common_manager(child, a, b)
        if result is not None:
            hits += 1
            found = result
            if hits == 2:
                # a and b are in two DIFFERENT subtrees -> this node splits them.
                return root

    # hits == 1 -> bubble that subtree's answer up unchanged
    # hits == 0 -> nothing here, report None
    return found


# ---------- Part 2: k employees (the follow-up) -------------------------
def lowest_common_manager_k(root: Optional[Employee],
                            targets: Iterable[Employee]) -> Optional[Employee]:
    """Generalisation: deepest node whose subtree contains ALL k targets."""
    target_set = set(id(t) for t in targets)   # id() so we compare by identity
    k = len(target_set)
    answer = None

    def dfs(node: Optional[Employee]) -> int:
        """Returns how many targets live in node's subtree (node included)."""
        nonlocal answer
        if node is None:
            return 0

        count = 1 if id(node) in target_set else 0
        for child in node.reports:
            count += dfs(child)

        # First time a subtree reaches k, we're at the DEEPEST such node, because
        # post-order means every strictly-deeper node has already been evaluated
        # and none of them reached k (otherwise answer would already be set).
        if count == k and answer is None:
            answer = node
        return count

    dfs(root)
    return answer
```

DRY RUN

```
          CEO
         /   \
       Ann    Bob
      /  \      \
    Cal  Dee    Eve
```

Query: `lowest_common_manager(CEO, Cal, Dee)`.

call	node	is target?	children results	hits	returns
1	CEO	no	see below		
2	Ann	no			
3	Cal	**yes (a)**	—	—	`Cal`
4	Dee	**yes (b)**	—	—	`Dee`
2 (cont.)	Ann		Cal✓, Dee✓	**2**	`Ann` ← split point
5	Bob	no			
6	Eve	no	none	0	`None`
5 (cont.)	Bob		Eve✗	0	`None`
1 (cont.)	CEO		Ann✓, Bob✗	1	`Ann` propagated up

Answer: **Ann**. ✅

Second query: `lowest_common_manager(CEO, Ann, Dee)` — ancestor case.

call	node	result
CEO	not a target, recurse	
Ann	**is target** `a` → return `Ann` immediately, never descends to Dee	`Ann`
Bob → Eve	no hits	`None`
CEO	hits = 1 → propagate	**`Ann`**

Answer: **Ann** — correct under the self-ancestor convention. Note we never even visited Dee; the early return is what makes that safe.

Friends Purchases — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given two APIs — `getFriends(person)` returning that person's friends, and `getPurchases(person)` returning the products a person bought — return the **most popular products among a user's friends**.

Popularity = the number of *distinct friends* who bought the product, not total quantity. So if one friend bought the same item five times, that counts as **1**.

Common variant: **exclude products the user already owns.**

- Clarify out loud: *"Is friendship one-hop only, or should I go transitively? Top-`k` or all products above a threshold? How do I break ties? Are the API calls remote/expensive?"* Every one of these changes the code.
- Edge cases: user has no friends → empty; friends bought nothing; the user is in their own friend list (exclude); duplicates within one friend's purchase list (the whole point).

BRUTE-FORCE APPROACH

Idea. Concatenate every friend's purchase list into one big list, count occurrences, sort descending, take the top `k`.

Data structures. A list of all purchases, then a `Counter`, then a sort.

Algorithm.

1. `all_items = []`; for each friend: `all_items += getPurchases(friend)`.
2. `counts = Counter(all_items)`.
3. `sorted(counts.items(), key=lambda kv: -kv[1])[:k]`.

Complexity. Time `O(P + U log U)` where `P` = total purchase records and `U` = distinct products. Space `O(P)`.

Why it's not ideal. Two separate problems: 1. **It's wrong.** Counting the raw concatenated list counts *purchases*, not *distinct friends*. A friend who bought the same product three times inflates it to 3. The spec says popularity is distinct-friend count — this is the trap the question is built around. 2. **It's over-sorted.** Sorting all `U` products to take the top `k` is `O(U log U)` when a size-`k` heap gives `O(U log k)`. When `k` is small and `U` is large (a real product catalogue), that's a large constant-factor win, and it's the answer the interviewer is fishing for.

OPTIMAL APPROACH

Idea. Two corrections to the brute force: 1. **Dedupe per friend before counting** — wrap each friend's purchases in a `set()`, so each friend contributes at most 1 to each product's count. This makes the count "number of distinct friends", by construction. 2. **Use `heapq.nlargest(k, …)`** instead of a full sort — a bounded min-heap of size `k`.

Also: exclude the user's own products via a set difference, and exclude the user from their own friend list.

Data structures. `set` per friend (dedupe) · `Counter` (product → distinct-friend count) · `set` of the user's own products · a size-`k` heap.

Algorithm.

1. `owned = set(getPurchases(user))`.
2. `counts = Counter()`.
3. For each `friend` in `getFriends(user)`: skip if `friend == user`; for each `product` in `set(getPurchases(friend))`: if `product not in owned`: `counts[product] += 1`.
4. Return `heapq.nlargest(k, counts.items(), key=…)` with a deterministic tie-break.

Tie-breaking. Always propose one explicitly — "count descending, then product id ascending" is a safe default and makes the output deterministic (interviewers test for this).

Complexity. Let `F` = number of friends, `P` = total purchase records across friends, `U` = distinct products. Time `O(P + U log k)`. Space `O(U + max per-friend purchases)`.

PYTHON CODE

```python
import heapq
from collections import Counter
from typing import Callable, Iterable, Hashable
```

```python
def most_popular_among_friends(
    user: Hashable,
    k: int,
    get_friends: Callable[[Hashable], Iterable[Hashable]],
    get_purchases: Callable[[Hashable], Iterable[Hashable]],
    exclude_owned: bool = True,
) -> list[Hashable]:
    """
    Popularity = number of DISTINCT friends who bought the product.
    Ties broken by product id ascending, so the output is deterministic.
    """
    # Products the user already has — excluded from recommendations if asked.
    owned = set(get_purchases(user)) if exclude_owned else set()

    counts = Counter()

    for friend in get_friends(user):
        if friend == user:
            continue                    # don't let the user recommend to themselves

        # set(...) is THE line that makes this correct: it collapses one friend's
        # repeat purchases of the same product into a single vote. Counting the raw
        # list instead would measure quantity, not distinct-friend popularity.
        for product in set(get_purchases(friend)):
            if product in owned:
                continue
            counts[product] += 1

    # nlargest keeps a heap of size k -> O(U log k), better than an O(U log U) sort
    # when k << U. Key: count descending, then product ascending for determinism.
    # (Negating the count lets us sort both fields "descending" in one tuple.)
    top = heapq.nlargest(k, counts.items(), key=lambda kv: (kv[1], _neg_key(kv[0])))
    return [product for product, _count in top]


def _neg_key(product):
    """Invert the tie-break direction for nlargest: we want SMALLER ids to win ties,
    but nlargest keeps the LARGEST keys. Works for ints and strings alike."""
    if isinstance(product, (int, float)):
        return -product
    return tuple(-ord(c) for c in str(product))


# --- Simpler, interview-friendly variant if ties don't need specifying ----
def most_popular_simple(user, k, get_friends, get_purchases):
    counts = Counter()
    owned = set(get_purchases(user))
    for friend in get_friends(user):
        counts.update(set(get_purchases(friend)) - owned)   # set() = distinct-friend vote
    return [p for p, _ in counts.most_common(k)]
```

DRY RUN

```
FRIENDS   = {"me": ["ann", "bob", "cal"]}
PURCHASES = {
    "me":  ["laptop"],
    "ann": ["kindle", "kindle", "kindle", "echo"],   # bought kindle x3
    "bob": ["kindle", "laptop"],
    "cal": ["echo", "laptop", "kindle"],
}
```

Query: `most_popular_among_friends("me", k=2, …)`

`owned = {"laptop"}`

friend	raw purchases	`set(...)`	after removing owned	counts after
ann	kindle×3, echo	{kindle, echo}	{kindle, echo}	`{kindle:1, echo:1}`
bob	kindle, laptop	{kindle, laptop}	{kindle}	`{kindle:2, echo:1}`
cal	echo, laptop, kindle	{echo, laptop, kindle}	{echo, kindle}	`{kindle:3, echo:2}`

`nlargest(2)` → `kindle (3)`, `echo (2)`.

Answer: `["kindle", "echo"]`. ✅

The trap, made visible: without the `set()` per friend, ann's three kindle purchases would count as 3, giving `kindle: 5` — the *rank* happens to survive here, but on input where one friend buys product X ten times and three friends each buy Y once, the wrong version ranks X above Y when the spec clearly ranks Y above X.

DAY 2 — Tue Aug 18 · Sliding Window & Prefix Sum

> **The pattern split for the whole day.** Sliding window works when the window's validity is **monotone** — growing the window can only make it "more invalid", shrinking can only make it "more valid". That holds for *positive* numbers and for *distinct-character* constraints. It breaks the moment negatives appear, and then you need **prefix sum + hashmap** instead. Problems 3/424/209/438/567/76/992/1658/239 are windows; 974/525 are prefix maps. Knowing which of the two a prompt implies, in ten seconds, is the day's real skill.

---

3. Longest Substring Without Repeating Characters ★★★

UNDERSTAND THE PROBLEM

Given a string `s`, return the **length** of the longest substring containing no repeated characters.

- Substring = contiguous. (Subsequence would be a totally different problem.)
- Edge cases: `""` → 0; all identical `"bbbb"` → 1; all distinct → `len(s)`; spaces and symbols count as characters.
- Clarify: *"Is the alphabet ASCII or Unicode? Do I return the length or the substring itself?"*

BRUTE-FORCE APPROACH

Idea. Generate every substring and check each for uniqueness.

Data structures. A `set` per substring check.

Algorithm.

1. For each start `i`, for each end `j ≥ i`: build `s[i:j+1]`, test `len(set(sub)) == len(sub)`.
2. Track the max valid length.

Complexity. Time `O(n³)` naively (`n²` substrings × `O(n)` uniqueness check), or `O(n²)` if you extend the set incrementally as `j` grows and break on the first repeat. Space `O(min(n, alphabet))`.

Why it's not ideal. Even the `O(n²)` version restarts the set from scratch at every new `i`, throwing away the fact that most of `[i+1, j]` was already verified unique on the previous round. That discarded work is exactly what a sliding window preserves.

OPTIMAL APPROACH

Idea (variable-size window with last-seen index). Maintain a window `[left, right]` that is always duplicate-free. Extend `right` one character at a time. When `s[right]` was already seen **inside the current window**, jump `left` to `last[s[right]] + 1` — one move, not a slow shrink loop.

The guard `last[c] >= left` matters: a character seen long ago but already *outside* the window must not drag `left` backwards.

Data structures. `dict` mapping character → its most recent index.

Algorithm.

1. `last = {}`, `left = 0`, `best = 0`.
2. For `right, c` in `enumerate(s)`:
3. If `c in last and last[c] >= left`: `left = last[c] + 1`.
4. `last[c] = right`.
5. `best = max(best, right - left + 1)`.
6. Return `best`.

Complexity. Time `O(n)` — `right` advances `n` times, `left` only ever moves forward. Space `O(min(n, |alphabet|))`.

PYTHON CODE

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        last = {}          # char -> most recent index at which it appeared
        left = 0           # window start (inclusive)
        best = 0

        for right, c in enumerate(s):
            # Only jump if the previous sighting is INSIDE the current window.
            # Without `last[c] >= left`, a stale sighting from before `left`
            # would drag the window backwards. Example: "abba" — when we reach
            # the final 'a', last['a'] == 0 but left is already 2; ignoring the
            # guard would reset left to 1 and report a wrong length of 3.
            if c in last and last[c] >= left:
                left = last[c] + 1

            last[c] = right                      # record/refresh this char's position
            best = max(best, right - left + 1)   # window is guaranteed valid here

        return best
```

DRY RUN

`s = "abcabcbb"`

right	c	`last[c]` before	≥ left?	left	last after	window	len	best
0	a	—	—	0	a:0	`a`	1	1
1	b	—	—	0	b:1	`ab`	2	2
2	c	—	—	0	c:2	`abc`	3	**3**
3	a	0	0 ≥ 0 ✓	**1**	a:3	`bca`	3	3
4	b	1	1 ≥ 1 ✓	**2**	b:4	`cab`	3	3
5	c	2	2 ≥ 2 ✓	**3**	c:5	`abc`	3	3
6	b	4	4 ≥ 3 ✓	**5**	b:6	`cb`	2	3
7	b	6	6 ≥ 5 ✓	**7**	b:7	`b`	1	3

Answer: `3`. ✅

The `>= left` guard, isolated. `s = "abba"`:

right	c	last[c]	≥ left?	left	window	best
0	a	—		0	`a`	1
1	b	—		0	`ab`	2
2	b	1	1 ≥ 0 ✓	**2**	`b`	2
3	a	0	0 ≥ 2 ✗ — **ignored**	2	`ba`	2

Answer `2`. Without the guard, step 3 would set `left = 1`, yielding window `"bba"` — which contains a duplicate `b`. The guard is the correctness of the whole solution.

---

424. Longest Repeating Character Replacement

UNDERSTAND THE PROBLEM

Given string `s` and integer `k`, you may change **at most** `k` characters to any other uppercase letter. Return the length of the longest substring consisting of a single repeated letter that you can produce.

- Reframe immediately: a window is **valid** iff `window_length - (count of the most frequent char in it) ≤ k`. Those are the characters you'd have to change.
- Edge cases: `k = 0` (→ longest run of one letter); `k ≥ len(s)` (→ `len(s)`); `s` all one letter.

BRUTE-FORCE APPROACH

Idea. For each substring, count its characters, find the most frequent, and check whether the remainder is `≤ k`.

Data structures. A count array per substring.

Algorithm. Two nested loops over start/end; maintain a 26-count array incrementally as `end` grows; test `(end-start+1) - max(count) <= k`.

Complexity. Time `O(26 · n²)` — `n²` windows × `O(26)` to recompute the max. Space `O(26)`.

Why it's not ideal. Quadratic on `n = 10⁵`. And structurally, the validity test is monotone in window size (a *larger* window is never easier to satisfy), which is the exact precondition for a one-pass sliding window.

OPTIMAL APPROACH

Idea. Grow `right`. Track `count[]` of the window and `max_freq` = the largest count seen *at any point*. Window is valid while `(right - left + 1) - max_freq <= k`. When invalid, advance `left` once (not a loop) — because we never need the window to shrink by more than one per step: the answer can only ever grow by one per iteration, so a window that never shrinks past its historical best is sufficient.

The subtle bit interviewers probe: `max_freq` is deliberately **never decreased** when `left` moves. That looks like a bug and isn't. A stale (too large) `max_freq` can only make the window look *more* valid, which can only let the window stay the same size — never grow. Since we only ever record `best` as the current window size, and the window size never exceeds a size that was genuinely achievable at some earlier point, the answer stays correct. Say this out loud; it's the whole reason the solution is `O(n)` rather than `O(26n)`.

Data structures. A 26-length count array (or `Counter`); two pointers; `max_freq` scalar.

Algorithm.

1. `count = [0]*26`, `left = 0`, `max_freq = 0`, `best = 0`.
2. For `right` in `0..n-1`:
3. `count[s[right]] += 1`; `max_freq = max(max_freq, count[s[right]])`.
4. If `(right - left + 1) - max_freq > k`: `count[s[left]] -= 1`; `left += 1`.
5. `best = max(best, right - left + 1)`.
6. Return `best`.

Complexity. Time `O(n)`. Space `O(26) = O(1)`.

PYTHON CODE

```python
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        count = [0] * 26     # frequency of each letter inside the window
        left = 0
        max_freq = 0         # highest single-letter count EVER seen in a window
        best = 0

        for right in range(len(s)):
            idx = ord(s[right]) - ord('A')
            count[idx] += 1
            max_freq = max(max_freq, count[idx])

            # Characters we'd have to change = window size minus the dominant letter.
            # If that exceeds k the window is invalid -> shed one char from the left.
            # `if`, not `while`: the window never needs to shrink by more than one
            # per step, because it only ever grew by one.
            if (right - left + 1) - max_freq > k:
                count[ord(s[left]) - ord('A')] -= 1
                left += 1
                # NOTE: max_freq is intentionally NOT recomputed here.
                # A stale-high max_freq only makes the validity test more lenient,
                # which can keep the window the same size but never grow it beyond
                # a size that was genuinely achievable earlier. So `best` stays correct
                # and we avoid an O(26) rescan on every shrink.

            best = max(best, right - left + 1)

        return best
```

DRY RUN

`s = "AABABBA"`, `k = 1`

right	c	count	max_freq	size	size−mf	> k?	left	best
0	A	A:1	1	1	0	no	0	1
1	A	A:2	2	2	0	no	0	2
2	B	A:2 B:1	2	3	1	no (1 ≤ 1)	0	**3**
3	A	A:3 B:1	3	4	1	no	0	**4**
4	B	A:3 B:2	3	5	2	**yes** → drop `s[0]='A'`	1	4
5	B	A:2 B:3	3	5	2	**yes** → drop `s[1]='A'`	2	4
6	A	A:2 B:3	3	5	2	**yes** → drop `s[2]='B'`	3	4

Answer: `4` — window `"AABA"` at indices 0–3: change the single `B` to `A`. ✅

Watch rows 4–6: the window size froze at 5→ shrinks to 4 each time, so `best` never rises. That's the stale-`max_freq` behaviour working exactly as argued — it never produces a *larger* answer than a genuinely achievable one.

209. Minimum Size Subarray Sum

UNDERSTAND THE PROBLEM

Given an array of **positive** integers and a target, return the **minimal length** of a contiguous subarray whose sum is `≥ target`. Return `0` if none exists.

- "Positive integers" is load-bearing: it makes the running sum strictly increasing as the window grows, which is what makes the shrink-from-the-left step valid.
- This is the **shortest**-window variant, so the shrink loop runs *while valid* (opposite of 3/424, which shrink *while invalid*).
- Edge cases: no subarray reaches the target → `0`; a single element already ≥ target → `1`.

BRUTE-FORCE APPROACH

Idea. For every start, extend the end until the sum reaches the target, record the length.

Data structures. None.

Algorithm. Nested loops with a running sum; break the inner loop on the first `sum >= target`.

Complexity. Time `O(n²)`. Space `O(1)`.

Why it's not ideal. When start `i` produced a valid window ending at `j`, start `i+1` re-adds almost all of `nums[i+1..j]` from scratch. Because all values are positive, we know the window from `i+1` needs to be *at least as long relatively* — the running sum can be repaired by subtracting `nums[i]` instead of recomputing.

OPTIMAL APPROACH

Idea (shrinking window). Expand `right`, adding to `total`. Whenever `total >= target`, the window is valid — record its length, then **shrink from the left as far as it stays valid**, since we want the minimum. Each element is added once and removed once.

Data structures. Two pointers, one running sum.

Algorithm.

1. `left = 0`, `total = 0`, `best = ∞`.
2. For `right` in `0..n-1`:
3. `total += nums[right]`.
4. While `total >= target`: `best = min(best, right-left+1)`; `total -= nums[left]`; `left += 1`.
5. Return `0 if best == ∞ else best`.

Complexity. Time `O(n)` — amortised, since `left` advances at most `n` times total across all iterations of the `while`. Space `O(1)`.

Follow-up worth naming: with negatives allowed this breaks entirely, and you'd need prefix sums with a monotonic deque (LeetCode 862) — `O(n log n)` or `O(n)`. There's also an `O(n log n)` prefix-sum + binary-search solution for the positive case, which is the answer if they ask for "another way".

PYTHON CODE

```python
class Solution:
    def minSubArrayLen(self, target: int, nums: list[int]) -> int:
        left = 0
        total = 0
        best = float('inf')

        for right in range(len(nums)):
            total += nums[right]

            # SHRINK WHILE VALID — this is the "minimum length" shape.
            # (Contrast with problems 3/424, which shrink while INVALID because
            # they want the maximum length.)
            while total >= target:
                best = min(best, right - left + 1)
                total -= nums[left]     # valid only because all values are positive:
                left += 1               # removing one can only DECREASE the sum

        return 0 if best == float('inf') else best
```

DRY RUN

`target = 7`, `nums = [2, 3, 1, 2, 4, 3]`

right	add	total	while `total ≥ 7`	best	left after
0	2	2	no	∞	0
1	3	5	no	∞	0
2	1	6	no	∞	0
3	2	8	**yes**: len 4 → best 4; total 8−2=6; left=1	4	1
4	4	10	**yes**: len 4 (1..4) → best 4; total 10−3=7; left=2	4	2
		7	**yes**: len 3 (2..4) → best 3; total 7−1=6; left=3	**3**	3
5	3	9	**yes**: len 3 (3..5) → best 3; total 9−2=7; left=4	3	4
		7	**yes**: len 2 (4..5) → best **2**; total 7−4=3; left=5	**2**	5
		3	no	2	5

Answer: `2` — the subarray `[4, 3]`. ✅

Count the pointer moves: `right` advanced 6 times, `left` advanced 5 times. Total work `≤ 2n` — that's the amortised argument in concrete numbers.

---

438. Find All Anagrams in a String

UNDERSTAND THE PROBLEM

Given strings `s` and `p`, return the start indices of every substring of `s` that is an anagram of `p`.

- Window size is **fixed** at `len(p)` — this is the fixed-window family, which is simpler than 3/209.
- Anagram ⇒ identical character multisets, so we compare frequency counts, not order.
- Edge cases: `len(p) > len(s)` → `[]`; overlapping matches must all be reported (`s="aaaa", p="aa"` → `[0,1,2]`).

BRUTE-FORCE APPROACH

Idea. Slide a fixed window of size `len(p)` and, for each position, count the window's characters and compare with `p`'s counts.

Data structures. Two `Counter`s.

Algorithm. For each `i` in `0..n-m`: `if Counter(s[i:i+m]) == Counter(p): append(i)`.

Complexity. Time `O((n-m) · m)` — rebuilding a size-`m` counter per position. Space `O(26)`.

Why it's not ideal. Consecutive windows differ by exactly **two** characters — one leaves on the left, one joins on the right. Rebuilding all `m` counts to capture a 2-character delta is the waste. Incremental update makes each step `O(1)`.

OPTIMAL APPROACH

Idea (fixed window + a `matches` counter). Keep `need` (counts for `p`) and `window` (counts for the current window). Rather than comparing 26 slots on every slide — which is `O(26)` and fine, but not the sharpest answer — maintain a scalar `matches` = how many of the 26 letters currently have `window[c] == need[c]`. A window is an anagram iff `matches == 26`. Each slide touches at most 2 letters, so it updates `matches` in `O(1)`.

(The simpler `if window == need` array comparison is `O(26) = O(1)` too and is perfectly acceptable to write — mention both, code whichever you can write cleanly under pressure.)

Data structures. Two 26-length arrays; one integer `matches`.

Algorithm.

1. If `len(p) > len(s)`: return `[]`.
2. Build `need` from `p`, `window` from `s[:m]`. Set `matches` = number of letters where the two agree.
3. If `matches == 26`: record index 0.
4. For `right` in `m..n-1`: add `s[right]`, remove `s[right-m]`, updating `matches` around each change; if `matches == 26`, record `right - m + 1`.

Complexity. Time `O(n + m)`. Space `O(26) = O(1)`.

PYTHON CODE

```python
class Solution:
    def findAnagrams(self, s: str, p: str) -> list[int]:
        n, m = len(s), len(p)
        if m > n:
            return []

        need = [0] * 26
        window = [0] * 26
        for ch in p:
            need[ord(ch) - ord('a')] += 1
        for i in range(m):                          # prime the first window
            window[ord(s[i]) - ord('a')] += 1

        # matches = how many of the 26 letters currently have the exact right count.
        # Anagram <==> matches == 26.
        matches = sum(1 for c in range(26) if window[c] == need[c])

        res = []
        if matches == 26:
            res.append(0)

        for right in range(m, n):
```

```
            # --- character entering on the right ---
            inc = ord(s[right]) - ord('a')
            if window[inc] == need[inc]:
                matches -= 1                    # was correct, about to break
            window[inc] += 1
            if window[inc] == need[inc]:
                matches += 1                    # became correct

            # --- character leaving on the left ---
            dec = ord(s[right - m]) - ord('a')
            if window[dec] == need[dec]:
                matches -= 1
            window[dec] -= 1
            if window[dec] == need[dec]:
                matches += 1

            if matches == 26:
                res.append(right - m + 1)       # window is s[right-m+1 .. right]

        return res


class SolutionSimple:
    """Same complexity class, fewer moving parts. O(26) array compare per slide,
    which is still O(1). Write this one if you're short on time."""
    def findAnagrams(self, s: str, p: str) -> list[int]:
        n, m = len(s), len(p)
        if m > n:
            return []
        need, window = [0] * 26, [0] * 26
        for ch in p:
            need[ord(ch) - ord('a')] += 1
        res = []
        for i, ch in enumerate(s):
            window[ord(ch) - ord('a')] += 1
            if i >= m:                                  # evict the character falling out
                window[ord(s[i - m]) - ord('a')] -= 1
            if i >= m - 1 and window == need:
                res.append(i - m + 1)
        return res
```

DRY RUN

`s = "cbaebabacd"`, `p = "abc"` → `m = 3`, `need = a:1, b:1, c:1`

Initial window `s[0:3] = "cba"` → `window = a:1, b:1, c:1` → all 26 slots agree → `matches = 26` → record **0**.

right	s[right] in	s[right−3] out	window (non-zero)	matches==26?	record
3	`e`	`c`	a:1 b:1 e:1	no (c=0≠1, e=1≠0)	—
4	`b`	`b`	a:1 b:1 e:1	no	—
5	`a`	`a`	a:1 b:1 e:1	no	—
6	`b`	`e`	a:1 b:2	no (b=2≠1)	—
7	`a`	`b`	a:2 b:1	no	—
8	`c`	`a`	a:1 b:1 c:1	**yes**	**6**
9	`d`	`b`	a:1 c:1 d:1	no	—

Answer: `[0, 6]`. ✅ (`s[0:3] = "cba"`, `s[6:9] = "bac"` — both anagrams of `"abc"`.)

567. Permutation in String

UNDERSTAND THE PROBLEM

Given `s1` and `s2`, return `True` if `s2` contains **any permutation of** `s1` as a substring.

- This is problem 438 with the return type changed from "all indices" to "does one exist" — same machinery, early exit on the first hit. Say that out loud; recognising the isomorphism is the point.
- Edge cases: `len(s1) > len(s2)` → `False`; `s1 == s2`; empty `s1` → `True` (clarify).

BRUTE-FORCE APPROACH

Idea. Generate every permutation of `s1` and search for each in `s2`.

Data structures. A permutation generator.

Complexity. Time `O(m! · n)` — catastrophic. Space `O(m)`.

Why it's not ideal. `m = 10` already gives 3.6 million permutations. The realisation: you never need to *enumerate* permutations — a permutation is fully characterised by its character counts, so "is some permutation here?" collapses to "does some fixed-size window have matching counts?".

Middle-tier brute force: slide a fixed window and rebuild a `Counter` each time — `O(n·m)`. Better, still wasteful for the same reason as 438.

OPTIMAL APPROACH

Idea. Fixed-size sliding window of length `len(s1)` over `s2`, with incremental count updates and a `matches` scalar (or a plain 26-array compare). Return `True` on the first window where counts equal `need`.

Data structures. Two 26-arrays; `matches` integer.

Algorithm. Identical to 438 except step 4 returns `True` immediately instead of appending; return `False` after the loop.

Complexity. Time `O(n + m)`. Space `O(26) = O(1)`.

PYTHON CODE

```python
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        m, n = len(s1), len(s2)
        if m > n:
            return False

        need = [0] * 26
        window = [0] * 26
        for ch in s1:
            need[ord(ch) - ord('a')] += 1
        for i in range(m):
            window[ord(s2[i]) - ord('a')] += 1

        matches = sum(1 for c in range(26) if window[c] == need[c])
        if matches == 26:
            return True                    # the very first window already works

        for right in range(m, n):
            # add the incoming character
            inc = ord(s2[right]) - ord('a')
            if window[inc] == need[inc]:
                matches -= 1
            window[inc] += 1
            if window[inc] == need[inc]:
                matches += 1

            # drop the outgoing character
            dec = ord(s2[right - m]) - ord('a')
            if window[dec] == need[dec]:
                matches -= 1
            window[dec] -= 1
            if window[dec] == need[dec]:
                matches += 1

            if matches == 26:
                return True                # EARLY EXIT — the only diff vs. LC 438

        return False
```

DRY RUN

`s1 = "ab"`, `s2 = "eidbaooo"` → `m = 2`, `need = a:1, b:1`

Initial window `"ei"` → `e:1, i:1` → `matches < 26`.

right	in	out	window	match?
2	`d`	`e`	i:1 d:1	no
3	`b`	`i`	d:1 b:1	no
4	`a`	`d`	b:1 a:1	**yes → return True**

Answer: `True` — `s2[3:5] = "ba"`, a permutation of `"ab"`. ✅

Negative case `s1 = "ab"`, `s2 = "eidboaoo"`: windows are `ei, id, db, bo, oa, ao, oo` — none has counts `a:1, b:1` → `False`. Note `b` and `a` both appear in `s2` but never adjacently, which is exactly what the *contiguous window* enforces.

---

76. Minimum Window Substring ★★★

UNDERSTAND THE PROBLEM

Given `s` and `t`, return the **shortest substring of** `s` that contains every character of `t`, **including duplicates**. Return `""` if none exists.

- "Including duplicates" is the whole problem: if `t = "AABC"`, the window needs two `A`s. A `set`-based solution is wrong.
- The window may contain **extra** characters — it just has to cover `t`.
- Edge cases: `len(t) > len(s)` → `""`; `s == t`; `t` has repeated chars; no valid window.
- Clarify: *"Is the answer guaranteed unique? Case-sensitive?"* (LeetCode: unique if it exists, case-sensitive.)

BRUTE-FORCE APPROACH

Idea. Check every substring for coverage; keep the shortest that qualifies.

Data structures. A `Counter` per substring.

Algorithm. For each `(i, j)`: build `Counter(s[i:j+1])` and verify it dominates `Counter(t)` slot-by-slot.

Complexity. Time `O(n² · Σ)` (or `O(n²)` with incremental counting, where `Σ` is alphabet size). Space `O(Σ)`.

Why it's not ideal. `n = 10⁵` kills it. Structurally: coverage is **monotone** — if `[i, j]` covers `t`, so does `[i, j+1]`. That monotonicity is the licence to use a two-pointer window that never backtracks.

OPTIMAL APPROACH

Idea (expand-then-contract with a `have` / `need` counter). Two scalars do all the bookkeeping: - `need` = number of **distinct** characters in `t` whose required count is not yet met. - Expand `right`, decrementing `want[c]`. When `want[c]` hits exactly `0`, one more distinct requirement is satisfied → `need -= 1`. - Whenever `need == 0` the window is valid — record it, then **contract from the left** while it stays valid, restoring `want[c]` and bumping `need` back when a requirement breaks.

Tracking `need` as a single integer, rather than comparing dictionaries, is what makes each step `O(1)`.

Data structures. `Counter` `want` (initialised from `t`, decremented by window characters); integer `need`; two pointers; `(best_len, best_left)`.

Algorithm.

1. If `not t or not s` or `len(t) > len(s)`: return `""`.
2. `want = Counter(t)`; `need = len(want)`; `left = 0`; `best = (∞, 0)`.
3. For `right, c` in `enumerate(s)`:
4. If `c in want`: `want[c] -= 1`; if `want[c] == 0`: `need -= 1`.
5. While `need == 0`:
   - If `right - left + 1 < best[0]`: `best = (right-left+1, left)`.
   - `d = s[left]`; if `d in want`: `want[d] += 1`; if `want[d] > 0`: `need += 1`.
   - `left += 1`.
6. Return `""` if `best[0] == ∞` else `s[best[1] : best[1]+best[0]]`.

Note `want[c]` is allowed to go **negative** — that's how surplus copies are tracked, and it's why the restore step only bumps `need` when the value climbs back **above** zero.

Complexity. Time `O(|s| + |t|)` — each index enters and leaves the window once. Space `O(|Σ|)`.

PYTHON CODE

```python
from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not s or not t or len(t) > len(s):
            return ""

        # want[c] = how many MORE copies of c the window still needs.
        # Goes negative when the window holds surplus copies — that's intended.
        want = Counter(t)
        need = len(want)          # distinct chars not yet fully satisfied

        left = 0
        best_len, best_left = float('inf'), 0

        for right, c in enumerate(s):
            # ---- expand ----
            if c in want:
                want[c] -= 1
                if want[c] == 0:          # EXACTLY zero: requirement just became met.
                    need -= 1             # (< 0 means surplus — already counted.)

            # ---- contract while the window is still valid ----
            while need == 0:
                if right - left + 1 < best_len:
                    best_len, best_left = right - left + 1, left

                d = s[left]
                if d in want:
                    want[d] += 1
                    if want[d] > 0:       # climbed ABOVE zero -> now short again
                        need += 1
                left += 1

        return "" if best_len == float('inf') else s[best_left:best_left + best_len]
```

DRY RUN

`s = "ADOBECODEBANC"`, `t = "ABC"` → `want = {A:1, B:1, C:1}`, `need = 3`

right	c	want after	need	valid?	contract	best
0	A	A:0 B:1 C:1	2	no		
1	D	—	2	no		
2	O	—	2	no		
3	B	A:0 B:0 C:1	1	no		
4	E	—	1	no		
5	C	A:0 B:0 C:0	**0**	**yes**	record `[0,5]` len 6 → best="ADOBEC"; drop `A` → A:1, need=1, left=1	(6, 0)
6	O	—	1	no		
7	D	—	1	no		
8	E	—	1	no		
9	B	B:−1 (surplus, `need` unchanged)	1	no		
10	A	A:0	**0**	**yes**	`[1,10]` len 10 — not better. Drop `D` (∉want), `O`, `B` →B:0 ok...	

Contract phase at `right = 10`, step by step: - window `[1,10]="DOBECODEBA"`, len 10, not < 6 → no update. Drop `D` (not in want). `left=2`. - `[2,10]` len 9 → no update. Drop `O`. `left=3`. - `[3,10]` len 8 → no. Drop `B`: `want[B] = -1+1 = 0`, not `> 0` → `need` stays 0 (surplus consumed). `left=4`. - `[4,10]` len 7 → no. Drop `E`. `left=5`. - `[5,10]="CODEBA"` len 6, not < 6 → no. Drop `C`: `want[C] = 1 > 0` → `need = 1`. `left=6`. Loop exits.

right	c	want	need	valid?	contract	best
11	N	—	1	no		(6,0)
12	C	C:0	**0**	**yes**	`[6,12]="ODEBANC"` len 7 → no. Drop `O`, `D`, `E` → left=9. `[9,12]="BANC"` **len 4 < 6 → best!** Drop `B` →need=1, left=10	**(4, 9)**

Answer: `s[9:13] = "BANC"`. ✅

The step at `right = 10` where dropping the surplus `B` left `need` at 0 is the one people get wrong — it's why `want` must be allowed to go negative and why the restore check is `> 0`, not `>= 0`.

---

992. Subarrays with K Different Integers ★★

UNDERSTAND THE PROBLEM

Given an array and `k`, count the contiguous subarrays containing **exactly** `k` distinct integers.

- Count of subarrays, not lengths.
- Edge cases: `k > len(set(nums))` → 0; `k = 1` (count runs of equal values); all distinct.

BRUTE-FORCE APPROACH

Idea. For every `(i, j)`, count the distinct values in that slice.

Data structures. A `set` extended as `j` grows.

Algorithm. Nested loops; maintain a set for the inner loop; `if len(set) == k: count += 1`; break when `len(set) > k`.

Complexity. Time `O(n²)`. Space `O(k)`.

Why it's not ideal. Quadratic on `n = 2·10⁴` is $4 \cdot 10^8$ — too slow. The deeper issue: **"exactly k" is not a monotone window property.** Growing a window can take the distinct count from below `k`, to `k`, to above `k` — so there's no single valid/invalid boundary for two pointers to converge on. That's precisely why the naive sliding window fails here, and it's the observation the problem is testing.

OPTIMAL APPROACH

Idea (the `atMost` trick). "At most `k` distinct" **is** monotone — extending a window can only increase the distinct count, so shrinking always restores validity. So write a clean `atMost(k)` sliding window that counts *all* subarrays with `≤ k` distinct values, then:

```
exactly(k) = atMost(k) - atMost(k-1)
```

Inside `atMost`, when the window `[left, right]` is valid, it contributes `right - left + 1` subarrays — namely all the ones **ending at** `right`. Summing that over every `right` counts each subarray exactly once, with no double-counting.

Data structures. `Counter` for window frequencies; two pointers.

Algorithm.

1. `atMost(k)`: `left = 0`, `count = Counter()`, `total = 0`.
2. For `right`: `count[nums[right]] += 1`.
3. While `len(count) > k`: `count[nums[left]] -= 1`; if zero, `del`; `left += 1`.
4. `total += right - left + 1`.
5. Return `atMost(k) - atMost(k-1)`.

Complexity. Time `O(n)` — two linear passes. Space `O(k)`.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def subarraysWithKDistinct(self, nums: list[int], k: int) -> int:
        # "Exactly k" is NOT monotone, so a single window can't decide it.
        # "At most k" IS monotone -> classic shrinking window. Subtract to isolate.
        return self._at_most(nums, k) - self._at_most(nums, k - 1)

    def _at_most(self, nums: list[int], k: int) -> int:
        """Number of contiguous subarrays containing at most k distinct integers."""
        if k <= 0:
            return 0                        # k-1 can be 0 when the caller passes k=1

        count = defaultdict(int)
        left = 0
        total = 0

        for right, x in enumerate(nums):
            count[x] += 1

            # Too many distinct values -> shed from the left until valid again.
            while len(count) > k:
                y = nums[left]
                count[y] -= 1
                if count[y] == 0:
                    del count[y]            # MUST delete, or len(count) stays inflated
                left += 1

            # Every subarray ENDING at `right` and starting anywhere in [left, right]
            # is valid, and there are exactly (right - left + 1) of them.
            # Counting "subarrays ending at right" over all right hits each once.
            total += right - left + 1

        return total
```

DRY RUN

`nums = [1, 2, 1, 2, 3]`, `k = 2`

`atMost(2)`:

right	x	count	shrink?	left	`right-left+1`	total
0	1	{1:1}	no	0	1	1
1	2	{1:1,2:1}	no (2 ≤ 2)	0	2	3
2	1	{1:2,2:1}	no	0	3	6
3	2	{1:2,2:2}	no	0	4	10
4	3	{1:2,2:2,3:1} → 3 distinct	yes: drop 1→{1:1,2:2,3:1} still 3; drop 2→{1:1,2:1,3:1} still 3; drop 1→{2:1,3:1} ✓	3	2	**12**

`atMost(2) = 12`.

`atMost(1)`:

right	x	count	shrink	left	contrib	total
0	1	{1:1}	no	0	1	1
1	2	{1:1,2:1}	drop 1 →	1	1	2
2	1	{2:1,1:1}	drop 2 →	2	1	3
3	2	{1:1,2:1}	drop 1 →	3	1	4
4	3	{2:1,3:1}	drop 2 →	4	1	**5**

`atMost(1) = 5`.

Answer: `12 - 5 = 7`. ✅

Verify by listing: `[1,2]`, `[2,1]`, `[1,2]` (idx 2–3), `[1,2,1]`, `[2,1,2]`, `[1,2,1,2]`, `[2,3]` — that's 7 subarrays with exactly 2 distinct values.

1658. Minimum Operations to Reduce X to Zero ★

UNDERSTAND THE PROBLEM

Given `nums` and `x`, you may repeatedly remove either the **leftmost** or the **rightmost** element, subtracting its value from `x`. Return the **minimum number of removals** that makes `x` exactly `0`, or `-1`.

- The removed elements always form a **prefix + a suffix**. That's the structural fact everything hinges on.
- Values are positive (LeetCode constraint) — needed for the window to be valid.
- Edge cases: `x` exceeds `sum(nums)` → `-1`; `x == sum(nums)` → `len(nums)`; `x` unreachable.

BRUTE-FORCE APPROACH

Idea. Recursive / BFS search over "take from the left or take from the right", tracking the remaining `x`.

Data structures. Recursion stack or a queue of `(lo, hi, remaining)` states.

Algorithm. From `(lo, hi, x)`: try `(lo+1, hi, x - nums[lo])` and `(lo, hi-1, x - nums[hi])`; stop at `x == 0` (success) or `x < 0` (dead end).

Complexity. Time `O(2ⁿ)` without memoisation, `O(n²)` with memo on `(lo, hi)`. Space `O(n²)`.

Why it's not ideal. Exponential without memo; and even memoised it's `O(n²)` on `n = 10⁵`. The reformulation that kills it: since removals are always a prefix and a suffix, what **remains** is always one contiguous **middle** block. So instead of searching over removal sequences, solve for the middle block directly.

OPTIMAL APPROACH

Idea (invert the problem). If the removed prefix + suffix sums to `x`, then the remaining middle subarray sums to `total - x`. Minimising removals = **maximising the length of the middle subarray** whose sum is exactly `total - x`.

That is now a standard positive-values sliding-window problem: find the longest subarray with a given sum. Answer = `n - longest`, or `-1` if no such subarray exists.

Special case: `target = total - x == 0` means remove *everything* — answer `n` (the empty middle is valid). The window code handles this naturally if `best` is initialised to `-1` and updated when `total_window == target` including the empty window; the clean way is to note `left` can reach `right+1`.

Data structures. Two pointers, one running sum.

Algorithm.

1. `target = sum(nums) - x`.
2. If `target < 0`: return `-1`. If `target == 0`: return `len(nums)`.
3. Sliding window: expand `right`, `while cur > target: cur -= nums[left]; left += 1`; if `cur == target`: `best = max(best, right-left+1)`.
4. Return `-1` if `best == -1` else `n - best`.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def minOperations(self, nums: list[int], x: int) -> int:
        # THE INVERSION: removals are always a prefix + a suffix, so whatever
        # survives is one contiguous middle block. Removing the minimum count
        # == keeping the LONGEST middle block whose sum is (total - x).
        total = sum(nums)
        target = total - x

        if target < 0:
            return -1                  # even removing everything can't reach x
        if target == 0:
            return len(nums)           # remove the whole array

        left = 0
        cur = 0
        best = -1                      # longest middle block found, -1 = none yet

        for right, v in enumerate(nums):
            cur += v

            # All values are positive, so an over-large sum is fixed by shrinking
            # from the left - the standard monotone-window argument.
            while cur > target and left <= right:
                cur -= nums[left]
                left += 1

            if cur == target:
                best = max(best, right - left + 1)

        return -1 if best == -1 else len(nums) - best
```

DRY RUN

`nums = [3, 2, 20, 1, 1, 3]`, `x = 10` → `total = 30`, `target = 30 - 10 = 20`

right	v	cur	shrink	left	`cur == 20` ?	best
0	3	3	no	0	no	−1
1	2	5	no	0	no	−1
2	20	25	25>20: −3→22, left=1; 22>20: −2→20, left=2	2	**yes** len 1	**1**
3	1	21	21>20: −20→1, left=3	3	no	1
4	1	2	no	3	no	1
5	3	5	no	3	no	1

`best = 1` → answer `6 - 1 = 5`. ✅

Check: remove `[3, 2]` from the left (5) and `[1, 1, 3]` from the right (5) = 10 in **5** operations, leaving the middle `[20]`.

Impossible case `nums = [1,1]`, `x = 3`: `total = 2`, `target = -1 < 0` → `-1`. ✅

---

239. Sliding Window Maximum ★★

UNDERSTAND THE PROBLEM

Given `nums` and window size `k`, return an array of the maximum of every window as it slides one position at a time. Output length is `n - k + 1`.

- Edge cases: `k = 1` (→ `nums` itself); `k = n` (→ one value); negatives; duplicates.
- Target complexity: `O(n)`. The `O(n log k)` heap solution is acceptable but the deque is the expected answer.

BRUTE-FORCE APPROACH

Idea. For each window position, scan its `k` elements for the max.

Data structures. None.

Algorithm. For `i` in `0..n-k`: `res.append(max(nums[i:i+k]))`.

Complexity. Time `O(n·k)`. Space `O(1)` extra.

Why it's not ideal. With `n = 10⁵` and `k = 10⁵` this is 10^{10} operations. Structurally, consecutive windows overlap in `k-1` elements, so re-scanning all `k` to absorb a 1-element change is the waste — and, crucially, most of those elements can be **permanently discarded**: if `nums[i] < nums[j]` for `i < j`, then `nums[i]` can never be the max of any window that also contains `j`, and every future window containing `i` contains `j`. So `nums[i]` is dead forever.

OPTIMAL APPROACH

Idea (monotone-decreasing deque of indices). Keep a deque holding indices whose values are **strictly decreasing** from front to back. Invariants: - The front is always the index of the current window's maximum. - Before appending `right`, pop from the **back** every index whose value is `≤ nums[right]` — those are the permanently-dead candidates argued above. - Before reading the answer, pop from the **front** any index that has slid out of the window (`< right - k + 1`).

Store **indices**, not values, so the expiry check is possible.

Data structures. `collections.deque` of indices.

Algorithm.

1. `dq = deque()`, `res = []`.
2. For `right, v` in `enumerate(nums)`:
3. While `dq` and `nums[dq[-1]] <= v`: `dq.pop()`.
4. `dq.append(right)`.
5. If `dq[0] <= right - k`: `dq.popleft()`.
6. If `right >= k - 1`: `res.append(nums[dq[0]])`.
7. Return `res`.

Complexity. Time `O(n)` — each index is appended once and popped at most once. Space `O(k)`.

Alternative to name: a max-heap of `(-value, index)` with lazy deletion of expired entries — `O(n log n)` time, `O(n)` space. Mention it, then say the deque is strictly better.

PYTHON CODE

```python
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        dq = deque()      # indices, values strictly DECREASING front -> back
        res = []

        for right, v in enumerate(nums):
            # 1) Kill dominated candidates at the BACK.
            # If nums[dq[-1]] <= v then that older index can never be the max of any
            # future window: every window still containing it also contains `right`,
            # which is at least as large and expires later. So drop it permanently.
            while dq and nums[dq[-1]] <= v:
                dq.pop()

            dq.append(right)

            # 2) Expire the FRONT if it has slid out of the window.
            # Window currently covers [right-k+1, right].
            if dq[0] <= right - k:
                dq.popleft()

            # 3) Once the first full window exists, its max is at the front.
            if right >= k - 1:
                res.append(nums[dq[0]])

        return res
```

DRY RUN

`nums = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`

right	v	pop back (≤ v)	dq after append	expire front?	emit
0	1	—	`[0]` (vals 1)	no	—
1	3	pop 0 (1 ≤ 3)	`[1]` (3)	no	—
2	−1	none (3 > −1)	`[1,2]` (3,−1)	no	`nums[1]=` **3**
3	−3	none (−1 > −3)	`[1,2,3]` (3,−1,−3)	`dq[0]=1 ≤ 0`? no	**3**
4	5	pop 3, pop 2, pop 1	`[4]` (5)	no	**5**
5	3	none (5 > 3)	`[4,5]` (5,3)	no	**5**
6	6	pop 5, pop 4	`[6]` (6)	no	**6**
7	7	pop 6	`[7]` (7)	no	**7**

Answer: `[3, 3, 5, 5, 6, 7]`. ✅

Total deque operations: 8 appends and 8 pops — linear, exactly as claimed. The expiry check fired zero times here because a bigger element always arrived first; on input like `[5,4,3,2,1]` with `k=2` it fires on every step.

974. Subarray Sums Divisible by K

UNDERSTAND THE PROBLEM

Given an array (which **may contain negatives**) and `k`, return the number of contiguous subarrays whose sum is divisible by `k`.

- Negatives ⇒ prefix-sum + hashmap, not a window. Same family as LeetCode 560.
- The twist over 560: we care about `sum % k == 0` rather than `sum == k`, so the map is keyed on the **remainder**.
- Edge case that actually bites: Python's `%` always returns a non-negative result for positive `k` (`-3 % 5 == 2`), which is exactly the behaviour we want. In Java/C++ you must normalise with `((r % k) + k) % k` — say this, it shows you've thought about it.

BRUTE-FORCE APPROACH

Idea. Enumerate every subarray with a running sum, test divisibility.

Data structures. None.

Algorithm. Nested loops; `if running % k == 0: count += 1`.

Complexity. Time `O(n²)`. Space `O(1)`.

Why it's not ideal. `n = 3·10⁴` → ~4.5·10⁸ operations. And the same reformulation as 560 applies: divisibility of a *difference* of prefixes is a *lookup* keyed on remainder, not a scan.

OPTIMAL APPROACH

Idea. `sum(i..j) = P[j] - P[i-1]`, and

```
(P[j] - P[i-1]) % k == 0   ⟺   P[j] % k == P[i-1] % k
```

So count how many earlier prefixes share the current prefix's **remainder class**. That's a frequency map over the `k` possible remainders.

Seed `freq[0] = 1` for the empty prefix, same as 560 — it's what lets subarrays starting at index 0 be counted.

Data structures. An array `freq` of length `k` (better than a dict here — the key space is bounded by `k`).

Algorithm.

1. `freq = [0]*k`; `freq[0] = 1`; `running = 0`; `total = 0`.
2. For each `x`: `running = (running + x) % k`; `total += freq[running]`; `freq[running] += 1`.
3. Return `total`.

Complexity. Time `O(n)`. Space `O(k)`.

PYTHON CODE

```python
class Solution:
    def subarraysDivByK(self, nums: list[int], k: int) -> int:
        # freq[r] = how many prefixes seen so far have running sum ≡ r (mod k)
        freq = [0] * k
        freq[0] = 1          # empty prefix — lets subarrays starting at index 0 count

        running = 0
        total = 0

        for x in nums:
            # Python's % on a positive modulus always yields a value in [0, k-1],
            # so negatives are handled for free: (-3) % 5 == 2.
            # In Java/C++ you'd need ((r % k) + k) % k here.
            running = (running + x) % k

            # Any earlier prefix in the SAME remainder class closes a divisible
            # subarray, because the difference of two equal remainders is ≡ 0.
            total += freq[running]

            freq[running] += 1     # record AFTER the lookup
        return total
```

DRY RUN

`nums = [4, 5, 0, -2, -3, 1]`, `k = 5`

x	running (mod 5)	`freq[running]` before	total	freq after
—	0	—	0	`[1,0,0,0,0]`
4	4	0	0	`[1,0,0,0,1]`
5	(4+5)%5 = 4	**1**	1	`[1,0,0,0,2]`
0	4	**2**	3	`[1,0,0,0,3]`
−2	(4−2)%5 = 2	0	3	`[1,0,1,0,3]`
−3	(2−3)%5 = **4** (Python: −1 % 5 = 4)	**3**	6	`[1,0,1,0,4]`
1	(4+1)%5 = 0	**1**	**7**	`[2,0,1,0,4]`

Answer: `7`. ✅

The `-3` row is the one to watch: `2 + (-3) = -1`, and Python gives `-1 % 5 = 4`, correctly landing in the same class as the earlier prefixes of value 4 — which is what produced 3 hits at once. A language that returns `-1` there would find 0 hits and produce a wrong answer.

---

525. Contiguous Array

UNDERSTAND THE PROBLEM

Given a binary array, return the length of the **longest** contiguous subarray with an equal number of 0s and 1s.

- Edge cases: no balanced subarray → `0`; all zeros or all ones → `0`; `[0,1]` → `2`.

BRUTE-FORCE APPROACH

Idea. For every subarray, count 0s and 1s and compare.

Data structures. Two counters maintained incrementally.

Algorithm. Nested loops; track `zeros` / `ones`; when equal, update the best length.

Complexity. Time `O(n²)`. Space `O(1)`.

Why it's not ideal. `n = 10⁵` → 10^{10}. The transformation that fixes it: "equal counts of 0 and 1" is a *balance* condition, and balance conditions become prefix-sum conditions the moment you map the two symbols to `+1` and `-1`.

OPTIMAL APPROACH

Idea (map 0 → −1). Replace each `0` with `-1`. Now a subarray has equal 0s and 1s **iff its sum is 0**, i.e. iff two prefix sums are equal:

```
sum(i..j) == 0   ⇔   P[j] == P[i-1]
```

So for each running value, remember the **earliest index** at which it occurred; the longest balanced subarray ending at `j` starts right after that earliest occurrence. Store first-occurrence only — never overwrite — because we want the *longest*, not the count.

That "store first occurrence, don't overwrite" is the one behavioural difference from 560/974, where we counted every occurrence. Name it explicitly.

Data structures. `dict` mapping running-sum value → earliest index, seeded with `{0: -1}`.

Algorithm.

1. `first = {0: -1}`; `running = 0`; `best = 0`.
2. For `i, v` in `enumerate(nums)`: `running += 1 if v == 1 else -1`.
3. If `running in first`: `best = max(best, i - first[running])`.
4. Else: `first[running] = i`.
5. Return `best`.

Complexity. Time `O(n)`. Space `O(n)` — running sums range over `[-n, n]`.

PYTHON CODE

```python
class Solution:
    def findMaxLength(self, nums: list[int]) -> int:
        # Map 0 -> -1 so "equal counts" becomes "sum == 0",
        # and "sum(i..j) == 0" becomes "two prefix sums are equal".
        first = {0: -1}      # running sum -> EARLIEST index it occurred at.
                             # Seed -1 so a balanced prefix starting at index 0
                             # yields length i - (-1) = i + 1.
        running = 0
        best = 0

        for i, v in enumerate(nums):
            running += 1 if v == 1 else -1

            if running in first:
                # Same running sum seen before -> everything between is balanced.
                best = max(best, i - first[running])
            else:
                # Record ONLY the first occurrence. Overwriting would shorten the
                # candidate span — the opposite of what "longest" wants.
                # (Contrast LC 560/974, which count every occurrence instead.)
                first[running] = i

        return best
```

DRY RUN

`nums = [0, 1, 0, 0, 1, 1, 0]`

i	v	delta	running	seen before?	length	best	first after
—	—	—	0	—	—	0	`{0:-1}`
0	0	−1	−1	no	—	0	`{0:-1, -1:0}`
1	1	+1	0	**yes @ −1**	1−(−1)=**2**	2	unchanged
2	0	−1	−1	**yes @ 0**	2−0=2	2	unchanged
3	0	−1	−2	no	—	2	`-2:3`
4	1	+1	−1	**yes @ 0**	4−0=**4**	4	unchanged
5	1	+1	0	**yes @ −1**	5−(−1)=**6**	**6**	unchanged
6	0	−1	−1	**yes @ 0**	6−0=6	6	unchanged

Answer: `6` — `nums[0:6] = [0,1,0,0,1,1]` has three 0s and three 1s. ✅

Row 5 is the payoff for seeding `{0: -1}`: without it, the balanced prefix `[0..5]` would have been invisible.

Destination Cache — 🟠 LRU shape with TTL (InterviewDB)

UNDERSTAND THE PROBLEM

The title is verified; the prompt is not. "Destination" suggests **cached routing results**, which in practice means entries that expire. Prepare **both**: plain LRU (in `00_PRIMITIVES.py` #25 and LeetCode 146, covered on Day 11) and the **TTL variant** here.

TTL cache spec: `get(key, now)` returns the value if present **and not expired**, else a miss. `put(key, value, now, ttl)` inserts or refreshes. Capacity-bounded: on overflow evict the least-recently-used entry. Expired entries must never be returned and should be reclaimed.

- Clarify first: *"Is time supplied by the caller or read from a clock? Does a* `get` *refresh the TTL or only the recency? Is capacity fixed? Is this thread-safe?"* Each answer changes the code. Default assumptions below: time is injected (testable), `get` refreshes recency but **not** expiry, capacity is fixed.
- Edge cases: `get` on an expired key (miss, and evict it); `put` on an existing key (update value + recency + expiry); capacity 0; all entries expired.

BRUTE-FORCE APPROACH

Idea. Store entries in a plain dict with `(value, expiry)`. For eviction, keep a `last_used` timestamp per entry and, when full, **scan every entry** to find the minimum. For expiry, scan the whole dict periodically.

Data structures. One dict; a linear scan for both eviction and expiry.

Algorithm.

1. `get` : dict lookup; check `expiry > now` ; update `last_used` .
2. `put` : if full, `min(entries, key=last_used)` → delete → insert.

Complexity. `get` is `O(1)` , but `put` on a full cache is `O(n)` because of the min-scan. A full sweep for expired entries is `O(n)` too.

Why it's not ideal. A cache's entire value proposition is `O(1)` operations; an `O(n)` eviction on every insert once the cache is warm defeats it. The fix is the standard one: maintain recency order **incrementally** in a structure that supports `O(1)` move-to-front and `O(1)` removal of the tail — a doubly linked list, or Python's `OrderedDict` , which is one.

OPTIMAL APPROACH

Idea. Two ideas layered: 1. **Recency in `O(1)`** — `OrderedDict` (hash map + doubly linked list). `move_to_end(key)` marks recent use; `popitem(last=False)` evicts the least-recently-used, both `O(1)` . 2. **Expiry lazily** — store `expires_at` alongside the value and check it on read. Don't run a background sweeper; **evict on touch**. Optionally add an amortised cleanup that drops a bounded number of expired entries per operation, so a cache full of stale entries doesn't hold memory forever.

Data structures. `OrderedDict[key] -> (value, expires_at)` .

Algorithm.

- `get(key, now)` : miss if absent; if `expires_at <= now` , delete and report miss; else `move_to_end(key)` and return the value.
- `put(key, value, now, ttl)` : if key exists, overwrite and `move_to_end` . Else, if `len == capacity` , first try evicting an expired entry (cheap scan from the LRU end, bounded), otherwise `popitem(last=False)` . Then insert at the MRU end.

Complexity. `get` and `put` are `O(1)` amortised. Space `O(capacity)` .

PYTHON CODE

```python
from collections import OrderedDict
from typing import Any, Optional


class DestinationCache:
    """
    LRU cache with per-entry TTL.

    Design choices, stated explicitly because an interviewer will ask:
```

```
      * Time is INJECTED by the caller (`now`), not read from a global clock.
        Makes the cache deterministic and unit-testable, and matches how a
        routing layer usually already has a request timestamp in hand.
      * Expiry is LAZY: entries are reclaimed when touched, plus a bounded
        opportunistic sweep on insert. No background thread, no timers.
      * `get` refreshes RECENCY but not EXPIRY — a hot key still goes stale on
        schedule, which is what you want for routing data.
      * Not thread-safe. Wrap ops in a single lock (or shard by key hash) if needed.
    """

    def __init__(self, capacity: int, default_ttl: float = float('inf')):
        if capacity < 0:
            raise ValueError("capacity must be non-negative")
        self.capacity = capacity
        self.default_ttl = default_ttl
        # key -> (value, expires_at). OrderedDict IS a hash map + doubly linked list,
        # which is exactly the LRU data structure; front = LRU, back = MRU.
        self._store: "OrderedDict[Any, tuple[Any, float]]" = OrderedDict()

    # ---------------- public API ----------------
    def get(self, key: Any, now: float) -> Optional[Any]:
        """Return the cached value, or None on miss/expiry."""
        entry = self._store.get(key)
        if entry is None:
            return None

        value, expires_at = entry
        if expires_at <= now:
            # Lazy expiry: reclaim on touch rather than sweeping in the background.
            del self._store[key]
            return None

        self._store.move_to_end(key)      # O(1) — mark as most recently used
        return value

    def put(self, key: Any, value: Any, now: float,
            ttl: Optional[float] = None) -> None:
        if self.capacity == 0:
            return                        # degenerate cache: never stores anything

        ttl = self.default_ttl if ttl is None else ttl
        expires_at = now + ttl

        if key in self._store:
            # Overwrite in place, then refresh recency. No eviction needed.
            self._store[key] = (value, expires_at)
            self._store.move_to_end(key)
            return

        if len(self._store) >= self.capacity:
            # Prefer evicting something already DEAD over something merely old —
            # a stale entry costs us nothing to lose. Bounded scan keeps put() O(1)
            # amortised rather than degrading to a full O(n) sweep.
            if not self._evict_one_expired(now):
                self._store.popitem(last=False)   # O(1) — drop the LRU entry

        self._store[key] = (value, expires_at)    # inserts at the MRU end

    def __len__(self) -> int:
        return len(self._store)

    # ---------------- internals ----------------
    def _evict_one_expired(self, now: float, max_checks: int = 8) -> bool:
        """Scan a BOUNDED number of entries from the LRU end for a dead one.
        Bounded so a single put() can never become O(n)."""
        for key in list(self._store.keys())[:max_checks]:
            if self._store[key][1] <= now:
                del self._store[key]
                return True
        return False


# --------- The plain-LRU fallback, in case the prompt has no TTL ----------
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
```

```
        self._store = OrderedDict()

    def get(self, key: int) -> int:
        if key not in self._store:
            return -1
        self._store.move_to_end(key)
        return self._store[key]

    def put(self, key: int, value: int) -> None:
        if key in self._store:
            self._store.move_to_end(key)
        self._store[key] = value
        if len(self._store) > self.capacity:
            self._store.popitem(last=False)     # evict LRU
```

DRY RUN

`cache = DestinationCache(capacity=2, default_ttl=10)`

op	now	effect	store (LRU → MRU)
`put("SEA", r1)`	0	insert, expires at 10	`SEA(exp 10)`
`put("PDX", r2)`	1	insert, expires at 11	`SEA(10), PDX(11)`
`get("SEA")`	2	hit (2 < 10) → move to end	`PDX(11), SEA(10)`
`put("LAX", r3)`	3	full; scan for expired from LRU end: `PDX` expires 11 > 3, not dead; `SEA` 10 > 3, not dead → no expired → evict LRU = **PDX**	`SEA(10), LAX(13)`
`get("PDX")`	4	absent → **miss**	unchanged
`get("SEA")`	11	present but `10 ≤ 11` → **expired**, delete, miss	`LAX(13)`
`put("SFO", r4)`	12	len 1 < 2, no eviction	`LAX(13), SFO(22)`

Note the two distinct miss reasons at `now=4` and `now=11`: eviction (capacity pressure) vs. expiry (TTL). A good answer distinguishes them — real caches emit different metrics for each, and mentioning that lands well.

Top Customers — 🟡 Heap aggregation (InterviewDB)

UNDERSTAND THE PROBLEM

Given a list of transactions, return the **top `k` customers by total spend**, with ties broken by **customer id**.

- Clarify: *"Ties broken by smaller id or larger id? Is the output ordered? Can amounts be negative (refunds)? Is `k` larger than the number of customers?"* Assume: smaller id wins ties, output sorted best-first, refunds possible, `k` may exceed the customer count (return everyone).
- Edge cases: empty transactions; `k = 0`; all customers tied; one customer with many transactions.
- The two-part structure — **aggregate, then select** — is what's being tested. Naming it before you code is worth as much as the code.

BRUTE-FORCE APPROACH

Idea. Aggregate totals into a dict, then **fully sort** all customers by `(-total, id)` and slice the first `k`.

Data structures. `defaultdict(int)`; a sorted list.

Algorithm.

1. For each transaction: `totals[cid] += amount`.
2. `sorted(totals.items(), key=lambda kv: (-kv[1], kv[0]))[:k]`.

Complexity. Time `O(T + C log C)` where `T` = transactions, `C` = distinct customers. Space `O(C)`.

Why it's not ideal. It's honestly *fine* — and you should say so, because over-engineering is its own failure mode. But when `k ≪ C` (top 10 of ten million customers), the full sort does `C log C` work to answer a question that needs only `C log k`. Sorting to take the top-`k` is the classic "did they think about it" tell. If `C` is large enough that the customer table doesn't fit in memory, the heap version is the one that generalises to streaming.

OPTIMAL APPROACH

Idea. Same aggregation, different selection: a **bounded min-heap of size `k`**. Push each customer; when the heap exceeds `k`, pop the smallest. What remains is the top `k`. Sort just those `k` at the end for a presentable order.

Handling the tie-break inside a heap is the fiddly part. We want *high spend* to win, and among equal spends *low id* to win. A min-heap pops the smallest tuple, so we want the "worst" candidate to compare smallest. Use the key `(total, -id)` when ids are integers — for equal totals, the largest id gives the smallest `-id` and is popped first, which is exactly "large ids lose ties".

For string ids, `-id` isn't available; simplest correct approach is `heapq.nsmallest` / `nlargest` with a comparison key, or just accept the `O(C log C)` sort and say why.

Data structures. `defaultdict(int)` for totals; `heapq` of `(total, -id, id)` tuples bounded at size `k`.

Algorithm.

1. Aggregate totals.
2. For each `(cid, total)`: `heappush(heap, (total, -cid, cid))`; if `len(heap) > k`: `heappop(heap)`.
3. Sort the surviving `k` by `(-total, cid)` and return the ids.

Complexity. Time `O(T + C log k + k log k)`. Space `O(C + k)`.

PYTHON CODE

```python
import heapq
from collections import defaultdict
from typing import Iterable, Sequence


def top_customers(transactions: Iterable[Sequence], k: int) -> list:
    """
    transactions: iterable of (customer_id, amount)
    Returns the k customer ids with the highest total spend,
    ties broken by SMALLER customer id, output ordered best-first.
    Assumes integer customer ids (see the note on the heap key below).
    """
    if k <= 0:
        return []

    # ---- Step 1: aggregate. O(T), unavoidable — every transaction must be read.
    totals = defaultdict(int)
    for cid, amount in transactions:
        totals[cid] += amount

    # ---- Step 2: select the top k with a BOUNDED min-heap. O(C log k).
    # Heap key = (total, -cid). A min-heap evicts the SMALLEST key, so the key
    # must order "worst candidate" first:
    #   * lower total  -> smaller key -> evicted first.   ✅ high spend wins
    #   * equal total  -> larger cid gives smaller -cid  -> evicted first.  ✅ low id wins ties
    heap = []
    for cid, total in totals.items():
        heapq.heappush(heap, (total, -cid, cid))
        if len(heap) > k:
            heapq.heappop(heap)      # drop the current worst; heap stays size k

    # ---- Step 3: present. Only k items, so this sort is cheap.
    heap.sort(key=lambda t: (-t[0], t[2]))     # total desc, then id asc
    return [cid for _total, _neg, cid in heap]


def top_customers_simple(transactions: Iterable[Sequence], k: int) -> list:
    """The O(C log C) version. Correct, shorter, works with string ids.
    Say out loud: 'this is fine unless k << C or C doesn't fit in memory.'"""
    totals = defaultdict(int)
    for cid, amount in transactions:
        totals[cid] += amount
    ranked = sorted(totals.items(), key=lambda kv: (-kv[1], kv[0]))
    return [cid for cid, _ in ranked[:k]]
```

DRY RUN

```
transactions = [(1, 50), (2, 30), (1, 20), (3, 70), (2, 40), (4, 70)]
k = 2
```

Step 1 — aggregate:

txn	totals after
(1, 50)	`{1: 50}`
(2, 30)	`{1: 50, 2: 30}`
(1, 20)	`{1: 70, 2: 30}`
(3, 70)	`{1: 70, 2: 30, 3: 70}`
(2, 40)	`{1: 70, 2: 70, 3: 70}`
(4, 70)	`{1: 70, 2: 70, 3: 70, 4: 70}`

Everyone tied at 70 — the tie-break is now the whole answer.

Step 2 — bounded heap (`k = 2`), keys `(total, -cid, cid)` :

push	heap contents	size > 2?	pop (smallest)	heap after
`(70,-1,1)`	`[(70,-1,1)]`	no	—	`[(70,-1,1)]`
`(70,-2,2)`	`[(70,-2,2), (70,-1,1)]`	no	—	both
`(70,-3,3)`	3 items	yes	smallest is `(70,-3,3)` → **id 3 evicted**	`{1, 2}`
`(70,-4,4)`	3 items	yes	smallest is `(70,-4,4)` → **id 4 evicted**	`{1, 2}`

Step 3 — sort the survivors by `(-total, cid)`: `[(70,-1,1), (70,-2,2)]` → ids `[1, 2]`.

Answer: `[1, 2]`. ✅ — all four tied at 70, and the two smallest ids won, which is the specified tie-break.

Trace the key design at the eviction step: candidates 3 and 4 had `-cid` of `-3` and `-4`, both smaller than `-1` and `-2`, so the min-heap discarded exactly the customers with the largest ids. Getting that sign right is the part that's easy to fumble under time pressure — sanity-check it with a two-element example before you commit.

DAY 3 — Wed Aug 19 · Stacks, Monotonic Stacks & Linked Lists

> **Two pattern families in one day. Stacks** answer "what is the most recent unresolved thing?" — matching brackets, nested state, and (monotonic variant) "the nearest greater/smaller element in some direction". If a problem says *next*, *previous*, *nearest*, combined with *greater* or *smaller*, it is a monotonic stack, full stop. **Linked lists** are almost entirely about three primitives — **dummy head**, **fast/slow pointers**, **reverse in place** — and the hard problems (143, 148) are just those three composed. Drill the primitives, not the problems.

---

20. Valid Parentheses

UNDERSTAND THE PROBLEM

Given a string of `()[]{}`, return whether every bracket is closed by the same type, in the correct order.

- "Correct order" means properly nested: `([])` is invalid even though the counts match.
- Edge cases: `""` → `True`; odd length → always `False`; a closing bracket first `")("`; leftover openers at the end `"(("`.

BRUTE-FORCE APPROACH

Idea. Repeatedly find and delete adjacent matching pairs (`"()"`, `"[]"`, `"{}"`) until nothing changes; the string is valid iff it ends up empty.

Data structures. Just the string.

Algorithm. `while` the string still shrinks: `s = s.replace("()","").replace("[]","").replace("{}","")`. Return `s == ""`.

Complexity. Time `O(n²)` — each pass scans the whole string and may only remove one pair; up to `n/2` passes. Space `O(n)`.

Why it's not ideal. Quadratic, and it re-scans regions that can't possibly have changed. But it does point at the right structure: the thing you always resolve is the **most recently opened** bracket. "Most recent unresolved item" is the definition of a stack.

OPTIMAL APPROACH

Idea. Push openers. On a closer, the top of the stack must be its partner — pop and verify. At the end the stack must be empty.

Data structures. A `list` used as a stack; a dict mapping closer → opener.

Algorithm.

1. `pairs = {')':'(', ']':'[', '}':'{'}`; `stack = []`.
2. For each `ch`: if it's an opener, push; else if `not stack or stack.pop() != pairs[ch]`: return `False`.
3. Return `not stack`.

Complexity. Time `O(n)`. Space `O(n)` — worst case all openers.

PYTHON CODE

```python
class Solution:
    def isValid(self, s: str) -> bool:
        # Map each CLOSER to the opener it requires. Keying on the closer means
        # `ch in pairs` is exactly the test "is this a closing bracket?".
        pairs = {')': '(', ']': '[', '}': '{'}
        stack = []

        for ch in s:
            if ch in pairs:
                # Closer. Two ways to fail:
                #   1. nothing open at all  -> ")" with an empty stack
                #   2. wrong type on top    -> "(]"
                # Python short-circuits `or`, so stack.pop() only runs when
                # the stack is non-empty. Order of the checks matters.
                if not stack or stack.pop() != pairs[ch]:
                    return False
            else:
                stack.append(ch)      # opener -> becomes the new "most recent"

        # Anything left open at the end is unmatched: "((" is invalid.
        return not stack
```

DRY RUN

`s = "([)]"` — the case counting alone would wrongly accept.

ch	type	action	stack after	result
`(`	opener	push	`['(']`	
`[`	opener	push	`['(', '[']`	
`)`	closer, needs `(`	pop → `'['` ≠ `'('`	—	**return False** ✅

`s = "{[]}"`:

ch	action	stack after
`{`	push	`['{']`
`[`	push	`['{','[']`
`]`	pop `'['` == `'['` ✓	`['{']`
`}`	pop `'{'` == `'{'` ✓	`[]`

Stack empty → `True`. ✅

155. Min Stack

UNDERSTAND THE PROBLEM

Design a stack supporting `push`, `pop`, `top`, and `getMin` — **all in `O(1)`**.

- `getMin` returns the minimum of the *current* contents, so it must react correctly to pops.
- Edge cases: duplicates of the minimum (popping one must not lose the other); popping down to empty; `getMin` after a sequence of pushes and pops.
- Clarify: *"Is it guaranteed that `pop` / `top` / `getMin` are only called on a non-empty stack?"* (LeetCode: yes.)

BRUTE-FORCE APPROACH

Idea. A plain list; `getMin` scans it.

Data structures. One list.

Algorithm. `push` / `pop` / `top` are trivial; `getMin` = `min(self.data)`.

Complexity. `push` / `pop` / `top` are `O(1)`, but `getMin` is `O(n)`.

Why it's not ideal. The problem explicitly demands `O(1)` for all four. Scanning discards a fact we could have maintained cheaply: the minimum only changes on push (possibly down) and on pop (possibly back up), and both changes are predictable. Since the stack is LIFO, the minimum's history is itself a stack.

OPTIMAL APPROACH

Idea (two options — know both).

A. Auxiliary min-stack. Keep a second stack whose top is always the minimum of the corresponding prefix. On push, push `min(x, current_min)`. On pop, pop both. `getMin` = top of the aux stack. Simple, `O(n)` extra space, and **handles duplicate minima automatically** because you push a min entry for every element.

B. Encoded single stack. Store `(value, min_so_far)` pairs — the same idea in one list.

C. `O(1)` extra space trick. Store `2*x - min` when a new minimum arrives, recovering the previous minimum on pop. Mention it as a flourish; it's fiddly and overflow-prone, so don't lead with it.

Go with A.

Data structures. Two lists: `data` and `mins`.

Algorithm.

- `push(x)`: `data.append(x)`; `mins.append(x if not mins else min(x, mins[-1]))`.
- `pop()`: pop both.
- `top()`: `data[-1]`.
- `getMin()`: `mins[-1]`.

Complexity. All four `O(1)`. Space `O(n)`.

PYTHON CODE

```python
class MinStack:
    def __init__(self):
        self.data = []     # the actual stack contents
        self.mins = []     # mins[i] = minimum of data[0..i], so tops stay in sync

    def push(self, val: int) -> None:
        self.data.append(val)
        # The new minimum is either the incoming value or the previous minimum.
        # Pushing an entry for EVERY element (even when the min doesn't change)
        # keeps the two stacks the same height, which is what makes pop() O(1)
        # and what makes duplicate minima safe: [0, 1, 0] stores mins [0, 0, 0],
        # so popping one zero still leaves a zero recorded.
        self.mins.append(val if not self.mins else min(val, self.mins[-1]))

    def pop(self) -> None:
        self.data.pop()
        self.mins.pop()        # always in lockstep

    def top(self) -> int:
        return self.data[-1]

    def getMin(self) -> int:
        return self.mins[-1]   # O(1) — the whole point of the aux stack


class MinStackPairs:
    """Variant B: one stack of (value, min_so_far). Identical complexity,
    fewer objects to keep in sync — pick whichever you can write cleanly."""
    def __init__(self):
        self.stack = []                       # list[tuple[int, int]]

    def push(self, val: int) -> None:
        cur_min = val if not self.stack else min(val, self.stack[-1][1])
        self.stack.append((val, cur_min))

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]
```

DRY RUN

Operations: `push(-2)`, `push(0)`, `push(-3)`, `getMin()`, `pop()`, `top()`, `getMin()`

op	data	mins	returns
`push(-2)`	`[-2]`	`[-2]`	
`push(0)`	`[-2, 0]`	`[-2, -2]` ← min unchanged, still pushed	
`push(-3)`	`[-2, 0, -3]`	`[-2, -2, -3]`	
`getMin()`			**-3**
`pop()`	`[-2, 0]`	`[-2, -2]`	
`top()`			**0**
`getMin()`			**-2** ✅ correctly restored

Duplicate-minimum case — `push(0)`, `push(1)`, `push(0)`, `pop()`, `getMin()`:

op	data	mins
`push(0)`	`[0]`	`[0]`
`push(1)`	`[0,1]`	`[0,0]`
`push(0)`	`[0,1,0]`	`[0,0,0]`
`pop()`	`[0,1]`	`[0,0]`
`getMin()`		→ **0** ✅

A "push only when strictly smaller" optimisation breaks exactly here: it would store `mins = [0]`, and popping the second zero would either pop the sole entry (losing the real minimum) or not (leaking it). Push unconditionally.

---

150. Evaluate Reverse Polish Notation

UNDERSTAND THE PROBLEM

Given tokens in postfix (RPN) notation, evaluate the expression. Operators are `+ - * /`; division **truncates toward zero**.

- Postfix means an operator applies to the two most recent operands — a stack, by construction.
- **Operand order matters** for `-` and `/`: the value popped *second* is the left operand. `["4","13","-"]` is `4 - 13`, not `13 - 4`.
- **Truncation toward zero**, not floor: `-7 / 2 == -3`, whereas Python's `//` gives `-4`. This is the trap.
- Edge cases: a single number `["5"]`; negative literals `"-11"`; deep nesting.

BRUTE-FORCE APPROACH

Idea. Repeatedly scan the token list for the leftmost operator whose two preceding tokens are both numbers, evaluate it, and splice the result back into the list. Repeat until one token remains.

Data structures. The token list, mutated in place.

Algorithm. `while len(tokens) > 1`: find the first operator at index `i` with numbers at `i-1`, `i-2`; replace `tokens[i-2:i+1]` with the computed value.

Complexity. Time `O(n²)` — each splice is `O(n)` and there are `O(n)` of them. Space `O(n)`.

Why it's not ideal. The repeated scanning and list splicing is pure overhead. The observation: the "leftmost evaluable operator" always operates on the two most recently seen *unconsumed* numbers — which is exactly LIFO, so a single left-to-right pass with a stack does it in one go.

OPTIMAL APPROACH

Idea. Push numbers. On an operator, pop two (right first, then left), apply, push the result. One token remains at the end.

Data structures. A list as a stack.

Algorithm.

1. `stack = []`.
2. For each token: if it's an operator, `b = pop()`, `a = pop()`, `push(apply(a, op, b))`; else `push(int(token))`.
3. Return `stack[0]`.

Truncation. Implement division as `int(a / b)` — Python's `int()` on a float truncates toward zero — or the explicit integer form `abs(a)//abs(b)` with the sign reapplied, which avoids float precision issues on very large operands. Prefer the explicit form and say why.

Complexity. Time `O(n)`. Space `O(n)`.

PYTHON CODE

```python
class Solution:
    def evalRPN(self, tokens: list[str]) -> int:
        stack = []
        ops = {'+', '-', '*', '/'}

        for tok in tokens:
            if tok in ops:
                # ORDER MATTERS: the stack pops right-to-left, so the first pop
                # is the RIGHT operand. Getting this backwards silently breaks
                # '-' and '/' while leaving '+' and '*' looking fine.
                b = stack.pop()      # right operand
                a = stack.pop()      # left operand

                if tok == '+':
                    stack.append(a + b)
                elif tok == '-':
                    stack.append(a - b)
                elif tok == '*':
                    stack.append(a * b)
                else:
                    # TRUNCATE TOWARD ZERO, not floor.
                    #   Python:  -7 // 2 == -4   (floor)     <- WRONG here
                    #   Required: -7  / 2 -> -3  (truncate)
                    # Doing it on magnitudes then reapplying the sign avoids both
                    # the floor behaviour and any float rounding on huge values.
                    quotient = abs(a) // abs(b)
                    if (a < 0) != (b < 0):
                        quotient = -quotient
                    stack.append(quotient)
            else:
                # int() handles a leading '-' correctly, so negative literals
                # like "-11" need no special case.
                stack.append(int(tok))

        return stack[0]        # a well-formed RPN expression leaves exactly one value
```

DRY RUN

`tokens = ["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"]`

token	action	stack after
`10`	push	`[10]`
`6`	push	`[10, 6]`
`9`	push	`[10, 6, 9]`
`3`	push	`[10, 6, 9, 3]`
`+`	b=3, a=9 → 12	`[10, 6, 12]`
`-11`	push	`[10, 6, 12, -11]`
`*`	b=−11, a=12 → −132	`[10, 6, -132]`
`/`	b=−132, a=6 → `abs: 6//132 = 0`; signs differ → `-0 = 0`	`[10, 0]`
`*`	b=0, a=10 → 0	`[0]`
`17`	push	`[0, 17]`
`+`	b=17, a=0 → 17	`[17]`
`5`	push	`[17, 5]`
`+`	b=5, a=17 → 22	`[22]`

Answer: `22`. ✅

Truncation trap, isolated. `tokens = ["-7","2","/"]`: `b=2, a=-7`. Correct: `abs(-7)//abs(2) = 3`, signs differ → `-3`. Python's `-7 // 2` would give `-4` — wrong. Worth writing this two-element test on the whiteboard unprompted; interviewers score the dry run.

394. Decode String

UNDERSTAND THE PROBLEM

Given an encoded string like `"3[a2[c]]"`, return the expansion (`"accaccacc"`). Nesting is arbitrary and repeat counts can be multi-digit.

- Grammar: `k[encoded]` repeats `encoded` `k` times; plain letters pass through; segments concatenate.
- `k` can be more than one digit — `"12[a]"` — so digits must be accumulated, not read one at a time.
- Input is always well-formed (no stray brackets), and `k ≥ 1`.
- Edge cases: no brackets at all `"abc"`; nesting three deep; digits immediately after a `]` — `"2[a]3[b]"`.

BRUTE-FORCE APPROACH

Idea. Repeatedly find the **innermost** `k[...]` (a bracket pair containing no other brackets) with a regex or a scan, expand it in place, and repeat until no brackets remain.

Data structures. The string, rebuilt each round.

Algorithm. `while '[' in s`: locate an innermost pair, expand, splice.

Complexity. Time `O(d · n · L)` where `d` = nesting depth and `L` = output length — each pass rescans and rebuilds the whole string. Space `O(L)`.

Why it's not ideal. Repeated full-string rebuilds. The structural insight it misses: this is a **parsing** problem, and the natural way to parse nesting in one pass is to stack the *enclosing context* when you descend and restore it when you ascend. One pass, no rescanning.

OPTIMAL APPROACH

Idea (stack of saved state). Walk the string maintaining `cur` (the segment being built) and `num` (the repeat count being accumulated). Four cases: - **digit** → `num = num*10 + int(ch)` (this is the multi-digit fix). - `[` → we're descending: push `(cur, num)` onto the stack, then reset `cur = ""`, `num = 0`. - `]` → we're ascending: pop `(prev, k)` and set `cur = prev + cur * k`. - **letter** → `cur += ch`.

The stack holds *what was being built in the enclosing scope*, so returning from a nested group is just concatenation.

Data structures. One stack of `(string, int)` pairs; two accumulators.

Algorithm. As above; return `cur`.

Complexity. Time `O(L)` where `L` is the length of the decoded output (you must at minimum write every output character). Space `O(L)` for the stack and result.

PYTHON CODE

```python
class Solution:
    def decodeString(self, s: str) -> str:
        stack = []          # saved (enclosing_string, repeat_count) at each '['
        cur = ""            # segment being built at the CURRENT nesting level
        num = 0             # repeat count being accumulated, digit by digit

        for ch in s:
            if ch.isdigit():
                # Multi-digit counts: "12[a]" must read 12, not 1 then 2.
                num = num * 10 + int(ch)

            elif ch == '[':
                # Descending one level. Park everything the enclosing level had
                # built so far, plus the count that applies to the group we're
                # about to read, then start the inner segment from scratch.
                stack.append((cur, num))
                cur, num = "", 0

            elif ch == ']':
                # Ascending. `cur` is the fully-decoded inner group; repeat it and
                # glue it onto whatever the enclosing level had before the '['.
                prev, k = stack.pop()
                cur = prev + cur * k

            else:
                cur += ch           # plain character, appended at this level

        return cur                  # stack is empty on well-formed input
```

DRY RUN

`s = "3[a2[c]]"`

ch	branch	num	cur	stack
`3`	digit	3	`""`	`[]`
`[`	descend	0	`""`	`[("", 3)]`
`a`	letter	0	`"a"`	`[("", 3)]`
`2`	digit	2	`"a"`	`[("", 3)]`
`[`	descend	0	`""`	`[("", 3), ("a", 2)]`
`c`	letter	0	`"c"`	same
`]`	ascend: pop `("a", 2)` → `"a" + "c"*2`	0	`"acc"`	`[("", 3)]`
`]`	ascend: pop `("", 3)` → `"" + "acc"*3`	0	`"accaccacc"`	`[]`

Answer: `"accaccacc"`. ✅

Sibling groups, `s = "2[a]3[b]"` — checks that state resets cleanly between groups:

ch	num	cur	stack
`2`	2	`""`	`[]`
`[`	0	`""`	`[("",2)]`
`a`	0	`"a"`	
`]`	0	`"aa"`	`[]`
`3`	3	`"aa"`	`[]`
`[`	0	`""`	`[("aa",3)]` ← `cur` correctly parked
`b`	0	`"b"`	
`]`	0	`"aa" + "bbb"` = `"aabbb"`	`[]`

Answer: `"aabbb"`. ✅ The `push(cur, num)` — pushing `cur`, not just the count — is what makes sibling groups concatenate correctly.

739. Daily Temperatures

UNDERSTAND THE PROBLEM

Given daily temperatures, return an array where `answer[i]` is the number of days you must wait after day `i` for a **warmer** temperature, or `0` if none ever comes.

- This is the canonical **"next greater element"** problem. Say those words out loud the moment you read it.
- Strictly warmer, so equal temperatures don't count.
- Edge cases: monotonically decreasing (all zeros); last day (always 0); all equal (all zeros).

BRUTE-FORCE APPROACH

Idea. For each day, scan forward until a warmer day appears.

Data structures. None.

Algorithm. For `i`: for `j` in `i+1..n-1`: if `T[j] > T[i]`: `ans[i] = j - i`; break.

Complexity. Time `O(n²)` worst case (a decreasing array makes every scan run to the end). Space `O(1)`.

Why it's not ideal. `n = 10⁵` → 10^{10}. The waste: when day `j` finally resolves day `i`, it usually also resolves several other pending days at once — but the brute force rediscovers each of them separately. Batching those resolutions is exactly what a monotonic stack does.

OPTIMAL APPROACH

Idea (monotonic decreasing stack of indices). Keep a stack of indices whose temperatures are **decreasing** from bottom to top — these are the days still waiting for a warmer day. When a new temperature arrives, it resolves **every** stacked day cooler than it: pop each and record the distance. Then push the new index.

Store **indices**, not temperatures, so you can compute `j - i`.

Why the stack stays decreasing: any day that a newer, warmer day resolves is popped immediately, so only days strictly warmer than the incoming one remain.

Data structures. A list of indices as a stack; the answer array pre-filled with 0 (which is also the correct default for "never warms up").

Algorithm.

1. `ans = [0]*n`; `stack = []`.
2. For `i, t` in `enumerate(temps)`:
3. While `stack` and `temps[stack[-1]] < t`: `j = stack.pop()`; `ans[j] = i - j`.
4. `stack.append(i)`.
5. Return `ans`.

Complexity. Time `O(n)` — every index is pushed once and popped at most once. That amortised argument ("each index is pushed and popped once") is the answer they want when they ask *why* it's linear; "one pass" is not an acceptable justification.

Space `O(n)`.

PYTHON CODE

```python
class Solution:
    def dailyTemperatures(self, temperatures: list[int]) -> list[int]:
        n = len(temperatures)
        ans = [0] * n          # 0 is already the right answer for "never gets warmer"
        stack = []             # indices of days still WAITING; temps strictly decreasing

        for i, t in enumerate(temperatures):
            # Today resolves every pending day that was cooler than today.
            # Note this may pop several at once — that batching is exactly the work
            # the O(n^2) version repeats one day at a time.
            while stack and temperatures[stack[-1]] < t:
                j = stack.pop()
                ans[j] = i - j         # distance in days, hence storing INDICES

            # `<` not `<=`: equal temperatures are not "warmer", so an equal day
            # must stay on the stack waiting for a strictly greater one.
            stack.append(i)

        # Anything still on the stack never warmed up -> keeps its 0. No cleanup needed.
        return ans
```

DRY RUN

`temperatures = [73, 74, 75, 71, 69, 72, 76, 73]`

i	t	pops (with distances)	stack after (indices/temps)	ans so far
0	73	—	`[0]` / 73	`[0,0,0,0,0,0,0,0]`
1	74	pop 0: `ans[0] = 1-0 = 1`	`[1]` / 74	`[1,0,…]`
2	75	pop 1: `ans[1] = 2-1 = 1`	`[2]` / 75	`[1,1,…]`
3	71	none (75 > 71)	`[2,3]` / 75,71	
4	69	none (71 > 69)	`[2,3,4]` / 75,71,69	
5	72	pop 4: `ans[4]=5-4=1` ; pop 3: `ans[3]=5-3=2`	`[2,5]` / 75,72	`[1,1,0,2,1,0,0,0]`
6	76	pop 5: `ans[5]=6-5=1` ; pop 2: `ans[2]=6-2=4`	`[6]` / 76	`[1,1,4,2,1,1,0,0]`
7	73	none (76 > 73)	`[6,7]`	

Leftover indices 6 and 7 never warm up → keep `0`.

Answer: `[1, 1, 4, 2, 1, 1, 0, 0]`. ✅

Step `i = 5` is the batching in action: one temperature resolved two pending days in a single step. Push count = 8, pop count = 6 → linear.

---

735. Asteroid Collision

UNDERSTAND THE PROBLEM

Given `asteroids` where the **sign is direction** (positive = moving right, negative = moving left) and the **magnitude is size**, simulate collisions: a right-mover and a left-mover collide when the right-mover is to the left of the left-mover; the smaller explodes; if equal, **both** explode. Return the survivors.

- A collision happens **only** for the pattern `positive` then `negative` (`+` moving right meets `-` moving left). `-` then `+` means they're moving apart — no collision, ever.
- One incoming asteroid may destroy several stacked survivors in a chain.
- Edge cases: `[8, -8]` → `[]` (both die); `[-2, -1, 1, 2]` → unchanged (nothing collides); `[10, 2, -5]` → `[10]` (the `-5` kills `2`, then loses to `10`).

BRUTE-FORCE APPROACH

Idea. Repeatedly scan the list for an adjacent colliding pair (`a[i] > 0 and a[i+1] < 0`), resolve it, and restart the scan — because resolving one collision can create a new adjacency.

Data structures. The list, mutated.

Algorithm. `while` a collision exists: find it, delete the loser(s), rescan from the start.

Complexity. Time `O(n²)` — up to `n` collisions, each costing an `O(n)` scan plus `O(n)` list deletion. Space `O(1)` extra.

Why it's not ideal. The rescans are unnecessary: after a collision, the only *new* adjacency is between the surviving asteroid and the one immediately to its left. "Only the thing immediately to the left can be affected next" is precisely the stack access pattern.

OPTIMAL APPROACH

Idea. Sweep left to right with a stack of survivors. For each incoming asteroid `a`: - If `a > 0` → it moves right, can never collide with anything already on the stack. Push. - If `a < 0` → it moves left, so it fights every **positive** asteroid on top of the stack, in order: - While the top is positive and **smaller** than `|a|`: pop it (it explodes) and keep fighting. - If the top is positive and **equal** to `|a|`: pop it, and `a` dies too. Stop. - If the top is positive and **larger**: `a` dies. Stop. - If the stack empties or the top is negative: `a` survives → push.

Writing that control flow cleanly is the whole exercise — a `while` with an `else` clause, or an explicit `alive` flag, both work.

Data structures. A list as a stack.

Complexity. Time `O(n)` — each asteroid is pushed at most once and popped at most once. Space `O(n)`.

PYTHON CODE

```python
class Solution:
    def asteroidCollision(self, asteroids: list[int]) -> list[int]:
        stack = []      # survivors so far, in order

        for a in asteroids:
            alive = True

            # A collision requires a RIGHT-mover on the stack meeting an incoming
            # LEFT-mover. Any other combination is moving apart or same-direction.
            # One incoming asteroid can chain through several stacked ones.
            while alive and a < 0 and stack and stack[-1] > 0:
                top = stack[-1]
                if top < -a:
                    stack.pop()          # stacked one is smaller -> it explodes, fight on
                elif top == -a:
                    stack.pop()          # equal -> BOTH explode
                    alive = False
                else:
                    alive = False        # stacked one is bigger -> incoming explodes

            if alive:
                stack.append(a)

        return stack
```

DRY RUN

`asteroids = [5, 10, -5]`

a	direction	fight?	stack after
5	right	no (positive → just push)	`[5]`
10	right	no	`[5, 10]`
−5	left	top = 10 > 5 → **incoming dies**, `alive=False`	`[5, 10]`

Answer: `[5, 10]`. ✅

`asteroids = [10, 2, -5]` — the chain case:

a	fight sequence	stack after
10	push	`[10]`
2	push	`[10, 2]`
−5	top=2 < 5 → pop 2, keep fighting. top=10 > 5 → incoming dies	`[10]`

Answer: `[10]`. ✅ — one incoming asteroid resolved two stacked ones without any rescan.

`asteroids = [8, -8]`:

a	fight	stack
8	push	`[8]`
−8	top=8 == 8 → pop AND `alive=False`	`[]`

Answer: `[]`. ✅ The equal case must do **both** things — pop and mark dead. Doing only one is the standard bug here.

`asteroids = [-2, -1, 1, 2]`:

a	why no fight	stack
−2	stack empty → push	`[-2]`
−1	top is negative → loop condition fails → push	`[-2,-1]`
1	positive → push	`[-2,-1,1]`
2	positive → push	`[-2,-1,1,2]`

Answer: unchanged. ✅ Correctly recognises that `-` followed by `+` means diverging paths.

84. Largest Rectangle in Histogram ★★

UNDERSTAND THE PROBLEM

Given bar heights of equal width 1, return the area of the largest rectangle that fits entirely inside the histogram.

- The rectangle spans a contiguous range of bars and its height is the **minimum** bar in that range: `area = min(h[i..j]) × (j − i + 1)`.
- Equivalently, and more usefully: for each bar `i`, the largest rectangle *with height exactly* `h[i]` extends left and right until it meets a **strictly smaller** bar. So the task is "find the previous smaller and next smaller element for every index" — a monotonic stack, twice over, or once if you're clever.
- Edge cases: single bar; all equal (`n × h`); strictly increasing; strictly decreasing; a zero-height bar.

BRUTE-FORCE APPROACH

Idea (a). Try every `(i, j)` pair, tracking the running minimum height. **Idea (b).** For each `i`, expand left and right while bars are `≥ h[i]`.

Data structures. None.

Complexity. Both `O(n²)` time, `O(1)` space. (A naive version of (a) that recomputes the min is `O(n³)`.)

Why it's not ideal. `n = 10⁵` → 10^{10}. And (b) is one step from the answer: it's asking, for each bar, "where is the nearest strictly smaller bar on each side?" — the textbook monotonic-stack query. The stack computes all `n` of those answers in one linear pass instead of `n` separate expansions.

OPTIMAL APPROACH

Idea (increasing stack + sentinel). Maintain a stack of indices whose heights are **increasing** from bottom to top. When a bar arrives that is shorter than the stack top, that bar is the **next smaller** element for the top — so the top's rectangle is now fully bounded and can be finalised: - pop index `t`; - height = `heights[t]`; - right boundary = the current index `i` (exclusive); - left boundary = the new stack top after popping (exclusive) — because the stack is increasing, the element below `t` is exactly `t`'s **previous smaller** element; - `width = i − stack[−1] − 1`, or `i` if the stack is now empty (meaning `t` was the minimum so far and extends to the very left).

The sentinel. Append a virtual bar of height `0` at the end. It's shorter than everything, so it forces every remaining stacked bar to be popped and evaluated. Without it you need a separate drain loop after the main loop — the sentinel folds that into the main logic and removes a whole class of off-by-one bugs. Say "I'll add a sentinel so I don't need a second drain loop" out loud; it reads as experience.

Data structures. A list of indices as a stack.

Algorithm.

1. `stack = []`, `best = 0`, `heights = heights + [0]`.
2. For `i, h` in `enumerate(heights)`:
3. While `stack` and `heights[stack[−1]] > h`:
   - `t = stack.pop()`; `height = heights[t]`;
   - `width = i if not stack else i − stack[−1] − 1`;
   - `best = max(best, height * width)`.
4. `stack.append(i)`.
5. Return `best`.

Complexity. Time `O(n)` — each index pushed and popped once. Space `O(n)`.

PYTHON CODE

```python
class Solution:
    def largestRectangleArea(self, heights: list[int]) -> int:
        # Sentinel: a virtual bar of height 0 at the end. Being shorter than every
        # real bar, it forces the stack to fully drain inside the main loop, so we
        # don't need a separate post-loop cleanup pass (a classic off-by-one source).
        heights = heights + [0]

        stack = []      # indices, heights strictly INCREASING bottom -> top
        best = 0

        for i, h in enumerate(heights):
            # Bar i is shorter than the top -> i is the top's "next smaller" bar,
            # so the top's rectangle is now fully bounded on the right. Finalise it.
            while stack and heights[stack[-1]] > h:
                t = stack.pop()
                height = heights[t]

                # Left bound: because the stack is increasing, whatever sits below t
                # is t's PREVIOUS SMALLER bar. The rectangle spans strictly between
                # that bar and bar i, i.e. indices (stack[-1], i) exclusive.
                # If the stack is now empty, nothing to the left was smaller, so the
                # rectangle reaches all the way to index 0 -> width is exactly i.
                width = i if not stack else i - stack[-1] - 1

                best = max(best, height * width)

            stack.append(i)

        return best
```

DRY RUN

`heights = [2, 1, 5, 6, 2, 3]` → with sentinel: `[2, 1, 5, 6, 2, 3, 0]`

i	h	pops → (height, width, area)	stack after	best
0	2	—	`[0]`	0
1	1	pop 0: h=2, stack empty → w = 1, area **2**	`[1]`	2
2	5	— (1 < 5)	`[1,2]`	2
3	6	—	`[1,2,3]`	2
4	2	pop 3: h=6, w = 4−2−1 = 1, area 6 → best 6 pop 2: h=5, w = 4−1−1 = 2, area **10**	`[1,4]`	**10**
5	3	— (2 < 3)	`[1,4,5]`	10
6	0 (sentinel)	pop 5: h=3, w = 6−4−1 = 1, area 3 pop 4: h=2, w = 6−1−1 = 4, area 8 pop 1: h=1, stack empty → w = 6, area 6	`[6]`	**10**

Answer: `10` — bars at indices 2 and 3 (`heights` 5 and 6), rectangle height 5 × width 2. ✅

Two things to point at in the trace: - At `i = 4`, popping index 2 gave `width = 2` because the stack below it held index 1 (`height 1`), the previous smaller bar. The stack *is* the previous-smaller lookup — that's why no second pass is needed. - The sentinel at `i = 6` drained three bars, including the height-1 bar spanning the entire histogram (`w = 6`). Without it, that rectangle is simply never evaluated.

206. Reverse Linked List

UNDERSTAND THE PROBLEM

Given the head of a singly linked list, reverse it and return the new head. Know **both** the iterative and recursive versions.

- Reverse the pointers, not the values.
- Edge cases: empty list (`None`); single node; two nodes.
- This is one of the three linked-list primitives — 143 and 234 (palindrome) both call it as a subroutine, so it must be automatic.

BRUTE-FORCE APPROACH

Idea. Walk the list collecting values into an array, then walk again writing the values back in reverse order.

Data structures. A Python list of length `n` .

Algorithm. `vals = [node.val for node in list]` ; walk again assigning `node.val = vals[n-1-i]` .

Complexity. Time `O(n)` , space `O(n)` .

Why it's not ideal. The time is already optimal; the **space** is the problem. It also mutates values rather than structure, which is disallowed outright in some variants (143 explicitly says values may not be changed) and is a semantic difference if nodes carry other payload. Reversing links needs no auxiliary storage — only three pointers.

OPTIMAL APPROACH

Idea (iterative, three pointers). Walk the list flipping each `next` pointer to point backwards. You need `prev` (the reversed portion built so far), `cur` (the node being processed), and a saved `nxt` — because overwriting `cur.next` destroys your only way forward.

Data structures. Three pointers.

Algorithm.

1. `prev, cur = None, head` .
2. While `cur` : `nxt = cur.next` ; `cur.next = prev` ; `prev = cur` ; `cur = nxt` .
3. Return `prev` .

Recursive version. Reverse the tail, then make `head.next.next = head` and `head.next = None` . Base case: empty or single node. `O(n)` time, `O(n)` stack space — worth knowing, but the iterative version is strictly better and is what you should write by default.

Complexity. Iterative: time `O(n)` , space `O(1)` . Recursive: time `O(n)` , space `O(n)` .

PYTHON CODE

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        """Iterative — O(1) space. This is the default; write this one."""
        prev = None       # head of the already-reversed portion
        cur = head        # node currently being re-pointed

        while cur:
            nxt = cur.next     # SAVE FIRST — the next line destroys this link
            cur.next = prev    # flip the pointer backwards
            prev = cur         # the reversed portion grows by one
            cur = nxt          # advance using the saved reference

        return prev            # cur is None; prev is the last node = the new head

    def reverseListRecursive(self, head: ListNode | None) -> ListNode | None:
        """Recursive — O(n) stack. Know it, but prefer the iterative version."""
        # Base case: empty list, or a single node (already reversed).
        if head is None or head.next is None:
            return head

        # Reverse everything after head; new_head is the tail of the ORIGINAL list
        # and stays the head of the result all the way back up the call stack.
        new_head = self.reverseListRecursive(head.next)

        # head.next is still the node that now sits at the END of the reversed
        # sublist, so pointing ITS next at head appends head to the reversed part.
        head.next.next = head
        head.next = None       # head is now the tail — cut the old forward link
                               # (skip this and you create a 2-node cycle)
        return new_head
```

DRY RUN

`1 → 2 → 3 → None`

step	prev	cur	nxt	action	list state
start	`None`	`1`	—		`1→2→3→None`
1	`None`	`1`	`2`	`1.next = None`	`1→None`, `2→3→None`
	`1`	`2`		pointers advance	
2	`1`	`2`	`3`	`2.next = 1`	`2→1→None`, `3→None`
	`2`	`3`		advance	
3	`2`	`3`	`None`	`3.next = 2`	`3→2→1→None`
	`3`	`None`		advance	
exit	`3`	`None`		`cur` falsy → return `prev`	**`3→2→1→None`** ✅

The `nxt = cur.next` save is the line people drop. Trace step 1 without it: after `cur.next = prev`, node 1 points at `None`, and node 2 is unreachable — the rest of the list is lost.

21. Merge Two Sorted Lists

UNDERSTAND THE PROBLEM

Given two sorted linked lists, splice their **nodes** together into one sorted list and return its head.

- Splice existing nodes; don't allocate new ones (that's the point of the "dummy head" primitive).
- Edge cases: one or both lists empty; lists of very different lengths; duplicate values across lists.
- Stability (which list wins on a tie) doesn't affect correctness here, but state your choice.

BRUTE-FORCE APPROACH

Idea. Collect all values from both lists into an array, sort it, and build a new list.

Data structures. An array of size `n + m`.

Algorithm. Traverse both, `sort()`, rebuild.

Complexity. Time `O((n+m) log(n+m))`, space `O(n+m)`.

Why it's not ideal. It throws away the fact that both inputs are **already sorted**. Merging two sorted sequences is `O(n+m)` with no sorting at all — the merge step of merge sort. And rebuilding allocates `n+m` new nodes when the existing ones can simply be relinked.

OPTIMAL APPROACH

Idea (dummy head + two pointers). Walk both lists, always appending the smaller head to the output, advancing that list. When one runs out, attach the remainder of the other **wholesale** — it's already sorted.

The dummy head is a throwaway node placed before the real head. It removes the "is this the first node?" special case: `tail.next = node` works uniformly from the very first append, and the real head is just `dummy.next` at the end. This is a primitive worth internalising — it simplifies 19, 21, 23, 148, and half the linked-list problems in the loop.

Data structures. One dummy node; one `tail` pointer.

Algorithm.

1. `dummy = ListNode()`, `tail = dummy`.
2. While `l1 and l2`: append the smaller, advance it, advance `tail`.
3. `tail.next = l1 or l2` — attach whatever remains.
4. Return `dummy.next`.

Complexity. Time `O(n + m)`. Space `O(1)` — no new nodes beyond the single dummy.

PYTHON CODE

```python
class Solution:
    def mergeTwoLists(self, list1: ListNode | None,
                      list2: ListNode | None) -> ListNode | None:
        # DUMMY HEAD: a throwaway node that sits before the real result.
        # It means `tail.next = ...` works identically for the first append and
        # every later one — no "if result is None" special case anywhere.
        dummy = ListNode()
        tail = dummy

        while list1 and list2:
            # `<=` keeps the merge stable (ties take from list1 first). Not required
            # by the problem, but stability matters when nodes carry extra payload,
            # and it's the kind of thing worth stating out loud.
            if list1.val <= list2.val:
                tail.next = list1
                list1 = list1.next
            else:
                tail.next = list2
                list2 = list2.next
            tail = tail.next

        # Exactly one list can be non-empty here, and it's already sorted and
        # entirely >= everything emitted so far — so attach it in one move
        # instead of walking it node by node.
        tail.next = list1 if list1 else list2

        return dummy.next      # skip past the dummy to the real head
```

DRY RUN

`list1 = 1 → 2 → 4`, `list2 = 1 → 3 → 4`

step	l1 head	l2 head	comparison	appended	output so far
1	1	1	`1 <= 1` → l1	l1's 1	`d→1`
2	2	1	`2 <= 1` ? no → l2	l2's 1	`d→1→1`
3	2	3	`2 <= 3` → l1	2	`d→1→1→2`
4	4	3	no → l2	3	`d→1→1→2→3`
5	4	4	`4 <= 4` → l1	4	`d→1→1→2→3→4`
6	`None`	4	loop exits (l1 empty)	attach l2's remaining `4`	`d→1→1→2→3→4→4`

Return `dummy.next` → `1 → 1 → 2 → 3 → 4 → 4`. ✅

Empty-input case, `list1 = None`, `list2 = 5`: the `while` never runs; step 3 attaches `list2` directly; return `dummy.next = 5`. The dummy makes this work with no special-casing at all — that's what it's for.

---

142. Linked List Cycle II ★

UNDERSTAND THE PROBLEM

Given a linked list, return the node where a cycle **begins**, or `None` if there's no cycle. Must use `O(1)` space.

- Not just "is there a cycle" (that's LC 141) — you must return the entry node.
- Edge cases: no cycle; the whole list is one cycle (entry = head); a self-loop of one node; empty list.
- The `O(1)` space rule is what rules out the obvious hash-set answer.

BRUTE-FORCE APPROACH

Idea. Walk the list adding each node to a set; the first node already in the set is the cycle entry.

Data structures. A `set` of node **identities** (not values — values can repeat).

Algorithm. `seen = set()`; for each node: `if node in seen: return node`; `seen.add(node)`; return `None`.

Complexity. Time `O(n)`, space `O(n)`.

Why it's not ideal. Linear time is already optimal; the `O(n)` set violates the space constraint. It's also the *right* answer if the interviewer doesn't impose `O(1)` — offer it first, then upgrade. The upgrade uses a counting argument rather than storage.

OPTIMAL APPROACH

Idea (Floyd's tortoise and hare, plus the reset step).

Phase 1 — detection: advance `slow` by 1 and `fast` by 2. If they ever meet, there's a cycle. If `fast` hits `None`, there isn't.

Phase 2 — locating the entry: **reset `slow` to `head`**, then advance both one step at a time. They meet exactly at the cycle entry.

Why phase 2 works (know this — it's the follow-up question). Let `L` = distance from head to the cycle entry, `C` = cycle length, and `k` = distance from the entry to the meeting point. When they meet, slow has travelled `L + k` (taking the first lap) and fast has travelled `L + k + mC` for some integer `m ≥ 1`. Since fast moves twice as fast, `2(L + k) = L + k + mC`, hence

```
L + k = mC   →   L = mC − k
```

`mC − k` is exactly the distance from the meeting point forward to the entry (going `m` laps and backing off `k`). So a pointer starting at the head and a pointer starting at the meeting point, both moving one step at a time, cover the same distance to the entry and meet there.

Data structures. Two pointers.

Algorithm.

1. `slow = fast = head`.
2. While `fast and fast.next`: `slow = slow.next`; `fast = fast.next.next`; if `slow is fast`: break.
3. If the loop ended without meeting (i.e. `not fast or not fast.next`): return `None`.
4. `slow = head`; while `slow is not fast`: advance both by one.
5. Return `slow`.

Complexity. Time `O(n)`, space `O(1)`.

PYTHON CODE

```python
class Solution:
    def detectCycle(self, head: ListNode | None) -> ListNode | None:
        # ---------- PHASE 1: detect ----------
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:          # `is`, not `==` - identity, values may repeat
                break
        else:
            # `while...else` runs when the loop exits WITHOUT break, i.e. fast ran
            # off the end -> no cycle. Using it avoids re-testing the condition.
            return None

        # ---------- PHASE 2: locate the entry ----------
        # Let L = head->entry, C = cycle length, k = entry->meeting point.
        # At the meeting: slow walked L+k, fast walked L+k+mC, and fast == 2*slow,
        # so L + k = mC, i.e. L = mC - k = the distance from the meeting point
        # forward to the entry. Two pointers at head and at the meeting point,
        # both moving one step, therefore arrive at the entry together.
        slow = head
        while slow is not fast:
            slow = slow.next
            fast = fast.next          # SAME speed now - one step each, not two

        return slow                   # == fast == the cycle entry
```

DRY RUN

List: `3 → 2 → 0 → -4`, with `-4.next` pointing back to the node `2` (index 1). So `L = 1`, `C = 3`.

Label nodes by index: `n0=3, n1=2, n2=0, n3=-4`, and `n3 → n1`.

Phase 1:

step	slow	fast	meet?
start	n0	n0	(not checked before moving)
1	n1	n2	no
2	n2	n1 (n2→n3→n1)	no
3	n3	n3 (n1→n2→n3)	**yes → break**

Meeting point = `n3`. Check the algebra: slow travelled 3 steps = `L + k` = `1 + k` → `k = 2` (entry `n1` → `n2` → `n3` is indeed 2 steps). And `L + k = 3 = mC = 1·3`. ✅

Phase 2: reset `slow = n0` (head), keep `fast = n3`.

step	slow	fast	equal?
start	n0	n3	no
1	n1	n1 (n3 wraps to n1)	**yes → return n1**

Answer: node with value `2` — the cycle entry. ✅

No-cycle case, `1 → 2 → None`: phase 1 runs `slow=n1, fast=None`; loop condition `fast and fast.next` fails without a `break`, so the `else` fires → `None`. ✅

---

19. Remove Nth Node From End of List

UNDERSTAND THE PROBLEM

Given a linked list and `n`, remove the `n`-th node **from the end** and return the head. Do it in **one pass**.

- `n` is guaranteed valid (`1 ≤ n ≤ length`).
- Edge case that breaks naive code: removing the **head** itself (`n == length`). That's what the dummy node is for.
- Other edges: single-node list with `n=1` → `None`; removing the tail.

BRUTE-FORCE APPROACH

Idea. Walk once to compute the length `L`, then walk again to node `L - n - 1` and unlink its successor.

Data structures. None.

Algorithm. Two passes: count, then re-traverse.

Complexity. Time `O(L)` (two passes, still linear), space `O(1)`.

Why it's not ideal. It's correct and the asymptotics are fine — say so. But the problem explicitly asks for **one pass**, and the two-pass version needs the whole list available twice, which fails if the input is a one-shot stream. The one-pass technique (a fixed gap between two pointers) generalises to streaming and is the answer being tested.

OPTIMAL APPROACH

Idea (two pointers with a fixed gap). Advance a `fast` pointer `n + 1` steps ahead of `slow` (starting both at a **dummy** before the head). Then move both until `fast` falls off the end. At that moment `slow` sits exactly on the node **before** the one to delete, so `slow.next = slow.next.next` removes it.

The `+1` in the gap is what lands `slow` on the *predecessor* rather than on the target — the classic off-by-one.

The dummy node handles removing the head: with `slow` starting at the dummy, "the node before the head" is a real node, so the head case needs no branch.

Data structures. Dummy node; two pointers.

Algorithm.

1. `dummy = ListNode(0, head)`; `slow = fast = dummy`.
2. Repeat `n + 1` times: `fast = fast.next`.
3. While `fast`: advance both.
4. `slow.next = slow.next.next`.
5. Return `dummy.next`.

Complexity. Time `O(L)` in one pass. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def removeNthFromEnd(self, head: ListNode | None, n: int) -> ListNode | None:
        # Dummy before the head so that "the node before the target" always exists,
        # even when the target IS the head (n == length). Without it you'd need a
        # separate `if target is head: return head.next` branch.
        dummy = ListNode(0, head)
        slow = fast = dummy

        # Open a gap of n+1 between them. The +1 is deliberate: it leaves `slow`
        # on the PREDECESSOR of the node to delete, which is what unlinking needs.
        for _ in range(n + 1):
            fast = fast.next

        # Slide the whole gap to the end. When fast falls off, slow is exactly
        # n+1 nodes back from None -> i.e. just before the nth-from-last node.
        while fast:
            slow = slow.next
            fast = fast.next

        slow.next = slow.next.next      # unlink

        return dummy.next               # dummy.next, not head — head may be gone
```

DRY RUN

`head = 1 → 2 → 3 → 4 → 5`, `n = 2` (remove the `4`)

With the dummy: `D → 1 → 2 → 3 → 4 → 5 → None`

Step 2 — advance `fast` by `n + 1 = 3`:

move	fast
start	D
1	1
2	2
3	**3**

`slow = D`, `fast = 3`.

Step 3 — advance both until `fast` is `None`:

step	slow	fast
1	1	4
2	2	5
3	**3**	`None` → stop

`slow = 3`, which is the predecessor of `4`. ✅

Step 4: `slow.next = slow.next.next` → `3.next = 5`.

Result: `1 → 2 → 3 → 5`. ✅

Head-removal case, `head = 1 → 2`, `n = 2`:

- `fast` advances 3 from `D`: `D → 1 → 2 → None`. So `fast = None` immediately.
- The `while` never runs; `slow` stays at `D`.
- `slow.next = slow.next.next` → `D.next = 2`.
- Return `dummy.next = 2`. ✅

The dummy is doing all the work in that case — without it, `slow` would have nowhere valid to stand.

---

143. Reorder List

UNDERSTAND THE PROBLEM

Given `L0 → L1 → … → Ln`, reorder in place to `L0 → Ln → L1 → Ln−1 → …`. **Values may not be changed** — you must relink nodes.

- The "values may not be changed" clause explicitly bans the array-of-values shortcut.
- This is three primitives composed: **find the middle** (fast/slow), **reverse the second half** (206), **merge alternately** (a variant of 21). Recognising the decomposition out loud is most of the score here.
- Edge cases: 0, 1, or 2 nodes (unchanged); odd vs even length — the split must put the extra node in the **first** half.

BRUTE-FORCE APPROACH

Idea. Copy all node references into an array, then relink by walking the array from both ends inward.

Data structures. An array of `n` node references.

Algorithm. `nodes = [...]`; `i, j = 0, n−1`; alternately link `nodes[i] → nodes[j] → nodes[i+1] → …`; terminate the final node.

Complexity. Time `O(n)`, space `O(n)`.

Why it's not ideal. Honestly, this one is *fine* and much easier to get right under pressure — if you're running out of time, write it and say "this is `O(n)` space; the `O(1)` version splits, reverses, and merges." But the `O(1)` version is the expected answer, and it's the one that demonstrates the three primitives are automatic for you.

OPTIMAL APPROACH

Idea (split → reverse → merge).

1. **Find the middle** with fast/slow. Starting both at `head` and looping `while fast.next and fast.next.next` leaves `slow` at the end of the **first** half, which is the split point that puts the extra node in the first half on odd lengths — exactly what the target ordering needs.
2. **Cut** the list: `second = slow.next`, `slow.next = None`.
3. **Reverse** `second` in place (primitive from 206).
4. **Merge alternately**, taking one node from each list until the second is exhausted.

Data structures. Pointers only.

Complexity. Time `O(n)` — three linear passes. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def reorderList(self, head: ListNode | None) -> None:
        """Reorders in place. Returns None (LeetCode's signature)."""
        if not head or not head.next:
            return                      # 0 or 1 node: already correct

        # ---- 1. Find the end of the FIRST half (fast/slow) ----
        # Condition `fast.next and fast.next.next` (rather than `fast and fast.next`)
        # leaves slow at the LAST node of the first half. For odd n that puts the
        # extra node in the first half, which is what the target ordering requires:
        #   [1,2,3,4,5] must split as [1,2,3] + [4,5], not [1,2] + [3,4,5].
        slow = fast = head
        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next

        # ---- 2. Cut into two independent lists ----
        second = slow.next
        slow.next = None                # terminate the first half

        # ---- 3. Reverse the second half (LC 206 inlined) ----
        prev = None
        cur = second
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        second = prev                   # new head of the reversed second half

        # ---- 4. Merge alternately: first, second, first, second, ... ----
        # The first half is >= the second in length, so `second` runs out first
        # and drives the loop.
        first = head
        while second:
            n1, n2 = first.next, second.next    # save both before overwriting
            first.next = second
            second.next = n1
            first, second = n1, n2
```

DRY RUN

`head = 1 → 2 → 3 → 4 → 5` (odd length)

Step 1 — find the middle:

iteration	slow	fast	condition `fast.next and fast.next.next`
start	1	1	`2` and `3` → true
1	2	3	`4` and `5` → true
2	3	5	`fast.next` is `None` → false, stop

`slow = 3` — the last node of the first half. ✅

Step 2 — cut: `second = 4 → 5`; first half becomes `1 → 2 → 3 → None`.

Step 3 — reverse `second`: `5 → 4 → None`.

Step 4 — merge:

iter	first	second	n1	n2	links made	list so far
1	1	5	2	4	`1→5`, `5→2`	`1→5→2→3→None`
	2	4				
2	2	4	3	`None`	`2→4`, `4→3`	`1→5→2→4→3→None`
	3	`None`			loop exits	

Answer: `1 → 5 → 2 → 4 → 3`. ✅

Even length, `1 → 2 → 3 → 4` : - Step 1: start `slow=1, fast=1` ; check `2 and 3` → true → `slow=2, fast=3` ; check `fast.next=4` and `fast.next.next=None` → false, stop. `slow = 2` . - Cut: `[1,2]` + `[3,4]` . Reverse → `4 → 3` . - Merge: `1→4→2→3` . ✅

Both parities come out right from the single loop condition — which is why that condition is worth memorising verbatim rather than re-deriving under pressure.

---

138. Copy List with Random Pointer ★★

UNDERSTAND THE PROBLEM

Each node has `next` **and** a `random` pointer to any node in the list or `None`. Return a **deep copy**: an entirely new set of nodes with the same structure.

- The difficulty: when copying node `X`, its `random` target may not have been created yet. You need old→new correspondence for arbitrary targets.
- Edge cases: empty list; `random` pointing to the node itself; `random = None`; all randoms pointing at the head.
- **Follow-up:** `O(1)` extra space — know the interleaving trick.

BRUTE-FORCE APPROACH

Idea. Two passes with a hash map: first create a copy of every node and record `old → new`; second pass, wire up `next` and `random` by looking up each original target in the map.

Data structures. A dict `dict[Node, Node]`.

Algorithm.

1. Pass 1: for each old node, `mapping[old] = Node(old.val)`.
2. Pass 2: `mapping[old].next = mapping.get(old.next)`; `mapping[old].random = mapping.get(old.random)`.
3. Return `mapping[head]`.

Complexity. Time `O(n)`, space `O(n)` for the map.

Why it's not ideal. This is genuinely a good solution and you should write it first — the time is optimal and the code is short. It's "not ideal" only against the stated follow-up: the `O(n)` map is auxiliary storage that can be eliminated, because the correspondence old→new can be encoded **in the list itself** by interleaving copies next to their originals.

(Handling `None` gracefully: use `mapping.get(x)` or seed `mapping[None] = None` so the lookups need no branching.)

OPTIMAL APPROACH — `O(1)` EXTRA SPACE, INTERLEAVING

Idea. Three passes, no map:

1. **Interleave.** After each original node `A`, splice its copy `A'`: `A → A' → B → B' → …`. Now `A.next` *is* the map entry for `A`.
2. **Assign randoms.** For each original `A`: `A.next.random = A.random.next` — because `A.random.next` is exactly the copy of `A.random`. Guard for `A.random is None`.
3. **Unweave.** Restore the original list's `next` pointers and extract the copies into their own chain.

Data structures. Pointers only.

Complexity. Time `O(n)` — three linear passes. Space `O(1)` extra.

The critical detail in pass 3: you must restore the **original** list too, not just extract the copy. Leaving the input interleaved is a mutation bug that graders catch.

PYTHON CODE

```python
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random


class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        """O(1) extra space — interleave, wire, unweave."""
        if not head:
            return None

        # ---- PASS 1: interleave copies:  A -> A' -> B -> B' -> ... ----
        # After this, for ANY original node X, X.next is its copy. That IS the
        # old->new mapping, stored inside the list instead of in a hash map.
        cur = head
```

```
        while cur:
            copy = Node(cur.val)
            copy.next = cur.next
            cur.next = copy
            cur = copy.next          # skip over the copy we just inserted

        # ---- PASS 2: wire up the random pointers ----
        cur = head
        while cur:
            if cur.random:
                # cur.next        is the copy of cur
                # cur.random.next is the copy of cur.random
                cur.next.random = cur.random.next
            # (leave it None otherwise - a fresh Node already has random=None)
            cur = cur.next.next
        # Note this MUST be a separate pass: during pass 1 the copy of a node
        # that appears later in the list doesn't exist yet.

        # ---- PASS 3: unweave, restoring the original AND extracting the copy ----
        cur = head
        copy_head = head.next
        while cur:
            copy = cur.next
            cur.next = copy.next                # restore the original's link
            copy.next = cur.next.next if cur.next else None   # relink the copy chain
            cur = cur.next

        return copy_head


class SolutionHashMap:
    """The O(n)-space version. Write this FIRST in an interview - it's shorter and
    harder to get wrong - then offer the interleaving upgrade."""
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None

        mapping = {None: None}       # seeding None removes every null check below

        cur = head                   # pass 1: create every node
        while cur:
            mapping[cur] = Node(cur.val)
            cur = cur.next

        cur = head                   # pass 2: wire both pointer types
        while cur:
            mapping[cur].next = mapping[cur.next]
            mapping[cur].random = mapping[cur.random]
            cur = cur.next

        return mapping[head]
```

DRY RUN

Original: `A(1) → B(2) → C(3)`, with `A.random = C`, `B.random = A`, `C.random = None`.

Pass 1 — interleave:

```
A → A' → B → B' → C → C' → None
```

Now `A.next = A'`, `B.next = B'`, `C.next = C'` — the mapping, in place.

Pass 2 — randoms:

cur	`cur.random`	`cur.random.next`	assignment
A	C	C'	`A'.random = C'` ✅
B	A	A'	`B'.random = A'` ✅
C	`None`	—	skipped, `C'.random` stays `None` ✅

Pass 3 — unweave:

cur	copy	`cur.next =`	`copy.next =`
A	A'	`A'.next` = B	`B.next` = B'
B	B'	`B'.next` = C	`C.next` = C'
C	C'	`C'.next` = `None`	`cur.next` is `None` → `C'.next = None`

Restored original: `A → B → C → None` (with original randoms intact). Extracted copy: `A' → B' → C' → None`, with `A'.random = C'`, `B'.random = A'`, `C'.random = None`.

Answer: `A'` as the new head. ✅

Verify no cross-contamination: `A'.random` points at `C'`, a *copy* node — not at the original `C`. That's the property a grader checks by mutating the original after the copy is made.

---

148. Sort List ★

UNDERSTAND THE PROBLEM

Sort a linked list in `O(n log n)` time and `O(log n)` space.

- The space bound is the interesting constraint: it permits the recursion stack of merge sort but forbids copying to an array (`O(n)`).
- **Merge sort is the right algorithm for linked lists** — unlike arrays, merging needs no auxiliary buffer (just relinking), and quicksort's random access is unavailable.
- Edge cases: empty; single node; already sorted; reverse sorted; duplicates.

BRUTE-FORCE APPROACH

Idea. Copy values into an array, sort it, write them back.

Data structures. An array of `n` values.

Algorithm. Traverse, `sort()` , traverse assigning.

Complexity. Time `O(n log n)` , space **`O(n)`** .

Why it's not ideal. The time is optimal; the space is `O(n)` , violating the `O(log n)` bound. It also sorts *values* rather than relinking nodes, which is wrong if nodes carry payload beyond `val` . (An insertion-sort-on-list approach avoids the array but is `O(n²)` — also rejected.)

OPTIMAL APPROACH

Idea (top-down merge sort on the list).

1. **Base case:** a list of 0 or 1 nodes is sorted.
2. **Split** in the middle using fast/slow, then **cut** the link so you have two independent lists.
3. **Recursively sort** each half.
4. **Merge** the two sorted halves with the dummy-head merge from LC 21.

The split must produce two **strictly smaller** lists or the recursion never terminates. Using `prev` to track the node before `slow` and cutting there is the safe formulation — with a 2-node list, `slow` must land on the second node so the halves are 1 and 1.

Data structures. Recursion stack (depth `log n`); a dummy node per merge.

Complexity. Time `O(n log n)` — `log n` levels, `O(n)` merging per level. Space `O(log n)` for the recursion stack.

Bottom-up merge sort achieves `O(1)` space by iterating over sublist widths 1, 2, 4, … — worth naming as the follow-up if they push past `O(log n)` .

PYTHON CODE

```python
class Solution:
    def sortList(self, head: ListNode | None) -> ListNode | None:
        # Base case: 0 or 1 nodes are already sorted.
        if not head or not head.next:
            return head

        # ---- SPLIT: find the middle and CUT into two independent lists ----
        # `prev` trails slow so we can sever the link before slow. With
        # `slow = fast = head` and this loop condition, a 2-node list gives
        # slow = second node -> halves of size 1 and 1. Any split that can yield
        # an empty half would recurse forever.
        prev, slow, fast = None, head, head
        while fast and fast.next:
            prev = slow
            slow = slow.next
            fast = fast.next.next
        prev.next = None              # sever: head..prev  |  slow..end

        # ---- CONQUER: sort each half ----
        left = self.sortList(head)
        right = self.sortList(slow)

        # ---- COMBINE ----
        return self._merge(left, right)

    def _merge(self, a: ListNode | None, b: ListNode | None) -> ListNode | None:
        """Exactly LC 21 - dummy head, splice the smaller, attach the remainder.
        Merging linked lists needs NO auxiliary buffer, which is why merge sort
        is the natural choice here and quicksort is not."""
        dummy = ListNode()
        tail = dummy

        while a and b:
            if a.val <= b.val:        # <= keeps the sort stable
                tail.next = a
                a = a.next
            else:
                tail.next = b
                b = b.next
            tail = tail.next

        tail.next = a if a else b     # attach whatever remains, already sorted
        return dummy.next
```

DRY RUN

`head = 4 → 2 → 1 → 3`

Level 1 — split `[4,2,1,3]`:

iter	prev	slow	fast
start	`None`	4	4
1	4	2	1
2	2	1	`None` (fast was 1, `fast.next` =3, `fast.next.next` =None → loop test `fast and fast.next` → 1 and 3 → true, so this iteration ran)

Careful trace: start `slow=4, fast=4`. Test `fast(4) and fast.next(2)` → true → `prev=4, slow=2, fast=1`. Test `fast(1) and fast.next(3)` → true → `prev=2, slow=1, fast=None`. Test fails.

`prev = 2`, so cut after node `2`: left = `4 → 2`, right = `1 → 3`.

Level 2a — sort `[4, 2]`: split → `prev=4, slow=2` (one iteration: `fast(4) and fast.next(2)` true → `prev=4, slow=2, fast=None`). Cut → `[4]` and `[2]`. Both base cases. Merge: `2 ≤ 4`? compare `a=4, b=2` → `4 <= 2` false → take `b` → `2`, then attach `4`. Result `2 → 4`.

Level 2b — sort `[1, 3]`: split → `[1]` and `[3]`. Merge → `1 → 3`.

Level 1 merge — `[2,4]` with `[1,3]`:

a	b	compare	take	output
2	1	`2 <= 1` ? no	b=1	`1`
2	3	`2 <= 3` ? yes	a=2	`1→2`
4	3	no	b=3	`1→2→3`
4	`None`	loop exits	attach `4`	`1→2→3→4`

Answer: `1 → 2 → 3 → 4`. ✅

Recursion depth was 2 for `n = 4` — matching `log₂ 4`, which is the `O(log n)` space claim made concrete.

VM Load — ✅ Verified pattern (InterviewDB)

UNDERSTAND THE PROBLEM

Route each request to the **least-loaded** server, breaking ties by **lowest index**. Return either the chosen server per request or the final load vector.

- Each request presumably adds some load (clarify: uniform `+1`, or a per-request weight?). Assume a weight per request; uniform is the special case.
- Clarify: *"Do loads ever decrease — do requests finish and release capacity? Is `n` (servers) or `m` (requests) larger? Do I return the assignment sequence or the final loads?"* If loads decrease, this becomes a scheduling problem needing a release event stream, which is a materially different design.
- Edge cases: one server; all loads equal (always picks index 0); zero requests.

BRUTE-FORCE APPROACH

Idea. For each request, scan all `n` servers to find the minimum load, tie-breaking on index.

Data structures. An array `load[0..n-1]`.

Algorithm. For each request `w`: `best = argmin(load)` scanning left to right (so the first minimum found is the lowest index); `load[best] += w`; record `best`.

Complexity. Time `O(m · n)`. Space `O(n)`.

Why it's not ideal. When `n` is large (a fleet of thousands) and `m` is large, `m·n` is prohibitive. The linear scan re-derives the minimum from scratch every time, when only **one** entry changed since the last query. "Repeatedly extract the minimum, then update one element" is the textbook heap workload.

Important caveat worth voicing: if `n` is *small* (say ≤ 64), the scan is genuinely faster than a heap due to cache behaviour and constant factors. Say that — knowing when *not* to reach for the fancy structure is a signal.

OPTIMAL APPROACH

Idea (min-heap keyed on `(load, index)`). Push all servers as `(load, index)`. For each request: pop the smallest, apply the load, push it back. The tuple ordering gives the index tie-break **for free**: Python compares tuples element-wise, so equal loads fall back to comparing indices, and the smaller index wins.

Data structures. `heapq` of `(load, index)` tuples; an array of final loads.

Algorithm.

1. `heap = [(0, i) for i in range(n)]` — already a valid heap since it's sorted.
2. For each request weight `w`: `load, i = heappop(heap)`; record `i`; `heappush(heap, (load + w, i))`.
3. Return the assignments and/or the final loads.

Complexity. Time `O(n + m log n)`. Space `O(n)`.

Follow-up — servers that release load. Then you need two structures: a min-heap of `(load, index)` for selection, plus **lazy deletion** (push updated entries and discard stale pops by checking against an authoritative `load[]` array), or an indexed priority queue / `SortedList` supporting decrease-key. Naming lazy deletion here is a strong signal.

PYTHON CODE

```python
import heapq
from typing import Sequence


def assign_requests(n: int, requests: Sequence[int]) -> tuple[list[int], list[int]]:
    """
    Route each request to the least-loaded server, ties broken by LOWEST index.

    Args:
        n:        number of servers, indexed 0..n-1, all starting at load 0
        requests: the load each request adds (use [1]*m for uniform requests)

    Returns:
        (assignment per request, final load per server)
    """
```

```python
    if n <= 0:
        raise ValueError("need at least one server")

    # (load, index). Python compares tuples left to right, so equal loads fall
    # through to comparing indices — the lowest-index tie-break comes for free
    # with no custom comparator.
    heap = [(0, i) for i in range(n)]
    # Already heap-ordered (all keys equal, indices ascending), but heapify is
    # O(n) and makes the invariant explicit for a reader.
    heapq.heapify(heap)

    loads = [0] * n
    assignment = []

    for w in requests:
        load, i = heapq.heappop(heap)      # O(log n) — the least-loaded server
        assignment.append(i)
        loads[i] = load + w
        heapq.heappush(heap, (loads[i], i))   # O(log n) — reinsert with new load

    return assignment, loads


def assign_requests_scan(n: int, requests: Sequence[int]) -> tuple[list[int], list[int]]:
    """O(m*n) linear-scan version. Genuinely FASTER than the heap for small n
    (cache locality, no tuple allocation). Worth saying out loud — reaching for a
    heap when n is 8 is over-engineering."""
    loads = [0] * n
    assignment = []
    for w in requests:
        best = 0
        for i in range(1, n):
            if loads[i] < loads[best]:     # strict < keeps the FIRST minimum,
                best = i                   # which is the lowest index
        assignment.append(best)
        loads[best] += w
    return assignment, loads
```

DRY RUN

`n = 3`, `requests = [2, 3, 1, 4]`

Initial heap: `[(0,0), (0,1), (0,2)]`, `loads = [0,0,0]`

request `w`	pop	why this one	new load	push	heap after	loads
2	`(0,0)`	all tied at 0 → **lowest index wins**	0+2=2	`(2,0)`	`(0,1),(0,2),(2,0)`	`[2,0,0]`
3	`(0,1)`	0 < 2; ties with server 2 → index 1 wins	0+3=3	`(3,1)`	`(0,2),(2,0),(3,1)`	`[2,3,0]`
1	`(0,2)`	0 is the minimum	0+1=1	`(1,2)`	`(1,2),(2,0),(3,1)`	`[2,3,1]`
4	`(1,2)`	1 < 2 < 3	1+4=5	`(5,2)`	`(2,0),(3,1),(5,2)`	`[2,3,5]`

Assignment: `[0, 1, 2, 2]`. Final loads: `[2, 3, 5]`. ✅

The tie-break is visible in rows 1 and 2: with three servers at load 0, servers were chosen in index order 0, then 1, then 2 — never arbitrarily. That determinism is what the `(load, index)` tuple buys, and it's the specific thing a grader will test with an all-equal input.

Block Words — 🟡 Trie / Aho–Corasick (InterviewDB)

UNDERSTAND THE PROBLEM

Given a text and a list of blocked words, determine whether the text contains any blocked word.

Clarify before writing a single line — these change the algorithm outright: 1. **Substring or whole-word matching?** `"class"` contains `"ass"`. Substring matching flags it; whole-word matching doesn't. This is the Scunthorpe problem and it's a real product decision. 2. **Case sensitivity?** Almost always case-insensitive → normalise both sides. 3. **Punctuation and tokenisation?** Is `"bad-word"` one token or two? Does `"b.a.d"` count? 4. **Return a boolean, the matches, or the positions?**

The prompt's own note says GPT's research confirmed the core but *not* these rules — so the interview value here is entirely in asking. State your assumptions explicitly and code to them.

- Edge cases: empty text or empty blocklist; a blocked word longer than the text; overlapping blocked words; one blocked word being a prefix of another (`"bad"` and `"badly"`).

BRUTE-FORCE APPROACH

Idea. For each blocked word, run a substring search over the text.

Data structures. A list of blocked words.

Algorithm. `any(w in text for w in blocked)` — or an explicit `O(n·m)` scan per word.

Complexity. Time `O(k · n · m)` in the worst case for `k` words, text length `n`, word length `m`. Python's `in` uses a tuned mixed algorithm, so in practice it's closer to `O(k · n)` — say that; claiming naive `O(k·n·m)` for `str.__contains__` is technically pessimistic.

Why it's not ideal. The text is re-scanned once per blocked word. With a blocklist of 10,000 entries and a large document, that's 10,000 full passes over the text. Every pass rediscovers the same character positions. A single pass that tracks *all* candidate words simultaneously is what Aho–Corasick provides.

Whole-word variant: brute force is actually excellent — tokenise the text and check each token against a `set`. That's `O(n)` total and beats the trie. Only reach for a trie when substring matching is required.

OPTIMAL APPROACH

Case A — whole-word matching: tokenise and use a **hash set**. `O(n)` time, `O(total blocklist size)` space. Simple, and correct. Don't over-engineer this case.

Case B — substring matching: build an **Aho–Corasick automaton** — a trie of the blocked words augmented with **failure links** (like KMP's failure function, generalised to many patterns). Then scan the text once, following `goto` edges and falling back along failure links on a mismatch. Every match is detected in a single pass.

- Build: `O(Σ|blocked words|)`.
- Search: `O(n + number of matches)`.

Data structures. Trie nodes with a `children` dict, a `fail` link, and an `output` flag/list. A BFS queue to construct the failure links.

Algorithm (construction).

1. Insert every blocked word into the trie, marking terminal nodes.
2. BFS from the root: for each node `u` and edge `u --c--> v`, set `v.fail` to the node reached by following `u.fail`'s chain until a `c` edge exists (or the root). Merge `v.fail.output` into `v.output` so a match at a suffix is reported too.

Algorithm (search). Walk the text; on character `c`, follow `cur.children[c]` if present, else follow `fail` links until you can move or reach the root. If `cur.output` is non-empty → a blocked word was found.

Complexity. Build `O(Σ|w|)`. Search `O(n)` plus output size. Space `O(Σ|w| · Σ_alphabet)` worst case, `O(Σ|w|)` with dict children.

PYTHON CODE

```
from collections import deque
from typing import Iterable
```

```python
import re


# ============ Case A: WHOLE-WORD matching — use a set, not a trie ============
def contains_blocked_word(text: str, blocked: Iterable[str]) -> bool:
    """Whole-word, case-insensitive. O(n) — a trie would be over-engineering here."""
    blocked_set = {w.lower() for w in blocked}
    # \w+ tokenises on non-word characters, so "bad-word" becomes ["bad", "word"].
    # Whether that's correct is a PRODUCT decision — confirm it with the interviewer.
    tokens = re.findall(r"\w+", text.lower())
    return any(tok in blocked_set for tok in tokens)


# ============ Case B: SUBSTRING matching — Aho-Corasick ============
class _Node:
    __slots__ = ("children", "fail", "output")

    def __init__(self):
        self.children: dict[str, "_Node"] = {}
        self.fail: "_Node | None" = None
        self.output: list[str] = []     # words ending here (incl. via suffix links)


class BlockedWordMatcher:
    """
    Multi-pattern substring matcher. Builds once, then scans any number of texts
    in a single pass each — the win over `any(w in text ...)`, which rescans the
    text once per blocked word.
    """

    def __init__(self, blocked: Iterable[str], case_sensitive: bool = False):
        self.case_sensitive = case_sensitive
        self.root = _Node()
        self._build_trie(blocked)
        self._build_failure_links()

    def _norm(self, s: str) -> str:
        return s if self.case_sensitive else s.lower()

    def _build_trie(self, blocked: Iterable[str]) -> None:
        for word in blocked:
            word = self._norm(word)
            if not word:
                continue                        # ignore empty patterns
            node = self.root
            for ch in word:
                node = node.children.setdefault(ch, _Node())
            node.output.append(word)            # terminal: this word ends here

    def _build_failure_links(self) -> None:
        """BFS. fail(v) = the node for the LONGEST proper suffix of v's prefix
        that is itself a prefix of some pattern. Exactly KMP's failure function,
        generalised from one pattern to many."""
        q = deque()

        # Depth-1 nodes all fail to the root: a 1-char prefix has no proper
        # suffix other than the empty string.
        for child in self.root.children.values():
            child.fail = self.root
            q.append(child)

        while q:
            u = q.popleft()
            for ch, v in u.children.items():
                # Walk up u's failure chain looking for someone with a `ch` edge.
                f = u.fail
                while f is not None and ch not in f.children:
                    f = f.fail
                v.fail = f.children[ch] if f else self.root
                if v.fail is v:                 # guard: never let a node fail to itself
                    v.fail = self.root

                # Inherit outputs: if v's failure node ends a word, then that word
                # is a suffix of what we've matched, so it must be reported too.
                # This is what makes overlapping patterns ("bad" inside "badly") work.
                v.output.extend(v.fail.output)
```

```
                q.append(v)

    def contains(self, text: str) -> bool:
        """True if ANY blocked word occurs as a substring. Single pass, O(n)."""
        return self.find_first(text) is not None

    def find_first(self, text: str) -> tuple[int, str] | None:
        """Returns (end_index, word) of the first match, or None."""
        text = self._norm(text)
        node = self.root

        for i, ch in enumerate(text):
            # On a mismatch, fall back along failure links instead of restarting
            # the scan — that's what keeps the whole thing linear in len(text).
            while node is not self.root and ch not in node.children:
                node = node.fail
            node = node.children.get(ch, self.root)

            if node.output:
                return (i, node.output[0])

        return None

    def find_all(self, text: str) -> list[tuple[int, str]]:
        """Every match as (end_index, word). O(n + number of matches)."""
        text = self._norm(text)
        node = self.root
        hits = []
        for i, ch in enumerate(text):
            while node is not self.root and ch not in node.children:
                node = node.fail
            node = node.children.get(ch, self.root)
            for w in node.output:
                hits.append((i, w))
        return hits
```

DRY RUN

`blocked = ["he", "she", "his", "hers"]`, `text = "ushers"`

Trie (paths from root): `h→e` (word `he`), `s→h→e` (word `she`), `h→i→s` (word `his`), `h→e→r→s` (word `hers`).

Node labels by the prefix they represent: `h`, `he`, `her`, `hers`, `hi`, `his`, `s`, `sh`, `she`.

Failure links (BFS):

node	fail →	reasoning
`h`	root	depth 1
`s`	root	depth 1
`he`	root	longest proper suffix `"e"` is not a prefix of any pattern
`hi`	root	`"i"` isn't a prefix
`sh`	`h`	suffix `"h"` **is** a prefix ✓
`her`	root	`"er"`, `"r"` aren't prefixes
`his`	`s`	suffix `"s"` is a prefix ✓
`she`	`he`	suffix `"he"` is a prefix ✓ — **and `he` is a terminal, so `she` inherits output `["he"]`**
`hers`	`s`	suffix `"s"` is a prefix ✓

Scan `"ushers"`:

i	ch	transition	node	output	match
0	`u`	root has no `u` → stay	root	—	
1	`s`	root→ `s`	`s`	—	
2	`h`	`s` → `sh`	`sh`	—	
3	`e`	`sh` → `she`	`she`	`["she", "he"]` ← inherited	**hit at i=3**
4	`r`	`she` has no `r`; fail to `he`; `he` has `r` → `her`	`her`	—	
5	`s`	`her` → `hers`	`hers`	`["hers"]`	**hit at i=5**

`find_all` → `[(3, "she"), (3, "he"), (5, "hers")]`. ✅

Two things the trace demonstrates: - At `i = 3`, both `"she"` and `"he"` are reported from a single node visit — that's the **output inheritance** via failure links. Without it, `"he"` inside `"she"` is silently missed. - At `i = 4`, the failure link from `she` → `he` let the scan continue without rewinding the text pointer. That non-rewinding property is exactly why the scan is `O(n)` rather than `O(n·m)`.

DAY 4 — Thu Aug 20 · Trees

> **The single organising question for every tree problem: does information flow UP or DOWN? UP (post-order)** — the answer for a node is computed *from its children's answers*. Height, diameter, balance, max path sum, LCA. Signature: `def dfs(node) -> something`, and you combine `dfs(left)` and `dfs(right)`. **DOWN (pre-order)** — each node receives context *from its ancestors*. BST validation (the allowed range), root-to-leaf paths, path-sum-with-prefix. Signature: `def dfs(node, context) -> None`, and you pass a modified context to the children. A few problems (543, 124) do **both**: they *return* one quantity upward while *tracking* a different quantity in an outer variable. Recognising that "return one, track another" shape is the day's highest-leverage skill.
>
> Shared node definition for this whole section: `python class TreeNode: def __init__(self, val=0, left=None, right=None): self.val = val self.left = left self.right = right`

---

104. Maximum Depth of Binary Tree

UNDERSTAND THE PROBLEM

Return the number of nodes along the longest root-to-leaf path.

- Depth counted in **nodes**, not edges: a single node has depth 1, empty tree has depth 0. (Problem 543 counts *edges* — the inconsistency between the two is deliberate and catches people.)
- Edge cases: empty tree; single node; a degenerate chain (depth = `n`).

BRUTE-FORCE APPROACH

Idea. Enumerate every root-to-leaf path explicitly, storing each as a list, then take the longest.

Data structures. A list of paths; a path list carried down the recursion.

Algorithm. DFS carrying the current path; at a leaf, append a copy of the path to a results list; return `max(len(p) for p in paths)`.

Complexity. Time `O(n · h)` — copying the path at each of up to `n/2` leaves. Space `O(n · h)` to hold all the paths.

Why it's not ideal. We materialise every path just to measure them. Length is an aggregate — it can be *computed* bottom-up without ever storing a path. This is the "don't build what you can fold" instinct.

OPTIMAL APPROACH

Idea (UP). `depth(node) = 1 + max(depth(left), depth(right))`, with `depth(None) = 0`. Three lines, and the recursion structure is the algorithm.

Data structures. Recursion stack only. (A BFS level-count is the iterative equivalent — worth naming, since it avoids stack overflow on a 10^5-node chain.)

Algorithm.

1. If `node is None`: return 0.
2. Return `1 + max(maxDepth(node.left), maxDepth(node.right))`.

Complexity. Time `O(n)` — each node visited once. Space `O(h)` for the stack: `O(log n)` balanced, `O(n)` degenerate.

PYTHON CODE

```python
class Solution:
    def maxDepth(self, root: TreeNode | None) -> int:
        # Classic UP/post-order: my answer is a function of my children's answers.
        if root is None:
            return 0                    # empty subtree contributes no depth
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))


from collections import deque

class SolutionBFS:
    """Iterative level-count. Same O(n) time, O(width) space, and immune to
    Python's ~1000-frame recursion limit on a degenerate chain — worth mentioning
    when the constraints say n can be 10^5."""
    def maxDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        q = deque([root])
        depth = 0
        while q:
            depth += 1
            for _ in range(len(q)):        # snapshot the level size BEFORE adding
                node = q.popleft()
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
        return depth
```

DRY RUN

```
      3
     / \
    9   20
       /  \
      15   7
```

call	node	left result	right result	returns
`depth(9)`	9	`depth(None)=0`	`depth(None)=0`	`1 + 0 = 1`
`depth(15)`	15	0	0	1
`depth(7)`	7	0	0	1
`depth(20)`	20	1 (from 15)	1 (from 7)	`1 + max(1,1) = 2`
`depth(3)`	3	1 (from 9)	2 (from 20)	**`1 + max(1,2) = 3`**

Answer: `3` — path `3 → 20 → 15`. ✅

110. Balanced Binary Tree

UNDERSTAND THE PROBLEM

Return whether every node's two subtrees differ in height by at most 1.

- The condition applies at **every** node, not just the root.
- **Watch:** the naive version is `O(n²)`.
- Edge cases: empty (balanced); single node; a chain of 3 (unbalanced).

BRUTE-FORCE APPROACH

Idea. For each node, compute the height of both subtrees and check `|hL − hR| ≤ 1`; recurse into both children.

Data structures. A separate `height()` helper.

Algorithm. `isBalanced(node) = abs(height(l) − height(r)) <= 1 and isBalanced(l) and isBalanced(r)`.

Complexity. Time `O(n²)` — `height()` is `O(n)` and is called once per node; on a degenerate chain that's `n + (n−1) + … = O(n²)`. Space `O(h)`.

Why it's not ideal. `height()` re-walks subtrees that the outer recursion is already walking. The two traversals are computing overlapping things at the same time. Merge them: have one post-order pass return the height **and** signal imbalance, so each node is touched once.

OPTIMAL APPROACH

Idea (UP, with a sentinel). One post-order function returns the subtree's height, or `-1` meaning "somewhere below me the tree is already unbalanced." A `-1` from either child short-circuits straight to `-1`.

Using an impossible height value as the failure flag avoids returning tuples — cleaner to write under pressure. (A `(height, is_balanced)` tuple is equally valid; say which you prefer and why.)

Data structures. Recursion stack.

Algorithm.

1. `check(None) = 0`.
2. `l = check(left)`; if `l == -1`: return `-1`.
3. `r = check(right)`; if `r == -1`: return `-1`.
4. If `abs(l − r) > 1`: return `-1`.
5. Return `1 + max(l, r)`.
6. Answer: `check(root) != -1`.

Complexity. Time `O(n)` — one visit per node. Space `O(h)`.

PYTHON CODE

```python
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        # -1 is a SENTINEL height meaning "already unbalanced below here".
        # Real heights are >= 0, so -1 can never be confused with a valid answer.
        # This fuses the height computation and the balance check into ONE pass,
        # turning the naive O(n^2) into O(n).
        def check(node: TreeNode | None) -> int:
            if node is None:
                return 0

            left = check(node.left)
            if left == -1:
                return -1               # short-circuit: stop climbing, answer is settled

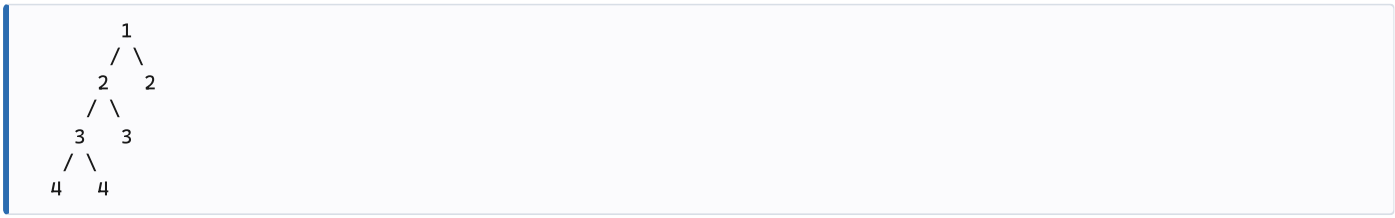
            right = check(node.right)
            if right == -1:
                return -1

            if abs(left - right) > 1:
                return -1               # THIS node is the violation

            return 1 + max(left, right)

        return check(root) != -1
```

DRY RUN



call	left	right	`|l-r|`	returns
`check(4)` (both)	0	0	0	1
`check(3)` (left)	1	1	0	2
`check(3)` (right)	0	0	0	1
`check(2)` (left)	2	1	1 ≤ 1 ✓	3
`check(2)` (right)	0	0	0	1
`check(1)`	3	1	**2 > 1** ✗	**−1**

`check(root) == -1` → return `False`. ✅

Short-circuit in action. Once any subtree returns `-1`, every ancestor returns `-1` immediately without doing further work — the `if left == -1: return -1` line before even calling `check(node.right)` means the right subtree is never traversed. On a large tree with an early violation, this is the difference between visiting a handful of nodes and visiting all of them.

---

543. Diameter of Binary Tree ★

UNDERSTAND THE PROBLEM

Return the length of the longest path between **any two nodes**, measured in **edges**. The path need not pass through the root.

- **Edges**, not nodes: a two-node tree has diameter 1, a single node has diameter 0.
- The path bends at exactly one node — its highest point. So: for every node, the best path *through* that node is `height(left) + height(right)` in edges.
- Edge cases: single node → 0; a chain of `n` → `n-1`; the optimal path skipping the root entirely.

BRUTE-FORCE APPROACH

Idea. For each node, compute `height(left) + height(right)`; take the max over all nodes.

Data structures. A `height()` helper called per node.

Complexity. Time `O(n²)` (same reason as 110 — `height()` re-walks). Space `O(h)`.

Why it's not ideal. Same redundancy as 110. And it exposes the fix: the outer traversal and the height computation want to be the **same** traversal.

OPTIMAL APPROACH

Idea ("return one, track another" — the pattern of the day). A single post-order function that: - **returns** the height of the subtree (what the *parent* needs), and - **updates an outer** `best` with `left_h + right_h`, the diameter of the path *bending at this node* (what the *answer* needs).

Those are two different quantities, and conflating them is the bug people write. The parent can only use a *downward* path; the answer wants a *bent* path. Say that out loud.

Data structures. Recursion stack; one `nonlocal` accumulator.

Algorithm.

1. `best = 0`.
2. `height(None) = 0`.
3. `l = height(left)`, `r = height(right)`; `best = max(best, l + r)`; return `1 + max(l, r)`.
4. Return `best`.

Complexity. Time `O(n)`. Space `O(h)`.

PYTHON CODE

```python
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode | None) -> int:
        best = 0        # longest bent path seen anywhere, in EDGES

        def height(node: TreeNode | None) -> int:
            """Returns the subtree height in edges below `node`, counting `node`
            itself as contributing 1 to its parent."""
            nonlocal best
            if node is None:
                return 0

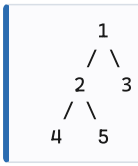
            left = height(node.left)
            right = height(node.right)

            # TRACK: the best path that BENDS here goes down-left and down-right.
            # In edges that's exactly left + right (no +1 — this node is the apex,
            # and the edges are already counted in the two child heights).
            best = max(best, left + right)

            # RETURN: the parent can only extend a path DOWNWARD through one side,
            # so it gets the taller branch plus the edge to this node.
            # Returning `left + right` here instead is the classic wrong answer —
            # a parent cannot use a path that already bent.
            return 1 + max(left, right)

        height(root)
        return best
```

DRY RUN



call	left h	right h	`best` update (`l+r`)	returns `1+max`
`height(4)`	0	0	`0+0 = 0` → best 0	1
`height(5)`	0	0	0 → best 0	1
`height(2)`	1	1	**`1+1 = 2`** → best 2	2
`height(3)`	0	0	0 → best 2	1
`height(1)`	2	1	**`2+1 = 3`** → best 3	3

Answer: `3` — the path `4 → 2 → 1 → 3` (or `5 → 2 → 1 → 3`), which is 3 edges. ✅

Path that skips the root. Add a long left chain: `4` gets children forming a depth-3 subtree. Then `height(2)` would record `l+r` larger than anything the root can produce, and `best` — being an outer variable updated at *every* node — captures it. That's precisely why the answer can't just be computed at the root.

---

102. Binary Tree Level Order Traversal

UNDERSTAND THE PROBLEM

Return the node values grouped level by level, top to bottom, left to right within a level.

- Output is a list of lists, one per level.
- Edge cases: empty tree → `[]`; single node → `[[1]]`; skewed tree (each level has one node).

BRUTE-FORCE APPROACH

Idea. Compute the tree's height, then for each depth `d` from 0 to `h-1`, run a full DFS collecting only the nodes at depth `d`.

Data structures. One list per level.

Algorithm. `for d in range(height): collect_at_depth(root, d)`.

Complexity. Time `O(n · h)` — a full traversal per level. Space `O(n)`.

Why it's not ideal. Traverses the whole tree once per level. BFS visits every node exactly once and produces the levels as a natural by-product — no repetition at all.

OPTIMAL APPROACH

Idea (BFS with a level snapshot). Use a queue. At the top of each outer iteration, **capture** `len(queue)` — that count is exactly the current level's size, because all of that level's nodes were enqueued before any of the next level's. Pop exactly that many, collecting values and enqueuing children.

That `for _ in range(len(q))` snapshot is the primitive worth memorising; it converts a flat BFS into a level-partitioned BFS with one line.

Data structures. `collections.deque`.

Algorithm.

1. If not root: return `[]`. `q = deque([root])`, `res = []`.
2. While `q`: `size = len(q)`; `level = []`; repeat `size` times: pop, append value, enqueue non-null children. Append `level` to `res`.
3. Return `res`.

Complexity. Time `O(n)`. Space `O(w)` where `w` is the max level width (`O(n)` for a full bottom level).

PYTHON CODE

```python
from collections import deque

class Solution:
    def levelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        res = []
        q = deque([root])

        while q:
            # SNAPSHOT the level size before the inner loop mutates the queue.
            # Everything currently in q belongs to this level, because a node's
            # children are only ever enqueued after all of its level-mates.
            size = len(q)
            level = []

            for _ in range(size):
                node = q.popleft()
                level.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)

            res.append(level)

        return res
```

DRY RUN

```
      3
     / \
    9   20
       /  \
      15   7
```

outer iter	`size`	popped	values collected	enqueued	queue after	res
1	1	3	`[3]`	9, 20	`[9, 20]`	`[[3]]`
2	2	9, 20	`[9, 20]`	(9 has none), 15, 7	`[15, 7]`	`[[3],[9,20]]`
3	2	15, 7	`[15, 7]`	none	`[]`	`[[3],[9,20],[15,7]]`

Answer: `[[3], [9,20], [15,7]]`. ✅

Note iteration 2: the queue *grows* to 4 entries mid-loop (`20, 15, 7` after popping 9), but `size` was frozen at 2, so exactly the right two nodes were consumed. Reading `len(q)` inside the loop instead is the standard bug.

---

199. Binary Tree Right Side View

UNDERSTAND THE PROBLEM

Return the values visible when the tree is viewed from the right — i.e. the **rightmost node of each level**, top to bottom.

- "Rightmost of each level", not "the right spine". If a node's right subtree is missing but its left subtree is deeper, a left-descendant becomes visible.
- Edge cases: empty; a left-only chain (every node is visible); a full tree (the right spine).

BRUTE-FORCE APPROACH

Idea. Run the full level-order traversal from problem 102, then take the last element of each level.

Data structures. The full level lists.

Algorithm. `[level[-1] for level in levelOrder(root)]`.

Complexity. Time `O(n)`, space `O(n)` — because it materialises every level.

Why it's not ideal. The time is already optimal, so this is a **space** critique only: we build `n` values of intermediate lists to keep `h` of them. A one-line change — record only the last node of each level — gets the auxiliary output down to `O(h)`. Worth saying explicitly, because "reuse 102 and slice" is a perfectly good answer and you should offer it before micro-optimising.

OPTIMAL APPROACH

Idea (BFS, keep the last of each level). Same level-snapshot BFS, but instead of collecting the whole level, append `node.val` only when `i == size - 1`.

Alternative (DFS, right-first). Traverse right before left, tracking depth; the **first** node seen at each depth is the visible one. `if depth == len(res): res.append(node.val)`. This is `O(h)` space and elegant — know both.

Data structures. A deque (BFS) or recursion (DFS).

Complexity. Time `O(n)`. Space `O(w)` BFS / `O(h)` DFS.

PYTHON CODE

```python
from collections import deque

class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        res = []
        q = deque([root])

        while q:
            size = len(q)
            for i in range(size):
                node = q.popleft()
                # The LAST node popped from this level is the rightmost one —
                # which is exactly what an observer to the right would see.
                if i == size - 1:
                    res.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)

        return res


class SolutionDFS:
    """Right-first DFS. O(h) space instead of O(width) — better on a wide tree."""
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        res = []

        def dfs(node: TreeNode | None, depth: int) -> None:
            if not node:
                return
            # First node reached at this depth wins. Because we always recurse
            # RIGHT first, that first arrival is the rightmost node of the level.
            if depth == len(res):
                res.append(node.val)
            dfs(node.right, depth + 1)     # right BEFORE left — the whole trick
            dfs(node.left, depth + 1)

        dfs(root, 0)
        return res
```

DRY RUN

```
      1
     / \
    2   3
     \   \
      5   4
```

BFS trace:

level	nodes popped (in order)	last one	res
0	1	**1**	`[1]`
1	2, 3	**3**	`[1,3]`
2	5, 4	**4**	`[1,3,4]`

Answer: `[1, 3, 4]`. ✅

The case that breaks "just follow the right spine":

```
      1
     / \
    2   3
   /
  4
```

The right spine is `1 → 3`, but level 2 contains only node `4`, so the answer is `[1, 3, 4]` — node `4` is visible from the right because nothing is to its right at that depth. Both solutions above handle it; a naive `while node.right` walk does not.

---

662. Maximum Width of Binary Tree ★

UNDERSTAND THE PROBLEM

Return the maximum width of any level, where width is measured **between the outermost non-null nodes of that level, counting the null gaps in between**.

- The nulls between the extremes count; nulls outside them do not.
- The catch: indices grow exponentially with depth. On a 3000-deep skewed tree, a naive heap index overflows in most languages — Python's arbitrary-precision ints save you, but you should still **normalise per level** and say why (it's the language-independent fix).
- Edge cases: single node → 1; full tree → width of the bottom level; a deep chain → 1.

BRUTE-FORCE APPROACH

Idea. Build the complete-tree array representation (fill nulls), then measure each level's span.

Data structures. An array of size `2^h`.

Complexity. Time and space `O(2^h)` — **exponential**.

Why it's not ideal. A tree of depth 3000 would need `2³⁰⁰⁰` slots. The realisation: we don't need the *array*, only the **indices**. Carrying an index alongside each node in the traversal gives the same arithmetic for `O(n)` space.

OPTIMAL APPROACH

Idea (heap indexing + per-level normalisation). Assign the root index 0; a node at index `i` has children at `2i` and `2i + 1`. A level's width is `last_index - first_index + 1`.

To keep indices small, **subtract the level's first index from every index on that level** before enqueuing children. Widths are differences, so shifting the whole level leaves them unchanged, but it resets the exponential growth at every level.

Data structures. `deque` of `(node, index)` pairs.

Algorithm.

1. `q = deque([(root, 0)])`, `best = 0`.
2. While `q`: `size = len(q)`; `_, first = q[0]`; loop `size` times popping `(node, i)`, normalising `i -= first`, enqueuing `(left, 2i)` and `(right, 2i+1)`; track `last = i`.
3. `best = max(best, last - 0 + 1)`.
4. Return `best`.

Complexity. Time `O(n)`. Space `O(w)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def widthOfBinaryTree(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        best = 0
        q = deque([(root, 0)])      # (node, heap index)

        while q:
            size = len(q)
            _, first = q[0]         # leftmost index on this level, BEFORE popping
            last = first

            for _ in range(size):
                node, i = q.popleft()

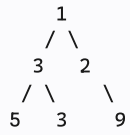
                # NORMALISE: shift every index on this level so the leftmost is 0.
                # Width is a DIFFERENCE, so a uniform shift can't change it — but
                # it stops indices doubling every level. Without this, a 3000-deep
                # skewed tree produces a 3000-bit integer (fine in Python, an
                # overflow in Java/C++). Do it anyway; it's the portable fix.
                i -= first
                last = i

                if node.left:
                    q.append((node.left, 2 * i))
                if node.right:
                    q.append((node.right, 2 * i + 1))

            # Nulls BETWEEN the extremes are counted (they occupy indices);
            # nulls outside are not (they were never enqueued).
            best = max(best, last + 1)

        return best
```

DRY RUN



level	queue entries `(node, raw idx)`	`first`	normalised	width	best
0	`(1, 0)`	0	`1→0`	`0+1 = 1`	1
1	`(3, 0)`, `(2, 1)`	0	`3→0`, `2→1`	`1+1 = 2`	2
2	`(5, 0)`, `(3, 1)`, `(9, 3)`	0	unchanged	**`3+1 = 4`**	**4**

Level 2 detail: node `3` (at normalised index 0) has children at `0` and `1` → nodes `5` and `3`. Node `2` (at index 1) has only a right child, at `2·1+1 = 3` → node `9`. Index 2 is a gap and **counts**, giving width 4.

Answer: `4`. ✅

124. Binary Tree Maximum Path Sum ★★

UNDERSTAND THE PROBLEM

Return the largest sum along any path between any two nodes. The path need not pass through the root, and must be a connected sequence of nodes where consecutive nodes share an edge — each node appearing at most once.

- Values can be **negative**, which is the entire difficulty. A negative subtree should simply be excluded.
- The path must contain at least one node, so an all-negative tree returns its largest single value (not 0).
- **Same shape as 543** — return one quantity, track another — with `max(0, …)` clamping added for the negatives.
- Edge cases: single negative node → that value; all negatives; a path bending at a deep node.

BRUTE-FORCE APPROACH

Idea. For every node, compute the best downward path sum in each direction by a fresh recursive scan, then combine.

Complexity. Time `O(n²)`, space `O(h)`.

Why it's not ideal. Same redundancy as 110 and 543: the per-node scan re-walks the same subtrees the outer traversal already covers. Fold both into a single post-order pass.

OPTIMAL APPROACH

Idea. One post-order function `gain(node)` returning **the best sum of a path that starts at `node` and goes strictly downward** — the only thing a parent can extend. Meanwhile, track `best` = the best path *bending* at each node, which is `node.val + left_gain + right_gain`.

Two design points: 1. **Clamp at zero:** `left = max(0, gain(node.left))`. A negative branch is never worth including — dropping it is always at least as good. This one `max(0, …)` is what handles negatives. 2. **Return `node.val + max(left, right)`**, not `node.val + left + right`. The parent can only pass *through* this node on one side; a bent path is a dead end from the parent's perspective.

Data structures. Recursion stack; one accumulator initialised to `-inf` (not 0 — an all-negative tree must not return 0).

Complexity. Time `O(n)`. Space `O(h)`.

PYTHON CODE

```python
class Solution:
    def maxPathSum(self, root: TreeNode | None) -> int:
        # -inf, NOT 0: the path must contain at least one node, so an all-negative
        # tree like [-3] must return -3. Seeding with 0 silently returns 0.
        best = float('-inf')

        def gain(node: TreeNode | None) -> int:
            """Best sum of a path that starts at `node` and descends one way only.
            That's the only shape a parent can extend."""
            nonlocal best
            if node is None:
                return 0

            # CLAMP AT ZERO: a branch with a negative best-gain is worth excluding
            # entirely — taking 0 instead means "don't go that way". This single
            # max() is the whole negative-number handling.
            left = max(0, gain(node.left))
            right = max(0, gain(node.right))

            # TRACK: the best path that BENDS at this node uses both sides.
            best = max(best, node.val + left + right)

            # RETURN: the parent can only continue through ONE side.
            # Returning left + right here would let an ancestor reuse a bent path,
            # producing a shape that isn't a path at all.
            return node.val + max(left, right)

        gain(root)
        return best
```

DRY RUN

```
      -10
      /  \
     9    20
         /  \
        15   7
```

call	left (clamped)	right (clamped)	`best` candidate = `val+l+r`	best	returns `val + max(l,r)`
`gain(9)`	0	0	`9+0+0 = 9`	9	`9 + 0 = 9`
`gain(15)`	0	0	15	15	15
`gain(7)`	0	0	7	15	7
`gain(20)`	`max(0,15)=15`	`max(0,7)=7`	**`20+15+7 = 42`**	**42**	`20 + max(15,7) = 35`
`gain(-10)`	`max(0,9)=9`	`max(0,35)=35`	`-10+9+35 = 34`	42	`-10 + 35 = 25`

Answer: `42` — the path `15 → 20 → 7`. ✅

Two things the trace shows: - The winning path (`42`) bends at node 20 and never touches the root. Only the outer `best` variable can capture it; the value returned to the root (35) is a different, smaller quantity. - At the root, `gain` returned 25 — but 25 was never a candidate for `best` beyond the `34` computed there. The distinction between returned value and tracked value is doing real work.

All-negative case, tree = single node `-3` : `left = right = 0` , `best = max(-inf, -3+0+0) = -3` . ✅ The `-inf` seed is what makes this right.

---

98. Validate Binary Search Tree

UNDERSTAND THE PROBLEM

Return whether a binary tree is a valid BST: every node's value is strictly greater than all values in its left subtree and strictly less than all values in its right subtree.

- **Watch:** comparing only against direct children is wrong — a violation can be against a distant ancestor.
- Duplicates are invalid under the standard LeetCode definition (strict inequalities). Clarify.
- Edge cases: single node; the classic counterexample `[5, 1, 4, null, null, 3, 6]`; values at `INT_MIN` / `INT_MAX` (which is why `float('-inf')` beats a hard-coded bound); a valid BST with negative values.

BRUTE-FORCE APPROACH

Idea. For each node, collect **all** values in the left subtree and check they're less than the node, and likewise for the right.

Data structures. A list per subtree.

Algorithm. `all(v < node.val for v in values(left)) and all(v > node.val for v in values(right))`, recursed.

Complexity. Time `O(n²)` — collecting the subtree at each node. Space `O(n)`.

Why it's not ideal. Correct, but quadratic and memory-hungry. The realisation: you don't need the whole set of subtree values — only the *bound* they must respect, and that bound is fully determined by the ancestors. So pass it **down**.

OPTIMAL APPROACH

Idea (DOWN — pass the allowed range). Recurse with `(low, high)`: `node.val` must lie strictly inside. Descending left tightens the upper bound to `node.val`; descending right tightens the lower bound to `node.val`.

This is the canonical **DOWN** pattern — context flows from ancestors to descendants — and it's the counterpart to the UP problems above. Naming the direction out loud is the fastest way to signal you understand the taxonomy.

Alternative (also `O(n)`): an in-order traversal must produce a strictly increasing sequence; track the previous value and compare. Equally good — know both, since the in-order framing generalises to problem 230.

Data structures. Recursion stack; two bound parameters.

Algorithm.

1. `valid(None, low, high) = True`.
2. If not `low < node.val < high`: return `False`.
3. Return `valid(left, low, node.val) and valid(right, node.val, high)`.
4. Start with `(-inf, +inf)`.

Complexity. Time `O(n)`. Space `O(h)`.

PYTHON CODE

```python
class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        # DOWN pattern: each node inherits the open interval it must live inside,
        # accumulated from ALL its ancestors — not just its parent. That's what
        # catches the violation a children-only check misses.
        def valid(node: TreeNode | None, low: float, high: float) -> bool:
            if node is None:
                return True             # an empty subtree satisfies any range

            # Strict inequalities: duplicates are invalid in the standard definition.
            if not (low < node.val < high):
                return False

            # Going LEFT tightens the ceiling; going RIGHT tightens the floor.
            # The other bound is inherited unchanged, which is exactly how an
            # ancestor's constraint reaches a distant descendant.
            return (valid(node.left, low, node.val) and
                    valid(node.right, node.val, high))

        # float inf, not INT_MIN/INT_MAX — node values may sit at the int extremes.
        return valid(root, float('-inf'), float('inf'))


class SolutionInorder:
    """In-order traversal must be strictly increasing. Same O(n)/O(h), and it's
    the framing that generalises to LC 230 (kth smallest)."""
    def isValidBST(self, root: TreeNode | None) -> bool:
        prev = float('-inf')

        def inorder(node: TreeNode | None) -> bool:
            nonlocal prev
            if node is None:
                return True
            if not inorder(node.left):
                return False
            if node.val <= prev:        # <= because equal values are invalid too
                return False
            prev = node.val
            return inorder(node.right)

        return inorder(root)
```

DRY RUN

The counterexample that defeats a children-only check:

```
      5
     / \
    1   4
       / \
      3   6
```

call	node	range `(low, high)`	in range?	notes
1	5	`(-inf, inf)`	✓	
2	1	`(-inf, 5)`	✓	left child, ceiling = 5
3	4	`(5, inf)`	✗ — **4 is not > 5**	**return False**

Answer: `False`. ✅

A children-only check would have passed node 4: its own children (3 and 6) straddle it correctly, and its parent comparison (`4` is the right child of `5`)… is precisely the check that fails. The trace shows the range `(5, inf)` arriving at node 4 from its parent — and it would have arrived just as correctly at a grandchild, which is the general case a parent-only check cannot express.

Valid tree, `[2,1,3]`:

node	range	ok?
2	`(-inf, inf)`	✓
1	`(-inf, 2)`	✓
3	`(2, inf)`	✓

→ `True`. ✅

---

230. Kth Smallest Element in a BST

UNDERSTAND THE PROBLEM

Given a BST and `k`, return the `k`-th smallest value (1-indexed). **Follow-up:** what if the tree is modified often and this query runs frequently?

- The BST property means **in-order traversal yields sorted order** — that's the whole solution.
- Edge cases: `k = 1` (leftmost node); `k = n` (rightmost); a skewed tree.

BRUTE-FORCE APPROACH

Idea. In-order traverse the entire tree into a list, return `arr[k-1]`.

Data structures. A list of all `n` values.

Complexity. Time `O(n)`, space `O(n)`.

Why it's not ideal. It fully materialises the tree to read one element. When `k` is small this is wasteful: the answer is known after `k` nodes. The fix is either an early-exit counter or an explicit iterative in-order traversal that can just stop.

OPTIMAL APPROACH

Idea (iterative in-order with an explicit stack). Push the entire left spine, then repeatedly: pop (this is the next smallest), decrement `k`, and if `k` hits 0 return the value; otherwise descend into the popped node's right child and push *its* left spine.

Iterative rather than recursive because it can **stop mid-traversal** cleanly, and it avoids stack-depth limits.

Data structures. An explicit stack of nodes.

Algorithm.

1. `stack = []`, `cur = root`.
2. While `cur or stack`:
3. While `cur`: `stack.append(cur)`; `cur = cur.left`.
4. `cur = stack.pop()`; `k -= 1`; if `k == 0`: return `cur.val`.
5. `cur = cur.right`.

Complexity. Time `O(h + k)` — descend to the leftmost node, then take `k` steps. Space `O(h)`.

Follow-up (frequent modification + frequent queries). Augment each node with `size` = the number of nodes in its subtree, maintained on insert/delete. Then the query becomes a descent: at each node, compare `k` against `size(left) + 1` and go left, stop, or go right with `k` reduced. That's `O(h)` per query and `O(h)` per update — the right answer, and naming it earns the follow-up.

PYTHON CODE

```python
class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        # In-order on a BST emits values in sorted order, so the kth pop IS
        # the kth smallest. Iterative (not recursive) so we can STOP the instant
        # we have the answer — recursion would keep unwinding.
        stack = []
        cur = root

        while cur or stack:
            # Push the whole left spine: the deepest one is the current smallest
            # unvisited value.
            while cur:
                stack.append(cur)
                cur = cur.left

            cur = stack.pop()          # next value in sorted order
            k -= 1
            if k == 0:
                return cur.val         # early exit — O(h + k), not O(n)

            cur = cur.right            # everything left of cur is now done
```

```
        return -1                       # unreachable when 1 <= k <= n


# ---------- Follow-up: many queries + many modifications ----------
class SizeAugmentedNode:
    """Each node caches its subtree size, maintained on insert/delete.
    Turns the query into an O(h) descent instead of an O(h + k) walk."""
    def __init__(self, val: int):
        self.val = val
        self.left = None
        self.right = None
        self.size = 1                   # nodes in this subtree, including self


def kth_smallest_augmented(root: SizeAugmentedNode | None, k: int) -> int | None:
    """O(h) per query. The update cost is pushed onto insert/delete, which is the
    right trade when reads dominate — say which side you're optimising for."""
    node = root
    while node:
        left_size = node.left.size if node.left else 0

        if k == left_size + 1:
            return node.val             # exactly this node
        elif k <= left_size:
            node = node.left            # answer is inside the left subtree
        else:
            k -= left_size + 1          # skip the left subtree AND this node
            node = node.right

    return None
```

DRY RUN

```
        5
       / \
      3   6
     / \
    2   4
   /
  1
```

`k = 3`

step	action	stack (bottom→top)	cur	k
1	push left spine from 5	`[5, 3, 2, 1]`	`None`	3
2	pop `1`	`[5, 3, 2]`	1	**2**
3	`cur = 1.right = None`; inner while doesn't run; pop `2`	`[5, 3]`	2	**1**
4	`cur = 2.right = None`; pop `3`	`[5]`	3	**0 → return 3**

Answer: `3`. ✅

The traversal stopped after visiting 4 nodes out of 6 — the early exit paying off. With `k = 1` it would stop after the initial spine push plus one pop, touching only the leftmost path.

---

105. Construct Binary Tree from Preorder and Inorder Traversal

UNDERSTAND THE PROBLEM

Given `preorder` and `inorder` traversals of a tree with **unique** values, reconstruct the tree.

- **Preorder** = `[root] + preorder(left) + preorder(right)` — so `preorder[0]` is always the root of the current subtree.
- **Inorder** = `inorder(left) + [root] + inorder(right)` — so locating the root inside `inorder` **splits** the remaining values into the two subtrees and tells you their sizes.
- Uniqueness is required; with duplicates the reconstruction is ambiguous. Say so.
- Edge cases: empty input; single node; a left-only or right-only chain.

BRUTE-FORCE APPROACH

Idea. Same recursive decomposition, but implemented with **list slicing** and a linear `inorder.index(root_val)` search at every level.

Data structures. New sublists at each recursive call.

Algorithm. `root = preorder[0]`; `i = inorder.index(root)`; recurse on `preorder[1:i+1]` / `inorder[:i]` and `preorder[i+1:]` / `inorder[i+1:]`.

Complexity. Time `O(n²)` — the `.index()` scan is `O(n)` per node, and each slice copies `O(n)` elements. Space `O(n²)` in slice allocations on a skewed tree.

Why it's not ideal. Two separate `O(n)`-per-node costs. Both vanish with the same two fixes: (1) precompute a `value → inorder index` dict so the lookup is `O(1)`; (2) pass **index ranges** instead of slices so nothing is copied.

OPTIMAL APPROACH

Idea. Precompute `idx = {value: position}` for `inorder`. Recurse over index bounds, consuming `preorder` left to right with a single moving pointer.

The moving pointer works because preorder visits nodes in exactly the order we construct them: root, then the entire left subtree, then the entire right subtree. So a shared cursor never needs to jump.

Data structures. A dict for `O(1)` root lookup; an integer cursor into `preorder`.

Algorithm.

1. `idx = {v: i for i, v in enumerate(inorder)}`; `self.pre = 0`.
2. `build(lo, hi)` over the inorder range `[lo, hi]`:
3. If `lo > hi`: return `None`.
4. `val = preorder[self.pre]`; `self.pre += 1`; `mid = idx[val]`.
5. `node.left = build(lo, mid-1)`; `node.right = build(mid+1, hi)` — **left first**, matching preorder's order.
6. Return `build(0, n-1)`.

Complexity. Time `O(n)`. Space `O(n)` for the dict plus `O(h)` recursion.

PYTHON CODE

```python
class Solution:
    def buildTree(self, preorder: list[int], inorder: list[int]) -> TreeNode | None:
        # O(1) root lookup instead of an O(n) inorder.index() scan per node.
        # Requires unique values — with duplicates the reconstruction is ambiguous.
        idx = {val: i for i, val in enumerate(inorder)}

        self.pre = 0        # cursor into preorder; only ever moves forward

        def build(lo: int, hi: int) -> TreeNode | None:
            """Builds the subtree occupying inorder[lo..hi]."""
            if lo > hi:
                return None                # empty range -> no node

            # Preorder gives the root of whatever subtree we're currently building,
            # and it does so in exactly the order this recursion needs — hence a
            # single shared cursor rather than slicing.
            val = preorder[self.pre]
            self.pre += 1

            node = TreeNode(val)
            mid = idx[val]                 # where this root sits in inorder

            # Inorder splits at the root: everything left of mid is the left
            # subtree, everything right is the right subtree.
            # LEFT MUST BE BUILT FIRST — preorder lists the entire left subtree
            # before the right one, so the cursor has to consume it in that order.
            node.left = build(lo, mid - 1)
            node.right = build(mid + 1, hi)

            return node

        return build(0, len(inorder) - 1)
```

DRY RUN

`preorder = [3, 9, 20, 15, 7]`, `inorder = [9, 3, 15, 20, 7]`

`idx = {9:0, 3:1, 15:2, 20:3, 7:4}`

call `build(lo,hi)`	`pre`	val	mid	left range	right range	node
`build(0,4)`	0→1	**3**	1	`(0,0)`	`(2,4)`	root 3
`build(0,0)`	1→2	**9**	0	`(0,-1)` → None	`(1,0)` → None	leaf 9
`build(2,4)`	2→3	**20**	3	`(2,2)`	`(4,4)`	node 20
`build(2,2)`	3→4	**15**	2	None	None	leaf 15
`build(4,4)`	4→5	**7**	4	None	None	leaf 7

Reconstructed:

```
    3
   / \
  9   20
     /  \
    15   7
```

✅

Trace the cursor: it consumed `preorder` in order `3, 9, 20, 15, 7` — never skipping or backtracking. That's why one shared integer suffices where slicing would copy.

---

236. Lowest Common Ancestor of a Binary Tree ★★

UNDERSTAND THE PROBLEM

Given a binary tree and two nodes `p` and `q`, return their lowest common ancestor. A node **may be its own ancestor**.

- Not a BST — you cannot use value comparisons to navigate.
- Both `p` and `q` are guaranteed present (LeetCode). If that guarantee is dropped, the algorithm below can return a wrong answer when only one is present — clarify, and if needed add a found-count check.
- Edge cases: `p` is an ancestor of `q`; `p == q`; the LCA is the root.
- Same shape as the Day-1 "Common Manager" problem, restricted to a binary tree.

BRUTE-FORCE APPROACH

Idea. For each node, test whether its subtree contains both `p` and `q`; return the deepest such node.

Data structures. A `contains()` helper.

Complexity. Time `O(n²)`, space `O(h)`.

Why it's not ideal. `contains()` re-walks overlapping subtrees, the same redundancy as 110/543/124. A single post-order pass computes "does my subtree contain p / q" for every node at once.

(A second, respectable approach: find the root-to-`p` and root-to-`q` paths and take the last common node. `O(n)` time, `O(h)` space, and it's the natural solution when nodes have parent pointers — which is a common follow-up. Mention it.)

OPTIMAL APPROACH

Idea (UP, three cases). Define `dfs(node)` returning "a witness": either `p`, `q`, or the LCA, or `None`. - `None` → return `None`. - `node is p or node is q` → return `node`. **Do not recurse further** — if the other target is below, this node is the LCA by the self-ancestor convention, and the parent's one-non-null-child rule propagates it correctly. - Otherwise recurse both ways: - **both non-null** → `p` and `q` are in different subtrees → this node is the split point → return `node`. - **exactly one non-null** → propagate it up. - **both null** → return `None`.

Data structures. Recursion stack.

Complexity. Time `O(n)`. Space `O(h)`.

PYTHON CODE

```python
class Solution:
    def lowestCommonAncestor(self, root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode:
        if root is None:
            return None

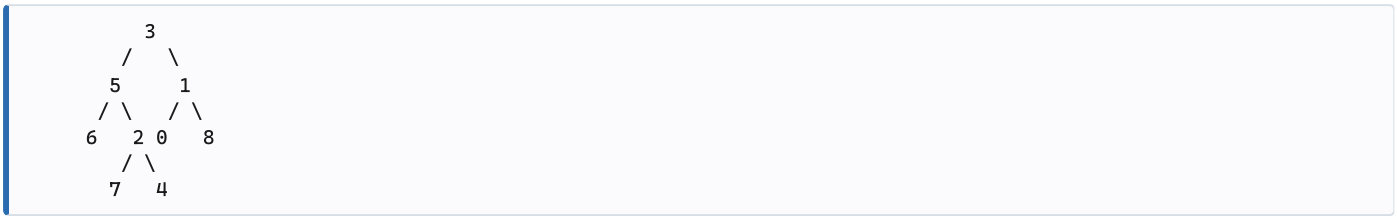
        # Found one of the targets. Return immediately WITHOUT descending further:
        # if the other target lies below us, we are the lowest common ancestor
        # (a node counts as its own ancestor), and the "exactly one non-null child"
        # case below carries us upward unchanged. Descending anyway would still be
        # correct but does needless work.
        if root is p or root is q:
            return root

        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        if left and right:
            # p and q were found in two DIFFERENT subtrees, so the paths from the
            # root diverge exactly here. This node is the LCA.
            return root

        # Exactly one side found something -> bubble it up unchanged.
        # Neither side found anything -> both are None, so this returns None.
        return left or right
```

DRY RUN



Query: `p = 5`, `q = 1` — the LCA is the root.

call	node	left	right	returns
`dfs(5)`	5	—	—	`5` (is `p`, early return)
`dfs(0)`	0	None	None	`None`
`dfs(8)`	8	None	None	`None`
`dfs(1)`	1	—	—	`1` (is `q`, early return)
`dfs(3)`	3	`5`	`1`	both non-null → `3`

Answer: `3`. ✅

Query: `p = 5`, `q = 4` — ancestor case.

call	node	left	right	returns
`dfs(5)`	5	—	—	`5` (early return; node 4 below is never visited)
`dfs(1)` subtree				`None` throughout
`dfs(3)`	3	`5`	`None`	one non-null → `5`

Answer: `5`. ✅ Correct under the self-ancestor convention — and note the early return meant node 4 was never even reached, which is safe precisely *because* of that convention.

---

297. Serialize and Deserialize Binary Tree ★

UNDERSTAND THE PROBLEM

Design `serialize(root) -> str` and `deserialize(str) -> root` that round-trip any binary tree. Any format is allowed.

- The critical insight: **a traversal alone is not enough** — preorder `[1,2,3]` is ambiguous without knowing where the nulls are. You must encode structure, and the standard way is **explicit null markers**.
- Values may be negative and multi-digit, so use a delimiter, not fixed-width characters.
- Edge cases: empty tree; single node; a skewed chain; negative values.
- Clarify: *"Does it have to be human-readable? Compact? Is there a size limit?"* — these are real design levers.

BRUTE-FORCE APPROACH

Idea. Serialize as a **complete-tree array** (heap layout): index `i` has children at `2i+1` and `2i+2`, with nulls filling the gaps.

Data structures. An array of size `2^h`.

Complexity. Time and space `O(2^h)` — **exponential** for a skewed tree. A 1000-node chain would need `2¹⁰⁰⁰` slots.

Why it's not ideal. Fine for near-complete trees, catastrophic otherwise. The fix: encode a **traversal with null markers**, which is `O(n)` regardless of shape — the size becomes proportional to the number of real nodes plus their null children (at most `n + 1` nulls).

OPTIMAL APPROACH

Idea (preorder with null markers). Serialize by preorder DFS, emitting `#` (or `null`) for every missing child. Deserialize by consuming the token stream in the **same** preorder order with a single cursor: read a token; if it's `#`, return `None`; else create the node, then recursively build the left subtree, then the right.

The reason a single cursor works is the same as in problem 105: preorder emits and consumes nodes in exactly matching order.

Alternative: BFS/level-order with null markers — same complexity, and it's what LeetCode's own test format uses. Either is acceptable; preorder is shorter to write.

Data structures. A list of tokens joined by a delimiter; an iterator or index cursor for deserialisation.

Algorithm (serialize). Preorder DFS appending `str(node.val)` or `"#"`; join with `","`. **Algorithm (deserialize).** Split on `","`; recursive `build()` pulling one token per call.

Complexity. Both directions `O(n)` time and `O(n)` space.

PYTHON CODE

```python
class Codec:
    """Preorder DFS with explicit null markers.

    Why markers are mandatory: preorder alone is ambiguous. Both of these trees
    have preorder [1, 2]:
        1          1
       /            \
      2              2
    The '#' tokens are what disambiguate them.
    """

    def serialize(self, root: TreeNode | None) -> str:
        out = []

        def dfs(node: TreeNode | None) -> None:
            if node is None:
                out.append('#')          # explicit null marker
                return
            out.append(str(node.val))    # str(), so negatives and multi-digit work
            dfs(node.left)
            dfs(node.right)

        dfs(root)
        # Comma delimiter, not fixed width — values can be "-1000" or "7".
```

```
        return ','.join(out)

    def deserialize(self, data: str) -> TreeNode | None:
        # An iterator gives us a single forward-only cursor shared across all
        # recursive calls, which is exactly what preorder consumption needs:
        # the tokens arrive in the same order the recursion wants them.
        tokens = iter(data.split(','))

        def build() -> TreeNode | None:
            val = next(tokens)
            if val == '#':
                return None
            node = TreeNode(int(val))
            node.left = build()          # left subtree consumes the next tokens...
            node.right = build()         # ...then the right one takes over
            return node

        return build()


from collections import deque

class CodecBFS:
    """Level-order variant — matches LeetCode's own display format, so it's easier
    to eyeball. Same O(n) time and space."""

    def serialize(self, root: TreeNode | None) -> str:
        if not root:
            return ''
        out, q = [], deque([root])
        while q:
            node = q.popleft()
            if node is None:
                out.append('#')
            else:
                out.append(str(node.val))
                q.append(node.left)      # append even if None — the marker is the point
                q.append(node.right)
        return ','.join(out)

    def deserialize(self, data: str) -> TreeNode | None:
        if not data:
            return None
        vals = data.split(',')
        root = TreeNode(int(vals[0]))
        q = deque([root])
        i = 1
        while q and i < len(vals):
            node = q.popleft()
            if vals[i] != '#':
                node.left = TreeNode(int(vals[i]))
                q.append(node.left)
            i += 1
            if i < len(vals) and vals[i] != '#':
                node.right = TreeNode(int(vals[i]))
                q.append(node.right)
            i += 1
        return root
```

DRY RUN

```
      1
     / \
    2   3
       / \
      4   5
```

Serialize (preorder, emitting nulls):

visit	emits	running output
1	`1`	`1`
2	`2`	`1,2`
2.left = None	`#`	`1,2,#`
2.right = None	`#`	`1,2,#,#`
3	`3`	`1,2,#,#,3`
4	`4`	`1,2,#,#,3,4`
4.left, 4.right	`#,#`	`1,2,#,#,3,4,#,#`
5	`5`	`…,5`
5.left, 5.right	`#,#`	`1,2,#,#,3,4,#,#,5,#,#`

Result: `"1,2,#,#,3,4,#,#,5,#,#"`

Deserialize — tokens consumed in order:

call depth	token	action
build()	`1`	create node 1; recurse left
├ build()	`2`	create node 2; recurse left
│ ├ build()	`#`	return None → `2.left = None`
│ └ build()	`#`	return None → `2.right = None`; node 2 done → `1.left = 2`
└ build()	`3`	create node 3; recurse left
.. ├ build()	`4`	create node 4
.. │ ├ build()	`#`	None
.. │ └ build()	`#`	None; node 4 done → `3.left = 4`
.. └ build()	`5`	create node 5
.... ├ build()	`#`	None
.... └ build()	`#`	None; node 5 done → `3.right = 5`

Reconstructed tree matches the original. ✅

Token count: 5 real nodes + 6 nulls = 11 — linear in `n`, not exponential in height. Compare the heap-array version for the same tree: it would need 7 slots here, but for a 5-node *chain* it would need 31.

---

863. All Nodes Distance K in Binary Tree ★

UNDERSTAND THE PROBLEM

Given a binary tree, a target node, and `k`, return every node exactly `k` edges from the target.

- "Distance" is undirected — you can move to children **and to the parent**. A binary tree node has no parent pointer, which is the entire obstacle.
- Edge cases: `k = 0` → `[target]`; `k` larger than the tree's radius → `[]`; the target is the root.

BRUTE-FORCE APPROACH

Idea. For every node in the tree, compute its distance to the target by finding the path between them (e.g. via the LCA), and keep the nodes at distance exactly `k`.

Data structures. Path lists or repeated searches.

Complexity. Time `O(n²)` — an `O(n)` distance computation per node. Space `O(h)`.

Why it's not ideal. Quadratic, and it computes `n` independent path searches over the same tree. The reframing that fixes it: distance-`k`-in-all-directions from a single source is precisely **BFS on an undirected graph**. The tree is already a graph; it's just missing the upward edges.

OPTIMAL APPROACH

Idea (tree → graph, then multi-directional BFS). Two phases: 1. **Add parent pointers.** One DFS/BFS annotating each node with its parent (or building an adjacency map). This converts the tree into an undirected graph. 2. **BFS from the target**, expanding to `left`, `right`, and `parent`, with a `visited` set to avoid bouncing back. After exactly `k` levels, the queue contains the answer.

The `visited` set is essential — without it, BFS oscillates between a node and its parent forever.

Data structures. `dict[node -> parent]`; a deque; a `visited` set (of node identities).

Algorithm.

1. DFS annotating parents.
2. `q = deque([target])`, `visited = {target}`, `dist = 0`.
3. While `q` and `dist < k`: pop the whole level, push unvisited neighbours (`left`, `right`, `parent`); `dist += 1`.
4. Return the values remaining in `q`.

Complexity. Time `O(n)` — one annotation pass plus one BFS. Space `O(n)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> list[int]:
        # ---- PHASE 1: turn the tree into an undirected graph ----
        # A binary tree node can't reach its parent, but "distance k" is measured
        # in ALL directions. Annotating parents supplies the missing edges.
        parent = {}

        def annotate(node: TreeNode | None, par: TreeNode | None) -> None:
            if not node:
                return
            parent[node] = par
            annotate(node.left, node)
            annotate(node.right, node)

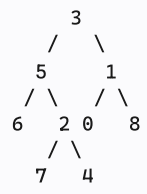
        annotate(root, None)

        # ---- PHASE 2: BFS outward from the target ----
        q = deque([target])
        visited = {target}       # MUST have this: without it BFS ping-pongs
                                 # between a node and its parent indefinitely
        dist = 0

        while q and dist < k:
            for _ in range(len(q)):              # level-snapshot, as in LC 102
                node = q.popleft()
                # Three neighbours now, not two: down-left, down-right, and UP.
                for nxt in (node.left, node.right, parent[node]):
                    if nxt and nxt not in visited:
                        visited.add(nxt)
                        q.append(nxt)
            dist += 1

        # Everything still in the queue sits exactly k edges away.
        # If the BFS exhausted the tree before reaching k, q is empty -> [].
        return [node.val for node in q]
```

DRY RUN



`target = 5`, `k = 2`

Phase 1 — parents: `{3:None, 5:3, 1:3, 6:5, 2:5, 0:1, 8:1, 7:2, 4:2}`

Phase 2 — BFS:

dist	queue at level start	expansions	visited additions
0	`[5]`	5's neighbours: `6` (left), `2` (right), `3` (parent)	6, 2, 3
1	`[6, 2, 3]`	6 → none new (parent 5 visited) 2 → `7`, `4` (parent 5 visited) 3 → `1` (child), `5` visited, parent None	7, 4, 1
2	`[7, 4, 1]`	loop stops: `dist == k`	

Answer: `[7, 4, 1]`. ✅

Verify by hand: `7` is `5 → 2 → 7` (2 edges), `4` is `5 → 2 → 4` (2 edges), `1` is `5 → 3 → 1` (2 edges, going **up** then down — the case a downward-only search misses entirely). That upward path is exactly what phase 1 exists to enable.

437. Path Sum III ★

UNDERSTAND THE PROBLEM

Given a binary tree and a target sum, return how many **downward** paths sum to the target. Paths may start and end at any node, but must run parent → child (no bending upward).

- Values may be **negative** — so you can't prune when the running sum exceeds the target.
- "Downward" means the path is a contiguous segment of some root-to-leaf path. That's what makes prefix sums applicable.
- Edge cases: `targetSum = 0`; negative values; a single node equal to the target; overlapping paths counted separately.

BRUTE-FORCE APPROACH

Idea. For every node, run a separate DFS counting downward paths that **start** at that node.

Data structures. Recursion only.

Algorithm. `count(root) = paths_from(root) + count(root.left) + count(root.right)`, where `paths_from` walks every downward path from that node.

Complexity. Time `O(n²)` (or `O(n·h)`, `O(n log n)` on a balanced tree — be precise: it's `O(n·h)`, which degenerates to `O(n²)` on a chain). Space `O(h)`.

Why it's not ideal. Each node's downward paths are recomputed independently, even though a path starting at a node is a **suffix** of a path starting at the root. That's the exact shape LeetCode 560 solved with prefix sums — and the same trick transfers to a tree, with one addition: you must **undo** the prefix entry when leaving a node (backtracking), because a prefix is only valid along the *current* root-to-node path.

OPTIMAL APPROACH

Idea (prefix sum + hashmap + backtracking). Carry a running sum from the root. Maintain `freq[prefix_value] = count` for prefixes **on the current root-to-node path only**. At each node: - `running += node.val`; - add `freq[running - target]` to the answer — each such earlier prefix marks the start of a qualifying downward path ending here; - record `freq[running] += 1`; - recurse into both children; - **`freq[running] -= 1` on the way back up** — this is the backtracking step, and omitting it lets a path in the left subtree pair with a prefix from the right subtree, which isn't a valid downward path.

Seed `freq[0] = 1` for the empty prefix, exactly as in LC 560, so paths beginning at the root are counted.

Data structures. `defaultdict(int)` for prefix frequencies; recursion stack.

Complexity. Time `O(n)`. Space `O(h)` for the map (bounded by the current path length) plus `O(h)` recursion.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def pathSum(self, root: TreeNode | None, targetSum: int) -> int:
        # freq[p] = how many prefix sums equal to p exist on the CURRENT
        # root-to-node path. Not the whole tree — that distinction is what the
        # backtracking step below enforces.
        freq = defaultdict(int)
        freq[0] = 1          # empty prefix -> lets paths starting at the root count
        total = 0

        def dfs(node: TreeNode | None, running: int) -> None:
            nonlocal total
            if node is None:
                return

            running += node.val

            # Any ancestor prefix equal to (running - target) starts a downward
            # path ending at THIS node with the required sum. Identical logic to
            # LC 560, just walking a tree path instead of an array.
            total += freq[running - targetSum]

            # Register this prefix for the descendants below us.
            freq[running] += 1

            dfs(node.left, running)
            dfs(node.right, running)

            # BACKTRACK — the single most important line here. This prefix is only
            # valid for nodes beneath us. Without the undo, a node in the right
            # subtree could pair with a prefix from the left subtree, counting a
            # "path" that bends upward and isn't a path at all.
            freq[running] -= 1

        dfs(root, 0)
        return total
```

DRY RUN

```
        10
       /  \
      5    -3
     / \     \
    3   2     11
```

`targetSum = 8`

Trace (showing `freq` as non-zero entries; `r` = running):

node	r	r - 8	freq[r-8]	total	freq after add
—	0	—	—	0	`{0:1}`
10	10	2	0	0	`{0:1, 10:1}`
5	15	7	0	0	`{0:1,10:1,15:1}`
3	18	10	**1** ← prefix at node 10	**1**	`{…,18:1}`
← backtrack 3					`{0:1,10:1,15:1}`
2	17	9	0	1	`{…,17:1}`
← backtrack 2					`{0:1,10:1,15:1}`
← backtrack 5					`{0:1,10:1}`
−3	7	−1	0	1	`{0:1,10:1,7:1}`
11	18	10	**1** ← prefix at node 10	**2**	

Wait — one more: at node `5`, `r = 15`, `r - 8 = 7`, and `freq[7]` was 0 at that time. But the path `5 → 3` sums to 8, found at node 3 above ✅. And the path `10 → -3 → 11`? That's `10 - 3 + 11 = 18`, not 8. The hit at node 11 is `r - 8 = 10`, matching the prefix at node 10 — meaning the path *after* node 10, i.e. `-3 → 11 = 8`. ✅

Also the direct hit: is there a path summing to 8 ending at node 2? `r = 17`, `17 - 8 = 9`, no prefix of 9 → no.

Answer: **2** — the paths `5 → 3` and `-3 → 11`. ✅

Why the backtrack matters, concretely. Delete the `freq[running] -= 1` line and re-run: after finishing node 3, `freq[18]` would still be 1 when the traversal reaches node 11 (also `r = 18`). Node 11 would then find `freq[18 - 8] = freq[10] = 1` correctly, but node 11's own registration would collide with a stale left-subtree entry in larger trees, producing counts for "paths" that pass through the root and back down — geometrically impossible. Run a two-branch example on the whiteboard if the interviewer asks why the undo is needed; it's the standard follow-up.

Tree Kyoukai — 🟡 Boundary of Binary Tree (LeetCode 545)

UNDERSTAND THE PROBLEM

Return the tree's boundary **anticlockwise**, starting from the root: the root, then the **left boundary top-down excluding leaves**, then **all leaves left to right**, then the **right boundary bottom-up excluding leaves**. No node appears twice.

Definitions, which are the whole problem — state them before coding: - **Left boundary:** start at the root's left child; at each step prefer `left`, and if there is no left child take `right`. Stop before reaching a leaf. - **Right boundary:** mirror — prefer `right`, fall back to `left`. Stop before a leaf. - **Leaves:** every node with no children, in left-to-right order. - If the root has no left child, the left boundary is **empty** (and symmetrically for the right).

- Edge cases: single node → `[root]`; a left-only chain (the chain is the left boundary, its last node is a leaf); a tree where the same node would qualify as both boundary and leaf — the de-duplication is why boundaries exclude leaves.

BRUTE-FORCE APPROACH

Idea. Compute the three sets separately (left boundary, leaves, right boundary), concatenate, then remove duplicates while preserving order.

Data structures. Three lists plus a `seen` set.

Complexity. Time `O(n)`, space `O(n)`.

Why it's not ideal. The de-duplication *by identity afterwards* is a patch, not a specification. It silently mis-handles the case where a boundary node is also a leaf and you actually want it counted once **in leaf order** — and it makes the code's correctness depend on a cleanup pass rather than on the traversal rules. The disciplined version encodes "boundary walks exclude leaves" directly, so no dedup is ever needed. Interviewers ask "what if the root is a leaf?" precisely to see whether your rules or your patch is doing the work.

OPTIMAL APPROACH

Idea (three targeted traversals, with leaves excluded by construction).

1. Emit the root (unless the root is itself a leaf — then it's the whole answer).
2. Walk the left boundary from `root.left`, appending each node **only if it isn't a leaf**.
3. DFS the whole tree collecting leaves left to right.
4. Walk the right boundary from `root.right`, collecting non-leaves into a list, then **append it reversed** (bottom-up).

Data structures. A result list; one auxiliary list for the right boundary.

Complexity. Time `O(n)` — the boundary walks are `O(h)` and the leaf collection is `O(n)`. Space `O(n)` for the output plus `O(h)` recursion.

PYTHON CODE

```python
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        def is_leaf(node: TreeNode) -> bool:
            return node.left is None and node.right is None

        # A single node is its own entire boundary. Handling it up front keeps
        # the rules below clean — otherwise the root would be emitted once as the
        # root and again as a leaf.
        if is_leaf(root):
            return [root.val]

        result = [root.val]

        # ---- 1. LEFT BOUNDARY, top-down, leaves EXCLUDED ----
        # Prefer the left child; fall back to the right when there is no left.
        # Excluding leaves here (rather than de-duplicating later) is what makes
        # the "no node twice" guarantee structural instead of accidental.
        node = root.left
        while node:
```

```
            if not is_leaf(node):
                result.append(node.val)
            node = node.left if node.left else node.right

        # ---- 2. ALL LEAVES, left to right ----
        def collect_leaves(n: TreeNode | None) -> None:
            if not n:
                return
            if is_leaf(n):
                result.append(n.val)
                return
            collect_leaves(n.left)      # left before right = left-to-right order
            collect_leaves(n.right)

        collect_leaves(root.left)
        collect_leaves(root.right)
        # Called on the two subtrees rather than on root, so the root can never
        # be re-emitted (and we already know it isn't a leaf).

        # ---- 3. RIGHT BOUNDARY, bottom-up, leaves EXCLUDED ----
        right_side = []
        node = root.right
        while node:
            if not is_leaf(node):
                right_side.append(node.val)
            node = node.right if node.right else node.left
        result.extend(reversed(right_side))    # bottom-up => reverse the top-down walk

        return result
```

DRY RUN

```
      1
     / \
    2   3
   / \ / \
  4  5 6  7
     / \
    8   9
```

Root: `1` → `result = [1]`

Left boundary from `root.left = 2`:

node	leaf?	appended	next (prefer left)
2	no	✓ `2`	`4`
4	**yes**	skipped	`4.left` is None, `4.right` is None → loop ends

`result = [1, 2]`

Leaves, DFS on `root.left` then `root.right`:

visit order	leaf?	emitted
2	no	
4	yes	`4`
5	no	
8	yes	`8`
9	yes	`9`
3	no	
6	yes	`6`
7	yes	`7`

`result = [1, 2, 4, 8, 9, 6, 7]`

Right boundary from `root.right = 3`:

node	leaf?	collected	next (prefer right)
3	no	✓ `3`	`7`
7	**yes**	skipped	ends

`right_side = [3]` → reversed → `[3]`.

Answer: `[1, 2, 4, 8, 9, 6, 7, 3]`. ✅

Node `4` illustrates the rule: it sits on the left boundary path *and* is a leaf. Excluding leaves from the boundary walk means it appears exactly once — in the leaf section, at the correct left-to-right position. A dedup-afterwards approach would have to decide which occurrence to drop, and the natural "keep the first" choice would place it in boundary position, which is the wrong output order.

Employee Directory — 🟡 Design (InterviewDB)

UNDERSTAND THE PROBLEM

Model employees (`id` , `name` , `manager` , `directReports`) and support three operations: 1. **Lookup by id** — exact, expected `O(1)` . 2. **Prefix search by name** — all employees whose name starts with a given prefix. 3. **"Everyone beneath employee X"** — the full transitive subtree of reports.

- Clarify: *"Is the org strictly a tree (one manager each) or can it be a DAG (dotted-line reports)? Is prefix search case-insensitive? Do we search first name, last name, or both? What's the read/write ratio? How large is the org — thousands or millions?"* The DAG question in particular changes operation 3 from "subtree" to "reachable set with a visited set".
- Assume for the code below: a strict tree, case-insensitive prefix search over the full name, reads vastly outnumber writes.
- Edge cases: an employee with no reports; the CEO (no manager); a prefix matching nothing; a prefix matching everyone (empty prefix).

BRUTE-FORCE APPROACH

Idea. Store employees in a flat list. Lookup by id = linear scan. Prefix search = scan all names calling `startswith` . Subtree = recursive walk following `directReports` .

Data structures. One list.

Complexity. Lookup `O(n)` · prefix search `O(n · L)` for `n` employees and prefix length `L` · subtree `O(size of subtree)` .

Why it's not ideal. Lookup and prefix search are the two hot read paths and both are linear. Lookup is trivially fixed with a dict (`O(1)`). Prefix search over a large directory typing character by character — an autocomplete box firing on every keystroke — makes a full `O(n)` scan per keystroke untenable; that's exactly the workload a **trie** exists for, giving `O(L + matches)` .

The subtree walk is already optimal (you must touch every result), so leave it alone — say that, rather than optimising something that doesn't need it.

OPTIMAL APPROACH

Idea. Three structures, one per access pattern: 1. `dict[id] -> Employee` for `O(1)` lookup. 2. A **trie over lowercased names**, each terminal node holding the list of employee ids with that name (names aren't unique), for `O(L + k)` prefix search. 3. The existing `directReports` links, walked with **BFS or an explicit stack** rather than recursion — an org chart can be deep, and more importantly an iterative walk lets you add a `visited` set cheaply if the structure turns out to be a DAG.

Complexity.

- `get(id)` : `O(1)` .
- `search_by_prefix(p)` : `O(L + k)` where `k` = number of results (plus the trie nodes traversed while collecting).
- `subordinates(id)` : `O(size of subtree)` — optimal.
- Build: `O(Σ name lengths)` . Space: `O(n + Σ name lengths)` .

PYTHON CODE

```python
from collections import deque
from typing import Iterator, Optional


class Employee:
    __slots__ = ("id", "name", "manager_id", "direct_reports")

    def __init__(self, emp_id: int, name: str, manager_id: Optional[int] = None):
        self.id = emp_id
        self.name = name
        self.manager_id = manager_id
        self.direct_reports: list[int] = []
```

```
class _TrieNode:
    __slots__ = ("children", "ids")

    def __init__(self):
        self.children: dict[str, "_TrieNode"] = {}
        # Employee ids whose full name ENDS at this node. A list, not a single id:
        # two people can share a name.
        self.ids: list[int] = []


class EmployeeDirectory:
    """
    Three access patterns, three structures — each chosen for one of them:
      * by id     -> hash map,  O(1)
      * by prefix -> trie,      O(L + results)   [autocomplete-shaped workload]
      * subtree   -> the report links themselves, walked iteratively

    Assumes a strict tree (one manager per employee). If the org is a DAG
    (dotted-line reporting), `subordinates` needs the `visited` set that is
    already present below — it costs nothing and makes the code DAG-safe.
    """

    def __init__(self):
        self._by_id: dict[int, Employee] = {}
        self._trie_root = _TrieNode()

    # ---------------- writes ----------------
    def add_employee(self, emp_id: int, name: str,
                     manager_id: Optional[int] = None) -> None:
        if emp_id in self._by_id:
            raise ValueError(f"employee {emp_id} already exists")

        emp = Employee(emp_id, name, manager_id)
        self._by_id[emp_id] = emp

        # Maintain the manager's report list so `subordinates` needs no scanning.
        if manager_id is not None:
            manager = self._by_id.get(manager_id)
            if manager is None:
                raise KeyError(f"manager {manager_id} not found")
            manager.direct_reports.append(emp_id)

        self._index_name(name, emp_id)

    def _index_name(self, name: str, emp_id: int) -> None:
        # Lowercased so search is case-insensitive. If the requirement were
        # "match on last name too", index each token separately instead.
        node = self._trie_root
        for ch in name.lower():
            node = node.children.setdefault(ch, _TrieNode())
        node.ids.append(emp_id)

    # ---------------- reads ----------------
    def get(self, emp_id: int) -> Optional[Employee]:
        """O(1)."""
        return self._by_id.get(emp_id)

    def search_by_prefix(self, prefix: str) -> list[Employee]:
        """All employees whose name starts with `prefix`. O(L + results)."""
        node = self._trie_root
        for ch in prefix.lower():
            node = node.children.get(ch)
            if node is None:
                return []                # prefix not present — early exit

        # Collect every id in the subtree rooted at the prefix node.
        out: list[Employee] = []
        stack = [node]
        while stack:
            cur = stack.pop()
            for emp_id in cur.ids:
                out.append(self._by_id[emp_id])
            stack.extend(cur.children.values())
        return out
```

```
    def subordinates(self, emp_id: int) -> list[Employee]:
        """Everyone transitively beneath `emp_id`, excluding the employee.
        O(size of subtree) — already optimal, since every result must be touched."""
        root = self._by_id.get(emp_id)
        if root is None:
            raise KeyError(f"employee {emp_id} not found")

        out: list[Employee] = []
        visited = {emp_id}                # cheap insurance; also makes this DAG-safe
        q = deque(root.direct_reports)

        # Iterative, not recursive: an org chart is usually shallow, but a
        # pathological one would blow Python's recursion limit, and BFS also
        # gives you level-by-level results for free if that's ever wanted.
        while q:
            cur_id = q.popleft()
            if cur_id in visited:
                continue
            visited.add(cur_id)
            emp = self._by_id[cur_id]
            out.append(emp)
            q.extend(emp.direct_reports)

        return out

    def chain_of_command(self, emp_id: int) -> Iterator[Employee]:
        """Bonus, and a very common follow-up: walk upward to the CEO."""
        cur = self._by_id.get(emp_id)
        while cur is not None:
            yield cur
            cur = self._by_id.get(cur.manager_id) if cur.manager_id is not None else None
```

DRY RUN

```
d = EmployeeDirectory()
d.add_employee(1, "Alice")               # CEO
d.add_employee(2, "Bob",     manager_id=1)
d.add_employee(3, "Alina",   manager_id=1)
d.add_employee(4, "Alan",    manager_id=2)
d.add_employee(5, "Charlie", manager_id=2)
```

Structures after the five inserts:

`_by_id = {1:Alice, 2:Bob, 3:Alina, 4:Alan, 5:Charlie}`

Report links: `Alice → [2, 3]`, `Bob → [4, 5]`, others empty.

Trie (paths, with terminal ids):

```
root
 ├─ a ─ l ─ i ─ c ─ e   ids=[1]
 │        │  └─ n ─ a   ids=[3]
 │        └─ a ─ n      ids=[4]
 ├─ b ─ o ─ b           ids=[2]
 └─ c ─ h ─ a ─ r ─ l ─ i ─ e   ids=[5]
```

`search_by_prefix("al")`:

step	char	node reached
1	`a`	node `a`
2	`l`	node `al`

Collect the subtree under `al`: reaches terminals `alice` (id 1), `alina` (id 3), `alan` (id 4).

Result: **Alice, Alina, Alan** ✅ — three matches found without examining Bob or Charlie at all. A flat scan would have tested all five names; on a directory of 100,000 people the difference is the whole point.

`search_by_prefix("z")`: node `z` absent under root → early return `[]`, `O(1)`.

`subordinates(1)`:

step	queue	popped	emitted	enqueued
1	`[2, 3]`	2 (Bob)	Bob	4, 5
2	`[3, 4, 5]`	3 (Alina)	Alina	—
3	`[4, 5]`	4 (Alan)	Alan	—
4	`[5]`	5 (Charlie)	Charlie	—

Result: `[Bob, Alina, Alan, Charlie]` ✅ — the full transitive subtree, in BFS (level) order. Note Alice herself is excluded, matching the stated contract.

`chain_of_command(4)`: Alan → Bob → Alice, stopping when `manager_id is None`. ✅

DAY 5 — Fri Aug 21 · Grids & Backtracking

> **Grid problems reduce to two questions.** (1) *Do I need shortest distance?* → **BFS** (and if there are many sources, multi-source BFS: seed the queue with all of them at once). (2) *Do I just need connectivity / area / flood fill?* → **DFS** is fine and usually shorter. **Backtracking is one template**, and every problem on this day is that template with a different pruning rule: `def backtrack(state): if is_complete(state): record(state); return for choice in choices(state): if not promising(choice): continue # <- THE PRUNE. this is the problem. make(choice); backtrack(state); undo(choice)` The `make` / `undo` pair is what distinguishes backtracking from plain DFS: in 200 you mark a cell visited **forever**, in 79 you un-mark it on the way out because a different path may legitimately reuse it.

---

200. Number of Islands ★★

UNDERSTAND THE PROBLEM

Given a grid of `'1'` (land) and `'0'` (water), count the islands — maximal groups of land connected **4-directionally**.

- Characters, not integers, in the LeetCode signature. Compare against `'1'`.
- 4-directional, not 8. Clarify — diagonal connectivity is a common variant.
- Edge cases: empty grid; all water (0); all land (1); a single cell.
- Clarify: *"May I mutate the input grid?"* If yes, sinking visited land saves the `O(mn)` visited set.

BRUTE-FORCE APPROACH

Idea. For every land cell, walk the entire grid to determine which previously-found island it connects to, merging groups as needed.

Data structures. A list of island member sets.

Complexity. Time `O((mn)²)`, space `O(mn)`.

Why it's not ideal. Quadratic in the cell count. The realisation: an island is a **connected component**, and connected components are found by one traversal per unvisited seed — total work `O(mn)` because each cell is entered once.

OPTIMAL APPROACH

Idea. Scan every cell. On finding an unvisited `'1'`, increment the counter and **flood-fill** the whole island (BFS or DFS), marking cells so they are never counted again. Because every cell is visited at most once across all floods, the total is linear.

Marking — two options: mutate the grid (`'1' → '0'`, "sinking the island"), which is `O(1)` extra space, or a `visited` set, which preserves the input. State which you're doing and why.

Data structures. DFS: a stack (explicit, to avoid recursion depth limits on a 300×300 grid). BFS: a deque.

Algorithm.

1. `count = 0`.
2. For each cell `(r, c)`: if `grid[r][c] == '1'`: `count += 1`; flood-fill from `(r, c)` marking all reachable land.
3. Return `count`.

Complexity. Time `O(m·n)` — each cell pushed and popped at most once. Space `O(min(m, n))` for BFS worst-case queue, `O(mn)` worst case for DFS on a snake-shaped island.

PYTHON CODE

```python
from collections import deque

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        if not grid or not grid[0]:
            return 0

        rows, cols = len(grid), len(grid[0])
        count = 0
        DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))     # 4-directional

        def sink(sr: int, sc: int) -> None:
            """Iterative BFS flood fill. Iterative, not recursive: a 300x300 grid
            that is entirely land would recurse 90,000 deep and blow the stack."""
            q = deque([(sr, sc)])
            grid[sr][sc] = '0'          # mark on ENQUEUE, not on dequeue —
                                        # otherwise a cell can be queued twice
            while q:
                r, c = q.popleft()
                for dr, dc in DIRS:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '1':
                        grid[nr][nc] = '0'
                        q.append((nr, nc))

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1':
                    count += 1          # a fresh, previously-unreached island
                    sink(r, c)          # consume all of it so it's counted once

        return count
```

DRY RUN

```
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1
```

scan position	cell	action	grid after
(0,0)	`'1'`	**count = 1**, flood: (0,0),(1,0),(0,1),(1,1) all sunk	rows 0–1 left block cleared
(0,1)...(1,1)	`'0'` now	skipped — already sunk	
(2,2)	`'1'`	**count = 2**, flood: only (2,2) (neighbours are water)	
(3,3)	`'1'`	**count = 3**, flood: (3,3),(3,4)	

Answer: `3`. ✅

Total cells enqueued across all three floods: 4 + 1 + 2 = 7 = the number of land cells. Every land cell entered exactly once — that's the linearity claim in concrete numbers.

Mark-on-enqueue detail: in the first flood, `(1,1)` is reachable from both `(1,0)` and `(0,1)`. Marking at dequeue time would enqueue it twice; marking at enqueue time makes it impossible.

---

695. Max Area of Island

UNDERSTAND THE PROBLEM

Given a binary grid, return the **area** (cell count) of the largest 4-directionally connected island, or `0` if there is none.

- Integers here (`1` / `0`), not characters — a small difference from 200 that trips people who pattern-match too fast.
- Edge cases: no land → `0` ; one big island; several equal-sized islands.

BRUTE-FORCE APPROACH

Idea. For each land cell, run a fresh flood fill from it (without global marking) to measure the island it belongs to; take the max.

Complexity. Time `O((mn)²)` — every cell re-explores its whole island. Space `O(mn)` .

Why it's not ideal. Each island of size `k` is explored `k` times. Marking globally means each island is explored exactly once, and the flood itself can *return* its size instead of a separate counting pass.

OPTIMAL APPROACH

Idea. Identical scan-and-flood to 200, but the flood **returns a count** rather than just marking. Track the maximum returned value.

Data structures. A stack (iterative DFS) — DFS is natural here because we want a returned aggregate and don't care about distances.

Algorithm.

1. `best = 0` .
2. For each cell: if land, `best = max(best, flood(r, c))` .
3. `flood` sinks each reached cell and counts them.

Complexity. Time `O(m·n)` . Space `O(m·n)` worst case for the stack.

PYTHON CODE

```python
class Solution:
    def maxAreaOfIsland(self, grid: list[list[int]]) -> int:
        if not grid or not grid[0]:
            return 0

        rows, cols = len(grid), len(grid[0])
        DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))
        best = 0

        def flood(sr: int, sc: int) -> int:
            """Sinks the island containing (sr, sc) and RETURNS its cell count.
            Counting during the same traversal that marks is what avoids the
            O((mn)^2) re-exploration of the brute force."""
            stack = [(sr, sc)]
            grid[sr][sc] = 0            # mark on push, so nothing is pushed twice
            area = 0

            while stack:
                r, c = stack.pop()
                area += 1               # count on POP: every pushed cell pops once
                for dr, dc in DIRS:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                        grid[nr][nc] = 0
                        stack.append((nr, nc))

            return area

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    best = max(best, flood(r, c))

        return best
```

DRY RUN

```
0 0 1 0
1 1 0 0
1 0 0 1
```

scan	cell	flood	area	best
(0,2)	1	pushes (0,2); neighbours (1,2)=0, (0,1)=0, (0,3)=0 → nothing more	**1**	1
(1,0)	1	(1,0) → (2,0), (1,1). From (1,1): (0,1)=0, (2,1)=0 → done. Cells: (1,0),(2,0),(1,1)	**3**	**3**
(2,3)	1	isolated	1	3

Answer: `3`. ✅

Each land cell was pushed exactly once across all three floods (1 + 3 + 1 = 5 = total land cells).

130. Surrounded Regions

UNDERSTAND THE PROBLEM

Given a board of `'X'` and `'O'`, flip every `'O'` region that is **not** connected to a border. Modify in place.

- The inverted framing is the whole trick: instead of finding surrounded regions (hard — you'd have to prove no escape route), find the regions that **touch the border** (easy — start the flood *from* the border) and flip everything else.
- Edge cases: no `'O'` at all; every `'O'` touches a border; a 1×N board (every cell is a border cell → nothing flips).

BRUTE-FORCE APPROACH

Idea. For each `'O'` region, flood-fill it and check whether any of its cells sits on the border; flip only if none does.

Data structures. A per-region cell list; a visited set.

Complexity. Time `O(mn)` actually — but with two passes per region (find, then flip) and awkward bookkeeping. Space `O(mn)`.

Why it's not ideal. The complexity is already linear, so this is a **clarity and bug-surface** critique rather than an asymptotic one — say that honestly. You must collect each region's cells, test a global property, then go back and mutate, and the collect-then-decide-then-mutate shape is where off-by-one and double-flip bugs live. The border-first inversion removes the "decide" step entirely: reachability from the border *is* the answer.

OPTIMAL APPROACH

Idea (start from the border). Three phases: 1. From every `'O'` on the border, flood-fill marking cells with a temporary sentinel `'#'` — these are the **safe** regions. 2. Sweep the whole board: every remaining `'O'` was unreachable from the border → flip to `'X'`. 3. Restore every `'#'` back to `'O'`.

Data structures. A stack; a temporary marker character.

Algorithm. As above.

Complexity. Time `O(m·n)` — three linear passes. Space `O(m·n)` worst case for the stack, `O(1)` auxiliary otherwise (marking is in place).

PYTHON CODE

```python
class Solution:
    def solve(self, board: list[list[str]]) -> None:
        """Modifies board in place."""
        if not board or not board[0]:
            return

        rows, cols = len(board), len(board[0])
        DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))

        def mark_safe(sr: int, sc: int) -> None:
            """Flood from a border 'O', tagging everything reachable as safe."""
            stack = [(sr, sc)]
            board[sr][sc] = '#'
            while stack:
                r, c = stack.pop()
                for dr, dc in DIRS:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] == 'O':
                        board[nr][nc] = '#'
                        stack.append((nr, nc))

        # ---- PHASE 1: seed from EVERY border 'O' ----
        # The inversion: "is this region surrounded?" is hard to test directly,
        # but "is it reachable from the edge?" is a plain flood fill. Anything
        # not reached is, by definition, surrounded.
        for r in range(rows):
            for c in (0, cols - 1):                  # left and right columns
                if board[r][c] == 'O':
                    mark_safe(r, c)
        for c in range(cols):
            for r in (0, rows - 1):                  # top and bottom rows
                if board[r][c] == 'O':
                    mark_safe(r, c)

        # ---- PHASES 2 & 3: one sweep does both ----
        for r in range(rows):
            for c in range(cols):
                if board[r][c] == 'O':
                    board[r][c] = 'X'     # unreached => surrounded => flip
                elif board[r][c] == '#':
                    board[r][c] = 'O'     # restore the safe cells
```

DRY RUN

```
X X X X
X O O X
X X O X
X O X X
```

Phase 1 — border seeds. Border cells: row 0, row 3, column 0, column 3. The only border `'O'` is `(3,1)`.

Flood from `(3,1)`: neighbours are `(2,1)='X'`, `(3,0)='X'`, `(3,2)='X'` → the region is just `(3,1)` → marked `'#'`.

Board now:

```
X X X X
X O O X
X X O X
X # X X
```

Phase 2/3 sweep:

cell	value	action
(1,1)	`'O'`	unreached → `'X'`
(1,2)	`'O'`	→ `'X'`
(2,2)	`'O'`	→ `'X'`
(3,1)	`'#'`	restore → `'O'`

Final:

```
X X X X
X X X X
X X X X
X O X X
```

✅

The interior region `{(1,1),(1,2),(2,2)}` was never reached from any border, so it flipped; `(3,1)` sits on the bottom row and survived. Note we never had to *prove* the interior region was enclosed — its absence from the flood is the proof.

---

417. Pacific Atlantic Water Flow ★

UNDERSTAND THE PROBLEM

Given a height grid, the Pacific touches the **top and left** edges, the Atlantic the **bottom and right**. Water flows from a cell to a 4-adjacent neighbour of **equal or lower** height. Return every cell from which water can reach **both** oceans.

- Same inversion as 130, one level cleverer: don't simulate flow *from* each cell — flood **backwards from each ocean**, moving to neighbours of **equal or greater** height (water could have come *down* from there).
- Edge cases: a 1×1 grid (touches both oceans → `[[0,0]]`); a flat grid (every cell reaches both); a strictly increasing grid.

BRUTE-FORCE APPROACH

Idea. From every cell, run a DFS following downhill-or-flat edges and check whether it reaches a Pacific edge and an Atlantic edge.

Data structures. A visited set per starting cell.

Complexity. Time `O((mn)²)` — a full traversal per cell. Space `O(mn)`.

Why it's not ideal. `m·n = 40,000` on a 200×200 grid gives 1.6 billion cell visits. And the searches overlap enormously: if `A` flows to `B` and `B` reaches the Pacific, then `A` does too — but the brute force rediscovers `B`'s whole downstream path for every upstream cell. Reversing the flow direction lets **one** flood per ocean settle every cell at once.

OPTIMAL APPROACH

Idea (reverse the flow, two floods).

1. Flood inward from all Pacific border cells, moving only to neighbours with height ≥ the current cell (reverse of downhill). Collect the reachable set `pacific`.
2. Same from the Atlantic border → `atlantic`.
3. The answer is the **intersection**.

Two `O(mn)` traversals total, instead of `mn` of them.

Data structures. Two `set`s of `(r, c)`; a stack per flood.

Algorithm. As above; return `[list(cell) for cell in pacific & atlantic]`.

Complexity. Time `O(m·n)` — each cell enters each flood at most once. Space `O(m·n)`.

PYTHON CODE

```python
class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))

        def flood(starts: list[tuple[int, int]]) -> set[tuple[int, int]]:
            """Reverse flow: walk UPHILL (or flat) from the ocean edge inward.
            A cell reachable this way is exactly a cell whose water could have
            flowed DOWN to that ocean. One flood settles every cell at once,
            replacing the brute force's mn separate searches."""
            seen = set(starts)
            stack = list(starts)

            while stack:
                r, c = stack.pop()
                for dr, dc in DIRS:
                    nr, nc = r + dr, c + dc
                    if (0 <= nr < rows and 0 <= nc < cols
                            and (nr, nc) not in seen
                            and heights[nr][nc] >= heights[r][c]):   # >= : flat counts
                        seen.add((nr, nc))
                        stack.append((nr, nc))

            return seen

        # Pacific touches the TOP row and the LEFT column.
        pacific_starts = ([(0, c) for c in range(cols)] +
                          [(r, 0) for r in range(rows)])
        # Atlantic touches the BOTTOM row and the RIGHT column.
        atlantic_starts = ([(rows - 1, c) for c in range(cols)] +
                           [(r, cols - 1) for r in range(rows)])

        pacific = flood(pacific_starts)
        atlantic = flood(atlantic_starts)

        # Cells draining to BOTH = set intersection.
        return [[r, c] for r, c in (pacific & atlantic)]
```

DRY RUN

```
1 2 2 3 5
3 2 3 4 4
2 4 5 3 1
6 7 1 4 5
5 1 1 2 4
```

Pacific flood starts at row 0 (`(0,0)…(0,4)`) and column 0 (`(0,0)…(4,0)`), walking to neighbours `≥` current height.

Sample propagation: from `(0,3)=3` → `(1,3)=4 ≥ 3` ✓ added → from `(1,3)=4` → `(1,4)=4 ≥ 4` ✓ (flat counts) added, `(2,3)=3 < 4` ✗. From `(0,4)=5` → `(1,4)=4 < 5` ✗ (but `(1,4)` already came in from `(1,3)`).

Pacific set includes: `(0,0..4)` , `(1,0)` , `(1,1)` , `(1,2)` , `(1,3)` , `(1,4)` , `(2,0)` , `(2,1)` , `(2,2)` , `(3,0)` , `(3,1)` , `(4,0)` .

Atlantic flood starts at row 4 and column 4. From `(4,0)=5` → `(3,0)=6 ≥ 5` ✓. From `(3,0)=6` → `(3,1)=7 ≥ 6` ✓. From `(2,4)=1` → `(1,4)=4 ≥ 1` ✓ → `(1,3)=4 ≥ 4` ✓ → `(0,3)=3 < 4` ✗, `(2,3)=3 < 4` ✗ … and `(1,2)=3 < 4` ✗.

Atlantic set includes: `(0,4)` , `(1,3)` , `(1,4)` , `(2,2)` , `(2,3)` , `(2,4)` , `(3,0)` , `(3,1)` , `(3,2)` , `(3,3)` , `(3,4)` , `(4,0..4)` .

Intersection: `(0,4)` , `(1,3)` , `(1,4)` , `(2,2)` , `(3,0)` , `(3,1)` , `(4,0)` .

Answer: `[[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]` ✅ (order unspecified).

Spot-check `(2,2) = 5` : downhill to the Pacific via `(2,1)=4 → (2,0)=2 → left edge` ✓; downhill to the Atlantic via `(3,2)=1 → (4,2)=1 → bottom edge` ✓. Both, as the intersection says.

994. Rotting Oranges ★★

UNDERSTAND THE PROBLEM

Grid cells are `0` (empty), `1` (fresh orange), `2` (rotten). Each minute, every rotten orange rots its 4-adjacent fresh neighbours. Return the minutes until no fresh oranges remain, or `-1` if some can never rot.

- **Multi-source BFS** — the archetype. All initially-rotten oranges start at time 0 *simultaneously*, so seed the queue with all of them before starting.
- The `-1` case is any fresh orange unreachable from any rotten one.
- Edge cases: no fresh oranges at all → `0` (not `-1`); no rotten oranges but some fresh → `-1`; empty grid.

BRUTE-FORCE APPROACH

Idea. Simulate minute by minute: scan the whole grid, find every rotten cell, rot its neighbours, repeat until no change.

Data structures. A full grid copy per minute (needed so oranges rotted *this* minute don't rot others in the same minute).

Complexity. Time `O(mn · T)` where `T` is the number of minutes — up to `O((mn)²)`. Space `O(mn)`.

Why it's not ideal. Every minute rescans all `mn` cells to find the handful that are actively spreading. BFS keeps exactly the active frontier in a queue, so total work is `O(mn)` regardless of how many minutes elapse. The "must not double-rot within a minute" problem also disappears: BFS's level boundary enforces it structurally.

OPTIMAL APPROACH

Idea (multi-source BFS).

1. Scan once: enqueue **every** rotten cell, and count the fresh ones.
2. If `fresh == 0`: return `0` immediately.
3. BFS level by level (the `for _ in range(len(q))` snapshot again). Each level = one minute. Rot fresh neighbours, decrement `fresh`, enqueue them.
4. After BFS, return `minutes` if `fresh == 0`, else `-1`.

Careful with the minute count: increment `minutes` only when a level actually produces new rot, or equivalently start at `-1` and increment per level — otherwise you over-count by one. The cleanest formulation: increment *before* processing a level only if the queue is non-empty **and** there are still fresh oranges.

Data structures. `deque` seeded with all sources; an integer `fresh` counter.

Complexity. Time `O(m·n)` — each cell enqueued at most once. Space `O(m·n)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def orangesRotting(self, grid: list[list[int]]) -> int:
        if not grid or not grid[0]:
            return 0

        rows, cols = len(grid), len(grid[0])
        DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))

        q = deque()
        fresh = 0

        # ---- Seed ALL sources at once. This is what "multi-source" means:
        # every initially-rotten orange is at time 0, so they must all be in the
        # queue before the first level is processed. Seeding them one at a time
        # would make later sources start late.
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 2:
                    q.append((r, c))
                elif grid[r][c] == 1:
                    fresh += 1

        if fresh == 0:
            return 0            # nothing to rot — 0 minutes, NOT -1

        minutes = 0
        while q and fresh > 0:
            minutes += 1        # one level == one minute
            # Snapshot: process exactly the cells that were rotten at the START
            # of this minute, so oranges rotted NOW don't spread until next minute.
            for _ in range(len(q)):
                r, c = q.popleft()
                for dr, dc in DIRS:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                        grid[nr][nc] = 2
                        fresh -= 1
                        q.append((nr, nc))

        # Any fresh orange left was unreachable from every rotten source.
        return minutes if fresh == 0 else -1
```

DRY RUN

```
2 1 1
1 1 0
0 1 1
```

Seed: rotten `(0,0)`; `fresh = 6`.

minute	queue at level start	rots this level	fresh after
1	`[(0,0)]`	`(0,1)`, `(1,0)`	4
2	`[(0,1),(1,0)]`	`(0,2)` from (0,1); `(1,1)` from (1,0) or (0,1)	2
3	`[(0,2),(1,1)]`	`(2,1)` from (1,1). `(0,2)` has neighbour `(1,2)=0` — empty, skipped	1
4	`[(2,1)]`	`(2,2)`	**0**

Loop exits (`fresh == 0`). Answer: `4`. ✅

Impossible case:

```
2 1 1
0 1 1
1 0 1
```

The `1` at `(2,0)` has neighbours `(1,0)=0` and `(2,1)=0` — both empty. BFS never reaches it, so after the queue drains `fresh == 1` → return `-1`. ✅

No-fresh case: `[[0,2]]` → `fresh = 0` → early return `0`, not `-1`. That early return is the edge case people miss.

---

79. Word Search

UNDERSTAND THE PROBLEM

Given a character grid and a word, return whether the word can be spelled by moving between 4-adjacent cells, **using each cell at most once per path**.

- "At most once **per path**" is the phrase that makes this backtracking rather than plain DFS: a cell used on a failed path must become available again for a different path.
- Edge cases: word longer than the grid's cell count → `False`; single-character word; the word requiring a cell revisit (must fail).

BRUTE-FORCE APPROACH

Idea. Generate every simple path in the grid of length `len(word)` and test each against the word.

Complexity. Time `O(mn · 4^L)` where `L = len(word)`, with no pruning — and generating paths first, then comparing, wastes almost all of them. Space `O(L)`.

Why it's not ideal. It defers the check to the end. Almost every path diverges from the word at the **first or second** character, so testing the match *as you extend* prunes the search tree by orders of magnitude. That's the difference between "generate then filter" and "prune during generation", which is the recurring lesson of the whole backtracking family.

OPTIMAL APPROACH

Idea (DFS + backtracking with immediate pruning). From each starting cell, DFS matching `word[i]` against `grid[r][c]`. Mark the cell as used (overwrite with a sentinel), recurse into the four neighbours with `i+1`, then **restore** the cell on the way out.

The prune is the very first line of the DFS: if `grid[r][c] != word[i]`, return `False` immediately — the subtree is dead.

Data structures. In-place marking with a sentinel character (`'#'`), which is `O(1)` extra space and faster than a `visited` set.

Algorithm.

1. For each cell as a start: if `dfs(r, c, 0)`: return `True`.
2. `dfs(r, c, i)`: bounds check; if `grid[r][c] != word[i]`: `False`; if `i == len(word)-1`: `True`. Mark; recurse 4 ways; unmark; return the OR.

Complexity. Time `O(m·n·4^L)` worst case (in practice far less due to pruning; more precisely `O(mn · 3^L)` since you never immediately go back the way you came). Space `O(L)` recursion.

PYTHON CODE

```python
class Solution:
    def exist(self, board: list[list[str]], word: str) -> bool:
        if not board or not board[0] or not word:
            return False

        rows, cols = len(board), len(board[0])

        def dfs(r: int, c: int, i: int) -> bool:
            # PRUNE FIRST — bail the instant this cell can't be word[i].
            # Checking here rather than after building a full path is the entire
            # difference between the brute force and this.
            if not (0 <= r < rows and 0 <= c < cols) or board[r][c] != word[i]:
                return False

            if i == len(word) - 1:
                return True                 # matched the final character

            # MAKE: mark this cell used. A sentinel that can never appear in the
            # word means neighbours can't step back onto it during THIS path.
            saved = board[r][c]
            board[r][c] = '#'

            found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                     dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))

            # UNDO: restore for OTHER paths. This is what makes it BACKTRACKING
            # rather than a flood fill — in LC 200 we never restore, because a
            # cell belongs to exactly one island; here a cell may legitimately be
            # reused by a different candidate path.
            board[r][c] = saved

            return found

        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0):
                    return True
        return False
```

DRY RUN

```
A B C E
S F C S
A D E E
```

`word = "ABCCED"`

Start at `(0,0)='A'` matches `word[0]='A'`. Mark `(0,0)='#'`.

depth	cell	`word[i]`	match?	action
1	(0,1)= `B`	`B`	✓	mark, descend
2	(0,2)= `C`	`C`	✓	mark, descend
3	try (1,2)= `C`	`C`	✓	mark, descend
4	try (2,2)= `E`	`E`	✓	mark, descend
5	try (2,1)= `D`	`D`	✓, and `i == 5 == len-1`	**return True**

All the way up: `True`. ✅

The undo doing real work. Take `word = "ABCB"` on the same grid. The path `A(0,0) → B(0,1) → C(0,2)` then needs a `B`; neighbours of `(0,2)` are `(0,1)='#'` (marked — correctly blocked), `(1,2)='C'`, `(0,3)='E'` → all fail. The recursion unwinds, **restoring** `(0,2)`, `(0,1)`, `(0,0)`, so the outer loop's next start cell begins with a clean board. Without the restore, every subsequent start would search a board littered with `'#'` and return spurious `False`s.

---

78. Subsets

UNDERSTAND THE PROBLEM

Given an array of **distinct** integers, return every possible subset (the power set).

- `2ⁿ` subsets, so output size is inherently exponential — that's the lower bound, not a flaw.
- Any order accepted, including the order of elements within a subset.
- Edge cases: `[]` → `[[]]` (the empty set is a subset); single element → `[[], [x]]`.

BRUTE-FORCE APPROACH

Idea. Iterate an `n`-bit counter from `0` to `2ⁿ − 1`; for each value, include element `i` when bit `i` is set.

Data structures. None beyond the output.

Complexity. Time `O(n · 2ⁿ)`, space `O(n · 2ⁿ)` output.

Why it's not ideal. Honestly, this one is **not worse** — it's the same complexity and is a perfectly good answer that some interviewers prefer. Offer it. The reason to still know the backtracking version is that bitmasking doesn't generalise: the moment the problem adds duplicates (LC 90), a target sum (LC 39), or a length constraint (LC 77), the mask has no place to prune and backtracking does. Frame it that way rather than calling the mask "wrong".

OPTIMAL APPROACH

Idea (backtracking template, no pruning). At each index, make a binary choice: include `nums[i]` or don't. Record the current partial set at **every** node of the recursion tree, not just the leaves — because every partial set *is* a valid subset.

Data structures. A `path` list built and unwound; the output list.

Algorithm.

1. `backtrack(start, path)`: append `path.copy()` to the result.
2. For `i` in `start..n-1`: `path.append(nums[i])`; `backtrack(i+1, path)`; `path.pop()`.

The `i + 1` (not `start + 1`, and not `i`) is what prevents duplicates and reuse.

Complexity. Time `O(n · 2ⁿ)` — `2ⁿ` subsets, each costing `O(n)` to copy. Space `O(n)` recursion plus the output.

PYTHON CODE

```python
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        res = []
        path = []

        def backtrack(start: int) -> None:
            # Record at EVERY node, not just leaves: every prefix of choices is
            # itself a valid subset. (Contrast LC 46, where only complete
            # permutations count and recording happens at the leaves only.)
            res.append(path[:])          # copy — `path` keeps mutating underneath us

            for i in range(start, len(nums)):
                path.append(nums[i])     # MAKE
                backtrack(i + 1)         # i+1: each element considered once, and
                                         # only elements AFTER i, which kills
                                         # permutation duplicates like [1,2]/[2,1]
                path.pop()               # UNDO

        backtrack(0)
        return res


class SolutionBitmask:
    """Iterative bitmask. Same complexity, no recursion. Perfectly acceptable —
    but it doesn't extend to duplicates/targets/length limits, which is why the
    backtracking template is the one to have automatic."""
    def subsets(self, nums: list[int]) -> list[list[int]]:
        n = len(nums)
        res = []
        for mask in range(1 << n):                    # 0 .. 2^n - 1
            res.append([nums[i] for i in range(n) if mask & (1 << i)])
        return res
```

DRY RUN

`nums = [1, 2, 3]`

Recursion tree (recording at every entry):

call	`path` on entry	recorded	loop over
`bt(0)`	`[]`	`[]`	i=0,1,2
├ `bt(1)` after +1	`[1]`	`[1]`	i=1,2
│ ├ `bt(2)` after +2	`[1,2]`	`[1,2]`	i=2
│ │ └ `bt(3)` after +3	`[1,2,3]`	`[1,2,3]`	(empty range)
│ └ `bt(3)` after +3 (i=2)	`[1,3]`	`[1,3]`	—
├ `bt(2)` after +2 (i=1)	`[2]`	`[2]`	i=2
│ └ `bt(3)` after +3	`[2,3]`	`[2,3]`	—
└ `bt(3)` after +3 (i=2)	`[3]`	`[3]`	—

Answer: `[[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]]` — 8 = 2^3 subsets. ✅

Trace the `path.pop()`: after `bt(3)` returns from the `[1,2,3]` branch, `path` unwinds to `[1,2]`, then `[1]`, letting the `i=2` iteration build `[1,3]` from a clean `[1]`. Skip the pop and `path` would grow monotonically, producing garbage.

---

46. Permutations

UNDERSTAND THE PROBLEM

Given an array of **distinct** integers, return every possible ordering.

- `n!` outputs — again, that's the intrinsic size.
- Unlike subsets, only **complete** arrangements count, so record at the leaves only.
- Edge cases: `[]` → `[[]]`; single element; `n = 3` → 6 permutations.

BRUTE-FORCE APPROACH

Idea. Generate all n^n sequences (each position independently picking any element), then filter out those with repeats.

Complexity. Time $O(n^n \cdot n)$ generated, of which only $n!$ survive. Space $O(n)$.

Why it's not ideal. For $n = 8$, $n^n = 16.7M$ vs $n! = 40{,}320$ — over 99.7% of the work is thrown away. Testing "have I used this already?" *while choosing*, rather than after building, prunes those branches at their root. Same generate-then-filter versus prune-during-generation lesson as 79.

OPTIMAL APPROACH

Idea (backtracking with a `used[]` array). At depth `d`, try every element not yet used. Mark used, recurse, unmark. When `len(path) == n`, record.

The `used` array is the standard mechanism; an alternative is swapping elements into place (`nums[i], nums[start] = nums[start], nums[i]`), which is $O(1)$ space but harder to reason about — mention it, code `used[]`.

Data structures. A boolean `used` list of length `n`; a `path` list.

Algorithm.

1. If `len(path) == n`: record a copy; return.
2. For `i` in `0..n-1`: if `used[i]`: continue. Mark; append; recurse; pop; unmark.

Complexity. Time $O(n \cdot n!)$ — $n!$ permutations, each $O(n)$ to copy. Space $O(n)$.

PYTHON CODE

```python
class Solution:
    def permute(self, nums: list[int]) -> list[list[int]]:
        n = len(nums)
        res = []
        path = []
        used = [False] * n          # used[i]: is nums[i] already in `path`?

        def backtrack() -> None:
            # Record at the LEAVES only — a partial arrangement is not a
            # permutation. (Contrast LC 78, which records at every node.)
            if len(path) == n:
                res.append(path[:])
                return

            for i in range(n):
                if used[i]:
                    continue        # THE PRUNE: skip elements already placed.
                                    # Testing this while choosing — rather than
                                    # filtering finished sequences — is what cuts
                                    # n^n down to n!.

                used[i] = True      # MAKE
                path.append(nums[i])

                backtrack()

                path.pop()          # UNDO — both parts, in reverse order
                used[i] = False

        backtrack()
        return res
```

DRY RUN

`nums = [1, 2, 3]`

depth	`path`	`used`	available	branch taken
0	`[]`	`FFF`	1,2,3	pick 1
1	`[1]`	`TFF`	2,3	pick 2
2	`[1,2]`	`TTF`	3	pick 3
3	`[1,2,3]`	`TTT`	—	**record** `[1,2,3]`
↑ undo to depth 2	`[1,2]`	`TTF`	(loop ends)	
↑ undo to depth 1	`[1]`	`TFF`	3	pick 3
2	`[1,3]`	`TFT`	2	pick 2
3	`[1,3,2]`	`TTT`	—	**record** `[1,3,2]`
↑ undo to depth 0	`[]`	`FFF`	2,3	pick 2
…				`[2,1,3]`, `[2,3,1]`
…				`[3,1,2]`, `[3,2,1]`

Answer: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]` — 6 = 3! ✅

The two-part undo (`path.pop()` **and** `used[i] = False`) must both happen. Dropping the `used[i] = False` means element `i` stays permanently unavailable after its first use, and the output collapses to a single permutation.

---

39. Combination Sum

UNDERSTAND THE PROBLEM

Given **distinct** candidates and a target, return all unique combinations summing to the target. **Each candidate may be reused unlimited times.**

- Combinations, not permutations: `[2,2,3]` and `[2,3,2]` are the same and must appear once.
- Unlimited reuse is the twist — the recursive call passes `i`, not `i+1`.
- All candidates are positive (LeetCode guarantee), which is what makes the `remaining < 0` prune valid.
- Edge cases: no combination reaches the target → `[]`; target 0 → `[[]]` (clarify); a candidate equal to the target.

BRUTE-FORCE APPROACH

Idea. Generate all sequences of candidates up to length `target / min(candidates)`, keep those summing to the target, then deduplicate by sorting each and using a set.

Complexity. Exponential with a large constant, plus an `O(k log k)` sort per candidate sequence for dedup. Space large.

Why it's not ideal. Deduplication *after the fact* again. The structural fix: enforce **non-decreasing index order** during generation — always recurse with `i` as the new lower bound — so `[2,3,2]` is never generated in the first place. And prune the moment the running sum exceeds the target, which the post-filter can't do.

OPTIMAL APPROACH

Idea (backtracking with a start index + reuse).

- Recurse with `i` (not `i + 1`) to allow reusing the current candidate.
- Recurse from `start` upward only, never revisiting earlier indices — this enforces non-decreasing order and kills permutation duplicates.
- Track `remaining = target - sum(path)`; prune when `remaining < 0`; record when `remaining == 0`.
- **Sorting the candidates first** upgrades the prune from `continue` to `break`: once one candidate overshoots, so do all later ones.

Data structures. `path` list; the sorted candidate array.

Complexity. Time `O(n^(T/m))` where `T` = target and `m` = the smallest candidate — exponential, but heavily pruned. Space `O(T/m)` recursion depth.

PYTHON CODE

```python
class Solution:
    def combinationSum(self, candidates: list[int], target: int) -> list[list[int]]:
        candidates.sort()          # enables the `break` prune below
        res = []
        path = []

        def backtrack(start: int, remaining: int) -> None:
            if remaining == 0:
                res.append(path[:])
                return

            for i in range(start, len(candidates)):
                # All candidates are positive AND sorted, so once one overshoots,
                # every later one does too -> break, not continue. On a sorted
                # array this turns a linear scan of dead branches into an O(1) exit.
                if candidates[i] > remaining:
                    break

                path.append(candidates[i])                # MAKE

                # `i`, NOT `i + 1` -> the same candidate may be reused.
                # But never an index BELOW i -> combinations stay non-decreasing,
                # so [2,3,2] is never generated and no dedup pass is needed.
                backtrack(i, remaining - candidates[i])

                path.pop()                                # UNDO

        backtrack(0, target)
        return res
```

DRY RUN

`candidates = [2, 3, 6, 7]` (already sorted), `target = 7`

call `bt(start, rem)`	path	loop	action
`bt(0, 7)`	`[]`	i=0 (2)	take 2 → `bt(0, 5)`
`bt(0, 5)`	`[2]`	i=0 (2)	take 2 → `bt(0, 3)`
`bt(0, 3)`	`[2,2]`	i=0 (2)	take 2 → `bt(0, 1)`
`bt(0, 1)`	`[2,2,2]`	i=0: `2 > 1` → **break**	return
back in `bt(0,3)`	`[2,2]`	i=1 (3)	take 3 → `bt(1, 0)` → **record** `[2,2,3]`
		i=2: `6 > 3` → break	
back in `bt(0,5)`	`[2]`	i=1 (3)	take 3 → `bt(1, 2)`
`bt(1, 2)`	`[2,3]`	i=1: `3 > 2` → break	return (nothing)
		i=2: `6 > 5` → break	
back in `bt(0,7)`	`[]`	i=1 (3)	take 3 → `bt(1, 4)`
`bt(1, 4)`	`[3]`	i=1 (3)	take 3 → `bt(1, 1)` : `3 > 1` break
		i=2: `6 > 4` → break	
back in `bt(0,7)`	`[]`	i=2 (6)	take 6 → `bt(2, 1)` : `6 > 1` break
		i=3 (7)	take 7 → `bt(3, 0)` → **record** `[7]`

Answer: `[[2,2,3], [7]]` . ✅

Two design decisions visible in the trace: - `bt(1, 2)` starting at index 1 means the path `[2,3]` can never go back and add another `2` — which is exactly why `[2,3,2]` never appears alongside `[2,2,3]` . - The `break` at `bt(0,1)` cut off indices 1, 2, 3 in one step. With unsorted candidates you'd need `continue` and would test all three.

22. Generate Parentheses ★

UNDERSTAND THE PROBLEM

Given `n`, return every well-formed combination of `n` pairs of parentheses.

- The count is the `n`-th Catalan number: for `n = 3` there are 5.
- "Well-formed" = at no prefix do closers exceed openers, and totals are equal.
- Edge cases: `n = 0` → `[""]` (clarify); `n = 1` → `["()"]`.

BRUTE-FORCE APPROACH

Idea. Generate all `2^(2n)` strings over `{'(' , ')'}` of length `2n`, then validate each with a counter or a stack.

Complexity. Time `O(2^(2n) · n)`, space `O(n)` per validation.

Why it's not ideal. For `n = 8`, `2¹⁶ = 65,536` candidates for 1,430 valid results — 98% waste. And it scales terribly: `n = 12` gives 16.7M candidates for 208K results. The fix is the theme of the whole day: **enforce validity while building**, using two counters, so no invalid prefix is ever extended.

OPTIMAL APPROACH

Idea (prune during generation with two counters). Track `open_count` and `close_count`. Two rules: - You may add `'('` if `open_count < n`. - You may add `')'` **only if** `close_count < open_count` — this is the well-formedness invariant, enforced at every step.

Record when the string reaches length `2n`. Every string generated is valid by construction; nothing is ever discarded.

Data structures. A list of characters (or a string) as the path; two counters.

Complexity. Time `O(4ⁿ / √n)` — proportional to the Catalan number times the `O(n)` cost of emitting each result. Space `O(n)` recursion.

PYTHON CODE

```python
class Solution:
    def generateParenthesis(self, n: int) -> list[str]:
        res = []
        path = []

        def backtrack(open_count: int, close_count: int) -> None:
            if len(path) == 2 * n:
                res.append(''.join(path))
                return

            # Rule 1: we may always open another pair while we have pairs left.
            if open_count < n:
                path.append('(')
                backtrack(open_count + 1, close_count)
                path.pop()

            # Rule 2: we may close only if there's an unmatched '(' to close.
            # THIS is the well-formedness invariant, enforced as a prune rather
            # than checked after the fact — so every string we build is valid and
            # nothing is ever generated then discarded.
            if close_count < open_count:
                path.append(')')
                backtrack(open_count, close_count + 1)
                path.pop()

        backtrack(0, 0)
        return res
```

DRY RUN

`n = 2`

path	open	close	allowed moves	result
`""`	0	0	`(` only (close 0 ⩤ open 0)	
`"("`	1	0	`(` (1<2) or `)` (0<1)	both branches
`"(("`	2	0	`)` only (open maxed)	
`"(()"`	2	1	`)` (1<2)	
`"(())"`	2	2	length 4 = 2n	**record** `"(())"`
`"()"`	1	1	`(` (1<2); `)` blocked (1 ⩤ 1) ✅	
`"()("`	2	1	`)`	
`"()()"`	2	2	length 4	**record** `"()()"`

Answer: `["(())", "()()"]` — 2 = Catalan(2). ✅

The blocked branch at `"()"` is the prune paying off: without the `close_count < open_count` guard the algorithm would build `"())"`, then `"())("` and `"()))"`, all dead. For `n = 2` that's a handful of wasted nodes; for `n = 10` it's the difference between ~1 million nodes and ~17,000.

---

212. Word Search II ★★

UNDERSTAND THE PROBLEM

Given a character grid and a **list** of words, return every word that can be spelled on the board (4-adjacent moves, no cell reused within a word).

- The naive answer is "run LC 79 once per word" — say that first, then explain why it doesn't scale.
- Edge cases: duplicate words in the list; words longer than the cell count; a word that is a prefix of another; empty word list.

BRUTE-FORCE APPROACH

Idea. For each word, run the LC 79 solution independently.

Complexity. Time `O(W · m · n · 4^L)` for `W` words of max length `L`. Space `O(L)`.

Why it's not ideal. With `W = 3·10⁴` words this is hopeless. The redundancy is specific and fixable: words sharing a prefix (`"oath"`, `"oats"`, `"oat"`) each re-explore the identical board path for that shared prefix. A **trie** lets a single DFS explore the board once while tracking *all* words consistent with the current path simultaneously — the same insight as Aho–Corasick on Day 3, applied to a grid.

OPTIMAL APPROACH

Idea (trie + DFS + aggressive pruning).

1. Build a trie of all words; each terminal node stores the complete word (avoids reconstructing it from the path).
2. DFS from every cell, walking the board and the trie **in lockstep**. If `grid[r][c]` isn't a child of the current trie node, the branch is dead → prune immediately.
3. On reaching a node with a stored word, record it and **clear the stored word** so duplicates aren't emitted.
4. **Prune the trie as you go:** after exploring a node, if it has no children left and no word, delete it from its parent. This shrinks the search space as words are found — the optimisation that makes the difference on large inputs.

Data structures. Trie nodes with a `children` dict and a `word` field; in-place board marking.

Complexity. Time `O(m · n · 4^L)` worst case, but in practice the trie prune makes it dramatically faster than `W` independent searches. Space `O(Σ|words|)` for the trie.

PYTHON CODE

```python
class TrieNode:
    __slots__ = ("children", "word")

    def __init__(self):
        self.children: dict[str, "TrieNode"] = {}
        # Store the COMPLETE word at its terminal node, not just a boolean flag.
        # Saves rebuilding the string from the DFS path on every hit.
        self.word: str | None = None


class Solution:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        if not board or not board[0] or not words:
            return []

        # ---- Build the trie: shared prefixes are explored ONCE, not once per word.
        root = TrieNode()
        for w in words:
            node = root
            for ch in w:
                node = node.children.setdefault(ch, TrieNode())
            node.word = w

        rows, cols = len(board), len(board[0])
        res = []

        def dfs(r: int, c: int, parent: TrieNode) -> None:
            ch = board[r][c]
            node = parent.children.get(ch)
```

```
            if node is None:
                return                  # PRUNE: no word continues with this char

            if node.word is not None:
                res.append(node.word)
                node.word = None        # de-duplicate: never emit the same word twice

            board[r][c] = '#'           # MARK — block reuse within this path
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != '#':
                    dfs(nr, nc, node)
            board[r][c] = ch            # UNDO — a different word may reuse this cell

            # TRIE PRUNING: a node with no remaining children and no word can
            # never contribute again, so unlink it. The search space physically
            # shrinks as words are found — this is the optimisation that separates
            # a passing solution from a timing-out one on large word lists.
            if not node.children:
                parent.children.pop(ch, None)

        for r in range(rows):
            for c in range(cols):
                dfs(r, c, root)

        return res
```

DRY RUN

```
o a a n
e t a e
i h k r
i f l v
```

`words = ["oath", "pea", "eat", "rain"]`

Trie: `o→a→t→h` (word `oath`), `p→e→a` (word `pea`), `e→a→t` (word `eat`), `r→a→i→n` (word `rain`).

Scan:

start cell	trie step	outcome
`(0,0)='o'`	root has `o` ✓	descend
→ `(0,1)='a'`	node `o` has `a` ✓	descend
→ `(1,1)='t'`	node `oa` has `t` ✓	descend
→ `(2,1)='h'`	node `oat` has `h` ✓, `word="oath"`	**record** `"oath"`, clear it
→ deeper	node `oath` has no children → after unwinding, **pruned from parent**	
`(0,1)='a'`	root has no `a`	**prune immediately** — no DFS at all
`(1,0)='e'`	root has `e` ✓	descend
→ `(1,1)='t'`	node `e` has only `a` → **prune**	
→ `(2,0)='i'`	node `e` has no `i` → **prune**	
`(2,3)='r'`	root has `r` ✓	descend
→ `(1,3)='e'`	node `r` has only `a` → prune	
→ `(3,3)='v'`	prune	
→ `(2,2)='k'`	prune	
`'p'`	never appears on the board	`"pea"` never found

Answer: `["oath"]`. ✅

Three prunes worth naming in the trace: - At `(0,1)`, the DFS died on its **first** character because root has no `'a'` child — zero board exploration for that start cell. Running LC 79 four times would have explored from there for every word. - After `"oath"` was found and its trie path had no other words, the `o→a→t→h` chain was unlinked, so any later start at another `'o'` would prune one level earlier. - `"eat"` exists as letters on the board (`e` at (1,0), `a` at (0,1), `t` at (1,1)) but they aren't 4-adjacent in that order — `(1,0)` and `(0,1)` are diagonal. The DFS correctly never finds it.

Gold Thief — 🟡 Path with Maximum Gold (LeetCode 1219)

UNDERSTAND THE PROBLEM

A grid where `0` is empty and positive values are gold. Start at **any** cell containing gold, move 4-directionally, collect the gold on visit, **never revisit a cell**, and **never step on a** `0`. Maximise the total gold collected.

- You may stop at any time — the path doesn't need to be maximal in length, just maximal in value. Since all values are positive, the optimum is always a maximal path, so this doesn't change the code.
- The `0` cells act as walls, which is what keeps the search space small in practice (LeetCode guarantees at most 25 gold cells).
- Edge cases: no gold anywhere → 0; a single gold cell; all cells gold (worst case for the search).

BRUTE-FORCE APPROACH

Idea. Enumerate every simple path in the grid and sum each.

Complexity. Time `O(mn · 4^k)` for `k` gold cells, space `O(k)`. Which is… essentially the same as the "optimal" here.

Why it's not ideal. This is the honest case where **the brute force and the intended solution are the same algorithm**, and you should say so rather than inventing a fake improvement. There is no polynomial algorithm — finding a maximum-weight simple path is NP-hard in general. What separates a good answer from a bad one is: 1. **Backtracking rather than path enumeration** — build incrementally and undo, so memory is `O(k)` rather than storing paths. 2. **Recognising the constraint** (≤ 25 gold cells) that makes exponential acceptable, and saying so out loud. 3. **Not attempting DP** — a naive "best from this cell" memo is *wrong*, because the answer depends on which cells the path has already consumed, and that history isn't in the state. Explaining why memoisation fails here is the strongest signal you can give on this problem.

OPTIMAL APPROACH

Idea (DFS + backtracking from every gold cell). For each gold cell, DFS collecting gold, marking the cell as consumed (set to `0`), recursing into the four neighbours, then restoring. Return the best total found.

Data structures. In-place marking (temporarily zeroing the cell), which doubles as the "don't step on `0`" check — one condition covers both rules.

Algorithm.

1. `best = 0`.
2. For each cell with gold: `best = max(best, dfs(r, c))`.
3. `dfs(r, c)`: if out of bounds or `grid[r][c] == 0`: return 0. Save the value, zero the cell, take the max over the four neighbours' `dfs`, restore, return `value + max_neighbour`.

Complexity. Time `O(mn · 4^k)` for `k` gold cells (in practice much less — the `0` walls prune hard). Space `O(k)` recursion.

PYTHON CODE

```python
class Solution:
    def getMaximumGold(self, grid: list[list[int]]) -> int:
        rows, cols = len(grid), len(grid[0])

        def dfs(r: int, c: int) -> int:
            """Best gold collectable on a path STARTING at (r, c), given the cells
            already consumed (encoded as zeros in `grid`)."""
            # Out of bounds, or an empty cell / already-consumed cell.
            # Zeroing visited cells means one condition enforces BOTH rules:
            # "never step on 0" and "never revisit".
            if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] == 0:
                return 0

            value = grid[r][c]
            grid[r][c] = 0              # MAKE: consume this cell

            best_onward = max(dfs(r + 1, c), dfs(r - 1, c),
                              dfs(r, c + 1), dfs(r, c - 1))

            grid[r][c] = value          # UNDO: restore for other starting cells
                                        # and for other paths through this cell

            return value + best_onward

        # Every gold cell is a candidate start - the problem lets us begin anywhere.
        best = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] != 0:
                    best = max(best, dfs(r, c))

        return best
```

Why memoisation would be wrong (have this ready — it's the follow-up): A cache keyed on `(r, c)` would claim "the best path from `(r,c)` is X" — but that value depends entirely on which cells the current path has already used. A path arriving at `(r,c)` from the north has different remaining options than one arriving from the south. To memoise correctly you'd need `(r, c, set_of_visited_cells)` as the key, which is `k · 2^k` states — no better than the search. This is the same reason the Travelling Salesman Problem has no simple DP over positions alone.

DRY RUN

```
0 6 0
5 8 7
0 9 0
```

Gold cells: `(0,1)=6`, `(1,0)=5`, `(1,1)=8`, `(1,2)=7`, `(2,1)=9`.

Start at `(0,1)=6`:

path	collected	notes
`6`	6	neighbours: `(1,1)=8` only (others are 0 or OOB)
`6 → 8`	14	from `(1,1)`: neighbours `(1,0)=5`, `(1,2)=7`, `(2,1)=9`
`6 → 8 → 9`	23	`(2,1)` neighbours all 0/consumed → stop
`6 → 8 → 7`	21	dead end
`6 → 8 → 5`	19	dead end

Best from `(0,1)`: **23**.

Start at `(2,1)=9`:

path	collected
`9 → 8 → 6`	23
`9 → 8 → 7`	24
`9 → 8 → 5`	22

Best from `(2,1)`: **24**.

Start at `(1,2)=7`: `7 → 8 → 9 = 24`, `7 → 8 → 6 = 21`, `7 → 8 → 5 = 20` → best **24**.

Start at `(1,0)=5`: `5 → 8 → 9 = 22`, `5 → 8 → 7 = 20`, `5 → 8 → 6 = 19` → best **22**.

Start at `(1,1)=8`: `8 → 9 = 17`, `8 → 7 = 15`, etc. → best **17**.

Answer: `24` — the path `9 → 8 → 7` (or `7 → 8 → 9`). ✅

The trace also shows why the undo matters: `(1,1)=8` is consumed and restored five separate times, once per starting cell. Without the restore, only the first start would see the 8 and every subsequent answer would be wrong.

XO Game — 🟡 Design Tic-Tac-Toe (LeetCode 348)

UNDERSTAND THE PROBLEM

Design an `n × n` tic-tac-toe. `move(row, col, player)` returns `0` (no winner yet), `1`, or `2` (that player has won).
Follow-up: `O(1)` per move, with no board rescan.

- Moves are guaranteed valid (no repeats, in bounds) — clarify, because handling invalid moves changes the API.
- A win is `n` in a row, column, main diagonal, or anti-diagonal.
- Once someone wins the game is over; clarify whether further moves are possible.
- Edge cases: `n = 1` (the first move wins); a win on the very last move; the anti-diagonal specifically (the condition people get wrong).

BRUTE-FORCE APPROACH

Idea. Store the full `n × n` board. After each move, scan that row, that column, and both diagonals for `n` identical marks.

Data structures. An `n × n` grid.

Algorithm. `move()` writes the cell, then runs four `O(n)` checks.

Complexity. Time `O(n)` per move, space `O(n²)`.

Why it's not ideal. The follow-up explicitly asks for `O(1)`. And the rescan re-derives information that changes by exactly **one cell** per move — the classic incremental-maintenance opportunity. The board itself is also unnecessary: nothing ever needs to be read back.

OPTIMAL APPROACH

Idea (counters with signed increments). Keep one counter per row, per column, and one each for the two diagonals. Encode player 1 as `+1` and player 2 as `-1`. On a move, add the player's delta to the relevant counters; if any counter reaches `±n`, that player has filled that line.

Why the signed trick beats two sets of counters: a single array does the job, and the win test is one `abs(...) == n` comparison. It also makes it impossible for the two players' marks to be conflated, since they cancel.

Diagonal membership:

- Main diagonal: `row == col`.
- Anti-diagonal: `row + col == n - 1`. (This is the one people write as `row == n - col`, which is off by one.)

Data structures. `rows[n]`, `cols[n]`, two scalars `diag` and `anti`. Total `O(n)` space, no board.

Complexity. `move()` is `O(1)` time. Space `O(n)`.

PYTHON CODE

```python
class TicTacToe:
    """
    O(1) per move, O(n) space, and no board at all — nothing ever needs reading back.

    Trick: player 1 contributes +1 and player 2 contributes -1 to each line's
    counter. A line is won exactly when its counter hits +n or -n, so one abs()
    test covers both players and the two players can never be confused (their
    contributions cancel rather than accumulate).
    """

    def __init__(self, n: int):
        self.n = n
        self.rows = [0] * n
        self.cols = [0] * n
        self.diag = 0        # main diagonal:  row == col
        self.anti = 0        # anti-diagonal:  row + col == n - 1

    def move(self, row: int, col: int, player: int) -> int:
        """Returns 0 (no winner), or 1 / 2 for the winning player. O(1)."""
        delta = 1 if player == 1 else -1

        self.rows[row] += delta
        self.cols[col] += delta

        if row == col:
            self.diag += delta
        # NOT elif — on an odd-sized board the centre cell sits on BOTH diagonals
        # (e.g. n=3, cell (1,1)), so it must update both counters.
        if row + col == self.n - 1:
            self.anti += delta

        n = self.n
        if (abs(self.rows[row]) == n or abs(self.cols[col]) == n
                or abs(self.diag) == n or abs(self.anti) == n):
            return player

        return 0
```

DRY RUN

`n = 3`

move	delta	rows	cols	diag	anti	win check	returns
`move(0,0,1)`	+1	`[1,0,0]`	`[1,0,0]`	+1 (0==0)	0	no	**0**
`move(0,2,2)`	−1	`[0,0,0]`	`[1,0,-1]`	+1	−1 (0+2==2)	no	**0**
`move(2,2,1)`	+1	`[0,0,1]`	`[1,0,0]`	**+2** (2==2)	−1	no	**0**
`move(1,1,2)`	−1	`[0,-1,1]`	`[0,-1,0]`	**+1**	**−2** (1+1==2)	no	**0**
`move(2,0,1)`	+1	`[0,-1,2]`	`[1,-1,0]`	+1	**−1** (2+0==2)	no	**0**
`move(1,0,2)`	−1	`[0,-2,2]`	`[0,-1,0]`	+1	−1	no	**0**
`move(2,1,1)`	+1	**`[0,-2,3]`**	`[0,0,0]`	+1	−1	`abs(rows[2])==3` ✓	**1**

Player 1 wins with row 2 (`(2,0), (2,1), (2,2)`). ✅

Note move 4 (`(1,1)` by player 2): the centre cell updated **both** `diag` and `anti`. Writing `elif` there is the standard bug — it would leave `anti` at `-1` instead of `-2`, and a player 2 win along the anti-diagonal would go undetected.

Also note the counters cancelling: after move 2, `rows[0]` is back to `0` because player 1 and player 2 each hold one cell in row 0 — correctly signalling that neither can win that row without the other's cells, which is exactly the semantics we want.

DAY 6 — Sat Aug 22 · Graphs

> **Pick the tool from the prompt's wording.** *"Is there an ordering / can all be finished / prerequisites"* → **topological sort (Kahn's)**. *"Shortest path, unweighted"* → **BFS**. *"Connected groups / are these two related / merge"* → **Union-Find**. *"Shortest path with a hop limit"* → **Bellman-Ford** (the limit breaks Dijkstra). *"Distance from many sources at once"* → **multi-source BFS**. **Kahn's algorithm is the single highest-value primitive today** — 207, 210, 802, and 310 are all it, wearing different hats.

---

207. Course Schedule ★

UNDERSTAND THE PROBLEM

Given `numCourses` and prerequisite pairs `[a, b]` meaning **b must come before a**, return whether all courses can be finished.

- Restated: does this directed graph have **no cycle**? A cycle means a mutual dependency, which can never be satisfied.
- Edge direction matters — get it wrong and everything downstream is wrong. `[a, b]` means an edge `b → a`.
- Edge cases: no prerequisites → `True`; a self-loop `[a, a]` → `False`; disconnected components; duplicate edges.

BRUTE-FORCE APPROACH

Idea. From every node, run a DFS looking for a path back to itself.

Data structures. Adjacency list; a visited set per start.

Complexity. Time `O(V · (V + E))`, space `O(V + E)`.

Why it's not ideal. Each start re-walks the same subgraphs. One traversal can decide the whole question, because "is this node on a cycle" and "is any node on a cycle" share almost all their work. Two standard single-pass answers exist: **Kahn's** (peel off nodes with no remaining prerequisites; if any remain, they're in a cycle) or **DFS with three colours** (white/grey/black; hitting a grey node means a back-edge, hence a cycle).

OPTIMAL APPROACH

Idea (Kahn's algorithm / BFS topological sort).

1. Build the adjacency list and the **in-degree** of every node.
2. Seed a queue with every node of in-degree 0 — courses with no unmet prerequisites.
3. Pop a node, "take" it, and decrement each neighbour's in-degree; any neighbour dropping to 0 becomes takeable and is enqueued.
4. Count how many nodes were taken. If that equals `numCourses`, no cycle. Otherwise the remaining nodes form a cycle — their in-degrees never reached 0 because they depend on each other.

Data structures. `defaultdict(list)` adjacency; an `indegree` array; a deque.

Complexity. Time `O(V + E)` — every node dequeued once, every edge relaxed once. Space `O(V + E)`.

PYTHON CODE

```python
from collections import defaultdict, deque

class Solution:
    def canFinish(self, numCourses: int, prerequisites: list[list[int]]) -> bool:
        # [a, b] means "b before a", i.e. a DIRECTED EDGE b -> a.
        # Reversing this is the single most common bug on this problem.
        graph = defaultdict(list)
        indegree = [0] * numCourses

        for course, prereq in prerequisites:
            graph[prereq].append(course)
            indegree[course] += 1        # `course` has one more unmet prerequisite

        # Seed with everything immediately takeable.
        q = deque(i for i in range(numCourses) if indegree[i] == 0)

        taken = 0
        while q:
            node = q.popleft()
            taken += 1

            for nxt in graph[node]:
                indegree[nxt] -= 1       # one prerequisite satisfied
                if indegree[nxt] == 0:
                    q.append(nxt)        # now unblocked

        # Any node never taken still has an unmet prerequisite. Since every edge
        # was relaxed, that can only happen if it sits on a cycle.
        return taken == numCourses
```

DRY RUN

`numCourses = 4`, `prerequisites = [[1,0],[2,0],[3,1],[3,2]]`

Edges: `0→1`, `0→2`, `1→3`, `2→3`. In-degrees: `[0, 1, 1, 2]`.

step	queue	popped	taken	in-degree updates
seed	`[0]`		0	
1	`[]`	0	1	`1: 1→0` ✓ enqueue; `2: 1→0` ✓ enqueue
2	`[1, 2]`	1	2	`3: 2→1`
3	`[2]`	2	3	`3: 1→0` ✓ enqueue
4	`[3]`	3	4	none

`taken == 4 == numCourses` → **`True`**. ✅

Cycle case, `prerequisites = [[1,0],[0,1]]`: edges `0→1` and `1→0`, in-degrees `[1, 1]`. The seed queue is **empty**, the loop never runs, `taken = 0 ≠ 2` → **`False`**. ✅ The empty seed is the signature of a graph that is entirely cyclic.

---

210. Course Schedule II

UNDERSTAND THE PROBLEM

Same setup as 207, but return a **valid completion order**, or `[]` if impossible.

- Any valid topological order is accepted; there are usually many.
- Edge cases: identical to 207, plus "no prerequisites" → any permutation is valid (returning `[0, 1, …, n-1]` is fine).

BRUTE-FORCE APPROACH

Idea. Generate permutations of the courses and test each against all prerequisite constraints.

Complexity. Time `O(n! · E)`. Space `O(n)`.

Why it's not ideal. Factorial for `n = 2000`. And it's the wrong mental model: an ordering is not something you *search* for here, it's something you *construct* greedily — at every step, any node with no unmet prerequisites is safe to place next, and Kahn's algorithm just does that repeatedly.

OPTIMAL APPROACH

Idea. Identical to 207, but **record the pop order**. If the recorded order has length `numCourses`, return it; otherwise a cycle blocked completion, so return `[]`.

The only change from 207 is appending to a list instead of incrementing a counter. Say that — recognising a problem as a trivial variant of one you've already solved is worth points.

Data structures. Same as 207, plus an output list.

Complexity. Time `O(V + E)`, space `O(V + E)`.

PYTHON CODE

```python
from collections import defaultdict, deque

class Solution:
    def findOrder(self, numCourses: int, prerequisites: list[list[int]]) -> list[int]:
        graph = defaultdict(list)
        indegree = [0] * numCourses

        for course, prereq in prerequisites:
            graph[prereq].append(course)      # prereq -> course
            indegree[course] += 1

        q = deque(i for i in range(numCourses) if indegree[i] == 0)
        order = []                            # the ONLY change from LC 207

        while q:
            node = q.popleft()
            order.append(node)                # pop order IS a valid topological order

            for nxt in graph[node]:
                indegree[nxt] -= 1
                if indegree[nxt] == 0:
                    q.append(nxt)

        # Short of numCourses => a cycle prevented some node from ever unblocking.
        return order if len(order) == numCourses else []
```

DRY RUN

`numCourses = 4`, `prerequisites = [[1,0],[2,0],[3,1],[3,2]]` (same graph as 207)

step	queue	popped	order
seed	`[0]`		`[]`
1	`[1, 2]`	0	`[0]`
2	`[2]`	1	`[0,1]`
3	`[3]`	2	`[0,1,2]`
4	`[]`	3	`[0,1,2,3]`

Answer: `[0, 1, 2, 3]`. ✅

Verify each constraint: `1` after `0` ✓, `2` after `0` ✓, `3` after `1` ✓, `3` after `2` ✓.

`[0, 2, 1, 3]` is equally valid — it would be produced if the seed order or adjacency order differed. Say "any topological order is acceptable" when you present it, so the interviewer doesn't think you got a different answer than expected.

---

133. Clone Graph

UNDERSTAND THE PROBLEM

Given a reference to a node in a **connected undirected** graph, return a deep copy. **Watch: cycles.**

- Undirected means every edge appears in both nodes' neighbour lists, so a naive recursion will bounce back and forth forever.
- Edge cases: `None` input; a single node with no neighbours; a two-node graph (each is the other's neighbour — the smallest cycle).

BRUTE-FORCE APPROACH

Idea. Recursively copy each node and its neighbours, without tracking what's been copied.

Complexity. Infinite — `A` copies `B`, `B` copies `A`, forever.

Why it's not ideal. It doesn't terminate. The fix is the general one for graph traversal: a visited map. But here it does double duty — it's both the cycle guard *and* the old→new correspondence, so a clone that's already been created gets reused rather than re-made. That "register before recursing" ordering is the entire problem.

OPTIMAL APPROACH

Idea (DFS with an old→new map, registering *before* recursing). 1. If `node` is already in the map, return the existing clone — this terminates cycles. 2. Otherwise create the clone and **put it in the map immediately**, before touching neighbours. 3. Then recurse on each neighbour and append the results to the clone's neighbour list.

Step 2 must precede step 3. If you create the clone, recurse, and only then register it, a neighbour that points back will not find it and will create a second copy — infinite recursion again.

Data structures. `dict[Node, Node]`.

Complexity. Time `O(V + E)` — each node created once, each edge traversed once per direction. Space `O(V)`.

PYTHON CODE

```python
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []


class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None

        old_to_new = {}      # doubles as the visited set AND the correspondence map

        def dfs(cur: 'Node') -> 'Node':
            # Already cloned -> return the SAME clone object. This is what
            # terminates cycles and keeps shared neighbours shared.
            if cur in old_to_new:
                return old_to_new[cur]

            clone = Node(cur.val)
            # REGISTER BEFORE RECURSING. Non-negotiable: in an undirected graph a
            # neighbour will immediately point back at `cur`, and it must find the
            # clone already in the map. Registering after the loop recurses forever.
            old_to_new[cur] = clone

            for nei in cur.neighbors:
                clone.neighbors.append(dfs(nei))

            return clone

        return dfs(node)
```

DRY RUN

Graph: `1 — 2`, `2 — 3`, `3 — 4`, `4 — 1` (a 4-cycle).

call	node	in map?	action
`dfs(1)`	1	no	create `1'`, **register** `{1:1'}`, recurse on neighbours `[2, 4]`
`dfs(2)`	2	no	create `2'`, register `{1:1', 2:2'}`, neighbours `[1, 3]`
`dfs(1)`	1	**yes**	return `1'` immediately — **cycle terminated**
`dfs(3)`	3	no	create `3'`, register, neighbours `[2, 4]`
`dfs(2)`	2	**yes**	return `2'`
`dfs(4)`	4	no	create `4'`, register, neighbours `[3, 1]`
`dfs(3)`	3	**yes**	return `3'`
`dfs(1)`	1	**yes**	return `1'`
← unwind			`4'.neighbors = [3', 1']`; `3'.neighbors = [2', 4']`; `2'.neighbors = [1', 3']`; `1'.neighbors = [2', 4']`

Answer: `1'`, with the full 4-cycle reconstructed. ✅

The `dfs(1)` hit at depth 3 is where the register-first ordering earns its keep: `1'` existed in the map even though `dfs(1)` had not yet finished building its neighbour list. Registering after the loop would have missed it and recursed forever.

---

547. Number of Provinces

UNDERSTAND THE PROBLEM

Given an `n × n` adjacency **matrix** of cities (`isConnected[i][j] == 1` means directly connected), return the number of connected groups.

- The relation is symmetric and reflexive — it's an equivalence relation, and provinces are its equivalence classes.
- Matrix input, not an edge list, so the graph is dense by representation: `O(n²)` is unavoidable just to read it.
- Edge cases: no connections → `n` provinces; all connected → 1; `n = 1`.

BRUTE-FORCE APPROACH

Idea. For each city, DFS/BFS over the matrix to mark its whole component, counting components.

Complexity. Time `O(n²)`, space `O(n)`.

Why it's not ideal. It **isn't** worse — DFS on this input is `O(n²)` and optimal, and it's the right answer if you're short on time. Say that. The reason to reach for **Union-Find** anyway is that it generalises: the moment the problem becomes *streaming* ("connections arrive over time, report the province count after each"), DFS must re-run from scratch on every query while Union-Find answers in near-`O(1)` incrementally. Framing it as "same complexity here, better under the obvious follow-up" is stronger than pretending DFS is wrong.

OPTIMAL APPROACH

Idea (Union-Find with path compression + union by size). Start with `n` disjoint sets. For every `(i, j)` with `i < j` and `isConnected[i][j] == 1`, union them, decrementing a component counter on each *successful* merge. The counter is the answer.

Two optimisations, both needed for the near-constant bound: - **Path compression** in `find` — flatten the tree as you search. - **Union by size (or rank)** — always hang the smaller tree under the larger, keeping depth logarithmic before compression.

Together they give amortised `O(α(n))` per operation, where `α` is the inverse Ackermann function — effectively ≤ 4 for any realistic `n`.

Data structures. `parent[]` and `size[]` arrays.

Complexity. Time `O(n² · α(n))` — dominated by reading the matrix. Space `O(n)`.

PYTHON CODE

```python
class DSU:
    """Union-Find with path compression and union by size.
    Amortised near-O(1) per operation (inverse Ackermann)."""

    def __init__(self, n: int):
        self.parent = list(range(n))     # each element starts as its own root
        self.size = [1] * n
        self.components = n

    def find(self, x: int) -> int:
        # PATH COMPRESSION: point every node on the path straight at the root,
        # so subsequent finds on this branch are O(1).
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]   # halving: skip a level
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> bool:
        """Returns True if a merge actually happened (they were separate)."""
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False                 # already together — no component change

        # UNION BY SIZE: hang the smaller tree under the larger so depth stays low.
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.components -= 1
        return True


class Solution:
    def findCircleNum(self, isConnected: list[list[int]]) -> int:
        n = len(isConnected)
        dsu = DSU(n)

        # Upper triangle only: the matrix is symmetric, so (j, i) adds nothing,
        # and the diagonal is a self-loop.
        for i in range(n):
            for j in range(i + 1, n):
                if isConnected[i][j] == 1:
                    dsu.union(i, j)

        return dsu.components
```

DRY RUN

```
isConnected = [[1,1,0],
               [1,1,0],
               [0,0,1]]
```

Initial: `parent = [0,1,2]`, `size = [1,1,1]`, `components = 3`.

pair	connected?	find(a), find(b)	merge?	parent after	components
(0,1)	1	0, 1 — different	**yes**	`[0,0,2]`, `size[0]=2`	**2**
(0,2)	0	—	skipped		2
(1,2)	0	—	skipped		2

Answer: `2` — provinces `{0,1}` and `{2}`. ✅

All-connected case, `[[1,1,1],[1,1,1],[1,1,1]]`:

pair	merge?	components
(0,1)	yes	2
(0,2)	yes	1
(1,2)	`find(1)=0`, `find(2)=0` — **same root**, no merge	1

Answer: `1`. ✅ The third pair returning `False` is exactly why `components` must decrement inside `union` rather than being counted from the edge list.

---

721. Accounts Merge

UNDERSTAND THE PROBLEM

Given accounts as `[name, email1, email2, …]`, merge accounts that share **any** email. Return each merged account as `[name, sorted emails…]`.

- The same person may appear in several accounts; the name is identical across their accounts but names are **not** unique across people, so you cannot merge on name.
- Emails are the identity keys. Two accounts merge if their email sets intersect — and merging is transitive (A shares with B, B shares with C ⇒ all three merge).
- Edge cases: an account with one email; two people with the same name and no shared email (must stay separate); duplicate emails within one account.

BRUTE-FORCE APPROACH

Idea. Repeatedly compare every pair of account email-sets; if they intersect, merge them; repeat until a full pass makes no merges.

Data structures. A list of sets.

Complexity. Time `O(A² · E)` per pass with up to `A` passes → `O(A³ · E)` worst case. Space `O(A · E)`.

Why it's not ideal. Transitive merging by repeated pairwise comparison is exactly the workload Union-Find was invented for. Union-Find handles transitivity for free — you never have to re-check, because `find` always returns the current representative of the whole merged group.

OPTIMAL APPROACH

Idea (Union-Find over emails).

1. Give every account an index; union all emails **within** an account (union each email with the account's first email).
2. Map each email to an account index so that an email seen in two accounts unions those accounts.
3. Group emails by their root, attach the name (from any account in that group), sort, and emit.

The neat formulation: union **account indices**, keyed by a `email → first account index that used it` map. Then each root collects its emails.

Data structures. `DSU` over account indices; `dict[email → account index]`; `defaultdict(set)` for grouping.

Complexity. Time `O(Σ E · α + Σ E log E)` — dominated by sorting each group's emails. Space `O(Σ E)`.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def accountsMerge(self, accounts: list[list[str]]) -> list[list[str]]:
        n = len(accounts)
        dsu = DSU(n)                          # union over ACCOUNT INDICES

        # email -> the first account index that claimed it.
        # When a later account uses the same email, the two accounts must merge —
        # and Union-Find makes that transitive for free (A~B, B~C => A~C).
        email_owner = {}

        for i, account in enumerate(accounts):
            for email in account[1:]:         # account[0] is the NAME, skip it
                if email in email_owner:
                    dsu.union(i, email_owner[email])
                else:
                    email_owner[email] = i

        # Group every email under its account's ROOT (not its original owner —
        # the root is the merged identity).
        groups = defaultdict(set)
        for email, owner in email_owner.items():
            groups[dsu.find(owner)].add(email)

        # Name comes from any account in the group; they're guaranteed identical
        # for a real person, and accounts only merge when they share an email.
        return [[accounts[root][0]] + sorted(emails)
                for root, emails in groups.items()]
```

DRY RUN

```
accounts = [
  ["John", "johnsmith@mail.com", "john_newyork@mail.com"],   # 0
  ["John", "johnsmith@mail.com", "john00@mail.com"],         # 1
  ["Mary", "mary@mail.com"],                                 # 2
  ["John", "johnnybravo@mail.com"],                          # 3
]
```

Phase 1 — build `email_owner` and union:

account	email	in map?	action
0	`johnsmith`	no	`email_owner[johnsmith] = 0`
0	`john_newyork`	no	`= 0`
1	`johnsmith`	**yes → 0**	`union(1, 0)` — accounts 0 and 1 merge
1	`john00`	no	`= 1`
2	`mary`	no	`= 2`
3	`johnnybravo`	no	`= 3`

DSU after: `{0,1}` merged (root 0), `{2}`, `{3}`.

Phase 2 — group by root:

email	owner	`find(owner)`	group
`johnsmith`	0	0	0
`john_newyork`	0	0	0
`john00`	**1**	**0** ← the merge showing up	0
`mary`	2	2	2
`johnnybravo`	3	3	3

Output:

- Root 0, name `accounts[0][0] = "John"`: `["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"]`

- Root 2: `["Mary", "mary@mail.com"]`
- Root 3: `["John", "johnnybravo@mail.com"]`

✅ Note the two separate "John" entries: accounts 0/1 and account 3 share no email, so they correctly remain distinct people despite the identical name. Merging on name would have been wrong — that's the trap the input is designed to catch.

---

127. Word Ladder ★★

UNDERSTAND THE PROBLEM

Given `beginWord`, `endWord`, and a dictionary, return the number of words in the **shortest** transformation chain where each step changes exactly one letter and every intermediate word is in the dictionary. Return `0` if impossible.

- The answer counts **words**, not steps — a chain `hit → hot → dot → dog → cog` has length 5.
- `beginWord` need not be in the dictionary; `endWord` **must** be, or the answer is 0.
- Shortest path in an unweighted graph → **BFS**, unambiguously.
- Edge cases: `beginWord == endWord`; `endWord` absent from the word list → `0`; no valid chain.

BRUTE-FORCE APPROACH

Idea. Build the graph explicitly by comparing every pair of words for a one-letter difference, then BFS.

Complexity. Building the graph is `O(N² · L)` for `N` words of length `L`. Space `O(N²)`.

Why it's not ideal. With `N = 5000` words, `N² = 25 million` pairwise comparisons just to build the graph — often slower than the search itself. The fix: **generate neighbours on demand** by mutating each of the `L` positions through 26 letters and checking dictionary membership. That's `O(L · 26)` per word instead of `O(N · L)`, and `26·L ≪ N` for realistic inputs.

OPTIMAL APPROACH

Idea (BFS with generated neighbours).

1. Put the word list in a `set` for `O(1)` membership; return `0` immediately if `endWord` isn't in it.
2. BFS from `beginWord`, tracking the level (= chain length).
3. For each word popped, generate all `L × 26` one-letter variants; any variant still in the set is an unvisited neighbour — enqueue it and **remove it from the set** (removal doubles as marking visited, and is faster than a separate set).
4. Return the level when `endWord` is reached; `0` if the queue drains.

Optimisation to name: bidirectional BFS — search from both ends and stop when the frontiers meet. It roughly square-roots the explored node count (`b^(d/2) + b^(d/2)` instead of `b^d`). Mention it; code the plain version unless there's time.

Data structures. A `set` for the dictionary; a deque.

Complexity. Time `O(N · L · 26)` = `O(N · L)`. Space `O(N · L)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: list[str]) -> int:
        words = set(wordList)           # O(1) membership; also our "unvisited" set
        if endWord not in words:
            return 0                    # unreachable by definition — early exit

        q = deque([beginWord])
        words.discard(beginWord)        # don't revisit the start
        length = 1                      # the chain includes beginWord itself

        while q:
            # Level-snapshot BFS: one level == one transformation step.
            for _ in range(len(q)):
                word = q.popleft()

                if word == endWord:
                    return length

                # GENERATE neighbours instead of precomputing the graph.
                # L*26 variants per word beats an O(N^2 * L) pairwise build
                # whenever 26*L << N, which is the realistic case.
                for i in range(len(word)):
                    for code in range(ord('a'), ord('z') + 1):
                        candidate = word[:i] + chr(code) + word[i + 1:]

                        if candidate in words:
                            # Removing from the set marks it visited AND keeps
                            # future membership tests cheap. One structure, two jobs.
                            words.discard(candidate)
                            q.append(candidate)

            length += 1                 # finished a level -> chain grew by one word

        return 0                        # queue drained without reaching endWord
```

DRY RUN

`beginWord = "hit"`, `endWord = "cog"`, `wordList = ["hot","dot","dog","lot","log","cog"]`

`words = {hot, dot, dog, lot, log, cog}`; `cog` present ✓.

level	queue at level start	popped	neighbours found in set	words removed	queue after
1	`[hit]`	`hit`	`hot` (`h_it` → `hot`)	`hot`	`[hot]`
2	`[hot]`	`hot`	`dot`, `lot`	both	`[dot, lot]`
3	`[dot, lot]`	`dot`	`dog`	`dog`	`[lot, dog]`
		`lot`	`log` (`dot` already removed)	`log`	`[dog, log]`
4	`[dog, log]`	`dog`	`cog`	`cog`	`[log, cog]`
		`log`	(`cog` already removed)	—	`[cog]`
5	`[cog]`	`cog`	**== endWord → return 5**		

Answer: `5` — the chain `hit → hot → dot → dog → cog`. ✅

Two details the trace makes concrete: - At level 3, `lot` tried to reach `dot` but `dot` had already been removed — so it wasn't re-enqueued. That's the visited-marking working, and without it the queue would grow exponentially with revisits. - Neighbour generation for `"hit"` produced 78 candidates (3 positions × 26 letters), of which exactly one was in the dictionary. That ratio is why generation beats pairwise graph construction: the wasted candidates cost a hash lookup each, whereas the pairwise build costs a full string comparison against every other word.

---

399. Evaluate Division ★

UNDERSTAND THE PROBLEM

Given equations like `a / b = 2.0` and a list of queries, return each query's value by chaining relations, or `-1.0` when undetermined.

- Model it as a **weighted directed graph**: an edge `a → b` with weight `2.0` means `a / b = 2.0`, and the reverse edge `b → a` carries `1 / 2.0 = 0.5`.
- Chaining multiplies edge weights along the path: `a/c = (a/b) · (b/c)`.
- `x / x = 1.0` **only if** `x` **appears in the equations**; an unknown variable yields `-1.0`. That distinction is the edge case interviewers test.
- Other edges: a query with an unseen variable; disconnected components; a query where one side is known and the other isn't.

BRUTE-FORCE APPROACH

Idea. For each query, try every possible sequence of equations to link the two variables.

Complexity. Exponential in the number of equations.

Why it's not ideal. It's a **path-finding** problem, and path-finding on a graph is linear per query with DFS or BFS — there's no need to enumerate sequences. Building the adjacency structure once and then running one traversal per query is the natural formulation.

OPTIMAL APPROACH

Idea (weighted graph + DFS accumulating a product).

1. Build `graph[a][b] = value` and `graph[b][a] = 1/value` for every equation.
2. For each query `(src, dst)`:
3. If either variable is unknown → `-1.0`.
4. If `src == dst` (and known) → `1.0`.
5. Otherwise DFS from `src` carrying the running product, with a `visited` set; on reaching `dst`, return the product.

Data structures. `defaultdict(dict)` for the weighted adjacency; a visited set per query.

Complexity. Time `O(E)` to build plus `O(V + E)` per query → `O(E + Q·(V+E))`. Space `O(V + E)`.

Follow-up worth naming: with very many queries, a **weighted Union-Find** (storing each node's ratio to its root) answers each query in near-`O(1)` after an `O(E·α)` build. That's the production answer.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def calcEquation(self, equations: list[list[str]], values: list[float],
                     queries: list[list[str]]) -> list[float]:
        # graph[a][b] = a / b. Every equation contributes BOTH directions:
        # a/b = v implies b/a = 1/v. Forgetting the reverse edge makes half the
        # queries unreachable.
        graph = defaultdict(dict)
        for (a, b), v in zip(equations, values):
            graph[a][b] = v
            graph[b][a] = 1.0 / v

        def dfs(src: str, dst: str, product: float, visited: set) -> float:
            if src == dst:
                return product           # reached the target; product is the ratio

            visited.add(src)
            for nxt, weight in graph[src].items():
                if nxt in visited:
                    continue             # cycle guard
                # Chaining MULTIPLIES: a/c = (a/b) * (b/c).
                result = dfs(nxt, dst, product * weight, visited)
                if result != -1.0:
                    return result        # found a path — no need to keep searching
            return -1.0

        out = []
        for src, dst in queries:
            # An unknown variable is undetermined even for x/x. This is the edge
            # case the problem is really testing: "x/x = 1" holds ONLY if x was
            # mentioned in some equation.
            if src not in graph or dst not in graph:
                out.append(-1.0)
            else:
                out.append(dfs(src, dst, 1.0, set()))

        return out
```

DRY RUN

`equations = [["a","b"], ["b","c"]]`, `values = [2.0, 3.0]`

Graph: `a→b: 2.0`, `b→a: 0.5`, `b→c: 3.0`, `c→b: 0.333…`

`queries = [["a","c"], ["b","a"], ["a","e"], ["a","a"], ["x","x"]]`

query	known?	DFS trace	result
`a/c`	both ✓	`a` (p=1.0) → `b` (p=1.0·2.0=2.0) → `c` (p=2.0·3.0=**6.0**), `src==dst`	**6.0**
`b/a`	✓	`b` (p=1.0) → `a` (p=1.0·0.5=**0.5**)	**0.5**
`a/e`	`e` not in graph	early exit	**−1.0**
`a/a`	✓	`src == dst` at depth 0 → return `product = 1.0`	**1.0**
`x/x`	`x` not in graph	early exit — **not** 1.0	**−1.0**

Answer: `[6.0, 0.5, -1.0, 1.0, -1.0]` . ✅

The last two rows are the whole point: `a/a` is `1.0` because `a` is a known quantity, while `x/x` is `-1.0` because `x` was never defined. A solution that special-cases `src == dst` **before** the known-variable check gets `x/x = 1.0` and fails.

Also note the visited set prevented `a → b → a` from looping, and the reverse edges were essential for `b/a` — a forward-only graph would have returned `-1.0` there.

684. Redundant Connection

UNDERSTAND THE PROBLEM

Given a graph that is a tree **plus one extra edge**, return the edge that can be removed to restore a tree. If several qualify, return the one appearing **last** in the input.

- `n` nodes and `n` edges means exactly one cycle exists.
- "Last in the input" is satisfied automatically by processing edges in order and returning the first one that closes a cycle — because a later edge closing the cycle means every earlier edge was still tree-building.
- Nodes are 1-indexed. Size the DSU as `n + 1`.
- Edge cases: the cycle involves the first two edges; a self-loop; the redundant edge is the last one.

BRUTE-FORCE APPROACH

Idea. For each edge (from last to first), remove it and test whether the remaining graph is a connected tree (DFS from any node, check all `n` nodes reachable and edge count is `n-1`).

Complexity. Time `O(n²)` — an `O(n)` connectivity check per edge. Space `O(n)`.

Why it's not ideal. Quadratic, and it repeatedly rebuilds and re-traverses almost the same graph. The direct observation: as you add edges one at a time, the **first** edge whose two endpoints are *already connected* is the one closing the cycle. Union-Find detects that in near-`O(1)` per edge — a single forward pass, no removals, no retries.

OPTIMAL APPROACH

Idea (Union-Find, return the first failed union). Process edges in input order. For each `(u, v)`: - If `find(u) == find(v)`, they're already connected, so this edge creates the cycle → **return it**. - Otherwise `union(u, v)` and continue.

Because there's exactly one extra edge, exactly one union fails, and processing in order guarantees it's the last-listed edge of the cycle.

Data structures. `DSU` over `n + 1` slots (1-indexed nodes).

Complexity. Time `O(n · α(n))` ≈ `O(n)`. Space `O(n)`.

PYTHON CODE

```python
class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        n = len(edges)
        dsu = DSU(n + 1)          # nodes are 1-indexed, so size n+1 and ignore slot 0

        for u, v in edges:
            # union() returns False when u and v already share a root, meaning
            # a path between them exists — so THIS edge closes a cycle.
            # Processing in input order means the first such edge is automatically
            # the last-listed cycle edge, satisfying the tie-break for free.
            if not dsu.union(u, v):
                return [u, v]

        return []                 # unreachable: the problem guarantees one exists
```

DRY RUN

`edges = [[1,2], [1,3], [2,3]]`

DSU over `{0..3}`, initially each its own root.

edge	`find(u)`	`find(v)`	same?	action	parent after
`[1,2]`	1	2	no	union → merge	`1` and `2` share a root
`[1,3]`	root(1)	3	no	union → merge	`1, 2, 3` all one set
`[2,3]`	root(2) = R	root(3) = R	**yes**	**return `[2,3]`**	

Answer: `[2, 3]`. ✅

Verify: removing `[2,3]` leaves `1–2` and `1–3`, a valid tree on 3 nodes with 2 edges. Removing `[1,2]` or `[1,3]` would also leave a tree — all three edges are on the cycle — so the tie-break matters, and returning the **last-listed** one (`[2,3]`) is what the problem asks for. The forward-scan gives that automatically.

---

802. Find Eventual Safe States ★

UNDERSTAND THE PROBLEM

Given a directed graph, return the sorted list of nodes from which **every possible path** reaches a terminal node (a node with no outgoing edges).

- A node is **unsafe** iff it can reach a cycle. Safe iff every path out of it terminates.
- Terminal nodes are trivially safe.
- "Every path", not "some path" — this is what makes it different from ordinary reachability.
- Edge cases: no edges at all → every node is safe; a fully cyclic graph → `[]`; a self-loop.

BRUTE-FORCE APPROACH

Idea. From each node, DFS to check whether any reachable path enters a cycle.

Complexity. Time `O(V · (V + E))` — a full cycle-detection traversal per node. Space `O(V)`.

Why it's not ideal. Each node re-explores the same downstream subgraph. Two clean single-pass fixes: 1. **Reverse graph + Kahn's:** reverse every edge, then topologically peel. In the reversed graph, terminal nodes have in-degree 0, and peeling propagates safety backwards. 2. **DFS with three colours** and memoised safety: white/grey/black, where reaching a grey node means a cycle.

Take (1) — it reuses the Kahn's primitive already drilled today.

OPTIMAL APPROACH

Idea (reverse the graph, then Kahn's).

1. Build the **reverse** graph: for every edge `u → v`, add `v → u`.
2. Compute `outdegree[u]` in the *original* graph — which is the in-degree in the reversed graph.
3. Seed the queue with all nodes of out-degree 0 (the terminal nodes — trivially safe).
4. Pop a safe node `v`; for each `u` in `reverse[v]`, decrement `outdegree[u]`; when it hits 0, **all** of `u`'s successors are safe, so `u` is safe → enqueue.
5. Everything dequeued is safe. Sort and return.

The counter semantics are the elegant part: `outdegree[u]` reaching 0 means "every one of my out-edges leads somewhere already proven safe" — exactly the "every path" requirement.

Data structures. Reverse adjacency list; an out-degree array; a deque; a boolean `safe` array.

Complexity. Time `O(V + E)`. Space `O(V + E)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def eventualSafeNodes(self, graph: list[list[int]]) -> list[int]:
        n = len(graph)

        # Reverse every edge. In the reversed graph, a node's in-degree equals
        # its ORIGINAL out-degree — which is the quantity we want to count down.
        reverse = [[] for _ in range(n)]
        outdegree = [0] * n

        for u in range(n):
            outdegree[u] = len(graph[u])
            for v in graph[u]:
                reverse[v].append(u)

        # Terminal nodes (no outgoing edges) are safe by definition.
        q = deque(u for u in range(n) if outdegree[u] == 0)
        safe = [False] * n

        while q:
            v = q.popleft()
            safe[v] = True

            for u in reverse[v]:
                outdegree[u] -= 1
                # Hitting zero means EVERY successor of u has been proven safe —
                # which is precisely the "all paths terminate" condition.
                # (Contrast ordinary reachability, which would fire on the FIRST
                # safe successor. The counter is what encodes "every".)
                if outdegree[u] == 0:
                    q.append(u)

        return [u for u in range(n) if safe[u]]     # already ascending
```

DRY RUN

`graph = [[1,2],[2,3],[5],[0],[5],[],[]]`

Original out-degrees: `[2, 2, 1, 1, 1, 0, 0]`

Reverse edges: `1→0`, `2→0`, `2→1`, `3→1`, `5→2`, `0→3`, `5→4`

step	queue	popped	safe	out-degree updates
seed	`[5, 6]`			node 5 and 6 are terminal
1	`[6]`	5	`safe[5]`	`reverse[5] = [2, 4]`: `outdeg[2]: 1→0` ✓ enqueue; `outdeg[4]: 1→0` ✓ enqueue
2	`[2, 4]`	6	`safe[6]`	`reverse[6] = []`
3	`[4]`	2	`safe[2]`	`reverse[2] = [0, 1]`: `outdeg[0]: 2→1`; `outdeg[1]: 2→1`
4	`[]`	4	`safe[4]`	`reverse[4] = []`

Queue empty. `safe = [F, F, T, F, T, T, T]`

Answer: `[2, 4, 5, 6]`. ✅

Nodes 0, 1, 3 form the cycle `0 → 1 → 3 → 0`, so their out-degrees never reached 0 — node 0 still had one unsatisfied edge (to node 1), and node 1 one (to node 3), and node 3 one (to node 0). The counters deadlock, exactly as they should.

Contrast with reachability: node 0 *can* reach the safe node 2 (via `0 → 2`), so a "some path terminates" test would wrongly mark it safe. The out-degree counter refuses because `0 → 1` still leads into the cycle.

---

310. Minimum Height Trees ★

UNDERSTAND THE PROBLEM

Given a tree as `n` nodes and `n - 1` edges, return every node that, chosen as root, minimises the tree's height. **There are at most two such nodes.**

- The answers are the **centroids** of the tree — the middle of its longest path (diameter). A path of even length has one centre; odd has two. That's why the answer is never larger than 2.
- Edge cases: `n = 1` → `[0]`; `n = 2` → `[0, 1]`; a star graph → the centre only.

BRUTE-FORCE APPROACH

Idea. For each node, BFS to compute the tree's height with that node as root; keep the minima.

Complexity. Time `O(n²)` — an `O(n)` BFS per node. Space `O(n)`.

Why it's not ideal. `n = 2·10⁴` gives $4 \cdot 10^8$ operations. And each BFS recomputes nearly the same distances. The structural insight: the answer is a property of the tree's *shape* (its diameter's midpoint), so it can be found by a single peeling process rather than `n` independent searches.

OPTIMAL APPROACH

Idea (peel leaves inward, layer by layer). Repeatedly remove all current leaves (degree-1 nodes) simultaneously. Each round strips one layer from the outside. Whatever remains when `n ≤ 2` nodes are left is the centre — one or two nodes.

Why it's correct: the tree's height from a node is dominated by the distance to the farthest leaf. Removing every leaf shortens every such distance by exactly 1, uniformly, so the set of optimal roots is unchanged. Iterating shrinks the tree until only the centre survives.

Data structures. Adjacency list (as sets or lists); a degree array; a deque of current leaves.

Algorithm.

1. If `n <= 2`: return `list(range(n))`.
2. Build adjacency and degrees; `leaves = [nodes with degree 1]`.
3. `remaining = n`. While `remaining > 2`: `remaining -= len(leaves)`; build the next leaf layer by decrementing neighbours' degrees and collecting those that drop to 1.
4. Return the final `leaves` list.

Complexity. Time `O(n)` — each node and edge processed once. Space `O(n)`.

PYTHON CODE

```python
from collections import defaultdict

class Solution:
    def findMinHeightTrees(self, n: int, edges: list[list[int]]) -> list[int]:
        # n == 1 has no edges; n == 2 makes both nodes centres. Handling these up
        # front keeps the peeling loop simple.
        if n <= 2:
            return list(range(n))

        graph = defaultdict(set)
        for u, v in edges:
            graph[u].add(v)
            graph[v].add(u)

        # Current outermost layer.
        leaves = [node for node in range(n) if len(graph[node]) == 1]
        remaining = n

        # Peel until 1 or 2 nodes remain — those are the tree's centroid(s),
        # i.e. the midpoint of its diameter. Stripping a whole layer reduces
        # EVERY node's distance-to-farthest-leaf by exactly 1, so the set of
        # optimal roots is preserved at every step.
        while remaining > 2:
            remaining -= len(leaves)
            next_leaves = []

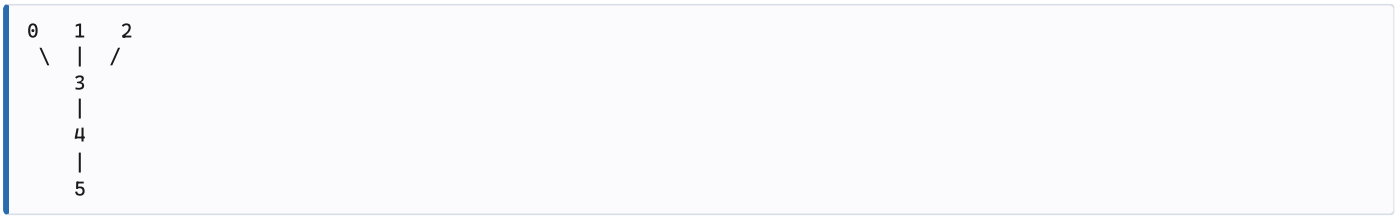
            for leaf in leaves:
                # A leaf has exactly one neighbour; detach in both directions.
                neighbor = graph[leaf].pop()
                graph[neighbor].discard(leaf)
                if len(graph[neighbor]) == 1:
                    next_leaves.append(neighbor)   # became a leaf for the NEXT round

            leaves = next_leaves

        return leaves       # 1 or 2 nodes — never more, by the centroid theorem
```

DRY RUN

`n = 6`, `edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]`



Degrees: `0:1, 1:1, 2:1, 3:4, 4:2, 5:1`

round	`remaining`	leaves	detach	resulting degrees	next leaves
start	6	`[0, 1, 2, 5]`			
1	`6 - 4 = 2`	`[0,1,2,5]`	0,1,2 detach from 3 → `deg(3) = 1`; 5 detaches from 4 → `deg(4) = 1`	`3:1, 4:1`	`[3, 4]` — note 3 is added once (when its degree first hits 1, after node 2) and 4 once

`remaining = 2`, loop exits.

Answer: `[3, 4]`. ✅

Check by hand: rooting at 3 gives height 2 (`3 → 4 → 5`). Rooting at 4 gives height 2 (`4 → 3 → 0`). Rooting at 0 gives height 3 (`0 → 3 → 4 → 5`). So 3 and 4 tie for the minimum — exactly the two centres of the diameter `0–3–4–5` (length 3, so two midpoints).

One subtlety in the code: `next_leaves` could receive a duplicate if two leaves shared the same neighbour and both pushed it. The `if len(graph[neighbor]) == 1` test fires only on the transition to degree 1, so each node enters `next_leaves` at most once — worth pointing out, because a naive version that appends whenever `len <= 1` does duplicate.

752. Open the Lock

UNDERSTAND THE PROBLEM

A 4-wheel lock, each wheel `0–9` and wrapping. Starting at `"0000"`, given a list of deadends and a target, return the minimum number of turns, or `-1`.

- Each move turns exactly one wheel one click, up or down → 8 neighbours per state.
- State space is exactly 10^4 = 10,000 combinations — small enough that plain BFS is comfortable.
- Deadends are states you may never occupy; if `"0000"` is a deadend the answer is `-1` immediately.
- Edge cases: target is `"0000"` → 0; `"0000"` in deadends → `-1`; target unreachable.

BRUTE-FORCE APPROACH

Idea. DFS from `"0000"` exploring turn sequences, tracking the shortest reaching the target.

Complexity. Exponential without memoisation; with a visited set it degenerates to a DFS over 10,000 states — but DFS gives **no shortest-path guarantee**, so you'd have to explore everything and take the minimum.

Why it's not ideal. DFS finds *a* path, not the shortest, so it can't stop early and must exhaust the space. BFS explores in order of distance, so the first time it touches the target that distance is provably minimal — it can return immediately. Whenever a prompt says "minimum number of moves" over an unweighted state space, BFS is the answer.

OPTIMAL APPROACH

Idea (BFS over the state space). Treat each 4-digit string as a graph node with 8 neighbours (each of 4 wheels, ±1 with wraparound). Standard level-by-level BFS from `"0000"`, skipping deadends and visited states.

Data structures. A `set` of deadends; a `set` of visited; a deque.

Algorithm.

1. If `"0000"` in deadends: `-1`. If target is `"0000"`: `0`.
2. BFS with level counting; generate 8 neighbours per state; skip if in deadends or visited.
3. Return the level on reaching the target; `-1` if the queue drains.

Optimisation to name: bidirectional BFS from `"0000"` and the target simultaneously — roughly halves the explored depth. Mention it as the follow-up.

Complexity. Time `O(10⁴ · 8 · 4)` = `O(N · b · L)` — bounded and constant-ish. Space `O(10⁴)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def openLock(self, deadends: list[str], target: str) -> int:
        dead = set(deadends)
        start = "0000"

        if start in dead:
            return -1               # can't even begin
        if target == start:
            return 0                # already there — check BEFORE the BFS loop

        visited = {start}
        q = deque([start])
        turns = 0

        while q:
            turns += 1              # one BFS level == one wheel turn
            for _ in range(len(q)):
                state = q.popleft()

                # 8 neighbours: 4 wheels x 2 directions, with wraparound.
                for i in range(4):
                    digit = int(state[i])
                    for delta in (1, -1):
                        # % 10 handles both wraps: 9+1 -> 0 and 0-1 -> 9.
                        new_digit = (digit + delta) % 10
                        nxt = state[:i] + str(new_digit) + state[i + 1:]

                        if nxt in dead or nxt in visited:
                            continue

                        if nxt == target:
                            return turns     # BFS => first arrival IS the shortest

                        visited.add(nxt)     # mark on ENQUEUE, not on dequeue
                        q.append(nxt)

        return -1                   # exhausted the reachable space
```

DRY RUN

`deadends = ["0201","0101","0102","1212","2002"]`, `target = "0202"`

`"0000"` not dead ✓, target ≠ start ✓.

turns	states at level start	sample neighbours generated	notable
1	`["0000"]`	`1000, 9000, 0100, 0900, 0010, 0090, 0001, 0009`	all valid, all enqueued
2	those 8	e.g. from `1000`: `2000, 0000(seen), 1100, 1900, …`	`0100`'s neighbour `0200` enqueued; `0101` is **dead** → skipped
3		from `0200`: **`0300, 0100(seen), 0210, 0290, 0201(dead), 0209`**	`0201` blocked — the direct route to the target is cut
…		the search must detour around the deadend wall	
6		`"0202"` first reached	**return 6**

Answer: `6`. ✅ One shortest path is `0000 → 1000 → 1100 → 1200 → 1201 → 1202 → 0202`.

The deadends `0201`, `0101`, `0102` form a wall around the target's cheap approaches, forcing the 6-turn detour instead of the 4-turn direct route (`0000 → 0100 → 0200 → 0201 → 0202`, which is blocked at `0201`). The BFS discovers that without any special handling — it simply never enqueues blocked states.

787. Cheapest Flights Within K Stops ★

UNDERSTAND THE PROBLEM

Given flights with prices, a source, a destination, and `k`, return the cheapest price using **at most `k` stops**, or `-1`.

- "At most `k` stops" means **at most `k + 1` edges** — the off-by-one that sinks people. Say it explicitly.
- **Watch: the stop cap breaks Dijkstra.** Dijkstra finalises a node the first time it's popped, assuming that's the cheapest way to reach it — but here a *more expensive* route with *fewer stops* may be the only one that can be extended within budget. The greedy invariant fails.
- Edge cases: `src == dst` → 0; no route → `-1`; `k = 0` (direct flights only).

BRUTE-FORCE APPROACH

Idea. DFS/BFS enumerating every path from `src` with at most `k+1` edges, tracking the minimum cost.

Complexity. Time `O(n^(k+1))` — exponential in the hop limit. Space `O(k)`.

Why it's not ideal. With `n = 100` and `k = 100`, hopeless. The fix isn't Dijkstra (it's invalid, as above) but **Bellman-Ford relaxed exactly `k+1` times** — after round `i`, `dist[v]` holds the cheapest cost to `v` using **at most `i` edges**. The hop limit maps directly onto the round count, which is why Bellman-Ford is the natural fit.

OPTIMAL APPROACH

Idea (Bellman-Ford, `k+1` rounds, relaxing from a snapshot).

1. `dist = [inf] * n`; `dist[src] = 0`.
2. Repeat `k + 1` times:
3. **Copy** `dist` into `tmp`.
4. For every flight `(u, v, w)`: `tmp[v] = min(tmp[v], dist[u] + w)`.
5. `dist = tmp`.
6. Return `dist[dst]` or `-1`.

The snapshot copy is mandatory. Relaxing in place would let a path use two edges within a single round — the round count would no longer bound the hop count, and you'd get answers that violate the stop limit. This one line is the difference between a correct and an incorrect solution, and it's the thing to point at during the dry run.

Data structures. A flat edge list (no adjacency needed) and two `dist` arrays.

Complexity. Time `O(k · E)`. Space `O(n)`.

PYTHON CODE

```python
class Solution:
    def findCheapestPrice(self, n: int, flights: list[list[int]],
                          src: int, dst: int, k: int) -> int:
        INF = float('inf')
        dist = [INF] * n
        dist[src] = 0

        # k STOPS == k+1 EDGES. Bellman-Ford's round count maps exactly onto the
        # edge budget: after round i, dist[v] is the cheapest cost to v using at
        # most i edges. That's why Bellman-Ford fits and Dijkstra does not —
        # Dijkstra finalises a node on first pop, but here a pricier route with
        # fewer stops may be the only one that can still be extended.
        for _ in range(k + 1):
            # SNAPSHOT: relax from the PREVIOUS round's distances only.
            # Relaxing in place would chain two edges inside one round, silently
            # exceeding the stop limit. This copy is the correctness of the whole
            # algorithm, not an optimisation.
            tmp = dist[:]

            for u, v, w in flights:
                if dist[u] != INF and dist[u] + w < tmp[v]:
                    tmp[v] = dist[u] + w

            dist = tmp

        return dist[dst] if dist[dst] != INF else -1
```

DRY RUN

`n = 4`, `flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]`, `src = 0`, `dst = 3`, `k = 1`

`k + 1 = 2` rounds. `dist = [0, inf, inf, inf]`.

Round 1 (`tmp` starts as `[0, inf, inf, inf]`):

flight	`dist[u]`	candidate	`tmp[v]` before	`tmp[v]` after
`0→1 (100)`	0	100	inf	**100**
`1→2 (100)`	**inf** (from the snapshot!)	—	inf	inf
`2→0 (100)`	inf	—	0	0
`1→3 (600)`	**inf**	—	inf	inf
`2→3 (200)`	inf	—	inf	inf

`dist = [0, 100, inf, inf]` — only 1-edge paths, correctly.

Round 2 (`tmp` starts as `[0, 100, inf, inf]`):

flight	`dist[u]`	candidate	`tmp[v]` after
`0→1 (100)`	0	100	100 (no change)
`1→2 (100)`	100	200	**200**
`2→0 (100)`	inf	—	0
`1→3 (600)`	100	**700**	**700**
`2→3 (200)`	**inf** (snapshot — round 1 had `dist[2] = inf`)	—	700

`dist = [0, 100, 200, 700]`.

Answer: `700` — the route `0 → 1 → 3` (1 stop). ✅

The snapshot doing its job: in round 2, the flight `2→3` read `dist[2] = inf` from the snapshot, even though `tmp[2]` had *already been set to 200* earlier in the same round. Without the copy, it would have computed `200 + 200 = 400` and returned `400` — the route `0 → 1 → 2 → 3`, which uses **2 stops** and violates `k = 1`. That single line is the entire difference.

For comparison, with `k = 2` the answer *is* `400`, found in round 3.

---

542. 01 Matrix

UNDERSTAND THE PROBLEM

Given a binary matrix, return each cell's distance to the nearest `0` (4-directional).

- Cells that are already `0` have distance 0.
- Guaranteed at least one `0` exists (clarify).
- Edge cases: all zeros → all zeros; a single `1` surrounded by zeros; a large region of `1`s.

BRUTE-FORCE APPROACH

Idea. For each `1` cell, BFS outward until the nearest `0` is found.

Complexity. Time `O((mn)²)` — an independent BFS per cell. Space `O(mn)`.

Why it's not ideal. With a 10^4-cell grid that's 10^8 operations, and the searches overlap almost completely — adjacent `1` cells explore nearly the same neighbourhood. **Invert the direction:** instead of `mn` searches outward from each `1`, run **one** search outward from *all* the `0`s at once. Multi-source BFS settles every cell in a single `O(mn)` pass, and by BFS's level property each cell is settled at exactly its true minimum distance.

OPTIMAL APPROACH

Idea (multi-source BFS).

1. Seed the queue with **every** `0` cell, distance 0.
2. Mark every `1` cell as unvisited (use `-1` or a separate `dist` grid seeded to `inf`).
3. Standard BFS: pop a cell, and for each unvisited neighbour set `dist = current + 1` and enqueue.

Because all sources start at distance 0 simultaneously, the first time a cell is reached is via the nearest source — the same argument as 994's rotting oranges. This is the same primitive; recognising it is the point.

Data structures. A `dist` grid; a deque seeded with all sources.

Complexity. Time `O(m·n)`. Space `O(m·n)`.

PYTHON CODE

```python
from collections import deque

class Solution:
    def updateMatrix(self, mat: list[list[int]]) -> list[list[int]]:
        rows, cols = len(mat), len(mat[0])
        DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))

        # -1 marks "not yet settled". Using the dist grid itself as the visited
        # marker saves a second structure.
        dist = [[-1] * cols for _ in range(rows)]
        q = deque()

        # MULTI-SOURCE seed: every 0 is a source at distance 0, and they all start
        # simultaneously. This is the inversion that replaces mn outward searches
        # from each 1 with ONE outward search from all the 0s.
        for r in range(rows):
            for c in range(cols):
                if mat[r][c] == 0:
                    dist[r][c] = 0
                    q.append((r, c))

        while q:
            r, c = q.popleft()
            for dr, dc in DIRS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and dist[nr][nc] == -1:
                    # First arrival wins: BFS expands in distance order, so this
                    # is provably the nearest source. No need to compare/minimise.
                    dist[nr][nc] = dist[r][c] + 1
                    q.append((nr, nc))

        return dist
```

DRY RUN

```
mat = 0 0 0
      0 1 0
      1 1 1
```

Seed: all `0` cells → `(0,0), (0,1), (0,2), (1,0), (1,2)` with distance 0.

`dist` after seeding:

```
 0  0  0
 0 -1  0
-1 -1 -1
```

BFS level	popped	settled neighbours
0	`(0,0)`	`(1,0)` already 0
	`(0,1)`	`(1,1)` → **1**
	`(0,2)`	`(1,2)` already 0
	`(1,0)`	`(2,0)` → **1**
	`(1,2)`	`(2,2)` → **1**
1	`(1,1)`	`(2,1)` → **2**
	`(2,0)`	`(2,1)` already settled at 2 — skipped
	`(2,2)`	`(2,1)` already settled — skipped
2	`(2,1)`	nothing new

Final:

```
0 0 0
0 1 0
1 2 1
```

✅

Check `(2,1)`: nearest `0` is `(1,0)` or `(1,2)`, both 2 steps away (`(2,1)→(2,0)→(1,0)`). Correct.

Note the two skipped attempts at level 1: `(2,0)` and `(2,2)` both tried to settle `(2,1)`, but it was already claimed by `(1,1)` in the same level. Because BFS settles in distance order, the first claim is always the minimum — no `min()` comparison is ever needed, which is what keeps the inner loop tight.

Conversion Rate — ✅ Verified (LeetCode 399 shape)

UNDERSTAND THE PROBLEM

Given rates like `USD → EUR = 0.9`, answer arbitrary conversion queries by chaining. Reverse edges are reciprocals. Return `-1` when no path exists.

- Structurally identical to LC 399, in currency clothing. Say so — recognising it costs you nothing and saves ten minutes.
- The domain adds real follow-ups worth raising: **rate staleness**, **floating-point drift** across long chains, **arbitrage** (a cycle whose product ≠ 1, meaning inconsistent data), and **bid/ask spreads** (the reverse rate isn't exactly the reciprocal in reality). Raising arbitrage detection unprompted is the strongest move available here.
- Edge cases: unknown currency → `-1`; `USD → USD` → `1.0` if USD is known, `-1` otherwise; disconnected currency groups.

BRUTE-FORCE APPROACH

Idea. For each query, try every sequence of rate applications.

Complexity. Exponential in the number of rates.

Why it's not ideal. Path-finding is linear per query with BFS/DFS; enumerating sequences is never necessary. And with many queries, even repeated linear searches are wasteful when the currency graph is static — which points at the better answer below.

OPTIMAL APPROACH

Idea. Two tiers, and knowing when to use each is the answer they want:

Tier 1 — DFS/BFS per query (LC 399). Build `graph[a][b] = rate`, `graph[b][a] = 1/rate`; DFS accumulating the product. `O(V + E)` per query. Right when queries are few or the rates change constantly.

Tier 2 — weighted Union-Find. Store, for each currency, its **rate relative to its component's root**. Then any query is two `find` calls and one division: `a/b = (a/root) / (b/root)`. Near-`O(1)` per query after an `O(E · α)` build. Right when queries vastly outnumber rate updates — the actual production shape for an FX service.

Tier 2 also gives **arbitrage detection** for free: when a union finds both currencies already in the same component, compare the implied rate against the stated one. A mismatch means the rate set is inconsistent.

Data structures. Tier 2: `parent[]` plus `weight[]` (ratio to parent), path-compressed together.

Complexity. Tier 2: build `O(E · α)`, query `O(α)` ≈ constant.

PYTHON CODE

```python
from collections import defaultdict


class CurrencyConverterDFS:
    """Tier 1 — LC 399 exactly. Right when rates change often or queries are few."""

    def __init__(self, rates: list[tuple[str, str, float]]):
        self.graph = defaultdict(dict)
        for a, b, rate in rates:
            self.graph[a][b] = rate
            self.graph[b][a] = 1.0 / rate     # reverse edge = reciprocal

    def convert(self, src: str, dst: str) -> float:
        if src not in self.graph or dst not in self.graph:
            return -1.0                       # unknown currency — even src==dst fails

        def dfs(cur: str, target: str, product: float, seen: set) -> float:
            if cur == target:
                return product
            seen.add(cur)
            for nxt, rate in self.graph[cur].items():
                if nxt not in seen:
                    got = dfs(nxt, target, product * rate, seen)
                    if got != -1.0:
                        return got
            return -1.0
```

```
        return dfs(src, dst, 1.0, set())


class CurrencyConverterDSU:
    """
    Tier 2 — weighted Union-Find. Near-O(1) per query after the build, which is
    the right trade for an FX service where reads massively outnumber rate updates.

    Invariant: weight[x] == value of x expressed in units of find(x)'s root.
    Then  a/b = weight[a] / weight[b]  whenever a and b share a root.
    """

    def __init__(self):
        self.parent: dict[str, str] = {}
        self.weight: dict[str, float] = {}     # x's rate relative to its ROOT

    def _add(self, x: str) -> None:
        if x not in self.parent:
            self.parent[x] = x
            self.weight[x] = 1.0               # a root's rate against itself is 1

    def find(self, x: str) -> tuple[str, float]:
        """Returns (root, x's rate relative to root), compressing the path."""
        if self.parent[x] != x:
            root, parent_weight = self.find(self.parent[x])
            # Recompute x's weight against the ROOT, not against its old parent —
            # this is path compression carrying the multiplicative weights along.
            self.weight[x] *= parent_weight
            self.parent[x] = root
        return self.parent[x], self.weight[x]

    def add_rate(self, a: str, b: str, rate: float, tolerance: float = 1e-6) -> None:
        """Record a / b = rate."""
        self._add(a)
        self._add(b)

        root_a, wa = self.find(a)
        root_b, wb = self.find(b)

        if root_a == root_b:
            # ARBITRAGE / CONSISTENCY CHECK — raise this unprompted, it's the
            # domain insight that separates a good answer here from a generic one.
            implied = wa / wb
            if abs(implied - rate) > tolerance:
                raise ValueError(
                    f"inconsistent rate: {a}/{b} stated {rate}, implied {implied}"
                )
            return

        # Attach root_b under root_a. We need weight[root_b] such that
        # b's value against root_a stays consistent:
        #   a/b = rate  and  a = wa * root_a,  b = wb * root_b
        #   => wa / (wb * w_rootb) = rate  => w_rootb = wa / (rate * wb)
        self.parent[root_b] = root_a
        self.weight[root_b] = wa / (rate * wb)

    def convert(self, src: str, dst: str) -> float:
        if src not in self.parent or dst not in self.parent:
            return -1.0
        root_s, ws = self.find(src)
        root_d, wd = self.find(dst)
        if root_s != root_d:
            return -1.0                        # different components — no chain
        return ws / wd
```

DRY RUN

Rates: `USD/EUR = 0.9`, `EUR/GBP = 0.8`

Tier 1 (DFS) — graph: `USD→EUR: 0.9`, `EUR→USD: 1.111…`, `EUR→GBP: 0.8`, `GBP→EUR: 1.25`

query	trace	result
`USD/GBP`	`USD` (p=1.0) → `EUR` (p=0.9) → `GBP` (p=0.9×0.8=**0.72**)	**0.72**
`GBP/USD`	`GBP` (1.0) → `EUR` (1.25) → `USD` (1.25×1.111…=**1.3888…**)	**1.3889**
`USD/JPY`	`JPY` not in graph	**−1.0**

Sanity: `0.72 × 1.3889 ≈ 1.0` ✓ — the reciprocal relationship holds.

Tier 2 (weighted DSU) — `add_rate("USD","EUR",0.9)`:

step	state
add USD, EUR	`parent={USD:USD, EUR:EUR}`, `weight={USD:1, EUR:1}`
`find(USD)` → `(USD, 1)`, `find(EUR)` → `(EUR, 1)`	roots differ
attach `EUR` under `USD`: `w_rootb = wa/(rate·wb) = 1/(0.9·1) = 1.111…`	`parent[EUR]=USD`, `weight[EUR]=1.111…`

Meaning: 1 EUR = 1.111… USD ✓ (since 1 USD = 0.9 EUR).

`add_rate("EUR","GBP",0.8)`:

step	state
`find(EUR)` → root `USD`, `wa = 1.111…`	
`find(GBP)` → `(GBP, 1)`	roots differ
`weight[GBP] = wa/(0.8·1) = 1.111…/0.8 = 1.3888…`; `parent[GBP] = USD`	

Query `USD/GBP`: `find(USD)` → `(USD, 1.0)`; `find(GBP)` → `(USD, 1.3888…)`. Result `1.0 / 1.3888… = 0.72` ✅ — matching Tier 1, in two `find` calls instead of a graph traversal.

Arbitrage check: now call `add_rate("USD","GBP", 0.75)`. Both are in the same component; implied `= 1.0/1.3888… = 0.72`, stated `0.75`, difference `0.03 > tolerance` → raises. That's a genuinely inconsistent rate set (you could cycle USD→EUR→GBP→USD and profit), and flagging it is the domain answer worth volunteering.

Friendship Path — 🟡 BFS (InterviewDB)

UNDERSTAND THE PROBLEM

Given friendships and two users, return the **shortest chain** connecting them.

- Friendship is symmetric → **undirected** graph.
- Return the actual path, not just its length — so you need parent pointers to reconstruct it.
- Clarify: *"Shortest by hop count, or is there a weight? If several shortest paths exist, which do I return? Return `[]` or `None` when disconnected? How large is the graph — do I need bidirectional BFS?"*
- Edge cases: `src == dst` → `[src]`; direct friends → 2 nodes; no connection → empty; a user not in the graph.

BRUTE-FORCE APPROACH

Idea. DFS enumerating all simple paths between the two users, keeping the shortest.

Complexity. Time `O(V!)` in the worst case — the number of simple paths is factorial. Space `O(V)`.

Why it's not ideal. Catastrophically exponential on a social graph, and unnecessary: BFS visits nodes in nondecreasing distance order, so the **first** time it reaches the target, that path is provably shortest. No enumeration, no comparison.

OPTIMAL APPROACH

Idea (BFS + parent map for reconstruction).

1. BFS from `src`, recording `parent[node] = the node we came from`.
2. On reaching `dst`, walk the parent chain back to `src` and reverse it.

The `parent` map serves as both the visited set and the reconstruction mechanism — seed it with `{src: None}` so the walk-back has a natural terminator.

Data structures. `defaultdict(set)` adjacency; `dict` parent map; a deque.

Complexity. Time `O(V + E)`. Space `O(V)`.

Follow-up worth naming: bidirectional BFS. On a social graph with branching factor `b` and distance `d`, unidirectional BFS explores `~b^d` nodes while bidirectional explores `~2·b^(d/2)`. For LinkedIn-style "degrees of separation" (`d ≈ 6`, `b ≈ 300`), that's the difference between 10^{15} and 10^{7} — the reason real systems all do it.

PYTHON CODE

```python
from collections import defaultdict, deque
from typing import Hashable, Iterable


class FriendshipGraph:
    def __init__(self, friendships: Iterable[tuple[Hashable, Hashable]] = ()):
        self.graph = defaultdict(set)
        for a, b in friendships:
            self.add_friendship(a, b)

    def add_friendship(self, a: Hashable, b: Hashable) -> None:
        self.graph[a].add(b)
        self.graph[b].add(a)        # undirected — friendship is symmetric

    def shortest_chain(self, src: Hashable, dst: Hashable) -> list:
        """Shortest chain of friends from src to dst, inclusive. [] if none."""
        if src not in self.graph or dst not in self.graph:
            return []
        if src == dst:
            return [src]

        # parent doubles as the VISITED set and the reconstruction map.
        # src maps to None, which terminates the walk-back naturally.
        parent = {src: None}
        q = deque([src])

        while q:
            node = q.popleft()

            for friend in self.graph[node]:
```

```
                if friend in parent:
                    continue                # already reached, and via a path
                                            # that is <= this one (BFS order)
                parent[friend] = node

                if friend == dst:
                    # BFS guarantees first arrival == shortest, so we can stop
                    # the entire search here rather than exploring further.
                    return self._rebuild(parent, dst)

                q.append(friend)

        return []                           # different components

    @staticmethod
    def _rebuild(parent: dict, dst: Hashable) -> list:
        path = []
        node = dst
        while node is not None:             # None marks the source
            path.append(node)
            node = parent[node]
        path.reverse()                      # we walked backwards
        return path

    def degrees_of_separation(self, src: Hashable, dst: Hashable) -> int:
        """Hop count, or -1. len(path) - 1 because the path counts NODES."""
        chain = self.shortest_chain(src, dst)
        return len(chain) - 1 if chain else -1


def bidirectional_shortest_chain(graph: dict, src, dst) -> list:
    """
    Follow-up: search from BOTH ends and stop when the frontiers meet.
    Explores ~2*b^(d/2) nodes instead of ~b^d - on a social graph with b~300
    and d~6 that is 10^7 vs 10^15, which is why every real system does this.
    """
    if src == dst:
        return [src]
    if src not in graph or dst not in graph:
        return []

    parent_f = {src: None}          # forward search tree
    parent_b = {dst: None}          # backward search tree
    frontier_f, frontier_b = {src}, {dst}

    while frontier_f and frontier_b:
        # Always expand the SMALLER frontier - keeps the branching balanced.
        if len(frontier_f) > len(frontier_b):
            frontier_f, frontier_b = frontier_b, frontier_f
            parent_f, parent_b = parent_b, parent_f
            # (After a swap the roles flip; the meeting logic below is symmetric,
            #  so we reassemble by orientation at the end.)

        next_frontier = set()
        for node in frontier_f:
            for nei in graph[node]:
                if nei in parent_f:
                    continue
                parent_f[nei] = node
                if nei in parent_b:
                    # Frontiers touched - splice the two half-paths.
                    left, right = [], []
                    n = nei
                    while n is not None:
                        left.append(n)
                        n = parent_f[n]
                    left.reverse()
                    n = parent_b[nei]
                    while n is not None:
                        right.append(n)
                        n = parent_b[n]
                    joined = left + right
                    # Orient so the path starts at src.
                    return joined if joined[0] == src else joined[::-1]
                next_frontier.add(nei)
        frontier_f = next_frontier
```

```
    return []
```

DRY RUN

```
friendships = [(A,B), (A,C), (B,D), (C,D), (D,E), (E,F)]
```

```
A -- B -- D -- E -- F
 \       /
  \_ C _/
```

Query: `shortest_chain(A, F)`

BFS step	queue	popped	new neighbours	parent map additions
seed	`[A]`			`{A: None}`
1	`[B, C]`	A	B, C	`B→A` , `C→A`
2	`[C, D]`	B	D (A seen)	`D→B`
3	`[D]`	C	(A, D both seen)	—
4	`[E]`	D	E (B, C seen)	`E→D`
5		E	**F == dst**	`F→E` → **reconstruct**

Reconstruction from `F` : `F → E → D → B → A` , reversed → `[A, B, D, E, F]` .

Answer: `[A, B, D, E, F]` — 4 hops. ✅

Note step 3: `C` found `D` already in `parent` (claimed by `B` in step 2), so it didn't overwrite. Both `A→B→D` and `A→C→D` are length-2 paths to `D` , so either is a valid shortest chain — the BFS keeps whichever arrives first, which is correct but non-deterministic if the adjacency is a `set` . If the interviewer asks for a deterministic tie-break (e.g. lexicographically smallest), sort the neighbour iteration — worth mentioning, and a one-line change.

`degrees_of_separation(A, F)` → `5 - 1 = 4` . ✅

`shortest_chain(A, "Zoe")` → `"Zoe"` not in the graph → `[]` . ✅

DAY 7 — Sun Aug 23 · ★★ MOCK LOOP #1

No new problems. Day 7 is a three-round simulated loop plus a debrief. There is nothing here to answer in the five-part format — the deliverable is the debrief, not a solution.

What to run (from `Day07_Aug23_Sun.md`): - **Round 1 (60 min) — Coding.** Two unseen problems, recorded. One LP question first, then the full Round Script. - **Round 2 (60 min) — Coding.** Two fresh problems: one linked list, one graph. Recorded. - **Break (30 min).** Take it properly — the real loop has breaks and you need to practise returning cold. - **Round 3 (60 min) — LLD.** One of the six briefs, on paper, narrating throughout. Then argue against one of your own decisions on camera and defend it.

Where to pull unseen problems from, given everything in this file is now "seen": use the ★ problems from days you haven't reached yet — Day 8's 1834 and 502, Day 9's 1552 and 528, Day 10's 759 and 918, Day 11's 494 and 588. Have someone else pick so you don't self-select the easy ones.

Debrief questions — write the answers down, they drive Days 12–13: 1. Which patterns did I freeze on? 2. Where did I go silent for more than 20 seconds? 3. Did I skip clarifying questions under pressure? 4. Did I dry-run without being prompted? 5. Which LP stories were weak or over-long? 6. **My three worst areas: __, _, ___**

DAY 8 — Mon Aug 24 · Heaps & Top-K

> **The one decision that defines the day: which direction does the heap face? k-th LARGEST → keep a MIN-heap of size k.** The root is the weakest survivor; when a bigger element arrives, evict the root. After the sweep the root *is* the k-th largest. **k-th SMALLEST / k CLOSEST → keep a MAX-heap of size k** (in Python: push negated values). Getting this backwards is the most common heap bug, and it produces plausible-looking wrong answers. Sanity-check the direction on a 3-element example *before* you write the loop.
>
> `heapq` **facts you must have automatic:** it is a **min-heap only**; `heappush` / `heappop` are `O(log n)`; `heapify` is `O(n)`, not `O(n log n)`; `heappushpop` is one `O(log n)` operation instead of two; `nlargest(k, it)` / `nsmallest(k, it)` are `O(n log k)`. For a max-heap, negate the key or wrap in a tuple whose first element is negated.

---

215. Kth Largest Element in an Array ★

UNDERSTAND THE PROBLEM

Given an unsorted array and `k`, return the **k-th largest** value in sorted order — **not** the k-th distinct value.

- "In sorted order, not distinct" means `[3,2,3,1,2,4,5,5,6]` with `k = 4` gives `4`, because the sorted-descending sequence is `6,5,5,4,…`.
- Edge cases: `k = 1` (the max); `k = n` (the min); all elements equal.
- Clarify: *"Can I mutate the input? Is `k` guaranteed valid?"*

BRUTE-FORCE APPROACH

Idea. Sort descending and index `k-1`.

Data structures. The sorted array.

Algorithm. `return sorted(nums, reverse=True)[k-1]`.

Complexity. Time `O(n log n)`, space `O(n)` (Timsort's).

Why it's not ideal. It computes a **total order** on all `n` elements to answer a question about one position. When `k ≪ n` — top 10 of a million — that's `n log n` work for what a size-`k` heap does in `n log k`. And for the true optimum there's Quickselect at `O(n)` average.

OPTIMAL APPROACH

Idea (min-heap of size k). Sweep the array maintaining a min-heap holding the `k` largest elements seen so far. The heap's root is the smallest of those `k` — i.e. the current k-th largest. When a new element exceeds the root, evict the root and insert the newcomer.

Why a **min**-heap for the k-th **largest**: the root is the element most at risk of being displaced, so it's the one you want cheap access to. A max-heap would put the wrong element at the root.

Data structures. `heapq` list of size `≤ k`.

Algorithm.

1. `heap = []`.
2. For each `x`: `heappush(heap, x)`; if `len(heap) > k`: `heappop(heap)`.
3. Return `heap[0]`.

Complexity. Time `O(n log k)`. Space `O(k)`.

Quickselect (the `O(n)` answer). Partition around a pivot like quicksort, but recurse into **only** the side containing the target index. Average `O(n)` (the recurrence `T(n) = T(n/2) + O(n)` sums to `2n`), worst case `O(n²)` — mitigated by a random pivot. Know both; lead with the heap, offer Quickselect when asked for better.

PYTHON CODE

```python
import heapq
import random

class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        """Min-heap of size k. O(n log k) time, O(k) space."""
        heap = []

        for x in nums:
            heapq.heappush(heap, x)
            if len(heap) > k:
                # Evict the SMALLEST of the k+1 candidates. Whatever survives is
                # always the k largest seen so far, and heap[0] is the smallest
                # of those — i.e. exactly the running k-th largest.
                heapq.heappop(heap)

        return heap[0]

    def findKthLargest_pushpop(self, nums: list[int], k: int) -> int:
        """Same idea, one heap op per element instead of two once warmed up."""
        heap = nums[:k]
        heapq.heapify(heap)              # O(k), not O(k log k)
        for x in nums[k:]:
            if x > heap[0]:
                heapq.heappushpop(heap, x)   # single O(log k) op
        return heap[0]


class SolutionQuickselect:
    """O(n) average. Recurse into ONE side only — that's the whole difference
    from quicksort, and it turns n log n into n."""
    def findKthLargest(self, nums: list[int], k: int) -> int:
        # k-th largest == index (n-k) in ascending order.
        target = len(nums) - k
        lo, hi = 0, len(nums) - 1

        while True:
            p = self._partition(nums, lo, hi)
            if p == target:
                return nums[p]           # pivot landed exactly on the target index
            elif p < target:
                lo = p + 1               # discard the left side entirely
            else:
                hi = p - 1               # discard the right side entirely

    @staticmethod
    def _partition(nums: list[int], lo: int, hi: int) -> int:
        # Random pivot: guards against the sorted-input worst case that makes
        # a fixed pivot O(n^2).
        r = random.randint(lo, hi)
        nums[r], nums[hi] = nums[hi], nums[r]

        pivot = nums[hi]
        store = lo
        for i in range(lo, hi):
            if nums[i] < pivot:
                nums[i], nums[store] = nums[store], nums[i]
                store += 1
        nums[store], nums[hi] = nums[hi], nums[store]
        return store                     # pivot's final, permanent index
```

DRY RUN

`nums = [3, 2, 1, 5, 6, 4]`, `k = 2` (expect `5`)

Min-heap of size 2:

x	push → heap	size > 2?	pop	heap after	root = running 2nd largest
3	`[3]`	no	—	`[3]`	—
2	`[2,3]`	no	—	`[2,3]`	2
1	`[1,3,2]`	yes	pop **1**	`[2,3]`	2
5	`[2,3,5]`	yes	pop **2**	`[3,5]`	3
6	`[3,5,6]`	yes	pop **3**	`[5,6]`	5
4	`[4,6,5]`	yes	pop **4**	`[5,6]`	5

Answer: `heap[0] = 5`. ✅

Sorted descending is `[6,5,4,3,2,1]`, so the 2nd largest is `5`. ✅

Direction check made concrete: at the `x = 4` step, the heap held `{5, 6}` and `4` was correctly rejected — because the root `5` was already larger. Had this been a max-heap, the root would have been `6`, `4 < 6` would also have "passed" a naive test, and `5` would have been evicted instead. Trace one step like this before committing to a direction.

---

347. Top K Frequent Elements

UNDERSTAND THE PROBLEM

Given an array and `k`, return the `k` most frequent values, in any order.

- Guaranteed unique answer (LeetCode) — no tie-break needed here, unlike problem 692.
- Edge cases: `k` equals the number of distinct values (return all); all elements identical; `k = 1`.

BRUTE-FORCE APPROACH

Idea. Count frequencies, then fully sort the `(value, count)` pairs by count descending and slice `k`.

Data structures. `Counter`; a sorted list.

Complexity. Time `O(n + d log d)` for `d` distinct values. Space `O(d)`.

Why it's not ideal. Same critique as 215 — a full sort to take a prefix. Two better options: a size-`k` heap at `O(d log k)`, or **bucket sort by frequency** at `O(n)`, which exploits the fact that any frequency is bounded by `n`.

OPTIMAL APPROACH

Idea (bucket sort by frequency — the `O(n)` answer). A frequency can only be an integer in `[1, n]`. So make `n + 1` buckets, put each value into `buckets[its frequency]`, then walk the buckets from high frequency down, collecting until you have `k`.

No comparisons, no heap — the frequency *is* the index. This is the answer that impresses.

Data structures. `Counter`; a list of `n + 1` lists.

Algorithm.

1. `counts = Counter(nums)`.
2. `buckets = [[] for _ in range(len(nums) + 1)]`; for each `(val, freq)`: `buckets[freq].append(val)`.
3. Walk `freq` from `n` down to `1`, extending the result; stop at `k`.

Complexity. Time `O(n)`. Space `O(n)`.

The heap version is `O(d log k)` and also fine — offer it, then present the bucket version as the improvement.

PYTHON CODE

```python
from collections import Counter
import heapq

class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        """Bucket sort by frequency. O(n) - no comparisons at all."""
        counts = Counter(nums)

        # A frequency is an integer in [1, n], so it can be used directly as an
        # array index. That bound is what lets us beat the O(d log k) heap.
        buckets = [[] for _ in range(len(nums) + 1)]
        for val, freq in counts.items():
            buckets[freq].append(val)

        res = []
        # Walk from the highest possible frequency downward.
        for freq in range(len(nums), 0, -1):
            for val in buckets[freq]:
                res.append(val)
                if len(res) == k:
                    return res          # early exit the moment we have k
        return res


class SolutionHeap:
    """O(d log k). Offer this first - it's the natural heap answer - then present
    the bucket version as the improvement."""
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        counts = Counter(nums)
        # nlargest keeps a size-k heap internally: O(d log k), beating a full sort.
        return [val for val, _ in heapq.nlargest(k, counts.items(),
                                                 key=lambda kv: kv[1])]
```

DRY RUN

`nums = [1, 1, 1, 2, 2, 3]`, `k = 2`

Counts: `{1: 3, 2: 2, 3: 1}`

Buckets (indices 0…6):

index (= frequency)	contents
0	`[]`
1	`[3]`
2	`[2]`
3	`[1]`
4–6	`[]`

Walk downward:

freq	bucket	action	res
6, 5, 4	empty	skip	`[]`
3	`[1]`	append 1	`[1]`
2	`[2]`	append 2 → **len == k**	`[1, 2]` → **return**

Answer: `[1, 2]`. ✅

Note the walk touched 4 empty buckets before finding anything. That's the `O(n)` bound's constant factor — bounded by `n` regardless of the data, and still linear. On an input where one value appears `n` times, the very first bucket checked is the answer.

---

692. Top K Frequent Words ★

UNDERSTAND THE PROBLEM

Given words and `k`, return the `k` most frequent, sorted by **count descending, then lexicographically ascending**.

- The **mixed sort direction** is the entire difficulty: one field descends, the other ascends. Negating a string isn't possible, so the trick used in "Top Customers" (Day 2) doesn't transfer directly.
- The output must be in that order, not merely contain the right `k` words.
- Edge cases: all words tied in count (pure lexicographic order); `k` equals the distinct count.

BRUTE-FORCE APPROACH

Idea. Count, then `sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))[:k]`.

Data structures. `Counter`; a sorted list.

Complexity. Time `O(n + d log d)`. Space `O(d)`.

Why it's not ideal. Correct and short — say so, and note that when `k ≈ d` it's actually the best choice. The improvement targets `k ≪ d`: a size-`k` heap gives `O(d log k)`. But the mixed direction makes the heap comparator genuinely tricky, which is exactly what's being tested.

OPTIMAL APPROACH

Idea (size-`k` MIN-heap with an inverted comparator). We keep the `k` best and evict the worst, so the heap's root must be the **worst** candidate. "Worst" here means *lowest count*, and among equal counts *lexicographically largest*.

So the heap key is `(count, ReverseString(word))` where the string comparison is inverted. Two clean ways: 1. A tiny wrapper class overriding `__lt__` to reverse string order — explicit and readable. 2. Push `(count, word)` into a min-heap but flip which one you evict... which doesn't work cleanly. Use the wrapper.

After the sweep, pop everything and reverse — pops come out worst-first.

Data structures. `Counter`; a size-`k` heap of wrapped tuples.

Complexity. Time `O(n + d log k)`. Space `O(d)`.

PYTHON CODE

```python
from collections import Counter
import heapq


class _Word:
    """Wraps a word so that LARGER lexicographic order compares as SMALLER.
    Needed because we keep a MIN-heap of the k best and evict its root, so the
    root must be the WORST candidate: lowest count, and among ties the word
    that should lose — the lexicographically LARGEST one."""
    __slots__ = ("word",)

    def __init__(self, word: str):
        self.word = word

    def __lt__(self, other: "_Word") -> bool:
        return self.word > other.word        # inverted on purpose


class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        counts = Counter(words)

        heap = []
        for word, count in counts.items():
            # Key = (count, inverted-word). Min-heap pops the smallest key:
            #   lower count           -> popped first  ✅ (high count wins)
            #   equal count, larger word -> smaller key ✅ (small word wins ties)
            heapq.heappush(heap, (count, _Word(word)))
            if len(heap) > k:
                heapq.heappop(heap)          # discard the current worst

        # Pops come out worst-first, so collect and reverse.
```

```
        out = []
        while heap:
            count, w = heapq.heappop(heap)
            out.append(w.word)
        out.reverse()
        return out


class SolutionSort:
    """O(d log d). Shorter, no comparator gymnastics, and genuinely the better
    choice when k is close to d. Say that rather than pretending it's wrong."""
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        counts = Counter(words)
        # -count sorts descending; word sorts ascending. One tuple, two directions.
        return sorted(counts, key=lambda w: (-counts[w], w))[:k]
```

DRY RUN

`words = ["i","love","leetcode","i","love","coding"]`, `k = 2`

Counts: `{i: 2, love: 2, leetcode: 1, coding: 1}`

Heap trace (showing keys as `(count, word↓)` where `↓` marks inverted comparison):

push	heap contents	size > 2?	pop (smallest key)	heap after
`(2, i↓)`	`[(2,i↓)]`	no	—	`{i}`
`(2, love↓)`	2 items	no	—	`{i, love}`
`(1, leetcode↓)`	3 items	yes	count 1 is lowest → **pop leetcode**	`{i, love}`
`(1, coding↓)`	3 items	yes	count 1 → **pop coding**	`{i, love}`

Now pop the remaining two. Both have count 2, so the inverted comparator decides: `"love" > "i"` means `love↓ < i↓`, so `(2, love↓)` is the smaller key and pops **first**.

Pops: `love`, then `i` → `out = ["love", "i"]` → reversed → **`["i", "love"]`**. ✅

Correct: both have count 2, and `"i" < "love"` lexicographically, so `"i"` comes first.

Why the inversion is necessary: with a plain `(count, word)` key, the tie between `i` and `love` would pop `i` first (since `"i" < "love"`), and if the heap ever had to evict one of them it would evict `i` — the word that should have *won*. The inversion is what aligns "smallest key" with "worst candidate" on both fields at once.

---

23. Merge k Sorted Lists ★★

UNDERSTAND THE PROBLEM

Given an array of `k` sorted linked lists, merge them into one sorted list.

- Splice existing nodes; don't allocate new ones.
- Edge cases: empty array of lists; some lists are `None`; all lists empty; `k = 1`.
- Let `N` = total node count across all lists.

BRUTE-FORCE APPROACH

Idea (a). Collect every value into an array, sort, rebuild. **Idea (b).** Merge sequentially: merge list 1 with 2, that result with 3, and so on.

Complexity. (a) `O(N log N)` time, `O(N)` space. (b) `O(k·N)` time — the accumulated list is re-walked on every merge, so it's `N/k + 2N/k + … + N` ≈ `N·k/2`.

Why it's not ideal. (a) discards the fact that the inputs are already sorted and allocates `N` new nodes. (b) is the more instructive failure: the *early* merged result gets traversed `k` times. Balancing the merge tree — pairing lists up rather than accumulating — drops it to `O(N log k)`, and that's the fix.

OPTIMAL APPROACH

Idea 1 — min-heap of the current heads. Push the head of each list. Repeatedly pop the smallest, append it, and push that node's successor. The heap never exceeds `k` entries.

Python tuple comparison will try to compare `ListNode` objects on ties, which raises `TypeError` — so push `(value, tiebreak_index, node)` with a unique integer in the middle. That detail is a real interview trap.

Idea 2 — divide and conquer (pairwise merge). Merge lists in pairs, halving the count each round: `k → k/2 → k/4 → … → 1`. Each round touches all `N` nodes, and there are `log k` rounds.

Both are `O(N log k)`. The D&C version uses `O(1)` extra space (vs the heap's `O(k)`) and is often cleaner — know both and state the trade.

Complexity. Time `O(N log k)`. Space: heap `O(k)`, D&C `O(log k)` recursion (or `O(1)` iterative).

PYTHON CODE

```python
import heapq


class Solution:
    def mergeKLists(self, lists: list[ListNode | None]) -> ListNode | None:
        """Min-heap of current heads. O(N log k) time, O(k) space."""
        heap = []

        # Seed with every non-empty head.
        # The middle element is a UNIQUE TIEBREAK INDEX. Without it, two equal
        # values make Python compare the ListNode objects, which raises
        # TypeError: '<' not supported between instances of 'ListNode'.
        # This is a real interview trap, not a style choice.
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, i, node))

        dummy = ListNode()
        tail = dummy
        counter = len(lists)      # keeps tiebreak values unique as we push more

        while heap:
            _val, _i, node = heapq.heappop(heap)
            tail.next = node      # splice the existing node, don't allocate
            tail = node

            if node.next:
                heapq.heappush(heap, (node.next.val, counter, node.next))
                counter += 1

        tail.next = None          # terminate cleanly
```

```
        return dummy.next


class SolutionDivideConquer:
    """Pairwise merge. Same O(N log k) time, O(log k) space, no tiebreak trickery.
    Often the cleaner answer — the heap version's payoff is that it also handles
    a STREAMING setting where lists arrive over time."""

    def mergeKLists(self, lists: list[ListNode | None]) -> ListNode | None:
        if not lists:
            return None

        # Halve the list count each round: k -> k/2 -> ... -> 1, i.e. log k rounds,
        # each touching all N nodes. The sequential version instead re-walks the
        # growing accumulator k times, which is where its O(kN) comes from.
        while len(lists) > 1:
            merged = []
            for i in range(0, len(lists), 2):
                first = lists[i]
                second = lists[i + 1] if i + 1 < len(lists) else None
                merged.append(self._merge_two(first, second))
            lists = merged

        return lists[0]

    @staticmethod
    def _merge_two(a: ListNode | None, b: ListNode | None) -> ListNode | None:
        dummy = ListNode()
        tail = dummy
        while a and b:
            if a.val <= b.val:
                tail.next, a = a, a.next
            else:
                tail.next, b = b, b.next
            tail = tail.next
        tail.next = a if a else b
        return dummy.next
```

DRY RUN

`lists = [[1,4,5], [1,3,4], [2,6]]`

Heap seeding: `(1, 0, node1a)`, `(1, 1, node1b)`, `(2, 2, node2)`

pop	value	appended	push successor	heap after (values)
`(1,0,·)`	1 (list 0)	`1`	`(4, 3, ·)`	`{1(l1), 2, 4}`
`(1,1,·)`	1 (list 1)	`1`	`(3, 4, ·)`	`{2, 3, 4}`
`(2,2,·)`	2	`2`	`(6, 5, ·)`	`{3, 4, 6}`
`(3,4,·)`	3	`3`	`(4, 6, ·)`	`{4, 4, 6}`
`(4,3,·)`	4 (list 0)	`4`	`(5, 7, ·)`	`{4, 5, 6}`
`(4,6,·)`	4 (list 1)	`4`	none (end of list 1)	`{5, 6}`
`(5,7,·)`	5	`5`	none	`{6}`
`(6,5,·)`	6	`6`	none	`{}`

Answer: `1 → 1 → 2 → 3 → 4 → 4 → 5 → 6`. ✅

The tiebreak earning its place: the first two pops both had value `1`. Python compared the tuples element-wise, found `1 == 1`, and moved to the second element — `0 < 1` — resolving cleanly. Without that integer it would have compared `node1a < node1b`, and `ListNode` defines no `__lt__`, so the program would crash on this exact input. Mention it before you write the line; it reads as experience rather than luck.

---

973. K Closest Points to Origin ★

UNDERSTAND THE PROBLEM

Given points and `k`, return the `k` closest to the origin, in any order.

- Distance is Euclidean, but you can compare **squared** distances — the square root is monotonic, so it never changes the ordering, and skipping it avoids float error and a `sqrt` call per point. Say this; it's the domain detail being probed.
- Edge cases: `k == len(points)`; duplicate points; points on the axes.

BRUTE-FORCE APPROACH

Idea. Compute every distance, sort, take the first `k`.

Complexity. Time `O(n log n)`, space `O(n)`.

Why it's not ideal. Full sort for a prefix again. `O(n log k)` with a heap, or `O(n)` average with Quickselect.

OPTIMAL APPROACH

Idea (max-heap of size k — note the direction flip). We want the `k` **smallest** distances, so the heap must hold the current `k` best with the **worst** (largest distance) at the root, ready for eviction. Python's `heapq` is a min-heap, so push **negated** squared distances.

This is the mirror of problem 215, and doing both on the same day is the point: k-th largest → min-heap; k closest (smallest) → max-heap.

Data structures. A size-`k` heap of `(-dist², x, y)`.

Algorithm.

1. For each point: `d = x² + y²`; `heappush(heap, (-d, x, y))`; if `len > k`: `heappop` (removes the most-negative = largest distance).
2. Return the surviving points.

Complexity. Time `O(n log k)`. Space `O(k)`.

Quickselect gives `O(n)` average on the squared distances — same structure as 215's. Offer it as the follow-up.

PYTHON CODE

```python
import heapq

class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        heap = []

        for x, y in points:
            # SQUARED distance - sqrt is monotonic, so it cannot change the
            # ordering. Skipping it avoids n sqrt calls and all float rounding.
            dist = x * x + y * y

            # MAX-heap via negation. We want the k SMALLEST distances, so the
            # root must hold the LARGEST distance among our k keepers - that's
            # the one to evict when something closer arrives.
            # (Contrast LC 215: k-th LARGEST wants a MIN-heap. Same day, opposite
            #  direction - check yours on a 3-element example before coding.)
            heapq.heappush(heap, (-dist, x, y))
            if len(heap) > k:
                heapq.heappop(heap)      # pops the most negative = farthest point

        return [[x, y] for _negdist, x, y in heap]


class SolutionNlargest:
    """One-liner using the same idea. nsmallest keeps a size-k heap internally."""
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        return heapq.nsmallest(k, points, key=lambda p: p[0] * p[0] + p[1] * p[1])
```

DRY RUN

`points = [[3,3], [5,-1], [-2,4]]`, `k = 2`

Squared distances: `[3,3] → 18`, `[5,-1] → 26`, `[-2,4] → 20`

point	dist²	push key	heap (keys)	size > 2?	pop	heap after
`[3,3]`	18	`(-18,3,3)`	`{-18}`	no	—	`{[3,3]}`
`[5,-1]`	26	`(-26,5,-1)`	`{-26, -18}`	no	—	`{[3,3],[5,-1]}`
`[-2,4]`	20	`(-20,-2,4)`	`{-26,-20,-18}`	yes	root is `-26` (most negative) → **pop `[5,-1]`**	`{[3,3],[-2,4]}`

Answer: `[[3,3], [-2,4]]`. ✅

Check: distances are 18, 20, 26 — the two closest are indeed `[3,3]` and `[-2,4]`.

Direction check made concrete: the eviction removed the point at distance **26**, the farthest. With a min-heap (no negation) the root would have been distance 18 — the *closest* point — and the algorithm would have thrown away exactly the answers it was supposed to keep. One three-point trace like this catches the error in ten seconds.

---

295. Find Median from Data Stream ★★

UNDERSTAND THE PROBLEM

Design a structure accepting numbers one at a time, able to return the running median at any point.

- Median of an even-sized set is the average of the two middle values.
- Both operations should be fast: `addNum` in `O(log n)`, `findMedian` in `O(1)`.
- Edge cases: the first element; even vs odd counts; duplicates; negatives.
- Clarify: *"Roughly how many `addNum` vs `findMedian` calls? If the values are bounded small integers, a counting/bucket approach beats the heaps."* That follow-up is a real one.

BRUTE-FORCE APPROACH

Idea. Keep a list; on each `findMedian`, sort and index the middle.

Complexity. `addNum` `O(1)`, `findMedian` `O(n log n)`. Space `O(n)`.

Improvement (still not enough): keep the list sorted by inserting with `bisect.insort` — `findMedian` becomes `O(1)`, but `addNum` becomes `O(n)` because a list insert shifts elements.

Why it's not ideal. Both variants make one operation linear or worse. The realisation: we never need the *whole* sorted order — only the **boundary between the lower and upper halves**. Two heaps expose exactly that boundary in `O(1)` while supporting `O(log n)` insertion.

OPTIMAL APPROACH

Idea (two heaps). Split the data at the median: - `low` — a **max-heap** (negated) holding the smaller half; its root is the largest of the small values. - `high` — a **min-heap** holding the larger half; its root is the smallest of the large values.

The median is either `low`'s root (odd total, with `low` holding the extra) or the average of both roots (even total).

Two invariants to maintain on every insert:

1. **Order:** every element in `low` ≤ every element in `high`.
2. **Balance:** `len(low) == len(high)` or `len(low) == len(high) + 1`.

The clean way to enforce both without branching: push onto `low`, immediately move `low`'s root to `high` (guarantees order), then if `high` got bigger, move its root back to `low` (restores balance). Three heap operations, always the same three — no case analysis, which is what makes it writable under pressure.

Data structures. Two `heapq` lists.

Complexity. `addNum` `O(log n)`, `findMedian` `O(1)`. Space `O(n)`.

PYTHON CODE

```python
import heapq


class MedianFinder:
    """
    Two heaps straddling the median.
      low  = MAX-heap (stored negated) of the smaller half
      high = MIN-heap of the larger half

    Invariants after every addNum:
      1. max(low) <= min(high)                    [order]
      2. len(low) == len(high) or len(high) + 1   [balance; low holds any extra]
    """

    def __init__(self):
        self.low = []      # max-heap via negation
        self.high = []     # min-heap

    def addNum(self, num: int) -> None:
        # Always the SAME three moves — no if/else on which heap to target.
        # Branch-free is what makes this reliably writable under time pressure.

        # 1. Tentatively place in the low half.
        heapq.heappush(self.low, -num)

        # 2. Move low's largest into high. This guarantees invariant 1
        #    unconditionally: whatever we just added is now on the correct side.
        heapq.heappush(self.high, -heapq.heappop(self.low))

        # 3. Rebalance if high overtook low, restoring invariant 2.
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def findMedian(self) -> float:
        # Odd total: low holds the extra element, so its root IS the median.
        if len(self.low) > len(self.high):
            return -self.low[0]
        # Even total: average the two middle values.
        return (-self.low[0] + self.high[0]) / 2.0
```

DRY RUN

Operations: `addNum(1)`, `addNum(2)`, `findMedian()`, `addNum(3)`, `findMedian()`

op	step 1 (push low)	step 2 (low→high)	step 3 (rebalance?)	low	high	median
`add(1)`	`low = [-1]`	move 1 → `high = [1]`, `low = []`	`len(high)=1 > len(low)=0` → move back	`[-1]` (={1})	`[]`	
`add(2)`	`low = [-2,-1]` (root −2 = value 2)	move 2 → `high = [2]`, `low = [-1]`	`1 > 1` ? no	`{1}`	`{2}`	
`findMedian`				sizes equal → `(1 + 2)/2`		**1.5** ✅
`add(3)`	`low = [-3,-1]` (root = 3)	move 3 → `high = [2,3]`, `low = [-1]`	`2 > 1` → move `high`'s root **2** back	`{1,2}`	`{3}`	
`findMedian`				`len(low)=2 > len(high)=1` → `-low[0]` = **2**		**2** ✅

Sorted data at the end is `[1, 2, 3]`; the median is `2`. ✅

Why step 2 can't be skipped: suppose we naively pushed `3` straight onto `high` at the last step. `high` would become `{2, 3}` and `low` `{1}` — balanced-ish and, here, correct by luck. Now feed `add(0)` instead: pushing `0` straight onto `low` gives `low = {0, 1}`, `high = {2}` — fine. But `add(1.5)` after `low={1}, high={2}`: pushing onto `low` gives `low = {1, 1.5}`, `high = {2}` — also fine. The failure comes with `low={5}, high={6}` and `add(7)`: a naive "push to the smaller heap" puts `7` in `low`, giving `max(low) = 7 > min(high) = 6` — invariant 1 violated, and the median becomes wrong. Step 2's unconditional transfer makes that impossible.

767. Reorganize String ★

UNDERSTAND THE PROBLEM

Rearrange a string so **no two adjacent characters are equal**. Return any valid result, or `""` if impossible.

- **Feasibility rule, state it first:** a valid arrangement exists iff `max_count ≤ (n + 1) // 2`. Intuition: the most frequent character must fit into the alternating slots `0, 2, 4, …`, of which there are exactly `(n+1)//2`.
- Edge cases: `"aa"` → `""`; `"aab"` → `"aba"`; a single character; all distinct.

BRUTE-FORCE APPROACH

Idea. Generate permutations and return the first with no equal adjacent pair.

Complexity. Time `O(n! · n)`. Space `O(n)`.

Why it's not ideal. Factorial. And it never surfaces the feasibility condition — you'd exhaust every permutation before concluding `""`. The greedy insight is much stronger: **always place the most frequent remaining character that isn't the one you just placed**. Always taking the most frequent prevents a character from piling up with no room left.

OPTIMAL APPROACH

Idea (max-heap + hold-back). Push `(-count, char)` for every character. Repeatedly: 1. Pop the most frequent character **that isn't the previous one**. 2. Append it, decrement its count. 3. **Hold it back** — don't push it again until *after* the next character is placed. That single-slot delay is what enforces non-adjacency.

Implementation: pop the top, append, then push the *previously held* character back (if its count is still > 0), then set the current one as held.

Data structures. `Counter`; a max-heap of `(-count, char)`; one held-back slot.

Algorithm.

1. Build counts; if `max(counts.values()) > (n+1)//2`: return `""`.
2. Heapify `[(-c, ch)]`.
3. `prev = None`. While heap: pop `(negc, ch)`; append `ch`; if `prev` has remaining count, push it; `prev = (negc + 1, ch)` if `negc + 1 < 0` else `None`.
4. Return the joined string.

Complexity. Time `O(n log |Σ|)` — `n` placements, each a heap op over at most 26 entries, so effectively `O(n)`. Space `O(|Σ|)`.

PYTHON CODE

```python
import heapq
from collections import Counter


class Solution:
    def reorganizeString(self, s: str) -> str:
        n = len(s)
        counts = Counter(s)

        # FEASIBILITY, checked up front: the most frequent character must fit in
        # the alternating slots 0, 2, 4, ..., and there are exactly (n+1)//2 of
        # them. Exceeding that makes two of them adjacent by pigeonhole.
        if max(counts.values()) > (n + 1) // 2:
            return ""

        # Max-heap via negated counts.
        heap = [(-count, ch) for ch, count in counts.items()]
        heapq.heapify(heap)                      # O(|alphabet|)

        out = []
        prev = None      # (remaining_negcount, char) held back for exactly one slot

        while heap:
            negcount, ch = heapq.heappop(heap)   # most frequent AVAILABLE char
            out.append(ch)
            negcount += 1                        # used one copy (moving toward 0)

            # Release the character held from the previous step — now that a
            # different character sits between them, it's legal again. This
            # one-slot delay IS the non-adjacency guarantee.
            if prev is not None:
                heapq.heappush(heap, prev)
                prev = None

            # Hold the current character back if copies remain.
            if negcount < 0:
                prev = (negcount, ch)

        # Greedy + the feasibility precheck guarantee success, so no length check
        # is strictly needed — but returning "" defensively costs nothing.
        return ''.join(out) if len(out) == n else ""
```

DRY RUN

`s = "aab"` → counts `{a: 2, b: 1}`, `n = 3`, `(n+1)//2 = 2`. `max_count = 2 ≤ 2` ✓ feasible.

Heap: `[(-2,'a'), (-1,'b')]`

step	pop	append	negcount after	release prev	new prev	heap	out
1	`(-2,'a')`	`a`	`-1`	none	`(-1,'a')`	`[(-1,'b')]`	`"a"`
2	`(-1,'b')`	`b`	`0`	push `(-1,'a')`	none (count 0)	`[(-1,'a')]`	`"ab"`
3	`(-1,'a')`	`a`	`0`	none	none	`[]`	`"aba"`

Answer: `"aba"`. ✅

Infeasible case, `s = "aaab"`: counts `{a:3, b:1}`, `n = 4`, `(n+1)//2 = 2`, `max_count = 3 > 2` → return `""` immediately. ✅ Correct — any arrangement of three `a`s in four slots forces two adjacent.

The hold-back doing its job at step 2: after placing `a`, the heap contained only `b` — `a` was deliberately withheld. Without the hold-back, `a` (count 1, still the tied-most-frequent) could have been popped again immediately, producing `"aa…"`. The delay is the entire mechanism.

621. Task Scheduler ★

UNDERSTAND THE PROBLEM

Given task labels and a cooldown `n`, identical tasks must be at least `n` intervals apart. Return the **minimum total intervals** including idles.

- **Watch: the answer is a formula, not a simulation.** Simulating a heap works and is `O(total_time)`, but the closed form is `O(m)` and is what's being tested.
- Edge cases: `n = 0` (→ `len(tasks)`); one task type; many task types (idles never needed).

BRUTE-FORCE APPROACH

Idea. Simulate with a max-heap and a cooldown queue: each interval, pop the most frequent available task; if none is available, emit an idle; push cooled-down tasks back.

Data structures. Max-heap plus a queue of `(count, ready_time)`.

Complexity. Time `O(total_time · log 26)`, space `O(26)`.

Why it's not ideal. It's correct, and worth knowing because it generalises to variants (per-task cooldowns, priorities). But `total_time` can be much larger than the task count when `n` is large and one task dominates — e.g. one task type repeated 1000 times with `n = 10⁶`. The closed form computes the same answer in `O(m)` regardless.

OPTIMAL APPROACH

Idea (closed form via the dominant task). Let `f` = the highest task frequency, and `c` = how many task types achieve that frequency.

Lay out the schedule as `f − 1` **complete frames**, each of length `n + 1`, followed by a final partial frame containing the `c` most-frequent tasks:

```
[A B _ _] [A B _ _] ... [A B]
 └─ f-1 frames of size (n+1) ─┘  └ c tasks ┘
```

So the length is `(f − 1) · (n + 1) + c`.

But if there are **many distinct tasks**, the idle slots get filled and can even overflow, making the schedule longer than the frame layout. In that case no idles are needed at all and the answer is just `len(tasks)`.

Final formula:

```
max((f − 1) · (n + 1) + c,  len(tasks))
```

Data structures. A `Counter`.

Complexity. Time `O(m)` for `m` tasks. Space `O(26) = O(1)`.

PYTHON CODE

```python
from collections import Counter


class Solution:
    def leastInterval(self, tasks: list[str], n: int) -> int:
        counts = Counter(tasks)
        f = max(counts.values())                # highest frequency
        c = sum(1 for v in counts.values() if v == f)   # how many tie at that max

        # Frame layout: the dominant task forces (f-1) gaps, each of width n+1
        # (the task itself plus n cooldown slots), then a final partial frame
        # holding the c tasks that share the maximum frequency.
        #
        #   [A B _ _][A B _ _]...[A B]
        #    <-- (f-1) frames --> <-c->
        framed = (f - 1) * (n + 1) + c

        # If there are enough DISTINCT tasks, every idle slot gets filled and the
        # frame bound is an underestimate — the schedule is simply back-to-back.
        # max() picks whichever constraint actually binds.
        return max(framed, len(tasks))
```

DRY RUN

`tasks = ["A","A","A","B","B","B"]`, `n = 2`

Counts: `{A: 3, B: 3}` → `f = 3`, `c = 2`, `len(tasks) = 6`.

`framed = (3 − 1) · (2 + 1) + 2 = 2 · 3 + 2 = 8`

`max(8, 6) = 8`. ✅

Concrete schedule: `A B idle A B idle A B` — 8 intervals, and every pair of identical tasks is 3 apart (≥ `n+1 = 3`). ✅

Case where `len(tasks)` binds instead — `tasks = ["A","A","A","B","B","B","C","C","C","D","D","E"]`, `n = 2`:

Counts: `{A:3, B:3, C:3, D:2, E:1}` → `f = 3`, `c = 3`, `len = 12`.

`framed = (3−1)·3 + 3 = 9`, but **`max(9, 12) = 12`**.

Correct: with five distinct task types there are always enough different tasks to fill every slot, so no idles occur and the answer is simply the task count. A formula without the `max` would return 9 — shorter than the number of tasks, which is obviously impossible. Point at that sanity check during the dry run; it's how you demonstrate the formula is understood rather than memorised.

`n = 0` case: `framed = (f−1)·1 + c`, which is `≤ len(tasks)`, so `max` returns `len(tasks)` ✓.

502. IPO ★

UNDERSTAND THE PROBLEM

Given `k` projects you may complete, starting capital `w`, and each project's `profits[i]` and `capital[i]`, return the **maximum final capital**.

- Completing a project **adds** its profit to your capital and doesn't consume the capital requirement — capital is a *threshold*, not a cost. That's the detail to confirm out loud.
- Since capital only grows, the set of affordable projects only ever expands.
- Edge cases: no project is affordable initially → return `w`; `k` exceeds the project count; all projects affordable from the start.

BRUTE-FORCE APPROACH

Idea. For each of the `k` rounds, scan all `n` projects for the most profitable affordable one, take it, mark it used.

Complexity. Time `O(k · n)`. Space `O(1)`.

Why it's not ideal. With `n = 10⁵` and `k = 10⁵` that's 10^{10}. Each scan re-examines projects it has already rejected as unaffordable, and re-examines the affordable pool it already knows about. Two heaps eliminate both: one keeps projects sorted by capital so newly-affordable ones stream in cheaply, the other keeps affordable projects sorted by profit so the best is `O(1)` to find.

OPTIMAL APPROACH

Idea (two heaps — the "unlock then choose" pattern).

1. Sort all projects by **capital ascending** (or use a min-heap keyed on capital).
2. Maintain a **max-heap of profits** for currently-affordable projects.
3. Repeat `k` times:
4. **Unlock:** move every project whose capital `≤ w` from the sorted list into the profit max-heap.
5. **Choose:** if the profit heap is empty, stop — nothing is affordable and capital can't grow. Otherwise pop the largest profit and add it to `w`.

Because capital is monotonically non-decreasing, the unlock pointer only moves forward — each project is unlocked exactly once across the whole run.

Greedy correctness: taking the highest available profit is optimal because capital never decreases, so any project affordable now stays affordable later. Taking the best now can only expand (never shrink) future options.

Data structures. A capital-sorted list plus an index pointer; a max-heap of profits.

Complexity. Time `O(n log n + k log n)`. Space `O(n)`.

PYTHON CODE

```python
import heapq


class Solution:
    def findMaximizedCapital(self, k: int, w: int,
                             profits: list[int], capital: list[int]) -> int:
        n = len(profits)

        # Sorted by capital ascending, so the "unlock" pointer only moves forward
        # and each project is unlocked at most once across all k rounds.
        projects = sorted(zip(capital, profits))

        available = []      # max-heap of profits (negated), for affordable projects
        i = 0               # how far into `projects` we have unlocked

        for _ in range(k):
            # UNLOCK: capital only grows, so everything newly affordable joins the
            # pool permanently. The pointer never rewinds.
            while i < n and projects[i][0] <= w:
                heapq.heappush(available, -projects[i][1])
                i += 1

            if not available:
                break       # nothing affordable, and w can no longer grow -> done

            # CHOOSE greedily: take the largest profit available.
            # Correct because capital is non-decreasing - anything affordable now
            # remains affordable later, so taking the best now never closes a door.
            w += -heapq.heappop(available)

        return w
```

DRY RUN

`k = 2`, `w = 0`, `profits = [1, 2, 3]`, `capital = [0, 1, 1]`

Sorted projects (by capital): `[(0, 1), (1, 2), (1, 3)]`

round	w before	unlock (capital ≤ w)	available (profits)	pop	w after
1	0	`(0,1)` → push 1. Next is `(1,2)`, `1 > 0` → stop	`{1}`	**1**	**1**
2	1	`(1,2)` → push 2; `(1,3)` → push 3	`{2, 3}`	**3**	**4**

Answer: `4`. ✅

Verify by hand: with `w = 0` only the first project is affordable, giving `w = 1`. Now both remaining projects are affordable; take the profit-3 one → `w = 4`. Taking profit 2 instead would have given `w = 3` and used up the last round — strictly worse. The greedy is right.

The unlock pointer's efficiency: across both rounds, `i` advanced from 0 to 3 — each project pushed exactly once. The brute force would have rescanned all three projects in both rounds, and on realistic inputs that difference is the whole complexity gap.

Early-break case: `w = 0`, `capital = [5, 5]` → round 1 unlocks nothing, `available` is empty → `break` → return `0`. Without that break you'd loop `k` times doing nothing, or worse, pop from an empty heap.

1834. Single-Threaded CPU ★

UNDERSTAND THE PROBLEM

Given `tasks[i] = [enqueueTime, processingTime]`, the CPU repeatedly picks the **shortest available** task (ties broken by **smallest index**) and runs it to completion. If nothing is available it idles until the next task arrives. Return the processing order (task indices).

- The CPU is non-preemptive: once started, a task runs to completion.
- "Available" means `enqueueTime ≤ current time`.
- **The idle jump is the trap:** when nothing is available, you must jump the clock forward to the next task's enqueue time rather than ticking one unit at a time.
- Edge cases: all tasks enqueue at time 0; a large gap forcing an idle; ties in processing time.

BRUTE-FORCE APPROACH

Idea. Simulate the clock one time unit at a time; at each tick, scan all tasks for available ones and pick the best.

Complexity. Time `O(T · n)` where `T` is the total timeline length — which can be `10⁹` given the constraints. Space `O(n)`.

Why it's not ideal. The timeline is enormous relative to the task count, and almost every tick does nothing. Two fixes: (1) **jump** the clock straight to the next enqueue time when idle, so time advances in `O(n)` discrete steps, not `O(T)`; (2) keep available tasks in a **min-heap** keyed on `(processingTime, index)` so selection is `O(log n)` rather than an `O(n)` scan.

OPTIMAL APPROACH

Idea (sort by enqueue time + min-heap on `(processing, index)`).

1. Sort tasks by `enqueueTime`, carrying the original index.
2. Maintain a pointer `i` into that sorted list and a min-heap of available tasks.
3. Loop until all tasks are done:
4. Push every task with `enqueueTime ≤ time` into the heap.
5. If the heap is empty, **jump**: `time = tasks[i].enqueueTime`, and continue (the next iteration will push it).
6. Otherwise pop `(processing, index)`, append `index` to the output, and `time += processing`.

The heap key `(processingTime, index)` gives both the shortest-first rule and the index tie-break in one tuple — the same technique as the VM Load problem on Day 3.

Data structures. Enqueue-sorted list; min-heap of `(processing, index)`.

Complexity. Time `O(n log n)`. Space `O(n)`.

PYTHON CODE

```python
import heapq


class Solution:
    def getOrder(self, tasks: list[list[int]]) -> list[int]:
        n = len(tasks)

        # Carry the ORIGINAL index — the answer is indices, and it's also the
        # tie-break key. Sort by enqueue time so the unlock pointer moves forward.
        indexed = sorted((enq, proc, i) for i, (enq, proc) in enumerate(tasks))

        available = []      # min-heap of (processingTime, originalIndex)
        order = []
        time = 0
        i = 0               # pointer into `indexed`

        while len(order) < n:
            # Admit everything that has arrived by now.
            while i < n and indexed[i][0] <= time:
                _enq, proc, idx = indexed[i]
                # (proc, idx): tuple comparison gives shortest-first, then
                # smallest-index on ties — no custom comparator needed.
                heapq.heappush(available, (proc, idx))
                i += 1

            if not available:
                # IDLE JUMP — the whole point of the problem. Advance the clock
                # straight to the next arrival instead of ticking one unit at a
                # time. Without this the loop is O(T) with T up to 1e9.
                time = indexed[i][0]
                continue

            proc, idx = heapq.heappop(available)
            order.append(idx)
            time += proc        # non-preemptive: runs to completion

        return order
```

DRY RUN

`tasks = [[1,2], [2,4], [3,2], [4,1]]` (indices 0–3)

Sorted by enqueue: `[(1,2,0), (2,4,1), (3,2,2), (4,1,3)]`

time	admit (enq ≤ time)	heap `(proc, idx)`	action	order	new time
0	none (`1 > 0`)	`{}`	**idle jump** → `time = 1`	`[]`	1
1	`(1,2,0)`	`{(2,0)}`	pop `(2,0)`	`[0]`	`1+2 = 3`
3	`(2,4,1)` , `(3,2,2)`	`{(2,2), (4,1)}`	pop `(2,2)` — shorter	`[0,2]`	`3+2 = 5`
5	`(4,1,3)`	`{(1,3), (4,1)}`	pop `(1,3)` — shortest	`[0,2,3]`	`5+1 = 6`
6	none left	`{(4,1)}`	pop `(4,1)`	`[0,2,3,1]`	10

Answer: `[0, 2, 3, 1]` . ✅

Two things worth pointing at: - **The idle jump at time 0** moved the clock to 1 in a single step. A tick-by-tick simulation would be identical here, but with `enqueueTime = 10⁹` it would run a billion no-op iterations. - **At time 5**, task 1 had been waiting since time 2 with `processing = 4` , but task 3 (just arrived, `processing = 1`) jumped ahead. That's shortest-job-first, and it's why the answer isn't simply arrival order — a good dry run has to include a case where a later arrival preempts an earlier one in the queue.

---

378. Kth Smallest Element in a Sorted Matrix

UNDERSTAND THE PROBLEM

Given an `n × n` matrix where each **row** and each **column** is sorted ascending, return the `k`-th smallest element in the whole matrix.

- Rows and columns are individually sorted, but the matrix is **not** globally sorted — `matrix[0][n-1]` may exceed `matrix[1][0]`. So you cannot flatten and binary-search positions.
- k-th smallest **in value order with duplicates**, not k-th distinct.
- Edge cases: `k = 1` (top-left); `k = n²` (bottom-right); all elements equal.

BRUTE-FORCE APPROACH

Idea. Flatten all `n²` elements, sort, index `k-1`.

Complexity. Time `O(n² log n²)` = `O(n² log n)`. Space `O(n²)`.

Why it's not ideal. It ignores the sortedness entirely. Two better routes: a **min-heap** merging the `n` sorted rows in `O(k log n)`, or — the answer being fished for — **binary search on the value range** in `O(n log(max-min))`, which doesn't even touch every element.

OPTIMAL APPROACH

Idea (binary search on the answer). Don't search positions — search **values**. Binary search `mid` in `[matrix[0][0], matrix[n-1][n-1]]` and ask: *how many elements are `≤ mid`?*

Counting is `O(n)` using the **staircase walk**: start at the bottom-left corner. If the current value `≤ mid`, the whole column above... no — precisely: if `matrix[row][col] <= mid`, then every element in that column *at or above* this row in the current column is also ≤ mid (column sorted), so add `row + 1` and move right. Otherwise move up.

Then: if `count < k`, the answer is larger → `lo = mid + 1`; else `hi = mid`. The loop converges on a value that is guaranteed to be **in the matrix**, because it converges on the smallest value whose count reaches `k`, and that value must be present.

Data structures. None — two integers and a walk.

Complexity. Time `O(n · log(max − min))`. Space `O(1)`.

The heap version is `O(k log n)` — better when `k` is tiny, worse when `k ≈ n²`. Mention the trade.

PYTHON CODE

```python
import heapq


class Solution:
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        """Binary search on the VALUE range. O(n log(max-min)) time, O(1) space."""
        n = len(matrix)
        lo, hi = matrix[0][0], matrix[n - 1][n - 1]

        def count_le(target: int) -> int:
            """How many elements are <= target. O(n) via a staircase walk from
            the bottom-left corner."""
            count = 0
            row, col = n - 1, 0

            while row >= 0 and col < n:
                if matrix[row][col] <= target:
                    # Column is sorted ascending downward, so every element in
                    # this column from row 0 down to `row` is also <= target.
                    # That's row+1 elements, counted in one step.
                    count += row + 1
                    col += 1            # move right to a larger column
                else:
                    row -= 1            # too big — move up to smaller values

            return count

        # Invariant: the answer is always in [lo, hi].
```

```
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if count_le(mid) < k:
                lo = mid + 1            # too few <= mid -> answer is strictly larger
            else:
                hi = mid                # enough <= mid -> answer is mid or smaller

        # lo == hi, and it is the SMALLEST value whose count reaches k.
        # Such a value must actually be present in the matrix: if it weren't, the
        # count would be unchanged at value-1, contradicting minimality.
        return lo


class SolutionHeap:
    """O(k log n) - merge the n sorted rows. Better than the binary search when
    k is small; worse when k approaches n^2. Know which you'd pick and why."""
    def kthSmallest(self, matrix: list[list[int]], k: int) -> int:
        n = len(matrix)
        # Seed with the first element of each row: (value, row, col)
        heap = [(matrix[r][0], r, 0) for r in range(min(n, k))]
        heapq.heapify(heap)

        val = 0
        for _ in range(k):
            val, r, c = heapq.heappop(heap)
            if c + 1 < n:
                heapq.heappush(heap, (matrix[r][c + 1], r, c + 1))
        return val
```

DRY RUN

```
matrix = [[ 1,  5,  9],
          [10, 11, 13],
          [12, 13, 15]]
k = 8
```

`lo = 1`, `hi = 15`. Expected answer: sorted values are `1,5,9,10,11,12,13,13,15` → the 8th is **13**.

iteration	lo	hi	mid	`count_le(mid)` staircase	count	`< k`?	new range
1	1	15	8	start (2,0)=12 > 8 → up; (1,0)=10 > 8 → up; (0,0)=1 ≤ 8 → +1, right; (0,1)=5 ≤ 8 → +1, right; (0,2)=9 > 8 → up, row=−1 stop	**2**	2 < 8 ✓	`lo = 9`
2	9	15	12	(2,0)=12 ≤ 12 → +3, right; (2,1)=13 > 12 → up; (1,1)=11 ≤ 12 → +2, right; (1,2)=13 > 12 → up; (0,2)=9 ≤ 12 → +1, right, col=3 stop	**6**	6 < 8 ✓	`lo = 13`
3	13	15	14	(2,0)=12 ≤ 14 → +3, right; (2,1)=13 ≤ 14 → +3, right; (2,2)=15 > 14 → up; (1,2)=13 ≤ 14 → +2, right, col=3 stop	**8**	8 < 8 ✗	`hi = 14`
4	13	14	13	(2,0)=12 ≤ 13 → +3; (2,1)=13 ≤ 13 → +3; (2,2)=15 > 13 → up; (1,2)=13 ≤ 13 → +2, col=3 stop	**8**	✗	`hi = 13`

`lo == hi == 13` → return **13**. ✅

Two points worth making during the dry run: - **The staircase counted 8 elements in 4 steps**, not by examining all 9 cells. On an `n = 300` matrix that's 300 steps instead of 90,000 — the source of the `O(n)` per check. - **`mid = 14` is not in the matrix**, yet the algorithm handled it gracefully: `count_le(14) = 8 ≥ k`, so `hi` moved to 14, and the next round narrowed to the actual present value 13. Explaining *why* the final `lo` is guaranteed to be a real matrix element (minimality argument in the code comment) is the follow-up question on this problem.

Bid Fulfillment — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Fixed `inventory` and a list of requests `[customerId, quantity, bid, timestamp]`. **Higher bid gets priority.** Within the **same bid**, allocate **round-robin in increasing timestamp**: each round gives **one unit** to every still-unfulfilled customer in that bid group. Drop a customer once fully fulfilled. Stop when inventory hits 0. Return the IDs of customers who received **zero** units, in **original input order**.

- **The trap, stated in the prompt itself:** it is *not* "sort by bid and give the top customer everything." The equal-bid round-robin is the entire question.
- Round-robin means: within a bid tier, cycle through customers by timestamp handing out one unit at a time, removing anyone whose `quantity` is met, until the tier is exhausted or inventory runs out.
- Output order is **original input order**, not sorted — a separate detail that's easy to lose.
- Edge cases: inventory 0; inventory exceeding total demand (nobody gets zero); all bids equal (one big round-robin); a customer requesting 0 units (clarify — treat as already fulfilled?); duplicate timestamps within a tier (clarify the tie-break; assume stable by input order).

BRUTE-FORCE APPROACH

Idea. Simulate literally: loop one unit at a time. For each unit, scan **all** requests to find the eligible customer — highest bid, then, among that bid, the one whose turn it is in the round-robin.

Data structures. The raw request list plus per-customer allocation counters.

Algorithm. `while inventory > 0`: scan all `n` requests, apply the priority rules, give one unit, decrement.

Complexity. Time `O(inventory · n)`. Space `O(n)`.

Why it's not ideal. `inventory` can be far larger than `n` — allocating a million units to five customers means a million scans of five records. Two improvements: (1) **group by bid** once and process tiers in descending order, so the priority rule is structural rather than re-derived per unit; (2) within a tier, allocate a **whole round at a time** — with `m` active customers and `r` units available, you can hand out `min(r // m, smallest_remaining)` units to everyone at once instead of looping unit by unit.

The full round-batching optimisation is worth *mentioning*; the per-unit round-robin within a tier is usually acceptable and much easier to get right. Say which you're doing and why — under interview time pressure, correct-and-simple beats clever-and-broken.

OPTIMAL APPROACH

Idea (group by bid, descending; round-robin by timestamp within each tier).

1. Group requests by `bid` into buckets.
2. Process bids in **descending** order.
3. Within a bid tier, sort the customers by `timestamp` ascending, then run rounds: in each round, walk the active list giving one unit each, dropping anyone whose `quantity` is met, stopping the moment inventory hits 0.
4. Track allocations in a dict; at the end, emit the input-order list of customers with zero allocation.

Data structures. `defaultdict(list)` bid → requests; a deque or index-cycling list for the round-robin; an allocation dict.

Complexity. Time `O(n log n + total_allocated)` — sorting dominates the setup; the allocation loop runs once per unit handed out, bounded by `min(inventory, total_demand)`. Space `O(n)`.

(With whole-round batching it becomes `O(n log n + n · rounds)`, where rounds is bounded by the largest quantity — mention it as the scaling answer.)

PYTHON CODE

```python
from collections import defaultdict, deque
from typing import Sequence


def bid_fulfillment(inventory: int, requests: Sequence[Sequence]) -> list:
```

```
    """
    requests: iterable of (customer_id, quantity, bid, timestamp)

    Rules:
      * higher bid is served first;
      * within one bid, ROUND-ROBIN by increasing timestamp — one unit per
        customer per round, dropping customers as they are fulfilled;
      * stop when inventory reaches 0.

    Returns: ids of customers allocated ZERO units, in ORIGINAL INPUT ORDER.
    """
    # allocated[cid] = units given so far. Seed every customer at 0 so the final
    # "who got nothing" query needs no special handling for untouched customers.
    allocated = {req[0]: 0 for req in requests}

    # Group by bid so the priority rule is structural rather than recomputed
    # for every single unit (which is what makes the brute force O(inventory*n)).
    by_bid = defaultdict(list)
    for cid, qty, bid, ts in requests:
        by_bid[bid].append((ts, cid, qty))

    # Highest bid tier first.
    for bid in sorted(by_bid, reverse=True):
        if inventory <= 0:
            break

        # Within a tier, order by timestamp — that defines the round-robin cycle.
        tier = sorted(by_bid[bid])            # sorts by (ts, cid, qty)
        queue = deque((cid, qty) for _ts, cid, qty in tier)

        # ROUND-ROBIN: hand out one unit, then move that customer to the BACK
        # of the queue. A deque makes "next in the cycle" O(1).
        # This is the whole question — NOT "give the top bidder everything".
        while queue and inventory > 0:
            cid, qty = queue.popleft()

            allocated[cid] += 1
            inventory -= 1

            # Requeue only if this customer still wants more. Dropping fulfilled
            # customers is what makes later rounds shorter.
            if allocated[cid] < qty:
                queue.append((cid, qty))

    # Original input order, and de-duplicated in case a customer appears twice.
    seen = set()
    zero = []
    for cid, _qty, _bid, _ts in requests:
        if cid not in seen and allocated[cid] == 0:
            seen.add(cid)
            zero.append(cid)
    return zero
```

DRY RUN

```
inventory = 5
requests = [
    (101, 3, 10, 1),    # cid 101, wants 3, bid 10, ts 1
    (102, 2, 10, 2),    # cid 102, wants 2, bid 10, ts 2
    (103, 4, 20, 3),    # cid 103, wants 4, bid 20, ts 3
    (104, 1,  5, 4),    # cid 104, wants 1, bid  5, ts 4
]
```

Grouping by bid: `{20: [103], 10: [101, 102], 5: [104]}` **Descending tiers:** `20`, then `10`, then `5`.

Tier bid = 20 (highest). Queue: `[(103, 4)]`

round step	pop	allocated[103]	inventory	requeue?
1	103	1	4	yes (1 < 4)
2	103	2	3	yes
3	103	3	2	yes
4	103	4	**1**	no — fulfilled, dropped

Queue empty; tier done. `inventory = 1`.

Tier bid = 10. Sorted by timestamp: `101 (ts 1)`, `102 (ts 2)`. Queue: `[(101,3), (102,2)]`

step	pop	allocated	inventory	requeue?
1	101	`101 → 1`	**0**	yes, but inventory is now 0

Loop exits on `inventory > 0` failing. `102` never got a unit.

Tier bid = 5. `inventory <= 0` → `break`. `104` never served.

Final allocations: `{101: 1, 102: 0, 103: 4, 104: 0}`

Zero-allocation customers in input order: `102` (index 1), `104` (index 3) → **`[102, 104]`**. ✅

Why the trap matters, made concrete. Suppose the tier-10 group had been served "top customer takes everything" instead of round-robin. With `inventory = 1` there'd be no visible difference. But raise inventory to `7`: correct round-robin gives `103` its 4, then alternates `101, 102, 101` → `101` gets 2, `102` gets 1, and **nobody** is at zero. The greedy "biggest first within a tier" would give `101` all 3 of its units, then `102` none — producing `[102, 104]` instead of `[104]`. That single input difference is what the question is built to detect, so run it as your second dry-run case out loud.

Minimum Possible Cost — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

There are `n` item types. The `j`-th unit bought of type `i` costs `a[i] + (j − 1) · b[i]`. Buy exactly `m` items in total, minimising the cost.

- Prices **escalate per type**: the first unit of type `i` costs `a[i]`, the second `a[i] + b[i]`, the third `a[i] + 2b[i]`, and so on. It's an arithmetic progression per type.
- You may buy any mix, including all `m` from one type.
- **Insight:** greedy with a min-heap — always buy the globally cheapest *next* unit, then push that type's new next price.
- Edge cases: `m = 0` → cost 0; `n = 1` (buy all `m` from it, cost is a closed-form arithmetic sum); `b[i] = 0` (price never escalates).
- Clarify: *"Can `b[i]` be negative (prices falling)?"* If so the greedy still works but you'd want to confirm no unbounded negative cost.

BRUTE-FORCE APPROACH

Idea. Enumerate every way to split `m` purchases across `n` types and evaluate the cost of each split.

Data structures. Recursion over `(type index, remaining purchases)`.

Complexity. The number of compositions of `m` into `n` non-negative parts is `C(m+n−1, n−1)` — combinatorial explosion. Even memoised as a DP over `(i, remaining)` it's `O(n · m)` states with `O(m)` transitions → `O(n · m²)`.

Why it's not ideal. With `m = 10⁵` the DP is 10^{10} operations. And it's over-general: the cost structure has a property the DP ignores — **each type's marginal price is non-decreasing** (assuming `b[i] ≥ 0`), which makes the problem an instance of "repeatedly take the globally cheapest marginal unit". That's a classic greedy, provable by an exchange argument.

Greedy correctness. Suppose an optimal solution doesn't take the globally cheapest available next unit at some step. Since each type's prices are non-decreasing, swapping one purchase for that cheaper unit does not increase the total and keeps the solution feasible. Repeating the swap converts any optimum into the greedy solution without raising the cost — so the greedy is optimal.

OPTIMAL APPROACH

Idea (min-heap of next prices).

1. Push each type's *first* unit price `a[i]` with its type index and a purchase counter.
2. Repeat `m` times: pop the cheapest `(price, i, j)`, add `price` to the total, and push that type's **next** price `a[i] + j · b[i]`.

The heap always holds exactly `n` entries — one "next price" per type — so each of the `m` purchases costs `O(log n)`.

Data structures. A min-heap of `(next_price, type_index, units_bought_so_far)`.

Complexity. Time `O(n + m log n)`. Space `O(n)`.

Scaling follow-up: when `m` is enormous (10^9), even `O(m log n)` is too slow. Then **binary search on the marginal price threshold `p`**: for a candidate `p`, each type contributes `floor((p − a[i]) / b[i]) + 1` units (clamped at 0), which is `O(n)` to evaluate; binary search `p` until the total unit count reaches `m`, then compute the cost in closed form with arithmetic-series sums. That's `O(n log(max_price))` — mention it; it's the difference between a good answer and an excellent one.

PYTHON CODE

```python
import heapq


def minimum_possible_cost(a: list[int], b: list[int], m: int) -> int:
    """
    The j-th unit of type i costs a[i] + (j-1)*b[i]. Buy exactly m units total,
    minimising the sum.

    Greedy: always take the globally cheapest NEXT unit. Correct by an exchange
```

```
    argument — each type's marginal prices are non-decreasing, so swapping any
    purchase for a currently-cheaper available unit never raises the total.
    """
    if m <= 0:
        return 0

    n = len(a)
    # (next price for this type, type index, units already bought from it)
    heap = [(a[i], i, 0) for i in range(n)]
    heapq.heapify(heap)                         # O(n), not O(n log n)

    total = 0
    for _ in range(m):
        price, i, bought = heapq.heappop(heap)
        total += price

        # Push this type's NEW next price. The heap therefore always holds
        # exactly one entry per type — n entries — so each purchase is O(log n).
        bought += 1
        heapq.heappush(heap, (a[i] + bought * b[i], i, bought))

    return total


def minimum_possible_cost_large_m(a: list[int], b: list[int], m: int) -> int:
    """
    Follow-up for huge m (say 1e9), where O(m log n) is too slow.
    Binary search the marginal PRICE THRESHOLD p: how many units cost <= p?
    Each type contributes floor((p - a[i]) / b[i]) + 1 units (0 if a[i] > p),
    so counting is O(n) and the search is O(n log(max_price)).
    Cost is then closed-form via arithmetic series.
    """
    n = len(a)

    def units_at_or_below(p: int) -> int:
        total_units = 0
        for i in range(n):
            if a[i] > p:
                continue
            if b[i] == 0:
                return float('inf')      # unlimited units at price a[i]
            total_units += (p - a[i]) // b[i] + 1
        return total_units

    lo, hi = min(a), max(a) + max(max(b), 1) * m
    while lo < hi:
        mid = (lo + hi) // 2
        if units_at_or_below(mid) < m:
            lo = mid + 1
        else:
            hi = mid
    threshold = lo

    # Take everything strictly below the threshold, then top up at the threshold.
    total_cost = 0
    taken = 0
    for i in range(n):
        if a[i] > threshold - 1:
            continue
        k = (threshold - 1 - a[i]) // b[i] + 1 if b[i] else 0
        # Arithmetic series: k terms starting at a[i] with step b[i]
        total_cost += k * a[i] + b[i] * (k * (k - 1) // 2)
        taken += k

    total_cost += (m - taken) * threshold     # remaining units all cost `threshold`
    return total_cost
```

DRY RUN

`a = [1, 5]`, `b = [2, 1]`, `m = 4`

Price schedules: - Type 0: `1, 3, 5, 7, 9, …` (start 1, step 2) - Type 1: `5, 6, 7, 8, …` (start 5, step 1)

Initial heap: `[(1, 0, 0), (5, 1, 0)]`

purchase	pop	price added	total	push next	heap after
1	`(1, 0, 0)`	1	**1**	`(1 + 1·2, 0, 1) = (3,0,1)`	`{(3,0,1), (5,1,0)}`
2	`(3, 0, 1)`	3	**4**	`(1 + 2·2, 0, 2) = (5,0,2)`	`{(5,0,2), (5,1,0)}`
3	`(5, 0, 2)`	5	**9**	`(1 + 3·2, 0, 3) = (7,0,3)`	`{(5,1,0), (7,0,3)}`
4	`(5, 1, 0)`	5	**14**	`(5 + 1·1, 1, 1) = (6,1,1)`	`{(6,1,1), (7,0,3)}`

Answer: `14`. ✅

Verify by hand: buying 3 from type 0 (`1 + 3 + 5 = 9`) and 1 from type 1 (`5`) totals **14**. Alternatives: 4 from type 0 = `1+3+5+7 = 16` ; 2 and 2 = `(1+3) + (5+6) = 15` . So 14 is indeed minimal. ✅

The tie at purchase 3 is worth noting: both `(5,0,2)` and `(5,1,0)` cost 5. The heap broke the tie on the type index, which is arbitrary but harmless — either choice yields the same total, since both units get bought anyway at `m = 4` . If the interviewer asks whether ties matter, the answer is no for the total cost, though it changes which type ends up with more units. Volunteering that shows you've thought about determinism.

DAY 9 — Tue Aug 25 · Binary Search (incl. binary search on the answer)

One template, used everywhere today. Write it once and never improvise the boundary logic again: `python lo, hi = <lowest possible answer>, <highest possible answer> while lo < hi: # NOT <=, and hi is never mid-1 mid = lo + (hi - lo) // 2 # floors; safe from overflow in other languages if feasible(mid): hi = mid # mid might be the answer - keep it else: lo = mid + 1 # mid definitely isn't - discard it return lo # lo == hi == the smallest feasible value` The invariant is **"the answer is always inside `[lo, hi]`"**, and the loop shrinks the interval without ever excluding it. Because `hi = mid` (not `mid - 1`), it terminates on `lo == hi` and needs no post-loop check.

Binary search on the answer (875, 1011, 410, 1552, Inconvenience, Optimal Packing) is the day's real subject. The recognition trigger: *"minimise the maximum X"*, *"maximise the minimum X"*, or *"smallest capacity/speed/size that fits within a budget"*. Whenever the answer is a number in a known range **and** checking a candidate is cheap **and** feasibility is **monotone** — if `k` works then `k+1` works — you binary search the answer rather than computing it. Say those three conditions out loud; they're the proof the technique applies.

---

704. Binary Search

UNDERSTAND THE PROBLEM

Given a sorted array and a target, return its index or `-1`, in `O(log n)`.

- All values distinct (LeetCode guarantee) — so there's exactly one answer or none.
- Edge cases: empty array; target smaller than everything or larger than everything; single element.

BRUTE-FORCE APPROACH

Idea. Scan left to right until the target is found.

Complexity. Time `O(n)`, space `O(1)`.

Why it's not ideal. It ignores sortedness entirely. Sortedness means one comparison eliminates **half** the remaining candidates, and the problem explicitly demands `O(log n)`.

OPTIMAL APPROACH

Idea. Maintain `[lo, hi]` as the range that could contain the target. Compare against the midpoint and discard half.

Here — unlike the template above — we want an **exact match**, so the `lo <= hi` / `hi = mid - 1` form is the natural fit, because there is a "not found" outcome to report.

Data structures. Two integer pointers.

Algorithm.

1. `lo, hi = 0, n-1`.
2. While `lo <= hi`: `mid = lo + (hi-lo)//2`; compare; return, or move `lo = mid+1` / `hi = mid-1`.
3. Return `-1`.

Complexity. Time `O(log n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        lo, hi = 0, len(nums) - 1

        # `<=` because lo == hi is still a valid one-element range to examine.
        # (The `lo < hi` template on the day header is for finding a BOUNDARY,
        #  where there is always an answer. Here there may be none, so we need
        #  the loop to be able to exit with lo > hi.)
        while lo <= hi:
            # lo + (hi-lo)//2 rather than (lo+hi)//2: identical in Python, but
            # the latter overflows in Java/C++ on large indices. Write the safe
            # form by habit - interviewers notice.
            mid = lo + (hi - lo) // 2

            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                lo = mid + 1        # target is strictly right of mid
            else:
                hi = mid - 1        # target is strictly left of mid

        return -1
```

DRY RUN

`nums = [-1, 0, 3, 5, 9, 12]`, `target = 9`

iter	lo	hi	mid	`nums[mid]`	comparison	action
1	0	5	2	3	`3 < 9`	`lo = 3`
2	3	5	4	9	**equal**	**return 4**

Answer: `4`. ✅

Not-found case, `target = 2`:

iter	lo	hi	mid	`nums[mid]`	action
1	0	5	2	3	`3 > 2` → `hi = 1`
2	0	1	0	−1	`-1 < 2` → `lo = 1`
3	1	1	1	0	`0 < 2` → `lo = 2`
exit	2	1			`lo > hi` → **return −1**

✅ Three comparisons for six elements — $\log_2 6 \approx 2.6$, as expected.

34. Find First and Last Position of Element in Sorted Array

UNDERSTAND THE PROBLEM

Given a sorted array **with duplicates** and a target, return the first and last index of the target, or `[-1, -1]`. `O(log n)`.

- Two separate boundary searches, not one. This is where the `lo < hi` / `hi = mid` template earns its keep.
- Edge cases: target absent; target appears once; the entire array is the target; target at index 0 or `n-1`.

BRUTE-FORCE APPROACH

Idea. Binary search for any occurrence, then linearly expand left and right to the run's edges.

Complexity. Time `O(log n + k)` where `k` is the number of occurrences — which degrades to `O(n)` when the whole array is the target. Space `O(1)`.

Why it's not ideal. The linear expansion breaks the `O(log n)` requirement on exactly the input the problem is designed around (many duplicates). Each boundary is itself a binary-searchable property: "is `nums[i] >= target`" is monotone, so `lower_bound` finds the first occurrence in `O(log n)`; likewise `upper_bound` for the last.

OPTIMAL APPROACH

Idea (two boundary searches — `lower_bound` and `upper_bound`).

- `lower_bound(target)` = the first index `i` with `nums[i] >= target`. That's the first occurrence, if the target exists.
- `upper_bound(target)` = the first index with `nums[i] > target`. The last occurrence is `upper_bound - 1`.

Both use the day's template verbatim, differing only in the comparison operator — which is the whole point of memorising it as a primitive.

Data structures. Two pointers.

Algorithm.

1. `first = lower_bound(target)`.
2. If `first == n` or `nums[first] != target`: return `[-1, -1]`.
3. `last = upper_bound(target) - 1`.
4. Return `[first, last]`.

Complexity. Time `O(log n)` — two searches. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def searchRange(self, nums: list[int], target: int) -> list[int]:
        def lower_bound(t: int) -> int:
            """First index i with nums[i] >= t. Returns len(nums) if none."""
            lo, hi = 0, len(nums)          # hi = n, not n-1: the answer may be
            while lo < hi:                 # "past the end", which is a valid result
                mid = lo + (hi - lo) // 2
                if nums[mid] >= t:
                    hi = mid               # mid may BE the boundary — keep it
                else:
                    lo = mid + 1           # mid is too small — discard it
            return lo

        def upper_bound(t: int) -> int:
            """First index i with nums[i] > t. The ONLY change is >= becoming >."""
            lo, hi = 0, len(nums)
            while lo < hi:
                mid = lo + (hi - lo) // 2
                if nums[mid] > t:
                    hi = mid
                else:
                    lo = mid + 1
            return lo

        first = lower_bound(target)
        # Two ways the target can be absent: the boundary ran off the end, or it
        # landed on a larger value. Check BOTH — `first == len(nums)` must come
        # first or the indexing raises.
        if first == len(nums) or nums[first] != target:
            return [-1, -1]

        return [first, upper_bound(target) - 1]
```

DRY RUN

`nums = [5, 7, 7, 8, 8, 10]`, `target = 8`

`lower_bound(8)`:

iter	lo	hi	mid	`nums[mid]`	`>= 8`?	action
1	0	6	3	8	yes	`hi = 3`
2	0	3	1	7	no	`lo = 2`
3	2	3	2	7	no	`lo = 3`
exit	3	3				return **3**

`nums[3] == 8` ✓ so the target exists.

`upper_bound(8)`:

iter	lo	hi	mid	`nums[mid]`	`> 8`?	action
1	0	6	3	8	no	`lo = 4`
2	4	6	5	10	yes	`hi = 5`
3	4	5	4	8	no	`lo = 5`
exit	5	5				return **5**

Last occurrence = `5 - 1 = 4`.

Answer: `[3, 4]`. ✅ (`nums[3] = nums[4] = 8`.)

Absent case, `target = 6`: `lower_bound(6)` returns `1` (first value `≥ 6` is `nums[1] = 7`), and `nums[1] = 7 ≠ 6` → `[-1, -1]`. ✅

All-duplicates case, `nums = [8,8,8]`, `target = 8`: `lower_bound = 0`, `upper_bound = 3`, answer `[0, 2]` — in `O(log n)`, whereas the linear-expansion brute force would have scanned all three. On `n = 10⁵` identical values that's the difference between 17 steps and 100,000.

33. Search in Rotated Sorted Array ★★

UNDERSTAND THE PROBLEM

A sorted array of **distinct** values was rotated at an unknown pivot. Given a target, return its index or `-1`, in `O(log n)`.

- Rotated means `[0,1,2,4,5,6,7]` became e.g. `[4,5,6,7,0,1,2]`.
- The array isn't globally sorted, so a plain binary search comparison is meaningless — but **at least one half of any split is always sorted**, and that's the whole insight.
- Edge cases: no rotation (`[1,2,3]`); rotation by 1; single element; target absent.

BRUTE-FORCE APPROACH

Idea. Linear scan for the target.

Complexity. Time `O(n)`, space `O(1)`.

Why it's not ideal. The problem demands `O(log n)`. A tempting two-pass fix — find the pivot by binary search, then binary search the correct segment — is `O(log n)` and perfectly valid; offer it if the one-pass version feels risky. The one-pass version is tighter and is the expected answer.

OPTIMAL APPROACH

Idea (one half is always sorted). At each step, split at `mid`. Compare `nums[lo]` with `nums[mid]`: - If `nums[lo] <= nums[mid]`, the **left half is sorted**. Check whether the target lies inside `[nums[lo], nums[mid])`; if so search left, else search right. - Otherwise the **right half is sorted**. Check whether the target lies inside `(nums[mid], nums[hi]]`; if so search right, else search left.

Every iteration still halves the range, so it stays `O(log n)`. The only skill is getting the two range tests' inclusivity right — use `<=` on the sorted end that includes an already-checked element carefully, and verify on a two-element example.

Data structures. Two pointers.

Complexity. Time `O(log n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        lo, hi = 0, len(nums) - 1

        while lo <= hi:
            mid = lo + (hi - lo) // 2

            if nums[mid] == target:
                return mid

            # KEY FACT: a rotated sorted array split anywhere always has at least
            # one SORTED half. Identify which, then use ordinary range logic on it.
            if nums[lo] <= nums[mid]:
                # LEFT half [lo, mid] is sorted.
                # `<=` (not `<`) matters when lo == mid: a single element counts
                # as sorted, and without the equality a 2-element array misroutes.
                if nums[lo] <= target < nums[mid]:
                    hi = mid - 1        # target is inside the sorted left half
                else:
                    lo = mid + 1        # must be in the (possibly rotated) right
            else:
                # RIGHT half [mid, hi] is sorted.
                if nums[mid] < target <= nums[hi]:
                    lo = mid + 1
                else:
                    hi = mid - 1

        return -1
```

DRY RUN

`nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 0`

iter	lo	hi	mid	`nums[mid]`	which half sorted?	target in it?	action
1	0	6	3	7	`nums[0]=4 <= 7` → **left** `[4..7]`	`4 <= 0 < 7` ? **no**	`lo = 4`
2	4	6	5	1	`nums[4]=0 <= 1` → **left** `[0..1]`	`0 <= 0 < 1` ? **yes**	`hi = 4`
3	4	4	4	0	—	**equal → return 4**	

Answer: `4`. ✅

Target absent, `target = 3`:

iter	lo	hi	mid	`nums[mid]`	sorted half	in range?	action
1	0	6	3	7	left `[4,7]`	`4 <= 3` ? no	`lo = 4`
2	4	6	5	1	left `[0,1]`	`0 <= 3 < 1` ? no	`lo = 6`
3	6	6	6	2	left `[2,2]`	`2 <= 3 < 2` ? no	`lo = 7`
exit	7	6					**return −1** ✅

Un-rotated case, `nums = [1,2,3]`, `target = 3`: iteration 1 has `mid = 1`, `nums[0]=1 <= 2` → left sorted `[1,2]`, `1 <= 3 < 2` ? no → `lo = 2`; then `mid = 2`, `nums[2] = 3` → return `2`. ✅ The algorithm degenerates gracefully to ordinary binary search when the rotation is zero.

---

153. Find Minimum in Rotated Sorted Array

UNDERSTAND THE PROBLEM

A sorted array of **distinct** values, rotated at an unknown pivot. Return the minimum, in `O(log n)`.

- The minimum is exactly the rotation point — the single place where the array "drops".
- Edge cases: no rotation (the minimum is `nums[0]`); single element; rotation by `n-1`.

BRUTE-FORCE APPROACH

Idea. `min(nums)` or a linear scan for the first descent.

Complexity. Time `O(n)`, space `O(1)`.

Why it's not ideal. `O(log n)` is required. The property that makes binary search work: comparing `nums[mid]` against `nums[hi]` tells you which side the drop is on — and that comparison is monotone in exactly the way binary search needs.

OPTIMAL APPROACH

Idea (compare against `hi`, not `lo`). The critical design choice: - If `nums[mid] > nums[hi]`, the drop is strictly **right** of `mid`, so the minimum is in `(mid, hi]` → `lo = mid + 1`. - Otherwise `nums[mid] <= nums[hi]`, meaning `[mid, hi]` is sorted, so the minimum is at `mid` or left of it → `hi = mid`.

Why compare against `hi` and not `lo`: with `nums[lo]` the un-rotated case is ambiguous. Take `[1,2,3]`: `nums[mid]=2 > nums[lo]=1`, which under a "left is sorted" reading would push you right — away from the actual minimum at index 0. Comparing against `hi` has no such blind spot. Say this out loud; it's the single question asked about this problem.

Data structures. Two pointers.

Complexity. Time `O(log n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def findMin(self, nums: list[int]) -> int:
        lo, hi = 0, len(nums) - 1

        # Template form: `lo < hi`, and hi = mid (never mid-1), so we converge
        # on the answer index without a post-loop check.
        while lo < hi:
            mid = lo + (hi - lo) // 2

            # COMPARE AGAINST hi, NOT lo. Against lo, the un-rotated array [1,2,3]
            # gives nums[mid]=2 > nums[lo]=1 and a naive reading sends you RIGHT,
            # away from the minimum at index 0. Against hi there is no such case.
            if nums[mid] > nums[hi]:
                # The drop is strictly to the right, so mid cannot be the minimum.
                lo = mid + 1
            else:
                # [mid, hi] is sorted, so the minimum is mid or further left.
                # KEEP mid - it might be the answer.
                hi = mid

        return nums[lo]      # lo == hi == the index of the minimum
```

DRY RUN

`nums = [4, 5, 6, 7, 0, 1, 2]`

iter	lo	hi	mid	nums[mid]	nums[hi]	mid > hi ?	action
1	0	6	3	7	2	yes	`lo = 4`
2	4	6	5	1	2	no	`hi = 5`
3	4	5	4	0	1	no	`hi = 4`
exit	4	4					return `nums[4] = 0` ✅

Un-rotated case, `nums = [1, 2, 3]` — the case that breaks the `lo` comparison:

iter	lo	hi	mid	`nums[mid]`	`nums[hi]`	`mid > hi ?`	action
1	0	2	1	2	3	no	`hi = 1`
2	0	1	0	1	2	no	`hi = 0`
exit	0	0					return `nums[0] = 1` ✅

Compare with the broken `lo`-based version at iteration 1: `nums[mid]=2 > nums[lo]=1` would set `lo = 2`, converge on index 2, and return `3` — wrong. Running exactly this two-case dry run in the interview is what proves you chose the comparison deliberately rather than by luck.

---

162. Find Peak Element

UNDERSTAND THE PROBLEM

Given an array where no two adjacent values are equal, return the index of **any** element strictly greater than both neighbours. Out-of-bounds counts as `-∞`. `O(log n)`.

- "Any peak" — multiple may exist, and returning any is fine.
- The `-∞` boundary convention guarantees a peak always exists: if the array is strictly increasing, the last element is a peak.
- Edge cases: single element (it's a peak); strictly increasing; strictly decreasing; a peak at index 0 or `n-1`.

BRUTE-FORCE APPROACH

Idea. Scan for the first index greater than both neighbours.

Complexity. Time `O(n)`, space `O(1)`.

Why it's not ideal. `O(log n)` is required. The surprising part is that binary search applies at all on an **unsorted** array — the enabling property isn't sortedness but the guarantee that **following the ascent always leads to a peak**.

OPTIMAL APPROACH

Idea (follow the ascent). Compare `nums[mid]` with `nums[mid+1]`: - If `nums[mid] < nums[mid+1]`, we're on an **upslope**. A peak must exist somewhere to the right — either the values keep rising until the boundary (where `-∞` makes the last element a peak) or they turn over. Either way → `lo = mid + 1`. - Otherwise we're on a **downslope or at a peak**. By the same argument a peak exists at `mid` or to its left → `hi = mid`.

This is a valid binary search because the predicate "a peak exists in `[lo, hi]`" is preserved as an invariant, even though the array has no global order.

Data structures. Two pointers.

Complexity. Time `O(log n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def findPeakElement(self, nums: list[int]) -> int:
        lo, hi = 0, len(nums) - 1

        while lo < hi:
            mid = lo + (hi - lo) // 2
            # mid+1 is always in bounds here: lo < hi means mid < hi, so mid+1 <= hi.

            if nums[mid] < nums[mid + 1]:
                # UPSLOPE. A peak must lie to the right: either the values keep
                # rising to the end (where the -inf boundary makes the last
                # element a peak) or they turn over somewhere. Discard [lo, mid].
                lo = mid + 1
            else:
                # DOWNSLOPE (or mid is itself a peak). Symmetric argument:
                # a peak exists at mid or to its left. KEEP mid.
                hi = mid

        # Note this works on an UNSORTED array — the enabling property is not
        # sortedness but "following the ascent always terminates at a peak".
        return lo
```

DRY RUN

`nums = [1, 2, 1, 3, 5, 6, 4]` — peaks exist at index 1 (value 2) and index 5 (value 6).

iter	lo	hi	mid	nums[mid]	nums[mid+1]	upslope?	action
1	0	6	3	3	5	yes	`lo = 4`
2	4	6	5	6	4	no	`hi = 5`
3	4	5	4	5	6	yes	`lo = 5`
exit	5	5					return **5**

Answer: `5` (`nums[5] = 6`, greater than `nums[4] = 5` and `nums[6] = 4`). ✅

Note it found the peak at index 5, not the one at index 1 — both are correct, and the problem explicitly allows either.

Strictly increasing case, `nums = [1, 2, 3]`:

iter	lo	hi	mid	comparison	action
1	0	2	1	`2 < 3` upslope	`lo = 2`
exit	2	2			return **2** ✅

`nums[2] = 3` is a peak because its right neighbour is out of bounds, i.e. `−∞`. That boundary convention is why the algorithm never fails to find something.

---

540. Single Element in a Sorted Array ★

UNDERSTAND THE PROBLEM

A sorted array where every value appears **exactly twice** except one, which appears once. Return the single value in `O(log n)` time and `O(1)` space.

- Sorted, so the pairs are adjacent.
- Edge cases: the single element is first; last; the array has one element.
- The `O(log n)` requirement rules out the elegant XOR-everything trick (`O(n)`), which you should still mention as the answer if the array *weren't* sorted.

BRUTE-FORCE APPROACH

Idea (a). XOR all elements — pairs cancel, leaving the single one. **Idea (b).** Scan in steps of 2 checking `nums[i] == nums[i+1]`.

Complexity. Both `O(n)` time, `O(1)` space.

Why it's not ideal. Both are linear and the problem demands logarithmic. XOR is the right answer for an *unsorted* array — say so, then explain that sortedness gives a stronger structural property: **index parity**.

OPTIMAL APPROACH

Idea (parity binary search). Before the single element, pairs start at **even** indices: `(0,1), (2,3), (4,5), …`. After it, pairs start at **odd** indices — everything has shifted by one.

So: force `mid` to be **even** (`mid -= mid & 1`). Then - if `nums[mid] == nums[mid+1]`, the pairing is still intact at `mid`, so the single element is to the **right** → `lo = mid + 2`; - otherwise the pairing has already broken, so the single element is at `mid` or to its **left** → `hi = mid`.

Data structures. Two pointers.

Complexity. Time `O(log n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def singleNonDuplicate(self, nums: list[int]) -> int:
        lo, hi = 0, len(nums) - 1

        while lo < hi:
            mid = lo + (hi - lo) // 2

            # Force mid EVEN. Before the single element, every pair starts at an
            # even index: (0,1), (2,3), ... After it, everything shifts by one and
            # pairs start at ODD indices. Anchoring on even indices turns that
            # shift into a clean binary-search predicate.
            # `mid & 1` is 1 when odd, so this subtracts at most 1.
            mid -= mid & 1

            if nums[mid] == nums[mid + 1]:
                # Pairing still intact here => the break is further right.
                # Skip TWO (past the whole pair), not one.
                lo = mid + 2
            else:
                # Pairing already broken => the single element is at mid or left.
                hi = mid

        return nums[lo]        # lo == hi == the single element's index


class SolutionXor:
    """O(n). The right answer if the array were NOT sorted — worth naming, then
    discarding because the problem demands O(log n)."""
    def singleNonDuplicate(self, nums: list[int]) -> int:
        result = 0
        for x in nums:
            result ^= x        # a ^ a == 0, so every pair cancels
        return result
```

DRY RUN

`nums = [1, 1, 2, 3, 3, 4, 4, 8, 8]` — the single element is `2` at index 2.

iter	lo	hi	raw mid	even mid	`nums[mid]`	`nums[mid+1]`	equal?	action
1	0	8	4	4	3	4	**no**	`hi = 4`
2	0	4	2	2	2	3	**no**	`hi = 2`
3	0	2	1	**0** (forced even)	1	1	**yes**	`lo = 0 + 2 = 2`
exit	2	2						return `nums[2] = 2` ✅

Iteration 3 is where the parity forcing shows its value: raw `mid = 1` is odd, and testing `nums[1] == nums[2]` (`1` vs `2`) would say "not equal" and set `hi = 1`, converging on index 1 and returning `1` — wrong. Snapping to index 0 correctly tests the intact pair `(0,1)` and pushes `lo` past it.

Single element at the end, `nums = [1,1,2,2,3]`:

iter	lo	hi	mid (even)	`nums[mid]` vs `nums[mid+1]`	action
1	0	4	2	`2 == 2` yes	`lo = 4`
exit	4	4			return `nums[4] = 3` ✅

74. Search a 2D Matrix

UNDERSTAND THE PROBLEM

A matrix where each row is sorted and **each row's first value exceeds the previous row's last**. Return whether a target exists, in `O(log(m·n))`.

- That second condition is the key: the matrix, read row by row, is one **globally sorted** sequence. So it can be treated as a flat sorted array.
- Contrast with problem 240 (next), where rows and columns are sorted but *not* globally — a distinction interviewers deliberately test back to back.
- Edge cases: empty matrix; single cell; target smaller/larger than everything.

BRUTE-FORCE APPROACH

Idea. Scan every cell, or binary search each row.

Complexity. Full scan `O(m·n)`; per-row binary search `O(m log n)`.

Why it's not ideal. Neither meets `O(log(mn))`. The per-row version is closer but still does `m` searches when the global ordering means **one** search suffices.

OPTIMAL APPROACH

Idea (treat it as a flat array). Index `i` in the virtual flat array of length `m·n` maps to `matrix[i // n][i % n]`. Run one ordinary binary search over `[0, m·n − 1]` using that mapping.

Data structures. Two integer pointers; integer division/modulo for the mapping.

Complexity. Time `O(log(m·n))`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        rows, cols = len(matrix), len(matrix[0])
        lo, hi = 0, rows * cols - 1

        # The guarantee "each row's first value > previous row's last" means the
        # matrix read row-by-row is ONE globally sorted sequence. So we can binary
        # search a VIRTUAL flat array and translate indices on the fly — no
        # flattening, no extra memory.
        while lo <= hi:
            mid = lo + (hi - lo) // 2
            value = matrix[mid // cols][mid % cols]     # flat index -> (row, col)

            if value == target:
                return True
            elif value < target:
                lo = mid + 1
            else:
                hi = mid - 1

        return False
```

DRY RUN

```
matrix = [[ 1,  3,  5,  7],
          [10, 11, 16, 20],
          [23, 30, 34, 60]]
target = 3
```

`rows = 3`, `cols = 4`, flat range `[0, 11]`.

iter	lo	hi	mid	`mid//4, mid%4`	value	compare	action
1	0	11	5	(1, 1)	11	`11 > 3`	`hi = 4`
2	0	4	2	(0, 2)	5	`5 > 3`	`hi = 1`
3	0	1	0	(0, 0)	1	`1 < 3`	`lo = 1`
4	1	1	1	(0, 1)	3	**equal**	**return True** ✅

Four comparisons for 12 cells — `log₂ 12 ≈ 3.6`. ✅

Absent case, `target = 13`: the search narrows to flat indices around 6–7 (`16` and `20`), never matches, and exits with `lo > hi` → `False`. ✅

---

240. Search a 2D Matrix II

UNDERSTAND THE PROBLEM

A matrix where each **row** and each **column** is sorted ascending, but rows are **not** globally ordered. Return whether a target exists.

- The difference from 74 is decisive: `matrix[0][3]` may exceed `matrix[1][0]`, so the flat-array trick is invalid.
- Edge cases: empty matrix; target smaller than `matrix[0][0]` or larger than the bottom-right; a non-square matrix.

BRUTE-FORCE APPROACH

Idea. Binary search each row independently.

Complexity. Time `O(m log n)`. Space `O(1)`.

Why it's not ideal. It's a genuine improvement over the `O(mn)` full scan and is acceptable — say so. But it discards the **column** ordering entirely, using only half the given structure. The staircase walk uses both and achieves `O(m + n)`, which beats `O(m log n)` whenever `n` is not tiny.

OPTIMAL APPROACH

Idea (staircase from a corner). Start at the **top-right** corner. That position is special: it is the **largest in its row** and the **smallest in its column**. So one comparison eliminates an entire row or an entire column: - `value == target` → found. - `value > target` → the target can't be anywhere in this **column** (everything below is larger) → move **left**. - `value < target` → the target can't be anywhere in this **row** (everything to the left is smaller) → move **down**.

Each step retires one full row or column, so at most `m + n` steps. Bottom-left works symmetrically; top-left and bottom-right do **not** — from the top-left both directions increase, so no elimination is possible. Say that; it's the follow-up.

Data structures. Two indices.

Complexity. Time `O(m + n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        rows, cols = len(matrix), len(matrix[0])

        # START AT THE TOP-RIGHT. That corner is simultaneously the LARGEST in its
        # row and the SMALLEST in its column, so a single comparison rules out an
        # entire row or an entire column.
        # Top-LEFT would not work: from there both moves increase, so no direction
        # eliminates anything. Bottom-left is the valid mirror image.
        row, col = 0, cols - 1

        while row < rows and col >= 0:
            value = matrix[row][col]

            if value == target:
                return True
            elif value > target:
                col -= 1        # too big: everything BELOW in this column is bigger
                                # still -> discard the whole column
            else:
                row += 1        # too small: everything LEFT in this row is smaller
                                # still -> discard the whole row

        return False
```

DRY RUN

```
matrix = [[ 1,  4,  7, 11, 15],
          [ 2,  5,  8, 12, 19],
          [ 3,  6,  9, 16, 22],
          [10, 13, 14, 17, 24],
          [18, 21, 23, 26, 30]]
target = 5
```

Start at `(0, 4) = 15`.

step	(row, col)	value	compare to 5	action	eliminated
1	(0,4)	15	`>`	`col = 3`	column 4
2	(0,3)	11	`>`	`col = 2`	column 3
3	(0,2)	7	`>`	`col = 1`	column 2
4	(0,1)	4	`<`	`row = 1`	row 0
5	(1,1)	5	=	**return True** ✅	

Five steps to search a 25-cell matrix. ✅

Absent case, `target = 20`:

step	(row,col)	value	action
1	(0,4)	15	`<` → row 1
2	(1,4)	19	`<` → row 2
3	(2,4)	22	`>` → col 3
4	(2,3)	16	`<` → row 3
5	(3,3)	17	`<` → row 4
6	(4,3)	26	`>` → col 2
7	(4,2)	23	`>` → col 1
8	(4,1)	21	`>` → col 0
9	(4,0)	18	`<` → row 5 → **out of bounds, return False** ✅

Nine steps, bounded by `m + n = 10`. ✅ Note the walk never revisits a row or column — that monotone retirement is the complexity proof.

---

875. Koko Eating Bananas ★

UNDERSTAND THE PROBLEM

Given banana piles and `h` hours, Koko eats at speed `k` bananas per hour, taking from **one pile at a time**; if a pile has fewer than `k` left, she finishes it and **wastes the rest of that hour**. Return the minimum `k` that finishes all piles within `h` hours.

- The wasted-hour rule means the time for a pile is `ceil(pile / k)`, not `pile / k`.
- **This is the archetype of binary search on the answer.** Check the three conditions out loud: (1) the answer is an integer in `[1, max(piles)]`; (2) checking a candidate `k` is `O(n)`; (3) feasibility is **monotone** — if speed `k` finishes in time, so does any speed `> k`.
- Edge cases: `h == len(piles)` (must eat each pile in one hour → `k = max(piles)`); `h` very large (→ `k = 1`); a single pile.

BRUTE-FORCE APPROACH

Idea. Try `k = 1, 2, 3, …` until one finishes within `h`.

Complexity. Time `O(max(piles) · n)` — up to `10⁹ · 10⁴`. Space `O(1)`.

Why it's not ideal. The candidate range is up to `10⁹`, so a linear scan over it is hopeless. But the scan reveals the structure that fixes it: the feasibility test flips from `False` to `True` exactly once as `k` increases and never flips back. A monotone predicate over an ordered range is precisely what binary search consumes.

OPTIMAL APPROACH

Idea (binary search on `k`).

1. Range: `lo = 1` (slowest meaningful speed), `hi = max(piles)` (fast enough to clear any pile in one hour, so always feasible).
2. `feasible(k)`: `sum(ceil(pile / k) for pile in piles) <= h`. Compute the ceiling as `(pile + k - 1) // k` or `-(-pile // k)` to stay in integers.
3. Apply the template: `feasible(mid)` → `hi = mid`, else `lo = mid + 1`.
4. Return `lo`.

Data structures. None.

Complexity. Time `O(n · log(max(piles)))`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def hours_needed(k: int) -> int:
            """Hours to clear every pile at speed k.
            ceil(pile / k) because a partially-eaten hour is still a whole hour —
            the 'wasted remainder' rule. `-(-p // k)` is integer ceiling division,
            avoiding floats entirely."""
            return sum(-(-pile // k) for pile in piles)

        # Range bounds, both justified:
        #   lo = 1            -> the slowest speed that makes any progress
        #   hi = max(piles)   -> guarantees every pile takes exactly 1 hour, so
        #                        hours_needed == len(piles) <= h is guaranteed
        #                        feasible (the problem promises h >= len(piles)).
        lo, hi = 1, max(piles)

        # MONOTONICITY (the reason binary search is valid): if speed k finishes
        # in time, every speed > k also does. So feasibility flips False->True
        # exactly once across the range, and we're finding that boundary.
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if hours_needed(mid) <= h:
                hi = mid            # mid works — it might be the minimum
            else:
                lo = mid + 1        # too slow — the answer is strictly larger

        return lo
```

DRY RUN

`piles = [3, 6, 7, 11]`, `h = 8`

`lo = 1`, `hi = 11`.

iter	lo	hi	mid	hours = Σ ceil(p/mid)	<= 8 ?	action
1	1	11	6	⌈3/6⌉+⌈6/6⌉+⌈7/6⌉+⌈11/6⌉ = 1+1+2+2 = 6	yes	hi = 6
2	1	6	3	1+2+3+4 = 10	no	lo = 4
3	4	6	5	1+2+2+3 = 8	yes (8 ≤ 8)	hi = 5
4	4	5	4	1+2+2+3 = 8	yes	hi = 4
exit	4	4				return **4**

Answer: `4`. ✅

Verify: at `k = 4`, hours are `⌈3/4⌉=1`, `⌈6/4⌉=2`, `⌈7/4⌉=2`, `⌈11/4⌉=3` → total 8 ≤ 8 ✓. At `k = 3`: `1+2+3+4 = 10 > 8` ✗. So 4 is minimal. ✅

Note the search made 4 feasibility checks over a range of 11 candidates. On the real constraint (`max(piles) = 10⁹`) that's ~30 checks instead of a billion — the entire point of the technique. And observe the monotonicity in the trace: results were `feasible` at 6, 5, 4, and `infeasible` at 3 — a single clean flip, exactly as the precondition requires.

1011. Capacity To Ship Packages Within D Days ★

UNDERSTAND THE PROBLEM

Given package weights that must be shipped **in order**, and `days`, return the minimum ship capacity that finishes within the deadline.

- "In order" means you may not reorder or split packages — you fill a day's ship until adding the next package would exceed capacity, then start a new day.
- Same three conditions as 875: the answer lies in `[max(weights), sum(weights)]`; checking a capacity is `O(n)`; feasibility is monotone.
- **Lower bound must be `max(weights)`**, not 1 — a capacity below the heaviest package can never ship it, so it's infeasible at any number of days. Getting this bound wrong produces an infinite loop or a wrong answer.
- Edge cases: `days == len(weights)` (→ `max(weights)`); `days == 1` (→ `sum(weights)`).

BRUTE-FORCE APPROACH

Idea. Try every capacity from `max(weights)` upward until one fits within `days`.

Complexity. Time `O(sum(weights) · n)`. Space `O(1)`.

Why it's not ideal. `sum(weights)` can be `5·10⁷`. Same monotone structure as 875 → binary search the capacity.

OPTIMAL APPROACH

Idea (binary search on capacity).

1. `lo = max(weights)`, `hi = sum(weights)`.
2. `days_needed(cap)`: greedily fill — walk the weights accumulating; when adding the next would exceed `cap`, close the day and start a new one with that package.
3. Template: `days_needed(mid) <= days` → `hi = mid`, else `lo = mid + 1`.

The greedy fill is optimal for a fixed capacity: since order is fixed, delaying a package to a later day can never reduce the day count.

Data structures. None.

Complexity. Time `O(n · log(sum − max))`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def shipWithinDays(self, weights: list[int], days: int) -> int:
        def days_needed(capacity: int) -> int:
            """Greedy fill: pack in order until the next item would overflow,
            then start a new day. Optimal for a FIXED capacity because the order
            is fixed — deferring a package can never reduce the day count."""
            d = 1
            load = 0
            for w in weights:
                if load + w > capacity:
                    d += 1              # close this day, open a new one
                    load = 0
                load += w
            return d

        # BOUNDS, and both matter:
        #   lo = max(weights) -> a smaller ship literally cannot carry the heaviest
        #                        package, so it is infeasible for ANY day count.
        #                        Starting at 1 instead is the classic bug here.
        #   hi = sum(weights) -> one giant ship, always feasible in 1 day.
        lo, hi = max(weights), sum(weights)

        while lo < hi:
            mid = lo + (hi - lo) // 2
            if days_needed(mid) <= days:
                hi = mid
            else:
                lo = mid + 1

        return lo
```

DRY RUN

`weights = [1,2,3,4,5,6,7,8,9,10]`, `days = 5`

`lo = 10`, `hi = 55`.

iter	lo	hi	mid	greedy packing	days	`<= 5` ?	action
1	10	55	32	`[1..7]=28`, `[8,9,10]=27`	2	yes	`hi = 32`
2	10	32	21	`[1..6]=21`, `[7,8]=15`, `[9,10]=19`	3	yes	`hi = 21`
3	10	21	15	`[1,2,3,4,5]=15`, `[6,7]=13`, `[8]`, `[9]` … let's compute: 1+2+3+4+5=15; +6 would be 21>15 → new day: 6+7=13, +8=21>15 → new day: 8+9=17>15 so 8 alone… careful: 8, +9=17>15 → new day: 9, +10=19>15 → new day: 10	**5**	yes	`hi = 15`
4	10	15	12	`1+2+3=6, +4=10, +5=15>12` → day1= `[1,2,3,4]` =10; day2: 5+6=11, +7=18>12 → `[5,6]`; day3: 7, +8=15>12 → `[7]`; day4: 8, +9=17>12 → `[8]`; day5: 9; day6: 10	**6**	no	`lo = 13`
5	13	15	14	day1: 1+2+3+4=10, +5=15>14 → `[1,2,3,4]`; day2: 5+6=11, +7=18>14 → `[5,6]`; day3: 7, +8=15>14 → `[7]`; day4: 8, +9=17>14→ `[8]`; day5: 9; day6: 10 → 6 days	6	no	`lo = 15`
exit	15	15					return **15**

Answer: `15`. ✅

Verify at capacity 15: `[1,2,3,4,5]` = 15, `[6,7]` = 13, `[8]`, `[9]`, `[10]` → exactly 5 days ✓. At capacity 14: 6 days ✗. So 15 is minimal. ✅

The trace shows the monotone flip cleanly: feasible at 32, 21, 15; infeasible at 14, 12. One boundary, found in 5 checks over a 46-wide range.

410. Split Array Largest Sum ★

UNDERSTAND THE PROBLEM

Given an array and `k`, split it into `k` **non-empty contiguous** subarrays. Return the **minimised largest subarray sum**.

- "Minimise the maximum" — the canonical binary-search-on-the-answer phrasing.
- **This is literally the same function as 1011**, with "days" renamed to "subarrays" and "capacity" to "largest sum". Say that out loud; recognising the isomorphism is worth more than re-deriving it.
- Edge cases: `k == 1` (→ `sum(nums)`); `k == len(nums)` (→ `max(nums)`).

BRUTE-FORCE APPROACH

Idea. DP over `(index, splits remaining)`, trying every cut position.

Data structures. A 2D DP table.

Complexity. Time `O(n² · k)`, space `O(n · k)`.

Why it's not ideal. With `n = 1000` and `k = 50` that's $5 \cdot 10^7$ — borderline, and the DP is fiddly to write correctly under time pressure. Binary search on the answer is `O(n log(sum))` — around 1000 × 30 = 30,000 operations, and it's ten lines. Mention the DP exists (it's the "principled" solution and some interviewers want it), then present binary search as strictly better here.

OPTIMAL APPROACH

Idea (binary search on the largest-sum value).

1. `lo = max(nums)` (no subarray can be smaller than its largest single element), `hi = sum(nums)` (one subarray holding everything).
2. `splits_needed(limit)`: greedily accumulate; when adding the next element would exceed `limit`, start a new subarray.
3. Template: `splits_needed(mid) <= k` → `hi = mid`, else `lo = mid + 1`.

Note the feasibility direction: **fewer** subarrays than `k` is still feasible, because you can always split further (each split can only lower the maximum). So `<= k`, not `== k`.

Data structures. None.

Complexity. Time `O(n · log(sum − max))`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        def subarrays_needed(limit: int) -> int:
            """Fewest contiguous subarrays with every sum <= limit. Greedy: extend
            the current subarray as far as possible before cutting. Optimal because
            cutting earlier can only ever require MORE subarrays."""
            count = 1
            total = 0
            for x in nums:
                if total + x > limit:
                    count += 1
                    total = 0
                total += x
            return count

        # Same bounds and same function as LC 1011 — this problem IS that problem
        # with 'days' renamed to 'subarrays'. Recognising the isomorphism is the
        # fastest route to a correct answer under time pressure.
        lo, hi = max(nums), sum(nums)

        while lo < hi:
            mid = lo + (hi - lo) // 2
            # `<= k`, not `== k`: needing FEWER subarrays is still feasible,
            # because any subarray can be split further and splitting never
            # raises the maximum.
            if subarrays_needed(mid) <= k:
                hi = mid
            else:
                lo = mid + 1

        return lo
```

DRY RUN

`nums = [7, 2, 5, 10, 8]`, `k = 2`

`lo = max = 10`, `hi = sum = 32`.

iter	lo	hi	mid	greedy split	count	`<= 2` ?	action
1	10	32	21	`7+2+5=14`, `+10=24>21` → cut; `10+8=18`	**2**	yes	`hi = 21`
2	10	21	15	`7+2+5=14`, `+10=24>15` → cut; `10`, `+8=18>15` → cut; `8`	**3**	no	`lo = 16`
3	16	21	18	`7+2+5=14`, `+10=24>18` → cut; `10+8=18`	**2**	yes	`hi = 18`
4	16	18	17	`14`, cut; `10`, `+8=18>17` → cut; `8`	**3**	no	`lo = 18`
exit	18	18					return **18**

Answer: `18`. ✅

Verify: the split `[7,2,5]` (sum 14) and `[10,8]` (sum 18) gives a maximum of 18. Any split with maximum 17 is impossible — `10` and `8` must be separated, forcing 3 subarrays. ✅

The `<= k` subtlety is visible if you take `k = 3`: at `mid = 18` the greedy uses 2 subarrays, which is `≤ 3`, so 18 is still marked feasible — correctly, since you can split `[7,2,5]` into `[7,2]` and `[5]` and the maximum stays 18. A `== k` test would reject it and return a needlessly larger answer.

1552. Magnetic Force Between Two Balls ★

UNDERSTAND THE PROBLEM

Given basket positions and `m` balls, place the balls in distinct baskets to **maximise the minimum gap** between any two balls. Return that largest possible minimum gap.

- **"Maximise the minimum"** — the *mirror* of 875/1011/410, which minimise a maximum. That flips the template's branch directions, and getting the flip right is the whole test.
- Positions must be **sorted** first — they're given unsorted.
- Edge cases: `m == 2` (→ `max - min`); `m == len(position)` (→ the smallest adjacent gap after sorting).

BRUTE-FORCE APPROACH

Idea. Try every subset of `m` baskets and compute each arrangement's minimum gap.

Complexity. Time `O(C(n, m) · m)` — combinatorial. Space `O(m)`.

Why it's not ideal. Astronomically large. But the same monotone structure applies, in reverse: if a minimum gap `d` is achievable, so is every smaller gap. So feasibility flips `True → False` as `d` grows, and we want the **largest** feasible value.

OPTIMAL APPROACH

Idea (binary search on the gap, with flipped branches).

1. Sort positions. `lo = 1` (smallest meaningful gap), `hi = position[-1] - position[0]` (the widest possible).
2. `can_place(d)`: greedily place the first ball at `position[0]`, then place the next ball at the first basket at least `d` beyond the last placed one. Count how many fit; feasible iff `count >= m`.
3. **Flipped template** — because we want the largest feasible value:
4. `can_place(mid)` → `lo = mid` (mid works; try bigger)
5. else → `hi = mid - 1`
6. and `mid` must be computed with a **ceiling** (`lo + (hi - lo + 1) // 2`) to avoid an infinite loop when `hi == lo + 1`.

That `+1` in the midpoint is the single most common bug in the maximise variant — without it, `lo = mid` can leave `lo` unchanged forever. Say it explicitly.

Data structures. The sorted array.

Complexity. Time `O(n log n + n · log(max_gap))`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def maxDistance(self, position: list[int], m: int) -> int:
        position.sort()          # gaps only make sense on sorted positions

        def can_place(d: int) -> bool:
            """Can we place m balls with every pair at least d apart?
            Greedy: always take the EARLIEST basket that satisfies the gap.
            Optimal because placing a ball later only reduces the room left."""
            count = 1
            last = position[0]           # first ball always goes leftmost
            for p in position[1:]:
                if p - last >= d:
                    count += 1
                    last = p
                    if count >= m:
                        return True      # early exit
            return count >= m

        lo, hi = 1, position[-1] - position[0]

        # MAXIMISE variant - branches are MIRRORED versus 875/1011/410:
        #   feasible -> lo = mid     (keep mid, try LARGER)
        #   else     -> hi = mid - 1 (mid fails, so does anything bigger)
        while lo < hi:
            # CEILING midpoint (+1). Without it, hi == lo + 1 gives mid == lo,
            # and `lo = mid` leaves lo unchanged -> INFINITE LOOP.
            # This one character is the classic bug in the maximise form.
            mid = lo + (hi - lo + 1) // 2

            if can_place(mid):
                lo = mid
            else:
                hi = mid - 1

        return lo
```

DRY RUN

`position = [1, 2, 3, 4, 7]`, `m = 3`

Sorted already. `lo = 1`, `hi = 7 - 1 = 6`.

iter	lo	hi	mid (ceil)	`can_place(mid)` greedy	count	feasible?	action
1	1	6	`1 + (5+1)//2 = 4`	place at 1; next ≥ 5 → 7 ✓ (count 2); no more	2	`2 >= 3` ? **no**	`hi = 3`
2	1	3	`1 + (2+1)//2 = 2`	place at 1; next ≥ 3 → 3 ✓ (2); next ≥ 5 → 7 ✓ (3)	3	**yes**	`lo = 2`
3	2	3	`2 + (1+1)//2 = 3`	place at 1; next ≥ 4 → 4 ✓ (2); next ≥ 7 → 7 ✓ (3)	3	**yes**	`lo = 3`
exit	3	3					return **3**

Answer: `3` — balls at positions 1, 4, 7, with gaps 3 and 3. ✅

Check `d = 4` is impossible: from 1, the next valid basket is ≥ 5, which is 7; that's only 2 balls, short of 3. ✅

The ceiling midpoint doing its job at iteration 3: `lo = 2`, `hi = 3`. A floor midpoint gives `mid = 2`, `can_place(2)` is true, `lo = mid = 2` — unchanged, and the loop spins forever. The ceiling gives `mid = 3`, which advances. Write a two-value trace like this whenever you use the maximise form; it takes fifteen seconds and catches the bug before the interviewer does.

---

528. Random Pick with Weight ★

UNDERSTAND THE PROBLEM

Given weights `w`, design `pickIndex()` returning index `i` with probability proportional to `w[i]`.

- Not a search problem on the surface, but the implementation is prefix sums plus binary search — which is why it sits on this day.
- Edge cases: a single weight; weights of very different magnitudes; many calls (the construction cost should be amortised).
- Clarify: *"Is the weight array fixed after construction, or can it be updated?"* Updates would call for a Fenwick tree — a strong follow-up to name.

BRUTE-FORCE APPROACH

Idea. Build an expanded array containing index `i` repeated `w[i]` times, then pick a uniformly random element.

Data structures. A list of length `sum(w)`.

Complexity. Construction `O(sum(w))` time and space; `pickIndex` `O(1)`.

Why it's not ideal. `sum(w)` can be `10⁸` per the constraints — the memory is prohibitive. The realisation: you don't need the expanded array, only the **boundaries** between each index's share of the number line. Prefix sums store those `n` boundaries instead of `sum(w)` entries, and locating a random point among sorted boundaries is a binary search.

OPTIMAL APPROACH

Idea (prefix sums + `lower_bound`).

1. Build `prefix[i] = w[0] + … + w[i]`. This partitions `[1, total]` into segments, where index `i` owns `(prefix[i-1], prefix[i]]`.
2. `pickIndex()`: draw a uniform integer `target` in `[1, total]`, then binary search for the **first** prefix `>= target`. That index is the answer.

Using `randint(1, total)` with a `>=` boundary search keeps the intervals half-open consistently and avoids off-by-one bias — the subtle bug that makes the distribution slightly wrong in a way tests rarely catch. Say why you chose that convention.

Data structures. A prefix-sum array; `bisect` (or a hand-written `lower_bound`).

Complexity. Construction `O(n)`. `pickIndex` `O(log n)`. Space `O(n)`.

PYTHON CODE

```python
import random
import bisect
import itertools


class Solution:
    def __init__(self, w: list[int]):
        # prefix[i] = w[0] + ... + w[i]. This carves [1, total] into segments:
        # index i owns the half-open range (prefix[i-1], prefix[i]], whose LENGTH
        # is exactly w[i] — which is what makes the distribution proportional.
        self.prefix = list(itertools.accumulate(w))
        self.total = self.prefix[-1]

    def pickIndex(self) -> int:
        # Uniform integer in [1, total]. Pairing randint(1, total) with a
        # "first prefix >= target" search keeps the intervals consistently
        # half-open. Mixing randint(0, total-1) with a `>` search is the classic
        # off-by-one that skews the distribution just enough to escape testing.
        target = random.randint(1, self.total)

        # bisect_left finds the first index with prefix[i] >= target — exactly
        # the segment owner. O(log n).
        return bisect.bisect_left(self.prefix, target)

    def pickIndex_manual(self) -> int:
        """Same thing with the boundary search written out — worth being able to
        produce, since 'use bisect' is not an answer if they ask you to implement it."""
        target = random.randint(1, self.total)
        lo, hi = 0, len(self.prefix) - 1
        while lo < hi:
            mid = lo + (hi - lo) // 2
            if self.prefix[mid] >= target:
                hi = mid          # mid may be the owner — keep it
            else:
                lo = mid + 1      # mid's segment ends before target — discard
        return lo
```

DRY RUN

`w = [1, 3, 2]` → `prefix = [1, 4, 6]`, `total = 6`.

Segment ownership:

index	weight	owns targets	count
0	1	`{1}`	1
1	3	`{2, 3, 4}`	3
2	2	`{5, 6}`	2

Sample picks:

`target`	`bisect_left(prefix, target)`	reasoning	returned index
1	first prefix `≥ 1` is `prefix[0]=1`	index 0	**0**
2	first `≥ 2` is `prefix[1]=4`	index 1	**1**
3	first `≥ 3` is `prefix[1]=4`	index 1	**1**
4	first `≥ 4` is `prefix[1]=4`	index 1	**1**
5	first `≥ 5` is `prefix[2]=6`	index 2	**2**
6	first `≥ 6` is `prefix[2]=6`	index 2	**2**

Distribution over the 6 equally-likely targets: index 0 → 1/6, index 1 → **3/6**, index 2 → **2/6**. Exactly proportional to `[1, 3, 2]`. ✅

Follow-up worth naming: if weights can be updated after construction, replace the prefix array with a **Fenwick tree (binary indexed tree)** — `O(log n)` updates and `O(log n)` prefix-sum queries, so `pickIndex` becomes an `O(log n)` search over the tree (or `O(log n)` via binary lifting on the tree directly). That's the production answer for a weighted sampler over changing weights, and volunteering it converts a routine problem into a strong signal.

Inconvenience — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

A binary grid where `1` marks a service centre. Distance is **Chebyshev**: `max(|dx|, |dy|)`. The grid's **inconvenience** is the maximum, over all `0` cells, of the distance to the nearest centre. You may flip **at most one** `0` to `1`. Minimise the resulting inconvenience.

- Chebyshev distance means a "ball of radius `d`" is a **square** of side `2d + 1` — that's what makes the feasibility check a rectangle test rather than something geometric and messy.
- **Insight (given in the prompt):** binary search the answer. For a candidate `d`, compute which cells are still uncovered by the existing centres; they must all fit inside **one** Chebyshev ball of radius `d`, which is a bounding-box test.
- Edge cases: no `0` cells (inconvenience 0); no `1` cells at all (one flip creates the only centre); already-optimal grid where flipping nothing helps.

BRUTE-FORCE APPROACH

Idea. For every `0` cell, flip it and recompute the whole grid's inconvenience by a multi-source BFS; take the minimum.

Data structures. A distance grid recomputed per candidate flip.

Complexity. Time `O((mn)²)` — an `O(mn)` BFS per candidate cell. Space `O(mn)`.

Why it's not ideal. On a 500×500 grid that's $6 \cdot 10^{10}$. The fix has two parts, and both are needed: 1. **Binary search the answer** `d` rather than computing it — feasibility is monotone (if inconvenience `d` is achievable with one flip, so is any larger `d`). 2. **Make the feasibility check** `O(mn)` using the Chebyshev-ball-is-a-square property: the set of cells not already within `d` of some centre must fit inside a single `(2d+1)×(2d+1)` square, which is true iff their **bounding box** has width and height `≤ 2d + 1`. No per-candidate BFS needed.

OPTIMAL APPROACH

Idea.

1. Precompute, once, the Chebyshev distance from every cell to the nearest existing centre — a **multi-source BFS with 8-directional moves** (8-connectivity is exactly Chebyshev distance).
2. `feasible(d)`: collect every cell whose precomputed distance `> d`. If there are none, `d` works with no flip. Otherwise compute their bounding box `[minR, maxR] × [minC, maxC]`; the flip is feasible iff `maxR − minR <= 2d` **and** `maxC − minC <= 2d` — meaning a single square of radius `d` can cover them all. (Place the new centre at the box's centre.)
3. Binary search `d` over `[0, max(m, n)]` with the standard minimise template.

Data structures. A distance grid from the initial BFS; four running extrema per check.

Complexity. Precompute `O(mn)`; each feasibility check `O(mn)`; total `O(mn · log(max(m,n)))`. Space `O(mn)`.

PYTHON CODE

```python
from collections import deque


def minimum_inconvenience(grid: list[list[int]]) -> int:
    """
    Chebyshev distance: max(|dx|, |dy|). Flip at most one 0 to 1 to minimise the
    maximum distance from any 0 cell to its nearest 1.
    """
    rows, cols = len(grid), len(grid[0])

    # ---- Precompute distance to the nearest EXISTING centre, once. ----
    # 8-directional BFS IS Chebyshev distance: one diagonal step costs the same
    # as one orthogonal step, which is exactly max(|dx|,|dy|) semantics.
    INF = float('inf')
    dist = [[INF] * cols for _ in range(rows)]
    q = deque()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                dist[r][c] = 0
                q.append((r, c))
```

```
    DIRS8 = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]
    while q:
        r, c = q.popleft()
        for dr, dc in DIRS8:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))

    def feasible(d: int) -> bool:
        """Can inconvenience <= d be achieved with at most one flip?

        A Chebyshev ball of radius d is a (2d+1)x(2d+1) SQUARE. So every cell
        still uncovered by the existing centres must fit inside ONE such square -
        which is true exactly when their BOUNDING BOX is at most 2d wide and tall.
        That turns an O(mn) BFS per candidate flip into four running extrema.
        """
        min_r, max_r = rows, -1
        min_c, max_c = cols, -1

        for r in range(rows):
            for c in range(cols):
                if dist[r][c] > d:                # not covered by existing centres
                    min_r = min(min_r, r); max_r = max(max_r, r)
                    min_c = min(min_c, c); max_c = max(max_c, c)

        if max_r == -1:
            return True                           # already within d - no flip needed

        # One new centre placed at the bounding box's centre covers everything
        # iff the box fits inside a square of side 2d+1.
        return (max_r - min_r) <= 2 * d and (max_c - min_c) <= 2 * d

    # ---- Binary search the answer. Monotone: if d works, so does d+1. ----
    lo, hi = 0, max(rows, cols)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if feasible(mid):
            hi = mid
        else:
            lo = mid + 1

    return lo
```

DRY RUN

```
grid = 1 0 0 0
       0 0 0 0
       0 0 0 0
       0 0 0 1
```

`rows = cols = 4`. Centres at `(0,0)` and `(3,3)`.

Precomputed Chebyshev distances (`max(|dr|,|dc|)` to the nearer centre):

	c=0	c=1	c=2	c=3
r=0	0	1	2	3
r=1	1	1	2	2
r=2	2	2	1	1
r=3	3	2	1	0

Current inconvenience without any flip = `3` (cells `(0,3)` and `(3,0)`).

Binary search over `[0, 4]`:

iter	lo	hi	mid	cells with `dist > mid`	bounding box	fits in `2·mid`?	feasible?	action
1	0	4	2	`(0,3)=3`, `(3,0)=3`	rows `[0,3]`, cols `[0,3]`	height 3 ≤ 4 ✓, width 3 ≤ 4 ✓	**yes**	`hi = 2`
2	0	2	1	all cells with dist ≥ 2: `(0,2),(0,3),(1,2),(1,3),(2,0),(2,1),(3,0),(3,1)`	rows `[0,3]`, cols `[0,3]`	height 3 ≤ 2? **no**	**no**	`lo = 2`
exit	2	2						return **2**

Answer: `2`. ✅

Verify: flipping the centre cell `(1,1)` or `(2,2)` … let's check `(1,2)`. Hmm — the feasibility test at `d = 2` said the uncovered cells `(0,3)` and `(3,0)` have a bounding box of height 3 and width 3, both ≤ `2d = 4`, so a single square of radius 2 covers both. Its centre would be at `((0+3)//2, (3+0)//2)` = roughly `(1,1)` or `(2,2)`. From `(1,2)`: distance to `(0,3)` is `max(1,1) = 1` ✓, to `(3,0)` is `max(2,2) = 2` ✓. And every other cell was already within 2. So inconvenience 2 is achieved. ✅

At `d = 1` the uncovered set spans the full 4×4 grid, needing a square of side 3 — impossible. ✅

The key move made concrete: the feasibility check at each `d` was four min/max updates over the grid — `O(mn)` with no search. The brute force would have run a full BFS for each of the 14 zero cells at every candidate `d`. That's the difference between `O((mn)²)` and `O(mn log n)`.

Optimal Packing — 🟡 Minimum Space Wasted From Packaging (LeetCode 1889)

UNDERSTAND THE PROBLEM

Choose a **single** supplier whose boxes can fit all packages (each package goes in one box of size ≥ the package), minimising total wasted space. Return `-1` if no supplier can fit everything, mod `10⁹ + 7`.

- **One supplier only** — you cannot mix boxes from different suppliers. That's what makes it a per-supplier evaluation rather than a global optimisation.
- Waste = `Σ (chosen box size − package size)` = `Σ box sizes chosen − Σ package sizes`. Since the second term is constant, minimising waste means minimising the total of the chosen box sizes.
- **Take the modulus only at the very end** — comparing already-reduced values would pick the wrong supplier. This is the trap the problem is built around.
- Edge cases: a supplier whose largest box is smaller than the largest package (infeasible); no feasible supplier at all → `-1`; one package.

BRUTE-FORCE APPROACH

Idea. For each supplier, for each package, linearly scan the supplier's boxes for the smallest that fits.

Complexity. Time `O(S · n · b)` for `S` suppliers, `n` packages, `b` boxes each. Space `O(1)`.

Why it's not ideal. With `n` and total boxes both up to `10⁵`, this is `10¹⁰`. Two fixes, both essential: 1. **Sort packages and boxes**, then for each box size use **binary search** to find how many packages it absorbs — turning the per-package scan into `O(log n)` per box. 2. **Prefix sums** over the sorted packages, so the total size of "all packages this box covers" is an `O(1)` lookup rather than a summation.

OPTIMAL APPROACH

Idea (sort + prefix sums + binary search per box).

1. Sort `packages`; build `prefix[i] = sum of the first i packages`.
2. For each supplier: sort its boxes. If the largest box `< max(packages)`, this supplier is infeasible — skip.
3. Walk the supplier's boxes in ascending order, tracking `prev` = how many packages are already assigned. For box size `b`, use `bisect_right(packages, b)` to find `idx` = the number of packages that fit in a box of size `b`. The packages `[prev, idx)` all go into boxes of size `b`, contributing `(idx − prev) · b` to the total box size.
4. Track the minimum total across suppliers; the answer is `min_total − sum(packages)`, taken mod `10⁹+7` **at the end**.

Data structures. Sorted arrays; a prefix-sum array; `bisect`.

Complexity. Time `O(n log n + Σ_s (b_s log b_s + b_s log n))`. Space `O(n)`.

PYTHON CODE

```python
import bisect
import itertools


class Solution:
    def minWastedSpace(self, packages: list[int], boxes: list[list[int]]) -> int:
        MOD = 10 ** 9 + 7

        packages.sort()
        n = len(packages)
        total_packages = sum(packages)

        # prefix[i] = sum of the first i packages. Not strictly needed for the
        # waste formula below (we only need COUNTS, since waste reduces to
        # sum(chosen boxes) - sum(packages)), but keeping it makes the
        # "waste of this box group" derivation obvious and is the shape you'd
        # want if the problem changed to per-group reporting.
        prefix = list(itertools.accumulate(packages, initial=0))

        best_total = float('inf')     # smallest achievable sum of chosen box sizes

        for supplier in boxes:
            supplier.sort()
```

```
            # Infeasible if the biggest box can't hold the biggest package.
            if supplier[-1] < packages[-1]:
                continue

            total = 0
            prev = 0                    # packages [0, prev) are already assigned

            for b in supplier:
                # How many packages fit in a box of size b? Since packages is
                # sorted, that's the insertion point for b — bisect_right so that
                # a package EQUAL to b counts as fitting (waste 0).
                idx = bisect.bisect_right(packages, b)

                if idx > prev:
                    # Packages [prev, idx) each go into a box of size b.
                    total += (idx - prev) * b
                    prev = idx

                if prev == n:
                    break               # every package placed — later boxes are moot

            best_total = min(best_total, total)

        if best_total == float('inf'):
            return -1                   # no supplier could fit everything

        # MOD ONLY AT THE END. Comparing modded totals across suppliers would let
        # a wrapped-around large value look small and select the wrong supplier.
        # This is the trap the problem is built around.
        return (best_total - total_packages) % MOD
```

DRY RUN

`packages = [2, 3, 5]`, `boxes = [[4, 8], [2, 8]]`

Sorted packages: `[2, 3, 5]`, `total_packages = 10`, `n = 3`.

Supplier 0: `[4, 8]`**.** Largest box 8 ≥ largest package 5 ✓ feasible.

box `b`	`bisect_right([2,3,5], b)`	`idx`	`prev`	packages assigned	contribution	running total	new prev
4	first index past values ≤ 4 → index 2	2	0	`[2, 3]` → 2 packages	`2 × 4 = 8`	8	2
8	past values ≤ 8 → index 3	3	2	`[5]` → 1 package	`1 × 8 = 8`	**16**	3

Supplier 0 total box size = **16**. Waste = `16 − 10 = 6`.

Supplier 1: `[2, 8]`**.** Largest box 8 ≥ 5 ✓.

box `b`	`idx`	`prev`	assigned	contribution	total	new prev
2	1 (only the package `2` is ≤ 2)	0	`[2]`	`1 × 2 = 2`	2	1
8	3	1	`[3, 5]`	`2 × 8 = 16`	**18**	3

Supplier 1 total = **18**. Waste = `18 − 10 = 8`.

`best_total = min(16, 18) = 16` → answer **`16 − 10 = 6`**. ✅

Verify supplier 0 by hand: package 2 → box 4 (waste 2), package 3 → box 4 (waste 1), package 5 → box 8 (waste 3). Total waste `2 + 1 + 3 = 6` ✓.

Infeasible case: `packages = [3, 5, 8, 10, 11, 12]`, `boxes = [[12], [11, 9], [10, 5, 14]]`. Supplier 1's largest box is 11 < 12 → skipped. Suppliers 0 and 2 both reach 12 or 14 and are evaluated normally.

If *every* supplier's largest box were below 12, `best_total` stays `inf` → return `-1`. ✅

Why `bisect_right` and not `bisect_left`: a package of size exactly `b` fits in a box of size `b` with zero waste. `bisect_left` would exclude it, pushing it into a larger box and inflating the answer. Test that with `packages = [4]`, `boxes = [[4]]`: `bisect_right` gives `idx = 1` → total 4 → waste 0 ✓; `bisect_left` gives `idx = 0` → the package never gets placed → wrong. One-element sanity checks like this are cheap and catch the whole class of boundary errors.

Travel Time — ✅ Verified (InterviewDB, Chinese source)

UNDERSTAND THE PROBLEM

`m` hubs arranged in a **circle** (hub 1 is adjacent to hub `m`). `transitionTime[i]` is the travel time on the edge between hub `i+1` and hub `i+2` (wrapping). Start at hub 1 and visit `requestedHubs` **in order**, choosing clockwise or counter-clockwise **per leg**. Return the minimum total time.

- Each leg is independent: you must visit the hubs in the given order, but for each hop you freely pick the shorter direction around the circle.
- **Insight:** prefix sums around the circle. For a leg from `a` to `b`, one direction costs `|pref[b] - pref[a]|` and the other costs `total - that`. Take the min.
- The index convention is fiddly and stated awkwardly — **restate it in your own words and confirm** before coding. Assume 1-indexed hubs and that `transitionTime[i]` (0-indexed array) is the edge from hub `i+1` to hub `i+2`, wrapping so the last entry connects hub `m` back to hub 1.
- Edge cases: empty `requestedHubs` → 0; a requested hub equal to the current position (cost 0); `m = 2` (both directions are the same single edge… or two distinct edges — clarify).

BRUTE-FORCE APPROACH

Idea. For each leg, walk hub by hub in both directions summing edge times, and take the smaller.

Complexity. Time `O(k · m)` for `k` legs — each leg walks up to the full circle. Space `O(1)`.

Why it's not ideal. With `m = 10⁵` and `k = 10⁵` that's 10^{10}. And every leg re-sums overlapping stretches of the same circle. A **prefix sum** computed once makes any arc's length an `O(1)` subtraction — the exact same move as LC 560/974, applied to a circular layout.

OPTIMAL APPROACH

Idea (prefix sums around the circle).

1. Build `pref[i]` = total travel time from hub 1 to hub `i` going clockwise. `pref[1] = 0`, `pref[i+1] = pref[i] + transitionTime[i-1]`.
2. `total` = the full circumference = sum of all transition times.
3. For a leg from `a` to `b`: `clockwise_or_counter = abs(pref[b] - pref[a])`; the other way round is `total - that`. Add `min(...)` to the answer.
4. Update the current position to `b`, and repeat.

Data structures. One prefix array of length `m + 1`.

Complexity. Time `O(m + k)`. Space `O(m)`.

PYTHON CODE

```python
from typing import Sequence


def minimum_travel_time(m: int,
                        transition_time: Sequence[int],
                        requested_hubs: Sequence[int]) -> int:
    """
    m hubs in a circle, 1-indexed. transition_time[i] is the edge between
    hub (i+1) and hub (i+2), wrapping so the last entry joins hub m back to hub 1.
    Start at hub 1, visit requested_hubs IN ORDER, choosing a direction per leg.

    CONFIRM THE INDEXING OUT LOUD before coding — the prompt's phrasing is
    ambiguous and an off-by-one here silently produces plausible wrong answers.
    """
    if not requested_hubs:
        return 0

    # pref[i] = clockwise distance from hub 1 to hub i. pref[1] = 0 by definition.
    # Length m+1 so hubs can be indexed 1..m directly, with pref[m+1] unused.
    pref = [0] * (m + 2)
    for i in range(1, m):
        pref[i + 1] = pref[i] + transition_time[i - 1]

    total = pref[m] + transition_time[m - 1]      # full circumference (closing edge)

    answer = 0
    current = 1                                   # start at hub 1

    for target in requested_hubs:
        # One arc between two points on a circle; the other arc is the complement.
        # Prefix sums make BOTH an O(1) computation instead of an O(m) walk.
        one_way = abs(pref[target] - pref[current])
        other_way = total - one_way
        answer += min(one_way, other_way)
        current = target                          # legs are sequential, not from hub 1

    return answer
```

DRY RUN

`m = 4`, `transition_time = [1, 2, 3, 4]`, `requested_hubs = [3, 1]`

Edges (per the stated convention): hub1—hub2 = 1, hub2—hub3 = 2, hub3—hub4 = 3, hub4—hub1 = 4.

Prefix sums:

hub	`pref`	derivation
1	0	start
2	1	`0 + transition_time[0] = 1`
3	3	`1 + transition_time[1] = 1 + 2`
4	6	`3 + transition_time[2] = 3 + 3`

`total = pref[4] + transition_time[3] = 6 + 4 = 10` (the full circle: `1+2+3+4`). ✅

Legs:

leg	from	to	`|pref[to] − pref[from]|`	other way	min	running answer
1	1	3	`|3 − 0| = 3`	`10 − 3 = 7`	**3**	3
2	3	1	`|0 − 3| = 3`	`10 − 3 = 7`	**3**	**6**

Answer: `6`. ✅

Verify by hand: hub 1 → hub 2 → hub 3 costs `1 + 2 = 3` (clockwise). The counter-clockwise route hub 1 → hub 4 → hub 3 costs `4 + 3 = 7`. So 3 ✓. The return leg is symmetric, another 3. Total 6 ✓.

A case where the long way wins isn't possible — `min` always picks the shorter arc — but a case where the *counter-clockwise* arc is shorter is worth tracing: leg from hub 1 to hub 4. `|pref[4] − pref[1]| = 6`, other way `= 10 − 6 = 4`. The algorithm picks **4**, correctly routing hub 1 → hub 4 directly over the closing edge rather than the long way round

through hubs 2 and 3. Run that as your second dry-run case; it's the one that proves the complement formula is right.

DAY 10 — Wed Aug 26 · Intervals, Greedy & DP Part 1

> **Intervals: the sort key IS the algorithm.** Get it wrong and nothing downstream can save you. **Sort by START** when you're *merging* or *inserting* (56, 57) — you need to know what overlaps what, in order. **Sort by END** when you're *selecting the most non-overlapping intervals* (435) — the classic activity-selection greedy: finishing earliest leaves the most room. **Sort by (start ↑, end ↓)** when *containment* matters (1288) — the longer interval must come first among equal starts, or you'd wrongly call it "covered". **Sweep line / two sorted arrays** when you need *concurrency* (253, 759) — decouple starts from ends and walk them as events.
>
> **DP Part 1: every problem here is one of two skeletons.** *"At each element, take it or skip it"* → `dp[i] = f(dp[i-1], dp[i-2])` (70, 198, 213). *"Reach a target amount using unlimited items"* → unbounded knapsack, and the **loop order decides combinations vs permutations** (322, 518). That loop-order fact is the single most-tested DP detail on this day.

---

56. Merge Intervals ★★

UNDERSTAND THE PROBLEM

Given a list of intervals, return them with all overlapping intervals merged.

- Intervals `[a,b]` and `[c,d]` overlap iff `c <= b` (once sorted by start). Touching intervals like `[1,4]` and `[4,5]` **do** merge under LeetCode's definition — confirm this, it's a real ambiguity.
- Merging is transitive: `[1,4]`, `[3,6]`, `[5,8]` all become `[1,8]`.
- Edge cases: empty input; one interval; nothing overlaps; everything overlaps into one; an interval fully containing another.

BRUTE-FORCE APPROACH

Idea. Repeatedly scan all pairs for an overlap, merge that pair, and restart — because merging can create new overlaps.

Data structures. The interval list, mutated.

Complexity. Time `O(n³)` — `O(n²)` pair scans, up to `n` merge rounds. Space `O(n)`.

Why it's not ideal. The rescans are the waste. **Sorting by start** makes overlap detection purely *local*: once sorted, an interval can only overlap the one immediately before it in the output, because everything earlier has a smaller start and was already absorbed. That collapses the whole thing into one linear pass.

OPTIMAL APPROACH

Idea (sort by start, then one pass).

1. Sort by start.
2. Walk the sorted list keeping an output list. For each interval, compare its start against the **last output interval's end**:
3. `start <= last_end` → they overlap, so extend: `last_end = max(last_end, end)`. The `max` is essential — the current interval may be fully contained.
4. otherwise → no overlap, append as a new interval.

Data structures. The sorted list; an output list.

Complexity. Time `O(n log n)` — dominated by the sort. Space `O(n)` for the output (`O(1)` extra if merging in place).

PYTHON CODE

```python
class Solution:
    def merge(self, intervals: list[list[int]]) -> list[list[int]]:
        if not intervals:
            return []

        # SORT BY START. This is the whole algorithm: it makes overlap detection
        # local, because after sorting, an interval can only ever overlap the most
        # recently emitted one — everything before that has a smaller start and
        # was already absorbed.
        intervals.sort(key=lambda iv: iv[0])

        merged = [intervals[0][:]]      # copy, so we never mutate the caller's data

        for start, end in intervals[1:]:
            last = merged[-1]

            if start <= last[1]:
                # Overlap (or touching — LC treats [1,4] and [4,5] as mergeable;
                # confirm this convention with the interviewer, it varies).
                # max() is REQUIRED: the current interval may be fully contained,
                # e.g. last = [1,10] and current = [2,3]. Assigning end blindly
                # would shrink the merged interval to [1,3].
                last[1] = max(last[1], end)
            else:
                merged.append([start, end])

        return merged
```

DRY RUN

`intervals = [[1,3], [2,6], [8,10], [15,18]]` (already sorted by start)

interval	`last` in output	`start <= last_end`?	action	merged after
`[1,3]`	—	seed	initialise	`[[1,3]]`
`[2,6]`	`[1,3]`	`2 <= 3` ✓	`last[1] = max(3, 6) = 6`	`[[1,6]]`
`[8,10]`	`[1,6]`	`8 <= 6` ✗	append	`[[1,6],[8,10]]`
`[15,18]`	`[8,10]`	`15 <= 10` ✗	append	`[[1,6],[8,10],[15,18]]`

Answer: `[[1,6], [8,10], [15,18]]`. ✅

Containment case, `intervals = [[1,10], [2,3], [4,5]]`:

interval	last	overlap?	`max(last_end, end)`	merged
`[1,10]`	—			`[[1,10]]`
`[2,3]`	`[1,10]`	`2 <= 10` ✓	**`max(10, 3) = 10`**	`[[1,10]]` — unchanged ✓
`[4,5]`	`[1,10]`	✓	`max(10, 5) = 10`	`[[1,10]]` ✓

Answer: `[[1,10]]`. ✅ Without the `max`, the second step would have set the end to `3`, and the third to `5`, returning `[[1,5]]` — silently dropping half the covered range. Trace this case out loud; it's the bug interviewers plant.

---

57. Insert Interval

UNDERSTAND THE PROBLEM

Given a **sorted, non-overlapping** interval list and one new interval, return the list after inserting and merging.

- The input is already sorted and clean — so no sort is needed, and the answer should be `O(n)`, not `O(n log n)`.
- **Three phases** structure the solution: intervals entirely before the new one, intervals overlapping it (merge them all into one), intervals entirely after.
- Edge cases: the new interval goes first or last; it overlaps everything; it overlaps nothing; the input list is empty.

BRUTE-FORCE APPROACH

Idea. Append the new interval, sort the whole list, and run problem 56's merge.

Complexity. Time `O(n log n)`, space `O(n)`.

Why it's not ideal. It re-sorts data that was **given sorted**, throwing away the problem's main gift. Because the list is already ordered, a single linear pass with three phases suffices — and the interviewer is specifically checking whether you notice.

OPTIMAL APPROACH

Idea (three phases, one pass).

1. **Before:** copy every interval whose `end < new_start` — they can't overlap.
2. **Overlap:** while the current interval's `start <= new_end`, absorb it: `new_start = min(new_start, start)`, `new_end = max(new_end, end)`. Then append the merged interval once.
3. **After:** copy the rest verbatim.

Data structures. An output list; one index pointer.

Complexity. Time `O(n)`. Space `O(n)` for the output.

PYTHON CODE

```python
class Solution:
    def insert(self, intervals: list[list[int]],
               newInterval: list[int]) -> list[list[int]]:
        res = []
        i, n = 0, len(intervals)
        start, end = newInterval

        # ---- PHASE 1: everything ending strictly BEFORE the new interval starts.
        # Strict `<` because touching intervals ([1,2] and [2,5]) should merge.
        while i < n and intervals[i][1] < start:
            res.append(intervals[i])
            i += 1

        # ---- PHASE 2: absorb every interval that overlaps.
        # No sort needed — the input is already ordered, which is exactly why this
        # is O(n) and not O(n log n). Re-sorting here would waste the problem's gift.
        while i < n and intervals[i][0] <= end:
            start = min(start, intervals[i][0])
            end = max(end, intervals[i][1])
            i += 1
        res.append([start, end])        # the single merged interval

        # ---- PHASE 3: everything starting strictly AFTER — copied verbatim.
        while i < n:
            res.append(intervals[i])
            i += 1

        return res
```

DRY RUN

`intervals = [[1,2], [3,5], [6,7], [8,10], [12,16]]`, `newInterval = [4,8]`

Phase 1 — while `intervals[i][1] < 4`:

i	interval	`end < 4`?	action
0	`[1,2]`	`2 < 4` ✓	append → `res = [[1,2]]`
1	`[3,5]`	`5 < 4` ✗	stop

Phase 2 — while `intervals[i][0] <= end` (starting `start=4, end=8`):

i	interval	`start <= end`?	new `start`	new `end`
1	`[3,5]`	`3 <= 8` ✓	`min(4,3) = 3`	`max(8,5) = 8`
2	`[6,7]`	`6 <= 8` ✓	`min(3,6) = 3`	`max(8,7) = 8`
3	`[8,10]`	`8 <= 8` ✓	`min(3,8) = 3`	**`max(8,10) = 10`**
4	`[12,16]`	`12 <= 10` ✗	stop	

Append `[3, 10]` → `res = [[1,2], [3,10]]`.

Phase 3: append `[12,16]`.

Answer: `[[1,2], [3,10], [12,16]]`. ✅

Two details the trace exposes: - At `i = 3`, `[8,10]` merged because `8 <= 8` — a **touching** interval. If the convention were "touching doesn't merge", that would need `<` and the answer would be `[[1,2],[3,8],[8,10],[12,16]]`. Confirm the convention before coding. - The `end` grew from 8 to 10 during phase 2, which is why phase 2 must be a loop rather than a fixed comparison — absorbing one interval can extend the reach and pull in more.

---

435. Non-overlapping Intervals

UNDERSTAND THE PROBLEM

Given intervals, return the **minimum number to remove** so that the rest don't overlap.

- Equivalent restatement: **maximise** the number of intervals kept, then subtract from `n`. That reframing is what makes the greedy obvious.
- Touching intervals `[1,2]` and `[2,3]` do **not** overlap (LeetCode convention).
- Edge cases: no overlaps → 0; all identical → `n - 1`; one interval → 0.

BRUTE-FORCE APPROACH

Idea. Try every subset, check whether it's overlap-free, keep the largest.

Complexity. Time `O(2ⁿ · n)`. Space `O(n)`.

Why it's not ideal. Exponential. And it hides a classic greedy: this is **activity selection**, where sorting by **end time** and always taking the earliest-finishing compatible interval is provably optimal.

Why sorting by end (not start) is correct — have this ready. Exchange argument: let the greedy pick interval `g` (earliest end) and suppose some optimal solution picks a different first interval `o`. Since `g` ends no later than `o`, swapping `o` for `g` leaves every later interval in the optimal solution still compatible. So the swap doesn't reduce the count, and by induction the greedy is optimal. Sorting by **start** fails outright: a very long interval starting first would be chosen and block everything after it.

OPTIMAL APPROACH

Idea (sort by end, count what you keep).

1. Sort by end ascending.
2. `kept = 0`, `last_end = -inf`. For each interval: if `start >= last_end`, keep it (`kept += 1`, `last_end = end`); otherwise it overlaps and is dropped.
3. Return `n - kept`.

Data structures. The sorted list; two scalars.

Complexity. Time `O(n log n)`. Space `O(1)` extra.

PYTHON CODE

```python
class Solution:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        if not intervals:
            return 0

        # SORT BY END — this is activity selection, and the end time is the key.
        # Exchange argument: the earliest-finishing interval leaves the most room
        # for everything after it, so swapping any optimal first choice for it
        # never reduces the count.
        # Sorting by START would be WRONG: one very long interval starting first
        # would be selected and block every later interval.
        intervals.sort(key=lambda iv: iv[1])

        kept = 0
        last_end = float('-inf')

        for start, end in intervals:
            # `>=` because touching intervals don't overlap under LC's convention:
            # [1,2] and [2,3] can both be kept.
            if start >= last_end:
                kept += 1
                last_end = end
            # else: overlaps the one we kept -> drop it (no counter needed;
            # we derive removals from n - kept at the end)

        return len(intervals) - kept
```

DRY RUN

`intervals = [[1,2], [2,3], [3,4], [1,3]]`

Sorted by end: `[[1,2], [2,3], [1,3], [3,4]]` — note `[2,3]` and `[1,3]` tie on end 3; either order works here.

interval	`start >= last_end` ?	action	kept	last_end
`[1,2]`	`1 >= -inf` ✓	keep	1	2
`[2,3]`	`2 >= 2` ✓ (touching)	keep	2	3
`[1,3]`	`1 >= 3` ✗	drop	2	3
`[3,4]`	`3 >= 3` ✓	keep	**3**	4

Answer: `4 - 3 = 1`. ✅ Removing `[1,3]` leaves `[1,2], [2,3], [3,4]`, which are pairwise non-overlapping.

Why start-sorting fails, concretely. Take `intervals = [[1,100], [2,3], [3,4], [4,5]]`. Sorted by **start** the greedy takes `[1,100]` first, then rejects all three others → keeps 1, removes 3. Sorted by **end** it takes `[2,3]`, `[3,4]`, `[4,5]` → keeps 3, removes 1. The correct answer is 1. Run this counterexample out loud when you state the sort key; it converts "I sorted by end" from an assertion into a justification.

---

253. Meeting Rooms II ★★

UNDERSTAND THE PROBLEM

Given meeting intervals, return the **minimum number of rooms** needed to hold them all.

- Restated: the maximum number of meetings **concurrently in progress** at any instant.
- A meeting ending at time `t` frees the room for one starting at `t` — confirm this convention.
- Edge cases: no meetings → 0; all meetings identical → `n`; no overlaps → 1.
- **Know both solutions** — the heap and the sweep line. Interviewers often ask for a second approach after you give one.

BRUTE-FORCE APPROACH

Idea. For every meeting, count how many others overlap it; the answer is the max.

Complexity. Time `O(n²)`, space `O(1)`.

Why it's not ideal. Quadratic on `n = 10⁴`. And it asks the wrong question: overlap counts per-meeting don't directly give the peak concurrency (a set of pairwise-overlapping meetings and a chain of overlaps look different). The right framing is **events on a timeline** — every start increments a counter, every end decrements it, and the peak is the answer.

OPTIMAL APPROACH — A: MIN-HEAP OF END TIMES

Idea. Sort meetings by start. Keep a min-heap of the **end times of currently-occupied rooms**. For each meeting: if the earliest-ending room has already freed (`heap[0] <= start`), pop it (reuse the room); then push this meeting's end. The heap size is the room count; its maximum is the answer — and since we only ever pop one before pushing one, the final heap size *is* the maximum.

Complexity. Time `O(n log n)`. Space `O(n)`.

OPTIMAL APPROACH — B: SWEEP LINE (TWO SORTED ARRAYS)

Idea. Separate the starts and the ends into two sorted arrays. Walk them with two pointers: whenever the next event is a start, `rooms += 1` and record the max; whenever it's an end, `rooms -= 1`. This is the same counting, with no heap.

Complexity. Time `O(n log n)`, space `O(n)`. Slightly faster in practice; also generalises to "how many resources at time T" queries.

PYTHON CODE

```python
import heapq


class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        """A: min-heap of the end times of occupied rooms."""
        if not intervals:
            return 0

        intervals.sort(key=lambda iv: iv[0])      # process meetings in start order

        heap = []       # end times of rooms currently in use; root = frees soonest

        for start, end in intervals:
            # If the soonest-freeing room is already free, reuse it.
            # `<=` because a meeting ending at t frees the room for one starting
            # at t — confirm this convention with the interviewer.
            if heap and heap[0] <= start:
                heapq.heappop(heap)

            heapq.heappush(heap, end)

        # We pop at most one per iteration and always push one, so the heap only
        # ever grows to the PEAK concurrency — its final size IS the answer.
        return len(heap)


class SolutionSweep:
    """B: sweep line. Same complexity, no heap, and it generalises to
    'how many rooms are in use at time T?' queries."""
```

```python
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        if not intervals:
            return 0

        # DECOUPLE starts from ends. We don't care WHICH meeting an event belongs
        # to, only that a room is claimed or released — so the pairing is
        # irrelevant and sorting them independently is valid.
        starts = sorted(iv[0] for iv in intervals)
        ends = sorted(iv[1] for iv in intervals)

        rooms = best = 0
        i = j = 0

        while i < len(starts):
            if starts[i] < ends[j]:
                rooms += 1                  # a meeting begins before the next ends
                best = max(best, rooms)
                i += 1
            else:
                rooms -= 1                  # a room frees up first
                j += 1

        return best
```

DRY RUN

`intervals = [[0,30], [5,10], [15,20]]`

Heap version (sorted by start: same order):

meeting	`heap[0]`	`heap[0] <= start` ?	pop?	push	heap after	size
`[0,30]`	—	heap empty	no	30	`[30]`	1
`[5,10]`	30	`30 <= 5` ✗	no	10	`[10, 30]`	**2**
`[15,20]`	10	`10 <= 15` ✓	**pop 10**	20	`[20, 30]`	2

Answer: `len(heap) = 2`. ✅

Sweep version: `starts = [0, 5, 15]`, `ends = [10, 20, 30]`

step	`starts[i]`	`ends[j]`	`start < end` ?	rooms	best
1	0	10	✓	1	1
2	5	10	✓	**2**	**2**
3	15	10	✗	1	2
4	15	20	✓	2	2
5	`i` exhausted				return **2** ✅

Both give 2 — meetings `[0,30]` and `[5,10]` overlap, and when `[15,20]` starts, `[5,10]` has finished so its room is reused.

The sweep's counter-intuitive step: sorting starts and ends *independently* breaks the pairing between them, which feels wrong. It's valid because a room is a fungible resource — we only need to know *how many* are claimed at each instant, never which meeting holds which. Say that when you present it; it's the objection interviewers raise.

1288. Remove Covered Intervals

UNDERSTAND THE PROBLEM

Given intervals, remove any interval **covered by another** (`[c,d]` covers `[a,b]` iff `c <= a` and `b <= d`). Return how many remain.

- Covering is about **containment**, not overlap. `[1,4]` and `[3,6]` overlap but neither covers the other.
- Edge cases: duplicate intervals (one covers the other — count once); no coverage; all covered by one big interval.

BRUTE-FORCE APPROACH

Idea. For each interval, check every other for coverage.

Complexity. Time `O(n²)`, space `O(1)`.

Why it's not ideal. Quadratic on `n = 10⁵`. The fix is a **sort key that makes coverage local**: sort by `(start ascending, end descending)`. Then an interval is covered iff its end is `<=` the largest end seen so far — a single scalar, one pass.

Why the end must sort descending is the entire trick. With equal starts, `[1,5]` and `[1,3]`, we need `[1,5]` processed **first** so that `[1,3]` sees `max_end = 5` and is correctly identified as covered. If `[1,3]` came first, `max_end` would be 3, and `[1,5]` would be counted as new — and `[1,3]` would already have been counted too, giving 2 instead of 1.

OPTIMAL APPROACH

Idea (sort by start ↑, end ↓; track `max_end`).

1. Sort with key `(start, -end)`.
2. `count = 0`, `max_end = -inf`. For each interval: if `end > max_end`, it isn't covered → `count += 1`, `max_end = end`. Otherwise it's covered → skip.
3. Return `count`.

Because starts are non-decreasing, any earlier interval already satisfies `start_earlier <= start_current`; so coverage reduces entirely to the end comparison.

Data structures. The sorted list; two scalars.

Complexity. Time `O(n log n)`. Space `O(1)` extra.

PYTHON CODE

```python
class Solution:
    def removeCoveredIntervals(self, intervals: list[list[int]]) -> int:
        # SORT KEY: start ASCENDING, end DESCENDING.
        # The descending end is the whole trick. With equal starts, the LONGER
        # interval must come first, so the shorter one sees a max_end that already
        # covers it. Reverse that order and [1,3] would be counted before [1,5],
        # yielding 2 where the answer is 1.
        intervals.sort(key=lambda iv: (iv[0], -iv[1]))

        count = 0
        max_end = float('-inf')

        for _start, end in intervals:
            # Starts are non-decreasing, so every earlier interval already has
            # start <= this start. Coverage therefore reduces entirely to the end
            # comparison — one scalar, no pairwise checks.
            if end > max_end:
                count += 1              # extends beyond everything so far -> keep
                max_end = end
            # else: fully inside some earlier interval -> covered, skip

        return count
```

DRY RUN

`intervals = [[1,4], [3,6], [2,8]]`

Sorted by `(start, -end)`: `[[1,4], [2,8], [3,6]]`

interval	`end > max_end` ?	action	count	max_end
`[1,4]`	`4 > -inf` ✓	keep	1	4
`[2,8]`	`8 > 4` ✓	keep	**2**	8
`[3,6]`	`6 > 8` ✗	**covered** by `[2,8]`	2	8

Answer: `2`. ✅ Check: `[3,6]` is inside `[2,8]` since `2 <= 3` and `6 <= 8` ✓.

The `-end` sort key isolated, `intervals = [[1,5], [1,3]]`:

Sorted by `(start, -end)` → `[[1,5], [1,3]]` (5 before 3, because `-5 < -3`).

interval	`end > max_end` ?	count	max_end
`[1,5]`	✓	1	5
`[1,3]`	`3 > 5` ✗ → covered	**1**	5

Answer: `1` ✅ — `[1,3]` is covered by `[1,5]`.

With a plain `(start, end)` sort the order would be `[[1,3], [1,5]]`, giving: `[1,3]` keeps (count 1, max_end 3), then `[1,5]` keeps (count 2). Answer 2 — **wrong**. That two-interval trace takes ten seconds and is the strongest possible justification for the sort key.

---

759. Employee Free Time ★

UNDERSTAND THE PROBLEM

Given each employee's sorted, non-overlapping busy intervals, return the **finite** intervals where **all** employees are free.

- "All free" means no employee is busy — so the answer is the **gaps** in the union of every employee's busy time.
- **Finite** excludes the infinite stretches before the first meeting and after the last.
- Edge cases: one employee (gaps in their own schedule); no gaps at all → `[]`; schedules that tile the timeline exactly (touching intervals leave no gap).

BRUTE-FORCE APPROACH

Idea. Discretise time into unit slots, mark every busy slot across all employees, then scan for runs of free slots.

Data structures. A boolean array over the timeline.

Complexity. Time and space `O(T)` where `T` is the timeline length — up to `10⁸` given the constraints.

Why it's not ideal. The timeline is enormous relative to the number of intervals. Only the **endpoints** matter, and there are at most `2n` of them. Flatten, merge (problem 56), and read off the gaps — `O(n log n)` regardless of how long the timeline is.

OPTIMAL APPROACH

Idea (flatten → merge → gaps).

1. Flatten every employee's intervals into one list. Employee identity is irrelevant: "someone is busy" is all that matters.
2. Sort by start and merge overlapping intervals (LC 56 verbatim).
3. The free time is the gap between consecutive merged intervals: `[merged[i].end, merged[i+1].start]`, emitted only when `end < start` (a strict gap).

Data structures. The flattened list; a merged list.

Complexity. Time `O(n log n)` for `n` total intervals. Space `O(n)`.

Follow-up worth naming: since each employee's schedule is *already sorted*, a **k-way merge with a min-heap** avoids the full sort, giving `O(n log k)` for `k` employees. That's the LC 23 primitive reused — mention it.

PYTHON CODE

```python
import heapq


class Interval:
    def __init__(self, start: int = 0, end: int = 0):
        self.start = start
        self.end = end


class Solution:
    def employeeFreeTime(self, schedule: list[list[Interval]]) -> list[Interval]:
        # FLATTEN: employee identity is irrelevant. The question is only
        # "is ANYONE busy at time t", so all intervals go into one pool.
        all_intervals = [iv for employee in schedule for iv in employee]
        if not all_intervals:
            return []

        all_intervals.sort(key=lambda iv: iv.start)

        # MERGE (LC 56 verbatim) to get the union of all busy time.
        merged = [Interval(all_intervals[0].start, all_intervals[0].end)]
        for iv in all_intervals[1:]:
            last = merged[-1]
            if iv.start <= last.end:
                last.end = max(last.end, iv.end)     # max() — iv may be contained
            else:
                merged.append(Interval(iv.start, iv.end))

        # GAPS between consecutive busy blocks are the common free time.
        # Only the gaps BETWEEN blocks are finite — before the first and after the
```

```
        # last are unbounded, which is why the problem says "finite intervals".
        free = []
        for i in range(1, len(merged)):
            if merged[i - 1].end < merged[i].start:      # strict: touching = no gap
                free.append(Interval(merged[i - 1].end, merged[i].start))

        return free


class SolutionHeap:
    """Follow-up: each employee's schedule is ALREADY sorted, so a k-way merge
    beats a full sort — O(n log k) instead of O(n log n). Same primitive as LC 23."""
    def employeeFreeTime(self, schedule: list[list[Interval]]) -> list[Interval]:
        # (start, employee index, interval index)
        heap = [(emp[0].start, e, 0) for e, emp in enumerate(schedule) if emp]
        heapq.heapify(heap)

        free = []
        prev_end = None

        while heap:
            _start, e, i = heapq.heappop(heap)
            iv = schedule[e][i]

            if prev_end is not None and prev_end < iv.start:
                free.append(Interval(prev_end, iv.start))

            prev_end = iv.end if prev_end is None else max(prev_end, iv.end)

            if i + 1 < len(schedule[e]):
                nxt = schedule[e][i + 1]
                heapq.heappush(heap, (nxt.start, e, i + 1))

        return free
```

DRY RUN

```
schedule = [[[1,2], [5,6]],     # employee 0
            [[1,3]],            # employee 1
            [[4,10]]]           # employee 2
```

Flatten: `[[1,2], [5,6], [1,3], [4,10]]`

Sort by start: `[[1,2], [1,3], [4,10], [5,6]]`

Merge:

interval	last merged	`start <= last.end` ?	action	merged
`[1,2]`	—	seed		`[[1,2]]`
`[1,3]`	`[1,2]`	`1 <= 2` ✓	`end = max(2,3) = 3`	`[[1,3]]`
`[4,10]`	`[1,3]`	`4 <= 3` ✗	append	`[[1,3],[4,10]]`
`[5,6]`	`[4,10]`	`5 <= 10` ✓	**`end = max(10,6) = 10`**	`[[1,3],[4,10]]` ✓

Gaps:

pair	`prev.end < next.start` ?	emit
`[1,3]` → `[4,10]`	`3 < 4` ✓	**`[3, 4]`**

Answer: `[[3, 4]]`. ✅

Verify: at time 3–4, employee 0 is free (busy 1–2 and 5–6), employee 1 is free (busy 1–3, and 3 is the end), employee 2 is free (busy from 4). ✓

The `max()` at the last merge step matters again: `[5,6]` sits entirely inside `[4,10]`, and assigning `end = 6` would have invented a phantom free interval `[6, ...]` that doesn't exist. Same bug class as problem 56 — worth flagging that it recurs.

---

55. Jump Game

UNDERSTAND THE PROBLEM

Given an array where `nums[i]` is the maximum jump length from index `i`, return whether the last index is reachable from index 0.

- You may jump **any** distance from 1 to `nums[i]`, not exactly `nums[i]`.
- A zero blocks progress only if you cannot jump *over* it.
- Edge cases: single element → `True` (already there); a leading zero with `n > 1` → `False`; all zeros.

BRUTE-FORCE APPROACH

Idea. DFS/backtracking from index 0, trying every jump length at every index.

Complexity. Time `O(2ⁿ)` without memoisation; `O(n²)` with memo on the index. Space `O(n)`.

Why it's not ideal. Even the memoised `O(n²)` is too slow at `n = 10⁴`. And it's over-general: we don't need to know *which* path reaches the end, only *whether* the end is within reach — and reachability here is captured by a single number, the **furthest index reachable so far**.

OPTIMAL APPROACH

Idea (greedy reach). Sweep left to right maintaining `reach` = the furthest index reachable using indices seen so far. - At index `i`, if `i > reach`, we can never get here → return `False`. - Otherwise `reach = max(reach, i + nums[i])`. - If `reach >= n - 1` at any point, return `True` early.

Correctness: reachability is *prefix-closed* — if index `j` is reachable then so is every index `< j` (jump lengths are flexible, so you can always stop short). So a single high-water mark fully characterises what's reachable.

Data structures. One integer.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def canJump(self, nums: list[int]) -> bool:
        reach = 0        # furthest index reachable using indices 0..i-1

        for i, jump in enumerate(nums):
            # If we've walked past the high-water mark, index i is unreachable -
            # and so is everything after it.
            if i > reach:
                return False

            reach = max(reach, i + jump)

            # Early exit: no need to examine the rest.
            if reach >= len(nums) - 1:
                return True

        # A single scalar suffices because reachability is PREFIX-CLOSED: jumps
        # may be any length up to nums[i], so if index j is reachable then so is
        # every index below it. One high-water mark captures the whole frontier.
        return True      # reached here only when n == 1
```

DRY RUN

`nums = [2, 3, 1, 1, 4]` — expect `True`.

i	nums[i]	i > reach?	i + nums[i]	reach	reach >= 4?
0	2	`0 > 0` ✗	2	2	no
1	3	`1 > 2` ✗	4	**4**	**yes → return True** ✅

Blocked case, `nums = [3, 2, 1, 0, 4]` — expect `False`.

i	nums[i]	i > reach ?	i + nums[i]	reach
0	3	✗	3	3
1	2	1 > 3 ✗	3	3
2	1	2 > 3 ✗	3	3
3	0	3 > 3 ✗	3	3
4	4	4 > 3 ✓	—	**return False** ✅

The zero at index 3 is the wall: nothing before it can reach past index 3, so index 4 is unreachable. The high-water mark froze at 3 for three consecutive steps, which is exactly the signature of being stuck.

---

45. Jump Game II

UNDERSTAND THE PROBLEM

Same array semantics as 55, but the last index is **guaranteed reachable**. Return the **minimum number of jumps** to reach it.

- Minimum count, so this is a shortest-path question on an implicit graph — but with a structure that admits a greedy.
- Edge cases: single element → 0 jumps; `nums[0] >= n-1` → 1 jump.

BRUTE-FORCE APPROACH

Idea. BFS over indices, or DP where `dp[i]` = the minimum jumps to reach `i`, filled by relaxing every reachable index.

Complexity. DP is `O(n²)` — each index relaxes up to `n` successors. Space `O(n)`.

Why it's not ideal. `n = 10⁴` gives 10^8 — borderline, and unnecessary. The realisation: BFS on this graph has a special shape. All indices reachable in exactly `k` jumps form a **contiguous window** `[current_end + 1, farthest]`. So instead of a queue, you can track the window's boundary with two integers — "BFS on an array".

OPTIMAL APPROACH

Idea (implicit BFS levels via two boundaries).

- `farthest` = the furthest index reachable from anything in the current level.
- `current_end` = the last index of the current level.
- Sweep `i` from 0 to `n-2`. Update `farthest = max(farthest, i + nums[i])`. When `i == current_end`, we've exhausted this level → `jumps += 1` and `current_end = farthest`.

Stop at `n-2`: arriving *at* the last index requires no further jump, and iterating to `n-1` would over-count by one.

Data structures. Three integers.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def jump(self, nums: list[int]) -> int:
        jumps = 0
        current_end = 0      # last index of the current BFS "level"
        farthest = 0         # furthest index reachable from this level

        # Stop at n-2: standing ON the last index needs no further jump.
        # Looping to n-1 would increment jumps one time too many.
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])

            # Exhausted every index in this level -> we must take another jump,
            # and the next level spans (current_end, farthest].
            # This is BFS without a queue: each level is a CONTIGUOUS window of
            # indices, so two boundaries replace the whole frontier.
            if i == current_end:
                jumps += 1
                current_end = farthest

        return jumps
```

DRY RUN

`nums = [2, 3, 1, 1, 4]` — expect `2`.

i	nums[i]	i + nums[i]	farthest	i == current_end ?	jumps	current_end
0	2	2	2	`0 == 0` ✓	**1**	2
1	3	4	**4**	`1 == 2` ✗	1	2
2	1	3	4	`2 == 2` ✓	**2**	4
3	1	4	4	loop ends at `i = 3` (`n-2 = 3`)	2	4

Answer: `2`. ✅ The path is index 0 → 1 (jump 1) → 4 (jump 2).

Levels made visible:

- Level 0 = `{0}`, reaching up to index 2.
- Level 1 = `{1, 2}`, reaching up to index 4.
- Index 4 is the target, inside level 2's window → 2 jumps.

The `i == current_end` check firing exactly twice corresponds to exactly two level transitions — which is why this is genuinely BFS and the answer is genuinely minimal, not just a plausible greedy.

Off-by-one check, `nums = [1, 2]`: the loop runs only for `i = 0`. `farthest = 1`, `i == current_end (0)` ✓ → `jumps = 1`, `current_end = 1`. Return **1** ✅. Had the loop gone to `i = 1`, it would have fired again at `i == current_end == 1` and returned 2 — wrong.

---

134. Gas Station ★

UNDERSTAND THE PROBLEM

Given `gas[i]` at each station and `cost[i]` to reach the next, on a **circular** route, return the starting index that lets you complete the loop, or `-1`. The answer is **unique**.

- You start with an empty tank and must never let it go negative *at any point*.
- Two facts drive the solution: (1) a solution exists **iff** `sum(gas) >= sum(cost)`; (2) if you run out of fuel travelling from `start` to `i`, then **no station between `start` and `i`** can be a valid start either.
- Edge cases: one station; `sum(gas) < sum(cost)` → `-1`; the answer is index 0.

BRUTE-FORCE APPROACH

Idea. Try every start index; simulate the full circular trip from each.

Complexity. Time `O(n²)`, space `O(1)`.

Why it's not ideal. Quadratic on `n = 10⁵`. And it re-derives information: fact (2) above means a failed attempt from `start` rules out **every** station up to the failure point in one shot, not just `start`. Exploiting that turns `n` attempts into one pass.

Proof of fact (2), have it ready. Suppose starting at `s` you reach station `i` and run dry. Take any `j` with `s < j <= i`. Arriving at `j` from `s`, your tank was `>= 0` (you got there), so starting *at* `j` with an empty tank gives you no more fuel than you had passing through — hence you'd also fail before `i`. So the next viable candidate is `i + 1`.

OPTIMAL APPROACH

Idea (one pass, restart on deficit).

1. `total = 0` (tracks the global surplus), `tank = 0` (current trip's fuel), `start = 0`.
2. For each `i`: `diff = gas[i] - cost[i]`; `total += diff`; `tank += diff`.
3. If `tank < 0`: the trip from `start` fails at `i`, so by fact (2) reset `start = i + 1` and `tank = 0`.
4. Return `start` if `total >= 0`, else `-1`.

Data structures. Three integers.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        total = 0        # global surplus: >= 0 iff SOME solution exists
        tank = 0         # fuel on the current candidate trip
        start = 0        # current candidate starting station

        for i in range(len(gas)):
            diff = gas[i] - cost[i]
            total += diff
            tank += diff

            if tank < 0:
                # Ran dry between `start` and `i`. KEY LEMMA: no station strictly
                # between them can work either — arriving at any such j from
                # `start` you had tank >= 0, so beginning AT j with an empty tank
                # gives you no more fuel and you'd fail no later. So skip them all
                # and restart at i+1.
                start = i + 1
                tank = 0

        # total >= 0 means the loop is completable by SOMEONE, and the problem
        # guarantees uniqueness — so the surviving `start` is that station.
        return start if total >= 0 else -1
```

DRY RUN

`gas = [1, 2, 3, 4, 5]`, `cost = [3, 4, 5, 1, 2]`

`diff = [-2, -2, -2, 3, 3]`, `sum = 0` → a solution exists.

i	diff	total	tank before	tank after	`tank < 0`?	start reset to	tank reset
0	−2	−2	0	−2	✓	**1**	0
1	−2	−4	0	−2	✓	**2**	0
2	−2	−6	0	−2	✓	**3**	0
3	3	−3	0	3	✗	3	3
4	3	**0**	3	6	✗	3	6

`total = 0 >= 0` → return **3**. ✅

Verify manually from station 3: tank `= 4 - 1 = 3`; at station 4: `3 + 5 - 2 = 6`; at station 0: `6 + 1 - 3 = 4`; at 1: `4 + 2 - 4 = 2`; at 2: `2 + 3 - 5 = 0` — never negative, loop completed ✓.

The lemma doing real work: the three resets at `i = 0, 1, 2` each eliminated exactly one candidate, but on input like `gas = [5,1,1,1]`, `cost = [1,1,1,10]` a single failure at `i = 3` would eliminate starts 0, 1, 2 and 3 all at once. That's the difference between `O(n²)` and `O(n)`.

Impossible case, `gas = [2,3,4]`, `cost = [3,4,3]`: `diff = [-1,-1,1]`, `total = -1 < 0` → return `-1` ✅, regardless of what `start` ended up as. Checking `total` separately from the greedy is what makes that correct.

---

70. Climbing Stairs

UNDERSTAND THE PROBLEM

Given `n` stairs, climbing 1 or 2 at a time, return the number of distinct ways to reach the top.

- Distinct **sequences** — `1+2` and `2+1` are different ways.
- It's Fibonacci: `ways(n) = ways(n-1) + ways(n-2)`, because the last step was either a 1 (from `n-1`) or a 2 (from `n-2`).
- Edge cases: `n = 1` → 1; `n = 2` → 2; `n = 0` → 1 (the empty way) — clarify.

BRUTE-FORCE APPROACH

Idea. Plain recursion: `climb(n) = climb(n-1) + climb(n-2)`.

Complexity. Time `O(2ⁿ)`, space `O(n)` stack.

Why it's not ideal. Exponential from recomputing the same subproblems — `climb(n-2)` is evaluated twice at every level. That's the textbook **overlapping subproblems** signature, and the whole reason DP exists. Memoisation makes it `O(n)`; noticing that only the last two values matter makes it `O(1)` space.

OPTIMAL APPROACH

Idea (bottom-up with two rolling variables). `dp[i] = dp[i-1] + dp[i-2]`, but since only the previous two entries are ever read, keep them in two scalars.

Data structures. Two integers.

Algorithm.

1. `prev2, prev1 = 1, 1` (ways to reach step 0 and step 1).
2. For `i` in `2..n`: `cur = prev1 + prev2`; shift.
3. Return `prev1`.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def climbStairs(self, n: int) -> int:
        # Recurrence: the last move was either a 1-step (from n-1) or a 2-step
        # (from n-2), and those two sets of sequences are disjoint and exhaustive.
        # So ways(n) = ways(n-1) + ways(n-2) — Fibonacci, shifted.
        if n <= 2:
            return n

        prev2, prev1 = 1, 2      # ways to reach step 1 and step 2

        for _ in range(3, n + 1):
            # Only the previous TWO values are ever read, so the O(n) dp array
            # collapses to two scalars. Recognising that rolling-window collapse
            # is the reusable skill here, not the Fibonacci itself.
            prev2, prev1 = prev1, prev1 + prev2

        return prev1
```

DRY RUN

`n = 5`

step `i`	prev2	prev1	new prev1 = `prev1 + prev2`
init	1 (ways to step 1)	2 (ways to step 2)	—
3	2	**3**	`2 + 1 = 3`
4	3	**5**	`3 + 2 = 5`
5	5	**8**	`5 + 3 = 8`

Answer: `8`. ✅

Verify by enumeration for `n = 4` (should be 5): `1+1+1+1`, `1+1+2`, `1+2+1`, `2+1+1`, `2+2` — five ways ✓. And `n = 5` extends each of those by a final 1 (5 ways) or extends each `n=3` way by a final 2 (3 ways) = 8 ✓.

---

198. House Robber

UNDERSTAND THE PROBLEM

Given amounts in houses along a street, you cannot rob two **adjacent** houses. Return the maximum you can take.

- Classic **skip-or-take** DP: at each house, either rob it (and skip the previous) or skip it (and keep the previous best).
- Edge cases: empty → 0; one house → that value; two houses → the max.

BRUTE-FORCE APPROACH

Idea. Recursively try robbing or skipping each house.

Complexity. Time `O(2^n)`, space `O(n)`.

Why it's not ideal. Exponential, with the same overlapping-subproblem structure as 70. The state that matters is just "best total considering houses `0..i`", and it depends only on `i-1` and `i-2`.

OPTIMAL APPROACH

Idea. `dp[i] = max(dp[i-1], dp[i-2] + nums[i])` : - **skip** house `i` → carry `dp[i-1]` forward; - **take** house `i` → its value plus the best from `i-2` (never `i-1`, that's the adjacency rule).

Only two previous values are read, so collapse to scalars.

Data structures. Two integers.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def rob(self, nums: list[int]) -> int:
        # rob_prev  = best total INCLUDING house i-1 as a candidate (i.e. dp[i-1])
        # rob_prev2 = best total up to house i-2                    (i.e. dp[i-2])
        rob_prev = rob_prev2 = 0

        for x in nums:
            # SKIP this house -> keep dp[i-1].
            # TAKE this house -> its value + dp[i-2]. Never dp[i-1]: that might
            # include the adjacent house, which the constraint forbids.
            rob_prev, rob_prev2 = max(rob_prev, rob_prev2 + x), rob_prev
            # The simultaneous assignment matters — computing them sequentially
            # would feed the NEW rob_prev into rob_prev2 and break the recurrence.

        return rob_prev
```

DRY RUN

`nums = [2, 7, 9, 3, 1]`

house	value	`rob_prev2 + x` (take)	`rob_prev` (skip)	new rob_prev	new rob_prev2
init			0	0	0
0	2	`0 + 2 = 2`	0	**2**	0
1	7	`0 + 7 = 7`	2	**7**	2
2	9	`2 + 9 = 11`	7	**11**	7
3	3	`7 + 3 = 10`	11	**11**	11
4	1	`11 + 1 = 12`	11	**12**	11

Answer: `12`. ✅ The optimal selection is houses 0, 2, 4 → `2 + 9 + 1 = 12`. ✓

Note house 3 (value 3): taking it would give 10, but skipping preserves 11 — and skipping is what makes house 4 available. That's the recurrence choosing correctly between local greed and global optimum, which a purely greedy "take the biggest available" would get wrong.

---

213. House Robber II

UNDERSTAND THE PROBLEM

Same as 198, but the houses are in a **circle** — the first and last are adjacent.

- The circular constraint means you can never rob **both** house 0 and house `n-1`.
- Edge cases: one house → that value (no circularity to violate); two houses → the max.

BRUTE-FORCE APPROACH

Idea. Recursively try every valid subset, checking the circular adjacency.

Complexity. Time `O(2^n)`.

Why it's not ideal. Exponential, and the circular constraint tempts people into a complicated single-pass DP with an extra "did I take the first house" dimension. That works but is fiddly. The clean reduction is to **split into two linear problems** — which is a much stronger answer because it reuses 198 unchanged.

OPTIMAL APPROACH

Idea (circular → two linear). Since houses 0 and `n-1` can't both be robbed, every valid solution falls into one of two cases: - **Exclude the last house:** solve 198 on `nums[0 : n-1]`. - **Exclude the first house:** solve 198 on `nums[1 : n]`.

The answer is the max of the two. This is exhaustive (any solution omits at least one of the two ends) and each subproblem is a plain linear House Robber.

Handle `n == 1` separately — both slices would be empty.

Data structures. Reuses the 198 helper.

Complexity. Time `O(n)` — two linear passes. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def rob(self, nums: list[int]) -> int:
        n = len(nums)
        if n == 1:
            return nums[0]      # no circularity to violate; both slices would be
                                # empty, so this case MUST be handled separately

        # Houses 0 and n-1 are adjacent, so they can never BOTH be robbed.
        # Therefore every valid solution omits at least one of them, and the two
        # cases below are exhaustive. Each is an ordinary LINEAR House Robber,
        # so we reuse LC 198 verbatim instead of adding a state dimension.
        return max(self._rob_linear(nums[:-1]),      # exclude the last house
                   self._rob_linear(nums[1:]))       # exclude the first house

    @staticmethod
    def _rob_linear(nums: list[int]) -> int:
        """LC 198, unchanged."""
        prev = prev2 = 0
        for x in nums:
            prev, prev2 = max(prev, prev2 + x), prev
        return prev
```

DRY RUN

`nums = [2, 3, 2]` (circular)

Case A — exclude the last house: `[2, 3]`

x	take	skip	prev	prev2
2	`0+2=2`	0	2	0
3	`0+3=3`	2	**3**	2

Result: **3**.

Case B — exclude the first house: `[3, 2]`

x	take	skip	prev	prev2
3	3	0	3	0
2	`0+2=2`	3	**3**	3

Result: **3**.

Answer: `max(3, 3) = 3`. ✅

Correct: robbing houses 0 and 2 would yield `2 + 2 = 4`, but they're **adjacent in the circle**, so that's forbidden. The best legal option is house 1 alone, worth 3. ✓ Notice that a naive linear House Robber on the full `[2,3,2]` returns exactly that forbidden 4 — running that comparison out loud is how you show the circular constraint is genuinely handled.

Second case, `nums = [1, 2, 3, 1]`: - Case A on `[1,2,3]` → best is `1 + 3 = 4`. - Case B on `[2,3,1]` → best is `2 + 1 = 3`. - Answer `max(4, 3) = 4` ✅ (houses 0 and 2, not adjacent in the circle since `n = 4`).

---

322. Coin Change ★★

UNDERSTAND THE PROBLEM

Given coin denominations and an amount, return the **fewest coins** summing to the amount, or `-1` if impossible. Coins are unlimited.

- **Watch: greedy is wrong.** Taking the largest coin first fails on `coins = [1, 3, 4]`, `amount = 6`: greedy gives `4 + 1 + 1 = 3` coins, but `3 + 3 = 2` coins is optimal. Say this out loud — it's the reason the problem exists.
- **Unbounded knapsack**: each coin may be used any number of times.
- Edge cases: `amount = 0` → 0 coins; no combination reaches the amount → `-1`; a coin larger than the amount.

BRUTE-FORCE APPROACH

Idea. Recursively try every coin at every step: `f(amount) = 1 + min(f(amount - c) for c in coins)`.

Complexity. Time `O(coins^amount)` without memoisation. Space `O(amount)` stack.

Why it's not ideal. Exponential from recomputing `f(k)` for the same `k` along many different paths. Memoising on the remaining amount collapses it to `O(amount × len(coins))` — and the bottom-up form of that memo is the DP below.

OPTIMAL APPROACH

Idea (bottom-up unbounded knapsack). `dp[a]` = fewest coins summing to exactly `a`. - `dp[0] = 0`. - `dp[a] = 1 + min(dp[a - c])` over all coins `c <= a`. - Initialise unreachable amounts to a sentinel (`inf` or `amount + 1`) so `min` naturally ignores them.

Iterate `a` from 1 to `amount`, and for each `a` try every coin. Because we read `dp[a - c]` which is already final, coins are implicitly reusable — that's what makes it *unbounded*.

Data structures. A 1-D array of length `amount + 1`.

Complexity. Time `O(amount × len(coins))`. Space `O(amount)`.

PYTHON CODE

```python
class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        # Sentinel: amount+1 is larger than any achievable coin count (the most
        # coins you could ever use is `amount` of value 1), so it acts as
        # infinity while staying an int. `float('inf')` works equally well.
        INF = amount + 1
        dp = [INF] * (amount + 1)
        dp[0] = 0                       # zero coins make zero

        for a in range(1, amount + 1):
            for c in coins:
                if c <= a:
                    # dp[a-c] is already FINAL when we read it, and we may reuse
                    # coin c freely — that's what makes this UNBOUNDED knapsack
                    # rather than 0/1.
                    dp[a] = min(dp[a], dp[a - c] + 1)

        # Untouched entries still hold the sentinel -> unreachable.
        return dp[amount] if dp[amount] != INF else -1
```

DRY RUN

`coins = [1, 3, 4]`, `amount = 6` — the greedy counterexample.

`dp = [0, INF, INF, INF, INF, INF, INF]` (`INF = 7`)

a	coins ≤ a	candidates `dp[a-c] + 1`	`dp[a]`
1	1	`dp[0]+1 = 1`	**1**
2	1	`dp[1]+1 = 2`	**2**
3	1, 3	`dp[2]+1 = 3` ; **`dp[0]+1 = 1`**	**1**
4	1, 3, 4	`dp[3]+1 = 2` ; `dp[1]+1 = 2` ; **`dp[0]+1 = 1`**	**1**
5	1, 3, 4	`dp[4]+1 = 2` ; `dp[2]+1 = 3` ; `dp[1]+1 = 2`	**2**
6	1, 3, 4	`dp[5]+1 = 3` ; **`dp[3]+1 = 2`** ; `dp[2]+1 = 3`	**2**

Answer: `2` — the coins `3 + 3`. ✅

Greedy comparison, made explicit: a largest-first greedy takes `4`, leaving 2, then `1 + 1` → **3 coins**. The DP found **2**. That single trace is the proof that greedy is wrong here, and it's exactly what an interviewer wants to hear before you start writing the table.

Impossible case, `coins = [2]`, `amount = 3`: `dp[1]` stays `INF` (no coin ≤ 1), `dp[2] = 1`, `dp[3] = min(INF, dp[1] + 1 = INF + 1)` — still effectively unreachable → return `-1` ✅. Using `amount + 1` rather than a true infinity means `dp[1] + 1` doesn't overflow into a spuriously valid value, because the final check compares against `INF` exactly; if you prefer absolute safety, use `float('inf')`.

---

518. Coin Change II

UNDERSTAND THE PROBLEM

Given an amount and coin denominations, return the number of **distinct combinations** making that amount.

- **Combinations, not permutations:** `1+2` and `2+1` are the same and count once.
- **Watch: loop order decides combinations vs permutations.** This is the single most-tested DP detail on the day.
- Edge cases: `amount = 0` → 1 (the empty combination); no combination → 0.

BRUTE-FORCE APPROACH

Idea. Recursively enumerate all coin sequences summing to the amount, then deduplicate by sorting each sequence.

Complexity. Exponential, plus a sort per sequence for dedup.

Why it's not ideal. Deduplication after the fact, again — the same anti-pattern as LC 15 and LC 39. The structural fix: enforce a **fixed coin order** during construction, so `2+1` is never generated as distinct from `1+2`. In the DP that means putting the **coin loop on the outside**.

OPTIMAL APPROACH

Idea (unbounded knapsack, coin loop outermost). `dp[a]` = number of combinations making amount `a`. - `dp[0] = 1` — the empty combination. - **For each coin (outer), for each amount (inner):** `dp[a] += dp[a - c]`.

Why the loop order is the whole problem:

- **Coin outer, amount inner** → each coin is fully "introduced" before the next, so combinations are counted in a canonical non-decreasing coin order. `2+1` is never reachable separately from `1+2`. → **combinations** ✅
- **Amount outer, coin inner** → at each amount every coin is tried, so orderings are counted separately. → **permutations** (which is what LC 377 asks for).

Being able to state that distinction cleanly is worth more than the code.

Data structures. A 1-D array of length `amount + 1`.

Complexity. Time `O(amount × len(coins))`. Space `O(amount)`.

PYTHON CODE

```python
class Solution:
    def change(self, amount: int, coins: list[int]) -> int:
        dp = [0] * (amount + 1)
        dp[0] = 1        # exactly one way to make 0: take nothing

        # ***THE LOOP ORDER IS THE PROBLEM.***
        # COIN OUTER, AMOUNT INNER -> COMBINATIONS.
        # Each coin is fully introduced before the next one, so every combination
        # is counted once in a canonical non-decreasing coin order: 1+2 and 2+1
        # are never both generated.
        #
        # Swapping the loops (amount outer, coin inner) counts PERMUTATIONS
        # instead - that is LeetCode 377, a different problem with a different
        # answer on the same input. Know which is which cold.
        for c in coins:
            for a in range(c, amount + 1):
                # Ascending inner loop -> dp[a-c] may already include coin c,
                # which is what makes the coin REUSABLE (unbounded knapsack).
                # A descending inner loop would give 0/1 knapsack instead (LC 416).
                dp[a] += dp[a - c]

        return dp[amount]
```

DRY RUN

`amount = 5`, `coins = [1, 2, 5]`

`dp = [1, 0, 0, 0, 0, 0]`

Coin 1 (`a` from 1 to 5): each `dp[a] += dp[a-1]`

a	`dp[a-1]`	`dp[a]` after
1	1	1
2	1	1
3	1	1
4	1	1
5	1	1

`dp = [1, 1, 1, 1, 1, 1]` — one way to make each amount using only 1s ✓

Coin 2 (`a` from 2 to 5): `dp[a] += dp[a-2]`

a	`dp[a]` before	`dp[a-2]`	`dp[a]` after
2	1	`dp[0]=1`	**2**
3	1	`dp[1]=1`	**2**
4	1	`dp[2]=2`	**3**
5	1	`dp[3]=2`	**3**

`dp = [1, 1, 2, 2, 3, 3]`

Coin 5 (`a = 5`): `dp[5] += dp[0] = 1` → **`dp[5] = 4`**

Answer: `4`. ✅

Enumerate to verify: `5`, `2+2+1`, `2+1+1+1`, `1+1+1+1+1` — four combinations ✓.

The loop-order difference, concretely. Swap the loops (amount outer, coin inner) on the same input and `dp[5]` becomes **9**, because it now counts `1+2+2`, `2+1+2`, and `2+2+1` as three distinct results. Nine is the answer to LC 377 (Combination Sum IV), not this problem. Running both on `amount = 5, coins = [1,2,5]` and getting 4 versus 9 is the fastest possible demonstration that you know which loop order you need — do it on the whiteboard.

Reuse direction: the inner loop ascends, so `dp[a-c]` may already have counted coin `c` — that's unbounded reuse. Descending the inner loop gives 0/1 knapsack (each item once), which is what LC 416 needs. Three problems, one table, distinguished entirely by loop direction and nesting.

Minimum Shipment — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

`warehouses[i]` is the warehouse holding item `i`. Each shipment sends either **two items from different warehouses** or **one item alone**. Minimise the number of shipments.

- Restated: pair up items so that paired items come from different warehouses; unpaired items ship solo. Minimise `pairs + singles`.
- **Insight (given):** closed form. Let `f` = the largest warehouse's item count and `n` = the total. If `f > n - f`, the surplus from that warehouse must ship alone, giving `f` shipments. Otherwise everything pairs up: `ceil(n / 2)`.
- Edge cases: all items in one warehouse (→ `n` shipments, all solo); `n = 1`; perfectly balanced warehouses.

BRUTE-FORCE APPROACH

Idea. Search over pairings — try every way to match items into cross-warehouse pairs and count the best.

Data structures. A matching/backtracking search, or greedy simulation with a max-heap (repeatedly pair items from the two largest warehouses).

Complexity. Exhaustive matching is exponential. The heap simulation is `O(n log k)` for `k` warehouses — correct, and a fine answer.

Why it's not ideal. The heap simulation *works* and you should mention it as your first instinct. But every step is forced: you always pair the two largest groups, and the outcome depends only on two numbers — the largest group's size and the total. So the whole simulation collapses to arithmetic. Recognising a closed form where you first saw a simulation is the signal being tested.

Deriving the formula. Let `f` = the largest group's size, `rest = n - f`. - If `f <= rest`, the groups are balanced enough that every item can find a cross-warehouse partner (a standard pairing argument), so the answer is `ceil(n/2)` — you can't do better, since each shipment carries at most 2 items. - If `f > rest`, every one of the `rest` items pairs with an item from the dominant warehouse, consuming `rest` shipments and `rest` dominant items. The remaining `f - rest` dominant items have no valid partner (they'd all be same-warehouse) and must ship alone: `rest + (f - rest) = f`.

OPTIMAL APPROACH

Idea. Count the frequency of each warehouse, take `f = max`, `n = total`, and return `max(f, ceil(n/2))`.

That single expression covers both branches: when `f <= n - f`, we have `f <= n/2 <= ceil(n/2)` so the ceiling wins; when `f > n - f`, `f > n/2` so `f` wins.

Data structures. A `Counter`.

Complexity. Time `O(n)`. Space `O(k)` for `k` distinct warehouses.

PYTHON CODE

```python
from collections import Counter
from typing import Sequence
import heapq


def minimum_shipments(warehouses: Sequence) -> int:
    """
    Each shipment carries either two items from DIFFERENT warehouses, or one
    item alone. Minimise the shipment count.

    Closed form: max(largest_group, ceil(total / 2)).
      * If one warehouse dominates (f > n - f), its surplus has no legal partner
        and must ship solo -> f shipments.
      * Otherwise everything pairs -> ceil(n/2), the information-theoretic floor
        since no shipment carries more than 2 items.
    """
    if not warehouses:
        return 0

    counts = Counter(warehouses)
    n = len(warehouses)
    f = max(counts.values())
```

```
    # One expression covers both branches:
    #   f <= n-f  =>  f <= n/2 <= ceil(n/2), so the ceiling term wins
    #   f >  n-f  =>  f >  n/2,              so f wins
    return max(f, (n + 1) // 2)


def minimum_shipments_simulation(warehouses: Sequence) -> int:
    """The greedy heap simulation - mention this first, then collapse it to the
    formula above. Always pair the two LARGEST remaining groups; that keeps the
    distribution as balanced as possible and never strands items unnecessarily."""
    counts = Counter(warehouses)
    heap = [-c for c in counts.values()]
    heapq.heapify(heap)

    shipments = 0
    while len(heap) >= 2:
        a = -heapq.heappop(heap)          # largest group
        b = -heapq.heappop(heap)          # second largest
        shipments += 1                    # one cross-warehouse pair
        if a - 1 > 0:
            heapq.heappush(heap, -(a - 1))
        if b - 1 > 0:
            heapq.heappush(heap, -(b - 1))

    if heap:
        shipments += -heap[0]             # leftovers all ship alone
    return shipments
```

DRY RUN

Case 1 — balanced. `warehouses = [1, 1, 2, 2, 3]`

Counts: `{1: 2, 2: 2, 3: 1}`. `n = 5`, `f = 2`.

`ceil(5/2) = 3`. **`max(2, 3) = 3`**. ✅

Verify by hand: pair `(item from W1, item from W2)`, pair `(W1, W2)`, then the lone `W3` item ships alone → 3 shipments ✓. Or pair `(W1,W3)`, `(W1,W2)`, `(W2)` alone — also 3. Can't do better: 5 items at ≤ 2 per shipment needs at least 3.

Case 2 — dominated. `warehouses = [1, 1, 1, 1, 2]`

Counts: `{1: 4, 2: 1}`. `n = 5`, `f = 4`, `rest = 1`.

`ceil(5/2) = 3`, but **`max(4, 3) = 4`**. ✅

Verify: the single W2 item pairs with one W1 item (1 shipment). The three remaining W1 items are all from the same warehouse, so none can pair with another — 3 solo shipments. Total `1 + 3 = 4` ✓. The `ceil(n/2) = 3` bound is unachievable here, which is exactly why the `max` is needed.

Case 3 — all one warehouse. `warehouses = [7, 7, 7]`: `f = 3`, `n = 3`, **`max(3, 2) = 3`** ✅ — every item ships alone.

Simulation cross-check on case 2: heap `[-4, -1]` → pop 4 and 1, ship 1, push back `-3` → heap `[-3]`, length 1 so the loop exits → add 3 → total **4** ✓. Both methods agree, which is the confidence check to run out loud before committing to the formula.

Minimize Absolute Differences — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given arrays `a` and `b` of length `n`, pair each `a[i]` with a **distinct** `b[j]`. Minimise `Σ |a[i] − b[π(i)]|` over all permutations `π`.

- It's an assignment problem on the surface, which suggests the Hungarian algorithm at `O(n³)` — but the cost structure here (absolute difference on a line) admits a much simpler answer.
- **Insight (given):** sort both and pair in order. The **exchange argument**: if `a1 < a2` are paired with `b1 > b2`, swapping them never increases the total.
- Edge cases: `n = 1`; identical arrays (cost 0); arrays with duplicates; negatives.

BRUTE-FORCE APPROACH

Idea. Try all `n!` permutations and take the minimum total.

Complexity. Time `O(n! · n)`. Space `O(n)`.

A better brute force worth naming: the **Hungarian algorithm** solves general assignment in `O(n³)`. Correct, and the right answer if the cost function were arbitrary.

Why neither is ideal. `n!` is hopeless, and `O(n³)` is unnecessary because our cost is not arbitrary — it's `|a − b|` on a number line, which satisfies the **Monge / inverse-triangle-inequality** condition that makes sorted pairing optimal. Naming *why* the general algorithm can be replaced is the whole point.

The exchange argument, in full — have this ready. Suppose an optimal pairing has `a1 < a2` matched to `b1 > b2`. Compare the two options:

```
crossed:  |a1 − b1| + |a2 − b2|
sorted:   |a1 − b2| + |a2 − b1|
```

For any reals with `a1 ≤ a2` and `b2 ≤ b1`, the identity `|a1 − b2| + |a2 − b1| ≤ |a1 − b1| + |a2 − b2|` holds — case-check on the six orderings of the four values, or observe that the crossed pairing's intervals overlap while the sorted one's do not. So swapping never increases the total. Repeat until no crossed pair remains: that's the fully sorted pairing.

OPTIMAL APPROACH

Idea. Sort both arrays ascending and pair element-wise, summing `|a[i] − b[i]|`.

Data structures. Two sorted arrays.

Complexity. Time `O(n log n)` — the two sorts. Space `O(1)` extra (or `O(n)` if you must not mutate the inputs).

PYTHON CODE

```python
def minimize_absolute_differences(a: list[int], b: list[int]) -> int:
    """
    Pair each a[i] with a distinct b[j] to minimise the sum of |differences|.

    Sort both, pair in order. EXCHANGE ARGUMENT: if a1 < a2 are paired with
    b1 > b2 (a "crossed" pairing), then
        |a1 − b2| + |a2 − b1|  <=  |a1 − b1| + |a2 − b2|
    for all real values − so uncrossing never increases the total. Repeating the
    uncrossing until none remain yields exactly the sorted pairing.

    This is why we do NOT need the O(n^3) Hungarian algorithm: the cost function
    |x − y| on a line has the Monge property, and sorted matching is optimal.
    """
    if len(a) != len(b):
        raise ValueError("arrays must be the same length")

    # sorted() rather than .sort() so the caller's arrays are untouched.
    sa = sorted(a)
    sb = sorted(b)

    return sum(abs(x − y) for x, y in zip(sa, sb))
```

DRY RUN

`a = [1, 5, 9]`, `b = [4, 2, 8]`

Sorted: `sa = [1, 5, 9]`, `sb = [2, 4, 8]`

pair	`|a - b|`
`1` ↔ `2`	1
`5` ↔ `4`	1
`9` ↔ `8`	1

Total: **3**. ✅

Verify against all 6 permutations of `b = [2,4,8]` against `a = [1,5,9]`:

pairing of (1,5,9) with	total
`(2,4,8)` ← sorted	`1+1+1 = 3`
`(2,8,4)`	`1+3+5 = 9`
`(4,2,8)`	`3+3+1 = 7`
`(4,8,2)`	`3+3+7 = 13`
`(8,2,4)`	`7+3+5 = 15`
`(8,4,2)`	`7+1+7 = 15`

Sorted pairing wins with 3 ✓ — and every crossed alternative is strictly worse, which is the exchange argument confirmed empirically.

Isolating one uncrossing: compare `(2,4,8)` at cost 3 with `(2,8,4)` at cost 9. The only difference is that `5` and `9` were matched to `8` and `4` crossed rather than `4` and `8` sorted: crossed costs `|5−8| + |9−4| = 3 + 5 = 8`, sorted costs `|5−4| + |9−8| = 1 + 1 = 2`. Uncrossing saved 6. Walking through exactly one swap like this, rather than asserting the lemma, is what makes the greedy convincing in the room.

Permutation Sorter — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given a permutation of `1..n`, two operations are allowed: 1. **Reverse** the whole array. 2. **Move the first element to the end** (a left rotation by one).

Return the **minimum operations** to sort the array ascending. The input is guaranteed sortable.

- **Insight (given):** the array must be a rotation of ascending order or a rotation of descending order — nothing else is reachable. Only a few shapes are cheaply reachable, so enumerate them.
- Note the asymmetry: rotation moves the **first** element to the **end** (left rotation only — you cannot rotate right).
- Edge cases: already sorted → 0; fully reversed; `n = 1` → 0; `n = 2`.

BRUTE-FORCE APPROACH

Idea. BFS over array states, applying both operations until the sorted state is reached.

Data structures. A queue of states plus a visited set of tuples.

Complexity. The reachable state space is `O(n)` rotations × 2 orientations = `O(n)` states, but each state is an `O(n)` tuple to hash and copy → `O(n²)` time and space.

Why it's not ideal. `O(n²)` on `n = 10⁵` is 10^{10} in both time and memory. And it's over-general: the reachable set is not an arbitrary graph, it's just `2n` states with a completely predictable structure. Once you characterise them, the answer is a small closed-form comparison.

Characterising the reachable states. Left rotations cycle through the `n` rotations of the current array; a reverse flips the orientation. So from a starting array `A`, the reachable set is `{rotations of A} ∪ {rotations of reverse(A)}`. For the sorted array to be reachable, `A` must be a rotation of `[1..n]` **or** a rotation of `[n..1]`.

OPTIMAL APPROACH

Idea (enumerate the small number of cheap plans).

Let `k` = the rotation offset such that rotating left `k` times sorts the array (if `A` is a rotation of ascending), and `k'` = the offset that sorts `reverse(A)`.

Plan A — `A` is a rotation of ascending order. Find the index `p` of value `1`. If `A` is exactly `[p..n-1] + [0..p-1]` of the sorted array, then `p` left rotations sort it. Cost = `p`.

Plan B — `A` is a rotation of descending order. Reverse once (cost 1), producing a rotation of ascending, then rotate. Find where value `1` sits in the reversed array; call it `q`. Cost = `1 + q`.

Take the minimum over whichever plans apply. Because both checks are `O(n)` and there are only two, the whole thing is linear.

Data structures. The array; a couple of index scans.

Complexity. Time `O(n)`. Space `O(n)` for the reversed copy (or `O(1)` with index arithmetic).

PYTHON CODE

```python
def min_operations_to_sort(arr: list[int]) -> int:
    """
    Operations: (1) reverse the whole array, (2) move the first element to the end.
    Return the minimum number of operations to sort ascending.

    Reachability: left rotations cycle through the n rotations of the current
    array, and a reverse flips orientation. So the reachable set is
        {rotations of arr}  U  {rotations of reverse(arr)}
    and the input is sortable only if it is a rotation of ascending or of
    descending order. That leaves just two plans to evaluate.
    """
    n = len(arr)
    if n <= 1:
        return 0

    target = list(range(1, n + 1))
```

```
    def rotations_to_sort(seq: list[int]) -> int | None:
        """If `seq` is a rotation of ascending order, return how many left
        rotations sort it; otherwise None."""
        # Value 1 must end up at index 0, so it must currently sit at the index
        # we rotate away — that index IS the rotation count.
        try:
            p = seq.index(1)
        except ValueError:
            return None

        # Verify the whole array really is that rotation, not just that 1 is
        # somewhere. O(n) check, no allocation of the rotated copy needed.
        for i in range(n):
            if seq[(p + i) % n] != target[i]:
                return None
        return p

    best = None

    # PLAN A: already a rotation of ascending -> pure rotations.
    a = rotations_to_sort(arr)
    if a is not None:
        best = a

    # PLAN B: a rotation of descending -> one reverse (cost 1), then rotations.
    b = rotations_to_sort(arr[::-1])
    if b is not None:
        cost = 1 + b
        best = cost if best is None else min(best, cost)

    # The problem guarantees sortability, so `best` is never None in valid input.
    return best if best is not None else -1
```

DRY RUN

Case 1 — already sorted. `arr = [1, 2, 3, 4]`

- Plan A: `index(1) = 0`. Verify rotation by 0: `seq[0..3] = [1,2,3,4]` matches target ✓ → cost **0**.
- Plan B: reversed is `[4,3,2,1]`, `index(1) = 3`. Verify: `seq[(3+0)%4]=1` ✓, `seq[(3+1)%4]=seq[0]=4` vs target `2` ✗ → `None`.

Answer: **0**. ✅

Case 2 — rotation of ascending. `arr = [3, 4, 1, 2]`

- Plan A: `index(1) = 2`. Verify: `seq[2]=1` ✓, `seq[3]=2` ✓, `seq[0]=3` ✓, `seq[1]=4` ✓ → cost **2**.
- Plan B: reversed is `[2,1,4,3]`, `index(1) = 1`. Verify: `seq[1]=1` ✓, `seq[2]=4` vs target `2` ✗ → `None`.

Answer: **2**. ✅ Trace it: `[3,4,1,2]` → rotate → `[4,1,2,3]` → rotate → `[1,2,3,4]` ✓ two operations.

Case 3 — fully reversed. `arr = [4, 3, 2, 1]`

- Plan A: `index(1) = 3`. Verify: `seq[3]=1` ✓, `seq[0]=4` vs target `2` ✗ → `None`.
- Plan B: reversed is `[1,2,3,4]`, `index(1) = 0`, verifies fully ✓ → cost `1 + 0 = 1`.

Answer: **1**. ✅ One reverse and you're done.

Case 4 — rotation of descending. `arr = [2, 1, 4, 3]`

- Plan A: `index(1) = 1`. Verify: `seq[1]=1` ✓, `seq[2]=4` vs target `2` ✗ → `None`.
- Plan B: reversed is `[3, 4, 1, 2]`, `index(1) = 2`, verifies (as in case 2) ✓ → cost `1 + 2 = 3`.

Answer: **3**. ✅ Trace: reverse `[2,1,4,3]` → `[3,4,1,2]`, then two rotations → `[1,2,3,4]` ✓.

The verification loop matters — without it, `index(1)` alone would accept arrays like `[3,1,4,2]` that aren't rotations of anything sorted, returning a confidently wrong answer on unsortable input.

DAY 11 — Thu Aug 27 · DP Part 2 & Design-in-Code

> **Four DP skeletons cover everything here.** Name the skeleton before writing a line. **1-D over prefixes** (139, 53, 152, 918, 300) — `dp[i]` describes the answer *ending at* or *up to* index `i`. **2-D over two strings** (1143, 72) — `dp[i][j]` compares prefixes of both; the recurrence branches on whether the characters match. **0/1 knapsack** (416, 494) — each item used at most once, so the inner loop runs **backwards**. Contrast Day 10's 322/518, where the forward inner loop gave unbounded reuse. Same table, opposite direction. **2-D grid** (62, 64) — `dp[r][c]` from the cell above and the cell to the left.
>
> **Design-in-code (146, 211, 380, 981, 588) is a different skill:** the algorithm is easy, the *data-structure choice* is the answer. For each one, state the required complexity per operation first, then pick structures that hit it. "Hash map plus doubly linked list" and "array plus index map" are the two combinations that recur.

---

139. Word Break

UNDERSTAND THE PROBLEM

Given a string and a dictionary, return whether the string can be segmented entirely into dictionary words (words reusable, any number of times).

- Every character must be consumed — partial segmentation doesn't count.
- Words may repeat, and a word may be a prefix of another.
- Edge cases: empty string → `True`; no valid split; a dictionary word longer than the string.

BRUTE-FORCE APPROACH

Idea. Recursively try every prefix that is a dictionary word, then recurse on the remainder.

Complexity. Time `O(2^n)` — every position is a potential cut point. Space `O(n)` stack.

Why it's not ideal. Exponential from re-solving the same suffix along many different cut paths: `"aaaa"` reaches the suffix `"aa"` via `a|a`, and again via `aa`. Memoising on the **start index** collapses it, and the bottom-up form is a clean 1-D DP over prefixes.

OPTIMAL APPROACH

Idea (1-D DP over prefixes). `dp[i]` = "can `s[0:i]` be fully segmented?" - `dp[0] = True` — the empty prefix is trivially segmented. - `dp[i] = True` if there exists `j < i` with `dp[j] == True` **and** `s[j:i]` in the dictionary.

Put the dictionary in a `set` for `O(1)` lookup. Two useful prunes: skip `j` values further back than the longest dictionary word, and `break` as soon as a valid `j` is found.

Data structures. A boolean array of length `n+1`; a `set` for the words.

Complexity. Time `O(n² · L)` where `L` is the average word length (from slicing and hashing), or `O(n · maxlen · L)` with the max-word-length prune. Space `O(n + dictionary)`.

PYTHON CODE

```python
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        words = set(wordDict)                  # O(1) membership
        max_len = max((len(w) for w in words), default=0)

        n = len(s)
        # dp[i] = "s[0:i] is fully segmentable"
        dp = [False] * (n + 1)
        dp[0] = True                           # empty prefix, vacuously true

        for i in range(1, n + 1):
            # PRUNE: a candidate word ending at i can be at most max_len long,
            # so j never needs to go further back than i - max_len. Without this
            # the inner loop is a full O(n) scan for every i.
            start = max(0, i - max_len)

            for j in range(start, i):
                # dp[j] means the prefix up to j is already solved; s[j:i] is the
                # candidate final word. Both must hold.
                if dp[j] and s[j:i] in words:
                    dp[i] = True
                    break                      # one witness is enough

        return dp[n]
```

DRY RUN

`s = "leetcode"`, `wordDict = ["leet", "code"]` → `max_len = 4`

`dp = [T, F, F, F, F, F, F, F, F]` (indices 0–8)

i	prefix `s[0:i]`	`j` range	checks	`dp[i]`
1	`l`	0	`dp[0]` ✓ and `"l"` in words? ✗	F
2	`le`	0–1	`"le"` ✗; `dp[1]=F` skip	F
3	`lee`	0–2	`"lee"` ✗; others `dp[j]=F`	F
4	`leet`	0–3	`dp[0]` ✓ and `"leet"` in words ✓	**T**
5	`leetc`	1–4	`dp[4]=T` and `"c"` ✗; rest `dp[j]=F`	F
6	`leetco`	2–5	`dp[4]=T` and `"co"` ✗	F
7	`leetcod`	3–6	`dp[4]=T` and `"cod"` ✗	F
8	`leetcode`	4–7	`dp[4]=T` and **`"code"`** ✓	**T**

Answer: `dp[8] = True`. ✅ — segmented as `"leet" + "code"`.

Negative case, `s = "catsandog"`, `wordDict = ["cats","dog","sand","and","cat"]`:

`dp[3] = T` (`cat`), `dp[4] = T` (`cats`), `dp[7] = T` (`cats` + `and`), `dp[8] = F`, `dp[9]`: candidates ending at 9 are `"og"`, `"dog"` from `j=6` — but `dp[6] = F` — and `"ndog"` from `j=5` with `dp[5]=F`. So `dp[9] = False` ✅.

The trap here is that `"cats"+"and"` reaches index 7 and `"og"` isn't a word, while `"cat"+"sand"` also reaches 7 — both dead ends. The DP explores both without re-deriving either, which is exactly what the exponential version fails to do.

53. Maximum Subarray

UNDERSTAND THE PROBLEM

Given an integer array, return the largest sum of any **contiguous** subarray (at least one element).

- "At least one element" means an all-negative array returns its largest element, **not** 0.
- Edge cases: all negatives; single element; all positives (→ the whole array).

BRUTE-FORCE APPROACH

Idea. Try every `(start, end)` pair with a running sum.

Complexity. Time `O(n²)` with the running-sum trick (`O(n³)` naive). Space `O(1)`.

Why it's not ideal. Quadratic at `n = 10⁵`. The insight (Kadane's): a subarray ending at index `i` either **extends** the best subarray ending at `i-1`, or **starts fresh** at `i`. That's a one-line recurrence with no nested loop.

OPTIMAL APPROACH

Idea (Kadane's algorithm). `cur = max(x, cur + x)` — extend or restart. Track `best = max(best, cur)` separately, because the running value may dip while the historical best stands.

Initialise both to `nums[0]`, **not** 0, so all-negative arrays are handled.

Data structures. Two scalars.

Complexity. Time `O(n)`. Space `O(1)`.

Divide and conquer gives `O(n log n)` — worth naming as the alternate framing, since some interviewers ask for it explicitly.

PYTHON CODE

```python
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        # Seed with nums[0], NOT 0. The subarray must be non-empty, so an
        # all-negative array like [-3, -1, -2] must return -1, not 0.
        cur = best = nums[0]

        for x in nums[1:]:
            # KADANE: a subarray ending at x either EXTENDS the best one ending
            # at the previous index, or STARTS FRESH at x. Whichever is larger.
            # Equivalent framing: if cur ever goes negative, it can only hurt
            # whatever follows, so discard it.
            cur = max(x, cur + x)

            # Track the historical maximum SEPARATELY — `cur` may dip below the
            # best we've already banked, and we must not lose it.
            best = max(best, cur)

        return best
```

DRY RUN

`nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`

x	`cur + x`	`max(x, cur+x)` → cur	best
−2	—	−2 (seed)	−2
1	`-2+1 = -1`	**`max(1, -1) = 1`** ← restart	1
−3	`1-3 = -2`	`max(-3, -2) = -2`	1
4	`-2+4 = 2`	**`max(4, 2) = 4`** ← restart	**4**
−1	`4-1 = 3`	`max(-1, 3) = 3`	4
2	`3+2 = 5`	`max(2, 5) = 5`	**5**
1	`5+1 = 6`	`max(1, 6) = 6`	**6**
−5	`6-5 = 1`	`max(-5, 1) = 1`	6
4	`1+4 = 5`	`max(4, 5) = 5`	6

Answer: `6` — the subarray `[4, -1, 2, 1]`. ✅

The two restarts (at `x = 1` and `x = 4`) are the algorithm discarding a negative prefix that could only drag down what follows. And the final two rows show why `best` must be tracked separately: `cur` fell to 1 and recovered only to 5, never beating the banked 6.

All-negative case, `nums = [-3, -1, -2]`: `cur = best = -3`; then `max(-1, -4) = -1` → best `-1`; then `max(-2, -3) = -2` → best stays `-1`. Answer **−1** ✅, which a 0-seeded version would have returned as 0.

---

152. Maximum Product Subarray

UNDERSTAND THE PROBLEM

Given an integer array (may contain **negatives and zeros**), return the largest product of any contiguous subarray.

- The difficulty over problem 53: a **large negative** product becomes the *best* the moment another negative arrives. So the minimum matters as much as the maximum.
- Zeros reset everything — the product through a zero is zero.
- Edge cases: a single negative; `[0, 2]`; an even vs odd count of negatives.

BRUTE-FORCE APPROACH

Idea. Every `(start, end)` with a running product.

Complexity. Time `O(n²)`, space `O(1)`.

Why it's not ideal. Quadratic. And the naive Kadane adaptation (`cur = max(x, cur * x)`) is **wrong**, which is the real lesson: with multiplication, the most negative running product is a latent maximum. You must carry both extremes.

OPTIMAL APPROACH

Idea (track both extremes). Maintain `cur_max` and `cur_min` — the largest and smallest products of subarrays ending at `i`. At each step the three candidates are `x`, `cur_max * x`, and `cur_min * x`: - `new_max = max(x, cur_max * x, cur_min * x)` - `new_min = min(x, cur_max * x, cur_min * x)`

Compute both from the OLD values simultaneously — updating `cur_max` first and then using it to compute `cur_min` is the classic bug.

Zeros need no special case: they force both extremes to 0, and the `x` term restarts the subarray at the next element.

Data structures. Three scalars.

Complexity. Time `O(n)`. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        cur_max = cur_min = best = nums[0]

        for x in nums[1:]:
            # WHY TRACK THE MINIMUM: multiplying by a negative SWAPS the roles —
            # the most negative running product becomes the largest. A plain
            # Kadane (`max(x, cur*x)`) silently loses that and is wrong here.
            candidates = (x, cur_max * x, cur_min * x)

            # Compute BOTH from the OLD cur_max/cur_min. Assigning cur_max first
            # and then reading it for cur_min is THE bug on this problem.
            cur_max, cur_min = max(candidates), min(candidates)

            best = max(best, cur_max)

        # Zeros need no branch: they drive both extremes to 0, and the bare `x`
        # candidate restarts the subarray on the next element.
        return best
```

DRY RUN

`nums = [2, 3, -2, 4]`

x	candidates (x, max·x, min·x)	cur_max	cur_min	best
2	seed	2	2	2
3	`(3, 6, 6)`	**6**	3	**6**
−2	`(-2, -12, -6)`	**−2**	**−12**	6
4	`(4, -8, -48)`	**4**	−48	6

Answer: `6` — the subarray `[2, 3]`. ✅

The case that proves the minimum is needed, `nums = [-2, 3, -4]`:

x	candidates	cur_max	cur_min	best
−2	seed	−2	−2	−2
3	`(3, -6, -6)`	**3**	**−6**	3
−4	**`(-4, 3·-4 = -12, -6·-4 = 24)`**	**24**	−12	**24**

Answer: `24` — the whole array `[-2, 3, -4]` → `-2 × 3 × -4 = 24` ✅.

The winning value came from `cur_min * x`, not `cur_max * x`. A single-extreme Kadane would have produced `max(-4, -12) = -4` and returned 3 — wrong by a factor of eight. Run this three-element trace out loud; it's the entire justification for the extra variable.

---

918. Maximum Sum Circular Subarray ★

UNDERSTAND THE PROBLEM

Given a **circular** array, return the maximum non-empty subarray sum, where a subarray may wrap around **once**.

- Two shapes are possible: a **normal** (non-wrapping) subarray, or a **wrapping** one.
- **Insight:** a wrapping subarray's complement is a *normal* subarray, so `wrapping_max = total - min_subarray_sum`.
- **The edge case that breaks the formula:** if every element is negative, the minimum subarray is the *whole array*, making `total - min = 0`, which corresponds to an **empty** subarray — forbidden. Detect it via `max_subarray < 0` and return that instead.
- Edge cases: all negatives; single element; no benefit to wrapping.

BRUTE-FORCE APPROACH

Idea. Duplicate the array (`nums + nums`) and run Kadane over every window of length ≤ `n`.

Complexity. Time `O(n²)` — Kadane alone would allow windows longer than `n`, so you must bound the length, which reintroduces a nested loop. Space `O(n)`.

Why it's not ideal. Quadratic, and the length bound is fiddly to get right. The complement trick replaces it with **two** linear Kadane passes and one subtraction.

OPTIMAL APPROACH

Idea (Kadane twice + complement).

1. Run Kadane for the **maximum** subarray sum → `max_sum`.
2. Run Kadane for the **minimum** subarray sum → `min_sum`. (Same code with `min` / `max` swapped.)
3. `total = sum(nums)`.
4. The wrapping candidate is `total - min_sum`.
5. If `max_sum < 0` (all elements negative), return `max_sum` — the complement would be the empty subarray.
6. Otherwise return `max(max_sum, total - min_sum)`.

Data structures. Four scalars.

Complexity. Time `O(n)` — one combined pass. Space `O(1)`.

PYTHON CODE

```python
class Solution:
    def maxSubarraySumCircular(self, nums: list[int]) -> int:
        total = 0
        # Two Kadanes running simultaneously in one pass.
        cur_max, best_max = 0, float('-inf')
        cur_min, best_min = 0, float('inf')

        for x in nums:
            total += x

            cur_max = max(x, cur_max + x)        # standard Kadane (maximum)
            best_max = max(best_max, cur_max)

            cur_min = min(x, cur_min + x)        # mirrored Kadane (minimum)
            best_min = min(best_min, cur_min)

        # ALL-NEGATIVE GUARD, and it is mandatory. If every element is negative,
        # the minimum subarray is the ENTIRE array, so total - best_min == 0 -
        # which corresponds to the EMPTY subarray, and the problem requires a
        # non-empty one. Return the (negative) best_max instead.
        if best_max < 0:
            return best_max

        # A wrapping subarray's COMPLEMENT is an ordinary contiguous subarray,
        # so maximising the wrap == minimising the complement.
        return max(best_max, total - best_min)
```

DRY RUN

`nums = [5, -3, 5]` — expect `10` (wrapping: `5 … 5`).

x	total	cur_max	best_max	cur_min	best_min
5	5	`max(5, 5) = 5`	5	`min(5, 5) = 5`	5
−3	2	`max(-3, 2) = 2`	5	**`min(-3, 2) = -3`**	**−3**
5	**7**	`max(5, 7) = 7`	**7**	`min(5, 2) = 2`	−3

`best_max = 7` (the whole array), `best_min = -3`, `total = 7`.

`best_max >= 0`, so answer = **`max(7, 7 - (-3)) = max(7, 10) = 10`**. ✅

Verify: the wrapping subarray is `nums[2], nums[0]` = `5 + 5 = 10` ✓. Its complement is `[-3]`, the minimum subarray — exactly as the formula says.

All-negative case, `nums = [-3, -2, -3]`:

x	total	cur_max	best_max	cur_min	best_min
−3	−3	−3	−3	−3	−3
−2	−5	`max(-2, -5) = -2`	**−2**	`min(-2, -5) = -5`	−5
−3	−8	`max(-3, -5) = -3`	−2	`min(-3, -8) = -8`	**−8**

`best_max = -2 < 0` → return **−2** ✅.

Without the guard, the formula would give `max(-2, -8 - (-8)) = max(-2, 0) = 0` — an empty subarray, which the problem forbids. That one branch is the difference between passing and failing this problem.

---

300. Longest Increasing Subsequence ★★

UNDERSTAND THE PROBLEM

Given an array, return the length of the longest **strictly increasing subsequence**. **Follow-up:** `O(n log n)`.

- Subsequence, not substring — elements need not be adjacent, only in order.
- Strictly increasing, so equal values don't extend a run.
- Edge cases: single element → 1; strictly decreasing → 1; all equal → 1.

BRUTE-FORCE APPROACH

Idea. Generate every subsequence and check whether it's increasing.

Complexity. Time `O(2ⁿ · n)`. Space `O(n)`.

Why it's not ideal. Exponential. The standard `O(n²)` DP — `dp[i]` = LIS ending at `i`, computed as `1 + max(dp[j])` over all `j < i` with `nums[j] < nums[i]` — is the obvious improvement and is worth writing first. But `n = 2500` makes `O(n²)` ~6 million (fine), and the follow-up explicitly wants `O(n log n)`.

OPTIMAL APPROACH — `O(N²)` DP (WRITE THIS FIRST)

`dp[i] = 1 + max(dp[j] for j < i if nums[j] < nums[i])`, default 1. Answer is `max(dp)`.

OPTIMAL APPROACH — `O(N LOG N)` PATIENCE SORTING

Idea. Maintain `tails`, where `tails[k]` = the **smallest possible tail value** of an increasing subsequence of length `k+1` seen so far. This array is always sorted ascending.

For each `x`, binary search for the first index with `tails[i] >= x`: - If found, **replace** `tails[i] = x` — same length, smaller tail, which can only help future extensions. - If not found (`x` exceeds every tail), **append** — the LIS just got longer.

The answer is `len(tails)`.

Critical caveat to state: `tails` is **not** a valid subsequence of the input — it's only a length bookkeeping device. If the problem asked for the actual subsequence, you'd need parent pointers. Volunteering that shows you understand the algorithm rather than having memorised it.

`bisect_left` gives strictly-increasing; `bisect_right` would give non-decreasing (LC 674-style variants).

Data structures. The `tails` array; `bisect`.

Complexity. Time `O(n log n)`. Space `O(n)`.

PYTHON CODE

```python
import bisect


class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        """O(n log n) patience sorting."""
        # tails[k] = the SMALLEST tail value among all increasing subsequences
        # of length k+1 found so far. Always sorted ascending, which is what
        # makes it binary-searchable.
        tails = []

        for x in nums:
            # bisect_left -> first index with tails[i] >= x, giving STRICTLY
            # increasing. bisect_right would allow equal values (non-decreasing),
            # which is a different problem - pick deliberately.
            i = bisect.bisect_left(tails, x)

            if i == len(tails):
                tails.append(x)      # x beats every tail -> the LIS grew
            else:
                tails[i] = x         # same length, smaller tail: strictly better
                                     # for anything that comes later

        # IMPORTANT: `tails` is NOT itself a valid subsequence of nums - it is a
        # length bookkeeping device only. Reconstructing the actual LIS needs
        # parent pointers. Say this; it distinguishes understanding from recall.
        return len(tails)


class SolutionQuadratic:
    """O(n^2) DP. Write this first - it's obvious, correct, and reconstructible."""
    def lengthOfLIS(self, nums: list[int]) -> int:
        n = len(nums)
        dp = [1] * n                 # every element alone is an LIS of length 1
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)
        return max(dp)
```

DRY RUN

`nums = [10, 9, 2, 5, 3, 7, 101, 18]`

x	`bisect_left(tails, x)`	action	tails after
10	0 (empty) → `== len`	append	`[10]`
9	0 (`10 >= 9`)	replace	`[9]`
2	0 (`9 >= 2`)	replace	`[2]`
5	1 (`2 < 5` , past end) → `== len`	append	`[2, 5]`
3	1 (`5 >= 3`)	replace	`[2, 3]`
7	2 → `== len`	append	`[2, 3, 7]`
101	3 → `== len`	append	`[2, 3, 7, 101]`
18	3 (`101 >= 18`)	replace	`[2, 3, 7, 18]`

Answer: **`len(tails) = 4`**. ✅ — e.g. the subsequence `[2, 3, 7, 18]` or `[2, 3, 7, 101]`.

The caveat made concrete: at the step `x = 3`, `tails` became `[2, 3]` — but `2` appears at index 2 and `3` at index 4, so that *is* a real subsequence here. Take `nums = [1, 5, 2]` instead: after processing, `tails = [1, 2]`, which is a valid subsequence. Now `nums = [4, 5, 1]`: `tails` becomes `[4]`, then `[4, 5]`, then `[1, 5]` — and `[1, 5]` is **not** a subsequence of `[4,5,1]`, since 1 comes after 5. The *length* 2 is still correct (`[4,5]`). That three-element example is the cleanest way to demonstrate the caveat if asked.

1143. Longest Common Subsequence ★

UNDERSTAND THE PROBLEM

Given two strings, return the length of their longest common **subsequence** (not substring).

- Subsequence: characters in order, not necessarily contiguous. `"ace"` is a subsequence of `"abcde"`.
- Edge cases: one string empty → 0; identical strings → their length; no common characters → 0.

BRUTE-FORCE APPROACH

Idea. Recursively compare: if the last characters match, `1 + LCS(m-1, n-1)`; otherwise `max(LCS(m-1, n), LCS(m, n-1))`.

Complexity. Time `O(2^(m+n))`. Space `O(m+n)`.

Why it's not ideal. Exponential from re-solving the same `(i, j)` prefix pair along different branches. There are only `m × n` distinct states, so memoisation collapses it to `O(mn)` — the canonical two-string DP.

OPTIMAL APPROACH

Idea (2-D DP over prefixes). `dp[i][j]` = LCS length of `text1[:i]` and `text2[:j]`. - If `text1[i-1] == text2[j-1]`: `dp[i][j] = dp[i-1][j-1] + 1` — both characters are consumed together. - Else: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])` — drop one character from either string. - Row 0 and column 0 are 0 (empty prefix).

Space optimisation: only the previous row is ever read, so two rows suffice — `O(min(m, n))` space.

Data structures. A 2-D table, or two rolling rows.

Complexity. Time `O(m · n)`. Space `O(m · n)`, reducible to `O(min(m,n))`.

PYTHON CODE

```python
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)

        # dp[i][j] = LCS of text1[:i] and text2[:j].
        # The (m+1) x (n+1) sizing gives a free zero row/column for empty
        # prefixes, which removes every boundary special case.
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    # Characters match -> consume BOTH and extend the diagonal.
                    dp[i][j] = dp[i - 1][j - 1] + 1
                else:
                    # No match -> drop one character from one string, take the
                    # better option. Both sub-answers are already computed.
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

        return dp[m][n]


class SolutionRolling:
    """O(min(m,n)) space — only the previous row is ever read."""
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        if len(text1) < len(text2):
            text1, text2 = text2, text1      # keep the inner dimension smaller
        m, n = len(text1), len(text2)

        prev = [0] * (n + 1)
        for i in range(1, m + 1):
            cur = [0] * (n + 1)
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    cur[j] = prev[j - 1] + 1
                else:
                    cur[j] = max(prev[j], cur[j - 1])
            prev = cur
        return prev[n]
```

DRY RUN

`text1 = "abcde"`, `text2 = "ace"` → expect `3` (`"ace"`).

	∅	a	c	e
∅	0	0	0	0
a	0	**1**	1	1
b	0	1	1	1
c	0	1	**2**	2
d	0	1	2	2
e	0	1	2	**3**

Key cells: - `dp[1][1]`: `'a' == 'a'` → `dp[0][0] + 1 = 1`. - `dp[2][1]`: `'b' vs 'a'` mismatch → `max(dp[1][1]=1, dp[2][0]=0) = 1`. - `dp[3][2]`: `'c' == 'c'` → `dp[2][1] + 1 = 2`. - `dp[5][3]`: `'e' == 'e'` → `dp[4][2] + 1 = 3`.

Answer: **`dp[5][3] = 3`**. ✅

The diagonal moves (at `(1,1)`, `(3,2)`, `(5,3)`) trace out exactly the matched characters `a`, `c`, `e` — which is how you'd reconstruct the actual subsequence if asked: walk backwards from `dp[m][n]`, moving diagonally on matches and toward the larger neighbour otherwise.

---

72. Edit Distance

UNDERSTAND THE PROBLEM

Given two strings, return the minimum number of **insert / delete / replace** operations to turn the first into the second.

- Also called Levenshtein distance.
- All three operations cost 1. (A variant adds transposition — Damerau-Levenshtein; mention it if they ask.)
- Edge cases: one string empty → the other's length; identical strings → 0.

BRUTE-FORCE APPROACH

Idea. Recursively try all three operations at each mismatch.

Complexity. Time `O(3^(m+n))`. Space `O(m+n)`.

Why it's not ideal. Exponential, same overlapping-subproblem structure as 1143 — only `m × n` distinct prefix pairs exist.

OPTIMAL APPROACH

Idea (2-D DP). `dp[i][j]` = edit distance between `word1[:i]` and `word2[:j]`. - **Base cases:** `dp[i][0] = i` (delete every character), `dp[0][j] = j` (insert every character). - **Match** (`word1[i-1] == word2[j-1]`): `dp[i][j] = dp[i-1][j-1]` — free, no operation needed. - **Mismatch:** `dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])`, which are **replace**, **delete**, and **insert** respectively.

Knowing which neighbour corresponds to which operation is what makes this writable rather than memorised — say it out loud as you fill the recurrence.

Data structures. A 2-D table (or two rolling rows).

Complexity. Time `O(m · n)`. Space `O(m · n)`, reducible to `O(min(m,n))`.

PYTHON CODE

```python
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        # BASE CASES: converting to/from an empty string costs one op per character.
        for i in range(m + 1):
            dp[i][0] = i        # delete all i characters
        for j in range(n + 1):
            dp[0][j] = j        # insert all j characters

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    # Characters already agree — no operation, just inherit.
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    # Each neighbour corresponds to ONE operation. Knowing which
                    # is which is what makes this derivable instead of memorised:
                    #   dp[i-1][j-1] -> REPLACE word1[i-1] with word2[j-1]
                    #   dp[i-1][j]   -> DELETE  word1[i-1]
                    #   dp[i][j-1]   -> INSERT  word2[j-1]
                    dp[i][j] = 1 + min(dp[i - 1][j - 1],
                                       dp[i - 1][j],
                                       dp[i][j - 1])

        return dp[m][n]
```

DRY RUN

`word1 = "horse"`, `word2 = "ros"` → expect `3`.

	∅	r	o	s
∅	0	1	2	3
h	1	1	2	3
o	2	2	**1**	2
r	3	**2**	2	2
s	4	3	3	**2**
e	5	4	4	**3**

Key cells: - `dp[1][1]` : `'h' vs 'r'` mismatch → `1 + min(dp[0][0]=0, dp[0][1]=1, dp[1][0]=1) = 1` (replace `h` → `r`). - `dp[2][2]` : `'o' == 'o'` → **`dp[1][1] = 1`** (free). - `dp[3][1]` : `'r' == 'r'` → **`dp[2][0] = 2`**. - `dp[5][3]` : `'e' vs 's'` mismatch → **`1 + min(dp[4][2]=3, dp[4][3]=2, dp[5][2]=4) = 3`**.

Answer: `3`. ✅

The three operations, reconstructed by walking back from `dp[5][3]` : replace `h` → `r`, delete `r` (the one in "horse" after `o`), delete `e`. Or equivalently: `horse` → `rorse` (replace) → `rose` (delete `r`) → `ros` (delete `e`). ✓

Being able to name the actual operation sequence from the table — not just the number — is the follow-up on this problem, and the neighbour-to-operation mapping in the comments is what makes it possible.

---

416. Partition Equal Subset Sum

UNDERSTAND THE PROBLEM

Given an array of **positive** integers, return whether it can be split into two subsets of **equal sum**.

- Reframe immediately: if `total` is odd, it's impossible. Otherwise the question is "can some subset sum to exactly `total // 2`?" — that's **subset-sum**, a 0/1 knapsack.
- **0/1**, not unbounded: each number is used at most once. That dictates a **backwards** inner loop.
- Edge cases: odd total → `False`; a single element → `False` (unless 0); all elements equal.

BRUTE-FORCE APPROACH

Idea. Enumerate all `2^n` subsets and check whether any sums to `total // 2`.

Complexity. Time `O(2^n)`, space `O(n)`.

Why it's not ideal. Exponential at `n = 200`. But the *reachable sums* are only `0..total/2`, and many subsets reach the same sum — so track reachable sums rather than subsets. That's `O(n · target)` states.

OPTIMAL APPROACH

Idea (0/1 knapsack, boolean, backwards inner loop). `dp[s]` = "is sum `s` reachable using some subset of the items processed so far". - `dp[0] = True`. - For each number `x`: for `s` from `target` **down to** `x`: `dp[s] |= dp[s - x]`.

Why backwards is mandatory. Iterating `s` ascending would read `dp[s - x]` *after* it was already updated with `x` in this same round — allowing `x` to be used twice. That's precisely the unbounded knapsack of LC 518. Descending guarantees `dp[s - x]` still reflects the state *before* `x` was considered, enforcing at-most-once. Same table, opposite direction, different problem — state this explicitly.

Data structures. A boolean array of length `target + 1`.

Complexity. Time `O(n · target)`. Space `O(target)`.

PYTHON CODE

```python
class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        total = sum(nums)

        # Odd total can never split into two equal integer halves.
        if total % 2 != 0:
            return False

        target = total // 2

        # dp[s] = "sum s is reachable with some subset of the items seen so far"
        dp = [False] * (target + 1)
        dp[0] = True              # the empty subset reaches 0

        for x in nums:
            # ***BACKWARDS.*** This is 0/1 knapsack: each number may be used AT
            # MOST ONCE. Descending guarantees dp[s-x] still holds the state from
            # BEFORE x was considered.
            # Ascending would read a dp[s-x] already updated with x in this same
            # round, letting x be reused — that is UNBOUNDED knapsack (LC 518).
            # Same table, opposite direction, different problem.
            for s in range(target, x - 1, -1):
                dp[s] = dp[s] or dp[s - x]

            if dp[target]:
                return True       # early exit once the target is reachable

        return dp[target]
```

DRY RUN

`nums = [1, 5, 11, 5]` → `total = 22`, `target = 11`.

`dp = [T, F, F, F, F, F, F, F, F, F, F, F]` (indices 0–11)

x = 1 (`s` from 11 down to 1): only `dp[1] |= dp[0] = T`. `dp` reachable sums: `{0, 1}`

x = 5 (`s` from 11 down to 5):

s	dp[s-5]	dp[s] after
6	`dp[1] = T`	**T**
5	`dp[0] = T`	**T**

Reachable: `{0, 1, 5, 6}`

x = 11 (`s` from 11 down to 11):

s	dp[s-11]	dp[s] after
11	`dp[0] = T`	**T** → early exit

Answer: `True` ✅ — the subset `{11}` sums to 11, and the complement `{1, 5, 5}` also sums to 11. ✓

The direction bug, made concrete. Take `nums = [3]`, `target = 6` (pretend). Backwards: `s = 6`, `dp[3]` is False → `dp[6]` stays False ✓ (you can't make 6 from one 3). Ascending: `s = 3` sets `dp[3] = True`; then `s = 6` reads `dp[3]` — now True — and sets `dp[6] = True` ✗, having used the single 3 twice. That two-line example is the fastest way to demonstrate why the loop runs backwards.

---

494. Target Sum ★

UNDERSTAND THE PROBLEM

Given an array and a target, assign `+` or `-` to each element. Return **how many** assignments reach the target.

- Every element must get a sign — you can't skip any.
- Negative targets are possible; so is a target unreachable in parity.
- Edge cases: `target` exceeding `sum(nums)` → 0; zeros in the array (each zero doubles the count, since `+0` and `-0` are distinct assignments).

BRUTE-FORCE APPROACH

Idea. Recursively try both signs at each index, counting the assignments that hit the target.

Complexity. Time `O(2^n)`, space `O(n)`.

Why it's not ideal. `n = 20` gives a million — borderline acceptable, which is worth saying. But the elegant reduction is what's being tested.

The reduction, in full. Let `P` be the set assigned `+` and `N` the set assigned `-`.

```
sum(P) - sum(N) = target
sum(P) + sum(N) = total
———————————————————————  (add)
2·sum(P) = target + total
sum(P) = (target + total) / 2
```

So the question becomes: **how many subsets sum to `(target + total) / 2`?** That's a counting subset-sum — a 0/1 knapsack with counts instead of booleans.

Two feasibility guards fall out: `(target + total)` must be **non-negative** and **even**, otherwise the answer is 0.

OPTIMAL APPROACH

Idea (count subsets summing to `s = (target + total) // 2`).

1. Guard: if `abs(target) > total` or `(target + total)` is odd → return 0.
2. `dp[0] = 1`, `dp[j] = 0` otherwise.
3. For each `x`: for `j` from `s` **down to** `x`: `dp[j] += dp[j - x]` (backwards — 0/1).
4. Return `dp[s]`.

Data structures. An integer array of length `s + 1`.

Complexity. Time `O(n · s)`. Space `O(s)`.

PYTHON CODE

```python
class Solution:
    def findTargetSumWays(self, nums: list[int], target: int) -> int:
        total = sum(nums)

        # REDUCTION (derive this out loud, don't just assert it):
        #   sum(P) - sum(N) = target
        #   sum(P) + sum(N) = total
        #   => 2*sum(P) = target + total
        #   => sum(P) = (target + total) / 2
        # So: count the subsets summing to that value.
        need = target + total

        # Two feasibility guards fall straight out of the algebra:
        if need < 0 or need % 2 != 0:
            return 0                  # non-integer or negative subset sum
        if abs(target) > total:
            return 0                  # target simply out of reach

        s = need // 2

        # dp[j] = number of subsets of the processed items summing to exactly j.
        dp = [0] * (s + 1)
        dp[0] = 1                     # exactly one way to make 0: the empty subset

        for x in nums:
            # BACKWARDS again - 0/1 knapsack, each number gets exactly one sign
            # and so is "used" at most once on the + side.
            for j in range(s, x - 1, -1):
                dp[j] += dp[j - x]

        return dp[s]
```

DRY RUN

`nums = [1, 1, 1, 1, 1]`, `target = 3` → expect `5`.

`total = 5`, `need = 3 + 5 = 8`, even ✓, `s = 4`.

`dp = [1, 0, 0, 0, 0]`

x = 1 (j from 4 down to 1): `dp[1] += dp[0]` → `dp = [1, 1, 0, 0, 0]`

x = 1 (2nd): `dp[4] += dp[3]=0`; `dp[3] += dp[2]=0`; `dp[2] += dp[1]=1` → 1; `dp[1] += dp[0]=1` → 2. `dp = [1, 2, 1, 0, 0]`

x = 1 (3rd): `dp[4] += dp[3]=0`; `dp[3] += dp[2]=1` → 1; `dp[2] += dp[1]=2` → 3; `dp[1] += dp[0]=1` → 3. `dp = [1, 3, 3, 1, 0]`

x = 1 (4th): `dp[4] += dp[3]=1` → 1; `dp[3] += dp[2]=3` → 4; `dp[2] += dp[1]=3` → 6; `dp[1] += 1` → 4. `dp = [1, 4, 6, 4, 1]`

x = 1 (5th): `dp[4] += dp[3]=4` → **5**; `dp[3] += dp[2]=6` → 10; `dp[2] += dp[1]=4` → 10; `dp[1] += 1` → 5. `dp = [1, 5, 10, 10, 5]`

Answer: **`dp[4] = 5`**. ✅

Verify: we need 4 of the five 1s to be `+` and 1 to be `-`, giving `4 - 1 = 3` ✓. There are `C(5,4) = 5` ways to choose which one is negative — matching exactly. And the final `dp` row `[1,5,10,10,5]` is the binomial coefficients `C(5,0..4)`, which is a satisfying sanity check to point out.

Parity guard case, `nums = [1, 2]`, `target = 2`: `total = 3`, `need = 5`, odd → return **0** ✅. Indeed `+1+2 = 3`, `+1-2 = -1`, `-1+2 = 1`, `-1-2 = -3` — 2 is unreachable.

---

62. Unique Paths

UNDERSTAND THE PROBLEM

On an `m × n` grid, a robot starts top-left and may move only **right or down**. Return how many distinct paths reach the bottom-right.

- Every path has exactly `(m-1) + (n-1)` moves, of which `m-1` are downs — so the answer is the binomial coefficient `C(m+n-2, m-1)`.
- Edge cases: `m = 1` or `n = 1` → exactly 1 path.

BRUTE-FORCE APPROACH

Idea. Recursively count: `paths(r, c) = paths(r+1, c) + paths(r, c+1)`.

Complexity. Time `O(2^(m+n))`, space `O(m+n)`.

Why it's not ideal. Exponential from recomputing the same cell along many prefixes. Only `m × n` distinct cells exist → memoise, or fill bottom-up.

OPTIMAL APPROACH

Idea (2-D grid DP). `dp[r][c] = dp[r-1][c] + dp[r][c-1]`, with the first row and first column all 1 (only one way to walk straight along an edge).

Space optimisation: only the previous row is read, so one rolling row of length `n` suffices: `dp[c] += dp[c-1]`, which reads the updated left neighbour and the not-yet-updated cell above — exactly the two terms needed.

The `O(min(m,n))`-time answer: compute `C(m+n-2, m-1)` directly with `math.comb`. Mention it; it's strictly better and shows you saw the combinatorial structure.

Data structures. A 1-D rolling array.

Complexity. DP: time `O(m · n)`, space `O(n)`. Combinatorial: time `O(min(m,n))`, space `O(1)`.

PYTHON CODE

```python
import math


class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # ROLLING ROW. dp[c] holds the path count for the current row.
        # Start as all 1s: the first row has exactly one path to each cell
        # (walk straight right).
        dp = [1] * n

        for _ in range(1, m):
            for c in range(1, n):
                # dp[c]     is still the value from the row ABOVE (not yet updated)
                # dp[c - 1] is already the value for THIS row (left neighbour)
                # So this single line is exactly dp[r][c] = dp[r-1][c] + dp[r][c-1].
                dp[c] += dp[c - 1]

        return dp[n - 1]

    def uniquePaths_combinatorial(self, m: int, n: int) -> int:
        """O(min(m,n)) - every path is a fixed sequence of (m-1) downs and (n-1)
        rights, so the count is just the number of ways to choose which moves are
        downs. Mention this; it shows you saw the structure, not just the table."""
        return math.comb(m + n - 2, m - 1)
```

DRY RUN

`m = 3`, `n = 7`

`dp = [1, 1, 1, 1, 1, 1, 1]` (row 0)

Row 1 (`c` from 1 to 6):

c	`dp[c]` before	`dp[c-1]`	`dp[c]` after
1	1	1	2
2	1	2	3
3	1	3	4
4	1	4	5
5	1	5	6
6	1	6	7

`dp = [1, 2, 3, 4, 5, 6, 7]`

Row 2:

c	before	`dp[c-1]`	after
1	2	1	3
2	3	3	6
3	4	6	10
4	5	10	15
5	6	15	21
6	7	21	**28**

Answer: `dp[6] = 28`. ✅

Cross-check combinatorially: `C(3 + 7 - 2, 3 - 1) = C(8, 2) = 28` ✓. Running both and confirming they agree is a strong closing move on this problem.

Note the rolling row's rows are the diagonals of Pascal's triangle — `[1,1,1,…]`, `[1,2,3,…]`, `[1,3,6,10,…]` — which is the same combinatorial fact showing up in the table.

---

64. Minimum Path Sum

UNDERSTAND THE PROBLEM

Given a grid of non-negative costs, moving only **right or down**, return the minimum total cost from top-left to bottom-right.

- Same movement model as 62, but minimising a sum rather than counting paths.
- Non-negative costs mean no negative-cycle concerns; a plain DP suffices (no Dijkstra needed).
- Edge cases: a 1×1 grid; a single row or column; all zeros.

BRUTE-FORCE APPROACH

Idea. Recursively explore both moves, taking the minimum.

Complexity. Time `O(2^(m+n))`, space `O(m+n)`.

Why it's not ideal. Exponential, same overlapping-cell structure as 62.

OPTIMAL APPROACH

Idea (2-D DP). `dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1])`, with the first row and column accumulated as prefix sums (only one way to reach them).

In-place variant: mutate `grid` directly for `O(1)` extra space — ask whether that's allowed.

Data structures. A rolling row, or the grid itself.

Complexity. Time `O(m · n)`. Space `O(n)` rolling, or `O(1)` in place.

PYTHON CODE

```python
class Solution:
    def minPathSum(self, grid: list[list[int]]) -> int:
        if not grid or not grid[0]:
            return 0

        rows, cols = len(grid), len(grid[0])

        # In-place: O(1) extra space. ASK whether mutating the input is allowed —
        # if not, use a rolling row of length `cols` instead.
        for r in range(rows):
            for c in range(cols):
                if r == 0 and c == 0:
                    continue                        # start cell: cost is itself
                elif r == 0:
                    grid[r][c] += grid[r][c - 1]    # top row: only from the left
                elif c == 0:
                    grid[r][c] += grid[r - 1][c]    # left column: only from above
                else:
                    # Interior: come from above or from the left, take the cheaper.
                    # Both are already final when we read them, because we fill
                    # row by row, left to right.
                    grid[r][c] += min(grid[r - 1][c], grid[r][c - 1])

        return grid[rows - 1][cols - 1]
```

DRY RUN

```
grid = [[1, 3, 1],
        [1, 5, 1],
        [4, 2, 1]]
```

Filling in place:

cell	rule	computation	value
(0,0)	start	—	**1**
(0,1)	top row	`3 + 1`	**4**
(0,2)	top row	`1 + 4`	**5**
(1,0)	left col	`1 + 1`	**2**
(1,1)	interior	`5 + min(4, 2) = 5 + 2`	**7**
(1,2)	interior	`1 + min(5, 7) = 1 + 5`	**6**
(2,0)	left col	`4 + 2`	**6**
(2,1)	interior	`2 + min(7, 6) = 2 + 6`	**8**
(2,2)	interior	`1 + min(6, 8) = 1 + 6`	**7**

Final grid:

```
1  4  5
2  7  6
6  8  7
```

Answer: **`grid[2][2] = 7`**. ✅

Trace the optimal path backwards from `(2,2)`: it came from `(1,2)=6`, which came from `(0,2)=5`, which came from `(0,1)=4`, from `(0,0)=1`. So the path is `1 → 3 → 1 → 1 → 1` = **7** ✓.

Note `(1,1)` chose the value from above-left correctly: `min(dp[0][1]=4, dp[1][0]=2) = 2`, giving 7 — the cheaper route into the middle came from the left column, not the top row. A greedy "always go where the raw grid value is smaller" would have picked differently and gotten a worse total; the DP compares *accumulated* costs, which is the distinction worth naming.

---

146. LRU Cache ★★

UNDERSTAND THE PROBLEM

Design a fixed-capacity cache with `get(key)` and `put(key, value)`, **both O(1)**, evicting the **least recently used** entry when full.

- Both `get` and `put` count as "uses" and refresh recency.
- `get` on a missing key returns `-1` and does **not** create an entry.
- `put` on an existing key updates the value **and** refreshes recency, without evicting.
- Edge cases: capacity 1; `put` on an existing key when full (no eviction); repeated `get` on the same key.

BRUTE-FORCE APPROACH

Idea. A dict plus a `last_used` timestamp per entry. On eviction, scan all entries for the minimum timestamp.

Data structures. One dict; a counter.

Complexity. `get` `O(1)`, `put` **`O(n)`** on a full cache.

Why it's not ideal. The problem demands `O(1)` for both, and a cache whose insert degrades to linear once warm has defeated its own purpose. The fix: maintain recency **order incrementally** in a structure supporting `O(1)` move-to-front and `O(1)` tail removal — a **doubly linked list**, paired with a hash map from key to node so lookup stays `O(1)`.

The hash-map-plus-doubly-linked-list combination is the answer, and being able to name *why each half is needed* is the point: the map gives `O(1)` **find**, the list gives `O(1)` **reorder**. Neither alone suffices.

OPTIMAL APPROACH

Idea. Doubly linked list with sentinel `head` and `tail` nodes (so no null checks anywhere), most-recent next to `head`, least-recent next to `tail`. A dict maps `key → node`.

- `get(key)`: dict lookup → unlink the node → re-insert right after `head` → return its value.
- `put(key, value)`: if present, update and move to front. Else create, insert at front, and if over capacity remove the node before `tail` and delete its key from the dict.

Sentinels matter: without them, every unlink/insert needs branches for "is this the head/tail?", which is where the bugs live. With them, `_remove` and `_add_front` are four unconditional pointer assignments each.

Data structures. `dict[key → Node]`; a doubly linked list with two sentinels.

Complexity. `get` and `put` both `O(1)`. Space `O(capacity)`.

PYTHON CODE

```python
class _Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=0, value=0):
        self.key = key          # stored so eviction can delete from the dict
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    """
    Hash map + doubly linked list. Each half does one job and neither alone works:
      * the MAP gives O(1) find-by-key
      * the LIST gives O(1) reorder and O(1) tail eviction
    Order: head <-> most-recent <-> ... <-> least-recent <-> tail
    """

    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache: dict[int, _Node] = {}

        # SENTINELS. head and tail are dummy nodes that always exist, so unlink
        # and insert are unconditional pointer writes — no "is this the first/last
        # node?" branching anywhere, which is where LRU bugs actually live.
```

```
        self.head = _Node()
        self.tail = _Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    # ---------- list primitives, both O(1) ----------
    def _remove(self, node: _Node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_front(self, node: _Node) -> None:
        """Insert directly after head — the most-recently-used position."""
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    # ---------- public API ----------
    def get(self, key: int) -> int:
        node = self.cache.get(key)
        if node is None:
            return -1               # miss: do NOT create an entry

        # A read counts as a use -> refresh recency.
        self._remove(node)
        self._add_front(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        node = self.cache.get(key)

        if node is not None:
            # Update in place and refresh recency. No eviction — the size is
            # unchanged, so a full cache must NOT evict on an overwrite.
            node.value = value
            self._remove(node)
            self._add_front(node)
            return

        if len(self.cache) >= self.capacity:
            # Evict the node just before the tail sentinel = least recently used.
            lru = self.tail.prev
            self._remove(lru)
            # THIS is why the node stores its key: without it we'd have no way to
            # delete the dict entry, and the map would leak forever.
            del self.cache[lru.key]

        node = _Node(key, value)
        self.cache[key] = node
        self._add_front(node)
```

DRY RUN

`LRUCache(2)`, then `put(1,1)`, `put(2,2)`, `get(1)`, `put(3,3)`, `get(2)`, `put(4,4)`, `get(1)`, `get(3)`, `get(4)`

op	action	list (head → tail)	cache keys	returns
`put(1,1)`	insert front	`1`	`{1}`	
`put(2,2)`	insert front	`2, 1`	`{1,2}`	
`get(1)`	hit → move to front	`1, 2`	`{1,2}`	**1**
`put(3,3)`	full → evict tail-prev = **2**; insert 3	`3, 1`	`{1,3}`	
`get(2)`	miss	`3, 1`	`{1,3}`	**−1**
`put(4,4)`	full → evict **1**; insert 4	`4, 3`	`{3,4}`	
`get(1)`	miss			**−1**
`get(3)`	hit → move to front	`3, 4`	`{3,4}`	**3**
`get(4)`	hit → move to front	`4, 3`	`{3,4}`	**4**

✅ All correct.

The critical step is `put(3,3)`: key `2` was evicted rather than key `1`, because the earlier `get(1)` had refreshed `1`'s recency. A cache that only tracked *insertion* order would have evicted `1` — that's the difference between LRU and FIFO, and it's the specific behaviour a test harness checks.

---

211. Design Add and Search Words Data Structure

UNDERSTAND THE PROBLEM

Design a structure supporting `addWord(word)` and `search(word)`, where the search string may contain `.` matching **any single letter**.

- `.` matches exactly one character, not zero or many.
- Words consist of lowercase letters; the search pattern may mix letters and dots.
- Edge cases: a pattern that is all dots (matches any word of that length); searching before adding anything; a word that is a prefix of another (`"bad"` vs `"badly"` — only exact-length matches count).

BRUTE-FORCE APPROACH

Idea. Keep a list (or set) of words; on search, scan every stored word and compare character by character, treating `.` as a wildcard.

Data structures. A list of strings, or a `dict[length → list of words]`.

Complexity. `addWord` `O(1)`; `search` `O(N · L)` for `N` stored words of length `L`.

Why it's not ideal. With up to `10⁴` words and `10⁴` searches that's `10⁸`+ character comparisons. The redundancy is specific: words sharing a prefix are compared against the pattern's prefix repeatedly. A **trie** shares that work — one traversal descends all matching prefixes at once, and a literal character prunes every non-matching branch instantly.

OPTIMAL APPROACH

Idea (trie with wildcard DFS).

- `addWord`: ordinary trie insertion, marking the terminal node.
- `search`: recursive DFS over `(node, index)`:
- If the character is a **letter**, follow that single child or fail — `O(1)` and fully pruned.
- If it's a **dot**, recurse into **every** child. This is the only branching case.
- At the end of the pattern, return the node's `is_word` flag.

Complexity depends on how many dots there are: with `k` dots the search can branch `26^k` in the worst case, but a leading dot is the expensive one — dots deeper in the pattern have already been pruned by the preceding letters.

Data structures. Trie nodes with a `children` dict and an `is_word` boolean.

Complexity. `addWord` `O(L)`. `search` `O(L)` with no dots; `O(26^d · L)` worst case with `d` dots. Space `O(total characters added)`.

PYTHON CODE

```python
class _TrieNode:
    __slots__ = ("children", "is_word")

    def __init__(self):
        self.children: dict[str, "_TrieNode"] = {}
        self.is_word = False        # True only at the END of an inserted word


class WordDictionary:
    def __init__(self):
        self.root = _TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, _TrieNode())
        node.is_word = True         # marks a complete word, not just a prefix

    def search(self, word: str) -> bool:
        def dfs(node: "_TrieNode", i: int) -> bool:
            if i == len(word):
                # Must be a COMPLETE word, not merely a prefix — searching "bad"
                # when only "badly" was added must return False.
                return node.is_word

            ch = word[i]

            if ch == '.':
                # WILDCARD: the only branching case. Try every child; one match
                # is enough. Worst case 26 branches per dot, which is why a
                # LEADING dot is expensive and a trailing one is nearly free
                # (the preceding letters have already pruned the tree).
                for child in node.children.values():
                    if dfs(child, i + 1):
                        return True
                return False

            # Literal character: follow exactly one edge, or fail immediately.
            # This is the pruning that beats scanning every stored word.
            nxt = node.children.get(ch)
            return dfs(nxt, i + 1) if nxt else False

        return dfs(self.root, 0)
```

DRY RUN

`addWord("bad")`, `addWord("dad")`, `addWord("mad")`, then several searches.

Trie after the three inserts:

```
root
 ├─ b ─ a ─ d*      (* = is_word)
 ├─ d ─ a ─ d*
 └─ m ─ a ─ d*
```

search	trace	result
`"pad"`	root has no `p` child → immediate fail	**False** ✅
`"bad"`	`b` ✓ → `a` ✓ → `d` ✓, `is_word = True`	**True** ✅
`".ad"`	dot → try `b` : `a` ✓ `d` ✓ `is_word` ✓ → **True** on the first branch	**True** ✅
`"b.."`	`b` ✓ → dot → only child `a` → dot → only child `d`, `is_word` ✓	**True** ✅
`"ba"`	`b` ✓ → `a` ✓, `i == len`, but node `ba` has `is_word = False`	**False** ✅

The `"pad"` case shows the pruning: it failed after **one** dict lookup, whereas the brute force would have compared `"pad"` against all three stored words character by character.

The `"b.."` case shows why dots deeper in the pattern are cheap: the leading `b` narrowed the search to a single branch, so each subsequent dot had only one child to try. Contrast `"..d"`, which branches three ways at the first dot. Pointing out that asymmetry is the follow-up answer on this problem.

The `"ba"` case is the `is_word` flag earning its keep — without it, every prefix of every stored word would match.

---

380. Insert Delete GetRandom O(1) ★

UNDERSTAND THE PROBLEM

Design a set supporting `insert`, `remove`, and **uniform-random** element access — all in average `O(1)`.

- `insert` returns whether the item was newly added; `remove` returns whether it was present.
- `getRandom` must be **uniformly** distributed over the current contents.
- Edge cases: removing the last element; removing the only element; `getRandom` on a single-element set.

BRUTE-FORCE APPROACH

Idea (a). A Python `set`: `insert` and `remove` are `O(1)`, but `getRandom` requires `random.choice(list(s))` — **`O(n)`** to materialise the list. **Idea (b).** A list: `getRandom` is `O(1)` by index, `insert` is `O(1)` by append, but `remove` requires finding the element (`O(n)`) and then shifting (`O(n)`).

Why neither is ideal. Each structure gives two of the three operations. A set has no *index* to sample by; a list has no *lookup* to remove by. The fix is to **combine them**: an array for indexable random access, plus a dict mapping value → its index in the array.

The remaining problem is that removing from the middle of an array is `O(n)`. That's solved by the **swap-with-last** trick: since the set is unordered, you may move the last element into the vacated slot and pop the tail — both `O(1)`.

OPTIMAL APPROACH

Idea (array + index map, remove by swap-with-last).

- `values: list` — the elements, in arbitrary order. Indexable → `getRandom` is `random.choice(values)`, `O(1)` and uniform.
- `index: dict[value → position in values]` — `O(1)` membership and location.
- `insert(v)`: if present, `False`. Else append to `values`, record `index[v] = len-1`, `True`.
- `remove(v)`: if absent, `False`. Else: look up its position `i`; move `values[-1]` into slot `i`; update that moved element's index; `pop()` the tail; delete `v` from the map.

The ordering inside `remove` is where bugs live: you must update the moved element's index *before* (or consistently with) popping, and you must handle the case where `v` **is** the last element — then the "moved" element is `v` itself, and a careless update writes a stale index into the map.

Data structures. A list and a dict.

Complexity. All three operations `O(1)` average. Space `O(n)`.

PYTHON CODE

```python
import random


class RandomizedSet:
    """
    Array + index map. Each half supplies what the other lacks:
      * the ARRAY gives O(1) uniform random access (you can sample an index)
      * the DICT  gives O(1) membership and location (you can find an element)
    Removing from the middle of an array is normally O(n); the SWAP-WITH-LAST
    trick makes it O(1), and it is legal precisely because a set is unordered.
    """

    def __init__(self):
        self.values: list[int] = []
        self.index: dict[int, int] = {}     # value -> its position in `values`

    def insert(self, val: int) -> bool:
        if val in self.index:
            return False                    # already present, no change

        self.index[val] = len(self.values)  # position it is about to occupy
        self.values.append(val)
        return True

    def remove(self, val: int) -> bool:
```

```
        if val not in self.index:
            return False

        i = self.index[val]                  # slot being vacated
        last = self.values[-1]

        # SWAP-WITH-LAST: overwrite the vacated slot with the tail element, then
        # pop the tail. Both O(1). Order is unspecified for a set, so relocating
        # an arbitrary element is harmless.
        self.values[i] = last
        self.index[last] = i
        # NOTE: when val IS the last element, `last == val` and the line above
        # writes index[val] = i — a stale entry. The delete below removes it, so
        # the sequence is still correct. Doing the delete FIRST would break the
        # general case instead. Trace both orderings before you commit.

        self.values.pop()
        del self.index[val]
        return True

    def getRandom(self) -> int:
        # Uniform over the current contents: every element occupies exactly one
        # array slot, and random.choice picks a slot uniformly.
        return random.choice(self.values)
```

DRY RUN

op	before	action	`values` after	`index` after	returns
`insert(1)`	empty	append at 0	`[1]`	`{1:0}`	**True**
`remove(2)`		`2` absent	`[1]`	`{1:0}`	**False**
`insert(2)`		append at 1	`[1, 2]`	`{1:0, 2:1}`	**True**
`getRandom()`		uniform over `[1,2]`			`1` or `2`, each 50%
`remove(1)`	`i = 0`, `last = 2`	`values[0] = 2`; `index[2] = 0`; pop; `del index[1]`	`[2]`	`{2:0}`	**True**
`insert(2)`		`2` present	`[2]`	`{2:0}`	**False**
`getRandom()`		only `2`			`2`

✅ All correct.

The remove-the-last-element edge case, starting from `values = [1, 2]`, `index = {1:0, 2:1}`, call `remove(2)`:

step	effect
`i = index[2] = 1`, `last = values[-1] = 2`	`last` **is** `val`
`values[1] = 2`	no-op
`index[2] = 1`	writes a stale-looking entry — but it's about to be deleted
`values.pop()`	`values = [1]`
`del index[2]`	`index = {1:0}` ✓

Final state is correct. Reversing the last two lines (`del` before the index update) would delete `index[2]` and then immediately re-add it, leaving a phantom entry pointing at a popped slot — and the next `getRandom` or `remove` would misbehave. Walking this exact two-element case is how you prove the ordering is deliberate.

981. Time Based Key-Value Store

UNDERSTAND THE PROBLEM

Design a store with `set(key, value, timestamp)` and `get(key, timestamp)` returning the value at the **largest timestamp ≤ the query**, or `""` if none.

- Timestamps for a given key are **strictly increasing** across calls to `set` (LeetCode guarantee) — so the per-key list is already sorted and needs no sorting.
- `get` with a timestamp before every stored one returns `""`.
- Edge cases: key never set; exact timestamp match; query timestamp beyond the latest.

BRUTE-FORCE APPROACH

Idea. Store a list of `(timestamp, value)` per key; on `get`, scan linearly backwards for the first timestamp `≤` the query.

Complexity. `set` `O(1)`; `get` `O(n)` per key.

Why it's not ideal. With `2·10⁵` calls, a linear scan per `get` is `O(n²)` overall. The per-key list is **already sorted** by the problem's guarantee, and "largest element ≤ x" in a sorted list is exactly `upper_bound − 1` — a binary search, `O(log n)`.

Note that the guarantee is what makes this cheap. If timestamps could arrive out of order, you'd need a sorted container (`SortedList`) or to sort lazily on first read — worth naming as the follow-up.

OPTIMAL APPROACH

Idea (per-key sorted list + binary search).

- `store: dict[key → list of (timestamp, value)]`, appended in increasing timestamp order.
- `get(key, t)`: binary search the key's list for the rightmost entry with `timestamp <= t`. Return its value, or `""` if the search lands at index 0.

Using `bisect_right` on a list of tuples with a sentinel value is subtle; the cleanest form is to keep **two parallel lists** (timestamps and values) so `bisect_right(times, t)` is unambiguous, or to hand-roll the boundary search.

Data structures. A dict of two parallel lists (or a list of tuples plus a manual search).

Complexity. `set` `O(1)`; `get` `O(log n)`. Space `O(total sets)`.

PYTHON CODE

```python
import bisect
from collections import defaultdict


class TimeMap:
    """
    Per key: two PARALLEL lists, kept in increasing timestamp order.
    The problem guarantees set() is called with strictly increasing timestamps
    per key, so appending keeps them sorted — no sorting needed anywhere.
    (If that guarantee were dropped you'd need a SortedList, or a lazy sort on
    first read. Worth naming — it's the natural follow-up.)
    """

    def __init__(self):
        self.times: dict[str, list[int]] = defaultdict(list)
        self.values: dict[str, list[str]] = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        # O(1) amortised append. Sorted by construction.
        self.times[key].append(timestamp)
        self.values[key].append(value)

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.times:
            return ""

        times = self.times[key]

        # bisect_right gives the first index with times[i] > timestamp, so
        # i-1 is the LARGEST timestamp <= the query — exactly what's asked for.
        # Parallel lists (rather than a list of tuples) keep this unambiguous:
        # bisecting tuples would require a sentinel value for the second field.
        i = bisect.bisect_right(times, timestamp)

        if i == 0:
            return ""          # every stored timestamp is strictly greater

        return self.values[key][i - 1]
```

DRY RUN

```
set("foo", "bar", 1)
get("foo", 1)     -> ?
get("foo", 3)     -> ?
set("foo", "bar2", 4)
get("foo", 4)     -> ?
get("foo", 5)     -> ?
get("foo", 0)     -> ?
```

State after both `set` calls: `times["foo"] = [1, 4]`, `values["foo"] = ["bar", "bar2"]`

query `t`	`bisect_right([1,4], t)`	`i`	`i == 0`?	result
1	first index with value `> 1` → index 1	1	no	`values[0] = "bar"` ✅
3	first index `> 3` → index 1	1	no	`values[0] = "bar"` ✅
4	first index `> 4` → index 2	2	no	`values[1] = "bar2"` ✅
5	first index `> 5` → index 2	2	no	`values[1] = "bar2"` ✅
0	first index `> 0` → index 0	**0**	**yes**	`""` ✅

All correct.

Why `bisect_right` and not `bisect_left`: at `t = 4`, `bisect_left([1,4], 4)` returns `1`, so `i-1 = 0` and we'd return `"bar"` — but timestamp 4 is an **exact match** and should return `"bar2"`. `bisect_right` returns `2`, giving `i-1 = 1` ✓. Testing the exact-match case is what catches this, and it's a one-line difference that's easy to get backwards under pressure.

588. Design In-Memory File System ★

UNDERSTAND THE PROBLEM

Design a file system supporting `ls(path)`, `mkdir(path)`, `addContentToFile(path, content)`, and `readContentFromFile(path)`.

- `ls` on a **file** returns just its name in a list; on a **directory** it returns its contents **sorted**.
- `mkdir` creates all intermediate directories (like `mkdir -p`).
- `addContentToFile` **appends** if the file exists, creates it (and any missing parents) if not.
- Paths are absolute, `/`-separated, starting with `/`. The root is `"/"`.
- Edge cases: `ls("/")` on an empty system → `[]`; a path with a single component; a file and directory with the same name (assume disallowed — clarify).

BRUTE-FORCE APPROACH

Idea. Store a flat `dict[full_path → content]` for files and a `set` of directory paths. `ls` scans every stored path for those whose parent matches the query.

Data structures. A flat dict plus a set.

Complexity. `mkdir` and `addContentToFile` `O(1)`; `ls` `O(N · L)` — scanning every path in the system.

Why it's not ideal. `ls` is the common operation and it's linear in the *entire* system, not in the directory's size. Worse, string-prefix matching on paths is error-prone (`/a/bc` versus `/a/b`). The natural structure mirrors the domain: a **tree of nested dictionaries** — which is exactly a trie keyed on path components. Then `ls` costs `O(depth + children)`, touching nothing outside the target directory.

OPTIMAL APPROACH

Idea (nested dicts = a trie over path components). Each node has: - `children: dict[name → node]` - `content: str` (non-empty only for files); a node is a file iff it has content, otherwise a directory.

Operations: - **Traverse:** split the path on `/`, drop empty parts (handles the leading slash and the root), and walk/create nodes. - `mkdir(path)`: traverse, creating every missing node — `mkdir -p` semantics come for free. - `addContentToFile`: traverse creating, then `node.content += content` (append). - `readContentFromFile`: traverse, return `node.content`. - `ls(path)`: traverse; if it's a file return `[last component]`; else return `sorted(children)`.

Data structures. A tree of nodes with dict children.

Complexity. All operations `O(depth)` plus, for `ls` on a directory, `O(k log k)` to sort `k` children. Space `O(total path components + content)`.

PYTHON CODE

```python
class _FSNode:
    __slots__ = ("children", "content")

    def __init__(self):
        self.children: dict[str, "_FSNode"] = {}
        self.content: str = ""       # non-empty => this node is a FILE

    @property
    def is_file(self) -> bool:
        # A node is a file iff it holds content. This conflates "empty file" with
        # "directory" — acceptable here because addContentToFile is the only way
        # to create a file and it always writes something. If empty files were
        # required, add an explicit `is_file` flag. Worth raising unprompted.
        return self.content != ""


class FileSystem:
    def __init__(self):
        self.root = _FSNode()

    @staticmethod
    def _parts(path: str) -> list[str]:
        """Split an absolute path into components.
```

```
        The filter drops the empty string produced by the leading '/', and makes
        the root path '/' yield [] — which correctly resolves to the root node."""
        return [p for p in path.split('/') if p]

    def _traverse(self, path: str, create: bool = False) -> "_FSNode | None":
        node = self.root
        for part in self._parts(path):
            if part not in node.children:
                if not create:
                    return None
                node.children[part] = _FSNode()     # mkdir -p semantics
            node = node.children[part]
        return node

    def ls(self, path: str) -> list[str]:
        node = self._traverse(path)
        if node is None:
            return []

        if node.is_file:
            # ls on a FILE returns just its own name — the last path component.
            return [self._parts(path)[-1]]

        # ls on a DIRECTORY returns its contents, SORTED.
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        # create=True builds every missing intermediate directory in one walk.
        self._traverse(path, create=True)

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self._traverse(filePath, create=True)
        node.content += content      # APPEND if it exists, create if it doesn't

    def readContentFromFile(self, filePath: str) -> str:
        node = self._traverse(filePath)
        return node.content if node else ""
```

DRY RUN

```
ls("/")                             -> ?
mkdir("/a/b/c")
addContentToFile("/a/b/c/d", "hello")
ls("/")                             -> ?
readContentFromFile("/a/b/c/d")     -> ?
ls("/a/b/c")                        -> ?
ls("/a/b/c/d")                      -> ?
addContentToFile("/a/b/c/d", " world")
readContentFromFile("/a/b/c/d")     -> ?
```

op	tree state	result
`ls("/")`	root has no children	`[]` ✅
`mkdir("/a/b/c")`	`_parts` → `["a","b","c"]`; creates all three	
`addContentToFile("/a/b/c/d", "hello")`	creates node `d` under `c`; `d.content = "hello"`	
`ls("/")`	root's children = `{a}`	`["a"]` ✅
`readContentFromFile("/a/b/c/d")`	traverse to `d`	`"hello"` ✅
`ls("/a/b/c")`	node `c` is a directory, children = `{d}`	`["d"]` ✅
`ls("/a/b/c/d")`	node `d` is a **file** → return last component	`["d"]` ✅
`addContentToFile("/a/b/c/d", " world")`	node exists; `content += " world"`	
`readContentFromFile(...)`		`"hello world"` ✅

Two behaviours the trace pins down: - `ls` on a file and `ls` on a directory containing one file both returned `["d"]` here — coincidentally identical, which is why you should also test `ls` on a directory with two entries to show the sort is real. Add `mkdir("/a/b/c/e")` and `ls("/a/b/c")` returns `["d", "e"]` sorted ✓. - `_parts("/")` returns `[]`, so `_traverse("/")` returns the root without any loop iterations — the root case needs no special branch. That's the payoff of filtering empty components rather than slicing off the leading slash.

Binary String Errors — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given `errorString` of `0`, `1`, `!` and integers `x`, `y`: replace every `!` with `0` or `1`. Each subsequence `01` costs `x`; each subsequence `10` costs `y`. Minimise total cost, mod `10⁹ + 7`.

- **Subsequence pairs, not adjacent pairs.** Every `0` before every `1` contributes `x`; every `1` before every `0` contributes `y`. So the cost is a double-counting over all index pairs.
- **Insight (the whole problem):** the optimal assignment of the `!`s has **threshold structure** — all `0`s then all `1`s, or all `1`s then all `0`s. Mixing among the `!`s themselves costs strictly more. So enumerate the split point and evaluate each in `O(1)`.
- Edge cases: no `!` at all (just compute the fixed cost); all `!`; `x == y`.

BRUTE-FORCE APPROACH

Idea. Try all `2^k` assignments of the `k` question marks; for each, count `01` and `10` subsequence pairs in `O(n)`.

Complexity. Time `O(2^k · n)`. Space `O(k)`.

Why it's not ideal. Exponential in the number of `!`s, which can be `n`. The fix has two parts: 1. **Counting pairs in one pass.** Sweeping left to right, maintaining `zeros_so_far` and `ones_so_far`: on a `1`, add `zeros_so_far · x`; on a `0`, add `ones_so_far · y`. That makes any fixed assignment `O(n)` instead of `O(n²)`. 2. **The threshold argument**, which reduces `2^k` assignments to `k + 1` candidates.

Proving the threshold structure — have this ready, it *is* the problem. Consider two `!` positions `i < j` assigned `1` then `0` (a "descent" among the question marks). Swapping them to `0` then `1` changes only the cost contributed by that pair and by their interactions with the fixed characters between them. Because every character between `i` and `j` interacts with both positions symmetrically, the net change depends only on the pair itself: the `10` pair costs `y`, the `01` pair costs `x`. So if `x < y` we should never leave a `1` before a `0` among the `!`s, and if `y < x` never the reverse. Either way, the optimum has all identical values first — a **threshold**. So we only need to try each split point: the first `t` question marks get one value and the rest get the other.

OPTIMAL APPROACH

Idea (enumerate the split point, evaluate incrementally).

1. Collect the positions of the `!`s.
2. For each split `t` in `0..k` (first `t` question marks → `0`, remaining → `1`), and also the mirrored family (first `t` → `1`, rest → `0`), build the assignment conceptually and compute the cost with the one-pass counter.
3. Doing that naively is `O(k · n)`, which is already fine for `n ≤ 10⁵` … but with prefix precomputation each split can be evaluated in `O(1)`, giving `O(n)` overall.

For interview purposes the `O(n · k)`-with-early-structure version is usually acceptable; the clean and safe implementation below evaluates each of the `k+1` splits in `O(n)` and states the `O(1)` refinement. Say which you're delivering and why.

Data structures. The `!` position list; running counters.

Complexity. As written: `O(n · k)` worst case, `O(n)` when `k` is small. With prefix arrays: `O(n)`. Space `O(n)`.

PYTHON CODE

```python
MOD = 10 ** 9 + 7


def min_cost_binary_string(error_string: str, x: int, y: int) -> int:
    """
    Replace every '!' with '0' or '1'. Every subsequence "01" costs x, every "10"
    costs y. Minimise the total, mod 1e9+7.

    THRESHOLD STRUCTURE (the whole problem): among the '!' positions the optimum
    never leaves a "descent" — if x < y you never place a 1 before a 0, and if
    y < x never a 0 before a 1. Swapping such a pair strictly improves the cost,
    because the characters between them interact symmetrically with both, so only
    the pair's own contribution (x vs y) changes. Hence the optimum assigns all
    identical values first, and we need only try each SPLIT POINT.
```

```
    """
    n = len(error_string)
    marks = [i for i, ch in enumerate(error_string) if ch == '!']
    k = len(marks)

    def cost_of(assignment: list[str]) -> int:
        """One-pass pair counting. Sweeping left to right:
             a '1' closes a "01" pair with every 0 seen so far  -> zeros * x
             a '0' closes a "10" pair with every 1 seen so far  -> ones  * y
        O(n) instead of the O(n^2) of enumerating index pairs."""
        zeros = ones = 0
        total = 0
        for ch in assignment:
            if ch == '1':
                total += zeros * x
                ones += 1
            else:
                total += ones * y
                zeros += 1
        return total

    if k == 0:
        return cost_of(list(error_string)) % MOD

    best = None
    base = list(error_string)

    # Only k+1 candidates per orientation, instead of 2^k assignments.
    for t in range(k + 1):
        for first, second in (('0', '1'), ('1', '0')):
            trial = base[:]
            for idx, pos in enumerate(marks):
                trial[pos] = first if idx < t else second
            c = cost_of(trial)
            best = c if best is None else min(best, c)

    # MOD ONLY AT THE END — comparing already-reduced costs could select a
    # wrapped-around large value as if it were small.
    return best % MOD
```

DRY RUN

`error_string = "0!1!"`, `x = 2`, `y = 3`

`marks = [1, 3]`, `k = 2`. Candidates (8 total, but many coincide):

t	orientation	assignment	pair counting	cost
0	`('0','1')`	`0 1 1 1`	`0`: zeros=1. `1`: `1·2=2`, ones=1. `1`: `1·2=2`, ones=2. `1`: `1·2=2`, ones=3	**6**
0	`('1','0')`	`0 0 1 0`	`0`: z=1. `0`: z=2. `1`: `2·2=4`, o=1. `0`: `1·3=3`, z=3	7
1	`('0','1')`	`0 0 1 1`	`0`, `0`: z=2. `1`: `2·2=4`. `1`: `2·2=4`	**8**
1	`('1','0')`	`0 1 1 0`	`0`: z=1. `1`: 2, o=1. `1`: 2, o=2. `0`: `2·3=6`	**10**
2	`('0','1')`	`0 0 1 0`	same as t=0 mirrored	7
2	`('1','0')`	`0 1 1 1`	same as t=0 first row	**6**

`best = 6`. ✅

Verify the winner `"0111"` by hand: `01` subsequence pairs — the single `0` at index 0 pairs with each of three `1`s → `3 × 2 = 6`. `10` pairs — no `1` precedes any `0` → 0. Total **6** ✓.

Compare with the naive "all zeros" assignment `"0010"`: cost 7. And `"0110"`: cost 10. The threshold argument is visible — the two best candidates (6 and 7) both have all their `!`s set to the same value, and the mixed assignment `"0110"` (a descent among the marks: `1` at index 1 before `0` at index 3) is the worst of the lot, exactly as the exchange argument predicts.

With `x` and `y` swapped (`x = 3`, `y = 2`), the mirror family wins instead: `"0000"` would give cost 0 here, and the algorithm finds it because both orientations are enumerated. Testing both orderings of `x` and `y` is the check that the enumeration is genuinely symmetric.

Warehouse Stock — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

`stock[0..n-1]`, each entry either `-1` (missing) or a known value in `1..200`. Replace every `-1` with a value in `1..200` so that: - `a[0] <= a[1]` - `a[n-1] <= a[n-2]` - `a[i] <= max(a[i-1], a[i+1])` for interior `i`

Count the valid completions, mod `998244353`.

- **Insight:** those three constraints say exactly **"no strict local maximum anywhere."** Every element must be `≤` at least one neighbour.
- **DP shape:** sweep left to right with a **two-state** DP. State 1 = "this element is already satisfied because it is `≤` its left neighbour"; state 0 = "it still needs the next element to be `≥` it". Prefix sums make each transition `O(V)` for `V = 200` values.
- Edge cases: `n = 1` (clarify — with no neighbours the constraints are vacuous or undefined); all values known (answer is 0 or 1); all `-1`.

BRUTE-FORCE APPROACH

Idea. Try all `200^k` assignments for the `k` missing entries, validating each in `O(n)`.

Complexity. Time `O(200^k · n)`. Space `O(k)`.

Why it's not ideal. Astronomically exponential. But it points at the DP: validity is checkable by a **left-to-right sweep** where the only thing you need to remember about the past is (a) the previous element's value and (b) whether the previous element is already satisfied. That's a state of size `200 × 2` — a linear DP.

OPTIMAL APPROACH

Idea (two-state DP over values, with prefix sums).

Let `dp[v][s]` after processing position `i` mean: the number of ways to fill `0..i` such that `a[i] = v` and `s` records whether `a[i]` is **already satisfied** (`s = 1` if `a[i] <= a[i-1]`, else `s = 0`, meaning it still needs `a[i+1] >= a[i]`).

Transition from position `i` to `i+1`, choosing `a[i+1] = w`: - The previous element's obligation must be discharged: if `s = 0`, we require `w >= v`. - The new element's state: `s' = 1` if `w <= v`, else `s' = 0`.

Base case at `i = 0`: the constraint `a[0] <= a[1]` means `a[0]` is *never* pre-satisfied, so it starts in state 0 for every allowed value.

Final answer: the constraint `a[n-1] <= a[n-2]` means the last element must be in state **1**. Sum `dp[v][1]` over all `v`.

Prefix sums make each transition `O(V)` rather than `O(V²)`: the "sum over all `v <= w`" and "sum over all `v >= w`" terms are prefix and suffix sums of the previous row.

Data structures. Two arrays of size `V+2` per state; prefix/suffix sum arrays.

Complexity. Time `O(n · V)`. Space `O(V)`.

PYTHON CODE

```python
MOD = 998244353
V = 200          # values are in 1..200


def count_valid_stock(stock: list[int]) -> int:
    """
    Fill every -1 with a value in 1..V so that NO ELEMENT IS A STRICT LOCAL
    MAXIMUM. (That is exactly what the three stated constraints amount to:
    a[0] <= a[1], a[n-1] <= a[n-2], and a[i] <= max(a[i-1], a[i+1]).)

    Two-state DP:
      dp1[v] = ways with a[i] == v and a[i] ALREADY satisfied (a[i] <= a[i-1])
      dp0[v] = ways with a[i] == v and a[i] still NEEDING a[i+1] >= a[i]
    """
    n = len(stock)
    if n == 0:
        return 0
    if n == 1:
        # No neighbours — constraints are vacuous. Clarify this with the
```

```
        # interviewer; the sensible reading is "any allowed value works".
        return V if stock[0] == -1 else 1

    def allowed(i: int):
        """Values position i may take."""
        return range(1, V + 1) if stock[i] == -1 else [stock[i]]

    # BASE CASE at i = 0. The constraint a[0] <= a[1] means position 0 can never
    # be pre-satisfied — it always owes the next element. So it starts in state 0.
    dp0 = [0] * (V + 2)
    dp1 = [0] * (V + 2)
    for v in allowed(0):
        dp0[v] = 1

    for i in range(1, n):
        ndp0 = [0] * (V + 2)
        ndp1 = [0] * (V + 2)

        # PREFIX/SUFFIX SUMS of the previous row, so each transition is O(1)
        # instead of an O(V) inner scan — this is what makes the whole DP O(nV).
        # pre_all[v]  = sum over u <= v of (dp0[u] + dp1[u])
        # suf0[v]     = sum over u >= v of dp0[u]     (state-0 entries only)
        pre_all = [0] * (V + 2)
        for u in range(1, V + 1):
            pre_all[u] = (pre_all[u - 1] + dp0[u] + dp1[u]) % MOD

        suf_all = [0] * (V + 3)
        suf0 = [0] * (V + 3)
        for u in range(V, 0, -1):
            suf_all[u] = (suf_all[u + 1] + dp0[u] + dp1[u]) % MOD
            suf0[u] = (suf0[u + 1] + dp0[u]) % MOD

        for w in allowed(i):
            # --- new element lands in state 1: w <= v (satisfied by the left) ---
            # The previous element must also be legal:
            #   if it was state 1, no obligation -> any v >= w works
            #   if it was state 0, it needed w >= v -> also v <= w
            # Since state 1 requires w <= v, a previous state-0 element would
            # need v <= w AND w <= v, i.e. v == w. Handle that exactly.
            ways1 = suf_all[w]              # all previous v >= w ...
            # ... but previous state-0 entries with v > w are ILLEGAL (they
            # required w >= v). Subtract them, keeping only v == w.
            illegal = (suf0[w + 1]) % MOD
            ndp1[w] = (ways1 - illegal) % MOD

            # --- new element lands in state 0: w > v (not satisfied yet) ---
            # Previous element with v < w. A previous state-0 element required
            # w >= v, which holds since w > v. So ALL previous v < w are legal.
            ndp0[w] = pre_all[w - 1] % MOD

        dp0, dp1 = ndp0, ndp1

    # FINAL: a[n-1] <= a[n-2] means the last element must be in state 1.
    return sum(dp1[1:V + 1]) % MOD
```

DRY RUN

`stock = [-1, -1]`, so `n = 2` and `V` conceptually small — use `V = 3` for a hand-checkable trace.

Constraints for `n = 2`: `a[0] <= a[1]` and `a[1] <= a[0]` → therefore `a[0] == a[1]`. With `V = 3` there should be exactly **3** valid completions: `(1,1)`, `(2,2)`, `(3,3)`.

Base (i = 0): `dp0 = [_, 1, 1, 1]`, `dp1 = [_, 0, 0, 0]` (state 0 for all three values).

Transition to i = 1:

Prefix/suffix over the previous row:

u	dp0[u]+dp1[u]	pre_all[u]	suf_all[u]	suf0[u]
1	1	1	3	3
2	1	2	2	2
3	1	3	1	1

For each `w`:

w	`ndp1[w] = suf_all[w] - suf0[w+1]`	`ndp0[w] = pre_all[w-1]`
1	`3 - suf0[2] = 3 - 2 = 1`	`pre_all[0] = 0`
2	`2 - suf0[3] = 2 - 1 = 1`	`pre_all[1] = 1`
3	`1 - suf0[4] = 1 - 0 = 1`	`pre_all[2] = 2`

Final: the last element must be state 1 → `sum(ndp1) = 1 + 1 + 1 = 3`. ✅

Matches the hand count exactly. The `ndp1` computation is where the subtlety lives: for `w = 1`, all three previous values are `>= 1`, but the two with `v > 1` were in state 0 and required `w >= v` — violated. Subtracting `suf0[2] = 2` removes exactly those, leaving only `v == w == 1`. That subtraction is the correctness of the whole transition, and tracing one `w` by hand is how you verify it before trusting the code.

Second check, `stock = [2, -1]` with `V = 3`: base gives `dp0 = [_,0,1,0]`. Transition: `ndp1[w] = suf_all[w] - suf0[w+1]`. For `w = 2`: `suf_all[2] = 1`, `suf0[3] = 0` → `1`. For `w = 1`: `suf_all[1] = 1`, `suf0[2] = 1` → `0`. For `w = 3`: `suf_all[3] = 0`, `suf0[4] = 0` → `0`. Sum = **1** ✅ — only `a[1] = 2` works, since `a[0] == a[1]` is forced.

Config Reconstruction — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given a digit string `config` and integers `x`, `y`: start with `curr = 0`; repeatedly add `x` **or** `y` to `curr`, appending `curr % 10` to a generated string each time. Find the **shortest** generated string that contains `config` as a **subsequence**.

- Only `curr % 10` is ever observed, and the next digit depends only on `curr % 10` (since `(curr + x) % 10` depends on `curr % 10` alone). So the state space collapses to ten last-digit values.
- **Insight:** BFS over `(last_digit, matched_prefix_length)` — `10 × (len(config) + 1)` states, tiny.
- We want the shortest length, and each step costs 1 → unweighted shortest path → **BFS**.
- Edge cases: empty `config` → 0; `x == y` (only one choice per step); `config` unreachable → `-1`.

BRUTE-FORCE APPROACH

Idea. DFS/BFS over the actual generated strings, exploring every sequence of `x` / `y` choices.

Complexity. Time `O(2^L)` for a generated length `L` — exponential, with unbounded `curr` values.

Why it's not ideal. The branching is exponential and `curr` grows without bound, so there's no natural termination. Two collapses fix it: 1. **`curr` matters only mod 10.** `(curr + x) % 10` is fully determined by `curr % 10` and `x`, so the running total can be discarded entirely. 2. **The match progress is a single integer.** Greedy subsequence matching means you only need to know how many characters of `config` are already matched — matching earlier is never worse.

Together the state is `(last_digit, matched)` — at most `10 × 101 = 1010` states.

OPTIMAL APPROACH

Idea (BFS over `(last_digit, matched)`).

1. Start state: `(0, 0)` — `curr % 10 == 0`, nothing matched. Note the initial `0` is **not** appended (we append only after adding `x` or `y`), so the start contributes no character.
2. From `(d, m)`, for each of `x` and `y`: `nd = (d + step) % 10`; the appended digit is `nd`. New match progress: `nm = m + 1` if `nd == int(config[m])` else `m`.
3. BFS level by level; the level number is the generated string's length.
4. Return the level at which `m == len(config)`.

Greedy matching is valid because subsequence matching is prefix-optimal: if a digit can advance the match, advancing is never worse than skipping. So `matched` is a single integer, not a set.

Data structures. A deque of states; a visited set of `(digit, matched)` pairs.

Complexity. Time `O(10 · L · 2)` = `O(L)`. Space `O(10 · L)`.

PYTHON CODE

```python
from collections import deque


def shortest_config_string(config: str, x: int, y: int) -> int:
    """
    Start curr = 0. Repeatedly add x or y, appending curr % 10 each time.
    Return the length of the SHORTEST generated string containing `config`
    as a subsequence, or -1 if impossible.

    STATE COLLAPSE — the whole problem:
      1. Only curr % 10 is ever observed, and (curr + x) % 10 depends solely on
         curr % 10. So the unbounded `curr` reduces to ten values.
      2. Subsequence matching is prefix-greedy: matching a character as early as
         possible is never worse. So progress is ONE integer, not a set.
    That gives 10 * (len(config)+1) states — BFS over them is trivial.
    """
    if not config:
        return 0

    target = [int(c) for c in config]
    L = len(target)

    start = (0, 0)          # (curr % 10, characters of config matched)
```

```
    visited = {start}
    q = deque([start])
    length = 0              # generated-string length == BFS level

    while q:
        length += 1         # every transition appends exactly one digit
        for _ in range(len(q)):
            d, m = q.popleft()

            for step in (x, y):
                nd = (d + step) % 10        # the digit that gets appended

                # GREEDY match advance — valid by the prefix-optimality argument.
                nm = m + 1 if nd == target[m] else m

                if nm == L:
                    # BFS => the first time we complete the match is the SHORTEST
                    # generated string. Return immediately.
                    return length

                state = (nd, nm)
                if state not in visited:
                    visited.add(state)
                    q.append(state)

    return -1               # state space exhausted without matching
```

DRY RUN

`config = "12"`, `x = 1`, `y = 3` → target digits `[1, 2]`, `L = 2`.

Start: `(0, 0)`, `visited = {(0,0)}`.

Level 1 (`length = 1`), popping `(0, 0)`:

step	`nd = (0 + step) % 10`	`target[0] = 1`	`nm`	complete?	state
`x=1`	1	matches ✓	1	`1 == 2` ? no	`(1, 1)` — enqueue
`y=3`	3	no	0	no	`(3, 0)` — enqueue

Level 2 (`length = 2`), popping `(1, 1)` then `(3, 0)`:

From `(1, 1)` — now matching `target[1] = 2`:

step	`nd = (1 + step) % 10`	matches `2` ?	`nm`	complete?
`x=1`	2	✓	2	`2 == L` → **return 2** ✅

Answer: **2**. ✅

Verify by hand: add `x=1` → `curr = 1`, append `1`. Add `x=1` → `curr = 2`, append `2`. Generated string `"12"`, which contains `"12"` as a subsequence, length **2** ✓. No shorter string is possible since `config` itself has length 2.

A case needing a detour, `config = "9"`, `x = 2`, `y = 4`:

level	states	transitions	found?
1	`(0,0)`	`+2 → 2`, `+4 → 4`	neither is 9
2	`(2,0)`, `(4,0)`	`2+2=4`, `2+4=6`, `4+2=6`, `4+4=8`	no
3	`(6,0)`, `(8,0)`	`6+2=8`, `6+4=0`, `8+2=0`, `8+4=2`	no
4	`(0,0)` seen, `(2,0)` seen...	all states already visited	queue drains

Return **−1** ✅ — with `x` and `y` both even and `curr` starting at 0, `curr % 10` is always even, so the digit `9` is unreachable. The visited set is what makes the search terminate instead of cycling forever, and it's what lets us confidently report impossibility rather than looping.

DAY 12 — Fri Aug 28 · ★★ MOCK LOOP #2 + Weak-Spot Repair

> **The four problems below are all ✅ Verified reported Amazon questions**, and they share a characteristic worth naming: **none of them is algorithmically hard.** They are *specification* problems. The difficulty is reading the rules precisely, asking the right clarifying question, and implementing exactly what was asked — which is precisely what an SDE-I loop screens for. Rushing to a clever data structure here is the failure mode; restating the spec back to the interviewer is the winning move.
>
> Day 12's other blocks (the three-round mock loop, the repair block, the six-LLD review) have no solutions to write — see the notes at the end of this section.

Smart Processor — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given array `arr` and a list of inclusive ranges `[l, r]`: in order, append `arr[l..r]` to a new array `efficient` (indices may repeat across ranges, so elements can be appended more than once). Then, for each **original** index `i`: - efficiency is `0` if `i` was covered by **any** range; - otherwise it is the count of elements in `efficient` **strictly less than** `arr[i]`.

Return the sum of all efficiencies.

- Two independent sub-problems: (1) which original indices are covered, and (2) for each uncovered index, how many `efficient` elements are strictly below its value.
- `efficient` **may contain duplicates** — an index appearing in two ranges contributes twice. That's explicit in the statement and it changes the counts.
- **Strictly** less than, not `≤`. Confirm.
- Edge cases: no ranges (every index uncovered, `efficient` empty → all efficiencies 0, sum 0); ranges covering everything (sum 0); overlapping ranges; a single element.

BRUTE-FORCE APPROACH

Idea. Build `efficient` by literally appending each range's slice. Mark covered indices by walking each range. Then, for each uncovered `i`, scan all of `efficient` counting elements `< arr[i]`.

Data structures. A list for `efficient`; a boolean array for coverage.

Complexity. Building `efficient` is `O(total range length)`, call it `M`. Marking coverage the same. The counting step is `O(n · M)`. Total `O(n · M)`, which with `n, M ~ 10⁵` is 10^{10}.

Why it's not ideal. The counting is the bottleneck, and it repeats an identical question — "how many elements below `v`" — for every uncovered index. **Sort `efficient` once**, then each query is a binary search (`bisect_left` gives exactly the count of strictly-smaller elements). That's `O(M log M + n log M)`.

The coverage marking can also be improved from `O(M)` to `O(n + k)` for `k` ranges using a **difference array** — increment at `l`, decrement at `r+1`, then prefix-sum. Worth doing, since ranges may overlap heavily and `M` can far exceed `n`.

OPTIMAL APPROACH

Idea (difference array for coverage + sort-once + binary search).

1. **Coverage:** `diff[l] += 1`, `diff[r+1] -= 1` for each range; prefix-sum; `covered[i] = prefix[i] > 0`. `O(n + k)`.
2. **Build `efficient`:** append `arr[l..r]` for each range in order. This is inherently `O(M)` — the array's contents are needed.
3. **Sort `efficient`** once: `O(M log M)`.
4. For each uncovered `i`: `bisect_left(efficient_sorted, arr[i])` — the count of strictly smaller elements. Sum them.

Data structures. A difference array; the `efficient` list; `bisect`.

Complexity. Time `O(n + M log M + n log M)`. Space `O(n + M)`.

PYTHON CODE

```python
import bisect
from typing import Sequence


def smart_processor(arr: Sequence[int], ranges: Sequence[Sequence[int]]) -> int:
    """
    Build `efficient` by appending arr[l..r] for each inclusive range, in order
    (duplicates are intended — an index in two ranges contributes twice).
    Then sum, over every ORIGINAL index i:
        0                                       if i is covered by any range
        count of elements in efficient < arr[i]  otherwise
    """
    n = len(arr)
    if n == 0:
        return 0

    # ---- 1. COVERAGE via a difference array: O(n + k) instead of O(total length).
    # Ranges may overlap heavily, so walking each one is wasteful when k is large.
    diff = [0] * (n + 1)
    for l, r in ranges:
        diff[l] += 1
        diff[r + 1] -= 1                # the n+1 slot is why diff is one longer

    covered = [False] * n
    running = 0
    for i in range(n):
        running += diff[i]
        covered[i] = running > 0

    # ---- 2. BUILD `efficient`. Inherently O(M) — we need the actual multiset.
    # Duplicates are deliberate: an index inside two ranges appears twice, and
    # that genuinely changes the counts below.
    efficient = []
    for l, r in ranges:
        efficient.extend(arr[l:r + 1])

    if not efficient:
        return 0                        # nothing to compare against

    # ---- 3. SORT ONCE. Every uncovered index asks the same kind of question,
    # so pay O(M log M) once rather than an O(M) scan per query.
    efficient.sort()

    # ---- 4. Count with binary search.
    total = 0
    for i in range(n):
        if covered[i]:
            continue                    # covered indices contribute 0
        # bisect_left returns the number of elements STRICTLY LESS than arr[i]
        # (bisect_right would include equals — confirm which the spec wants).
        total += bisect.bisect_left(efficient, arr[i])

    return total
```

DRY RUN

`arr = [4, 1, 7, 2, 5]`, `ranges = [[0, 1], [1, 2]]`

Step 1 — coverage. `diff` starts `[0,0,0,0,0,0]`. - Range `[0,1]`: `diff[0] += 1`, `diff[2] -= 1` → `[1,0,-1,0,0,0]` - Range `[1,2]`: `diff[1] += 1`, `diff[3] -= 1` → `[1,1,-1,-1,0,0]`

Prefix sums:

i	running	covered
0	1	**yes**
1	2	**yes**
2	1	**yes**
3	0	no
4	0	no

Step 2 — build `efficient`.

- Range `[0,1]` → append `arr[0..1] = [4, 1]`
- Range `[1,2]` → append `arr[1..2] = [1, 7]`

`efficient = [4, 1, 1, 7]` — note `1` appears **twice**, because index 1 sits in both ranges.

Step 3 — sort: `efficient = [1, 1, 4, 7]`

Step 4 — sum over uncovered indices:

i	`arr[i]`	covered?	`bisect_left([1,1,4,7], arr[i])`	contribution
0	4	yes	—	0
1	1	yes	—	0
2	7	yes	—	0
3	**2**	no	elements `< 2` are `1, 1` → **2**	**2**
4	**5**	no	elements `< 5` are `1, 1, 4` → **3**	**3**

Answer: `2 + 3 = 5`. ✅

Two spec details the trace pins down: - **Duplicates matter.** If `efficient` had been de-duplicated to `[1, 4, 7]`, index 3 would have contributed 1 instead of 2, and the answer would be 4. The statement says indices may repeat, so the multiset is correct. - **Strictly less.** At index 4 (`arr[i] = 5`), the value 4 counts but if `efficient` had contained a 5 it would **not**. Using `bisect_right` would have included it. Confirm the strictness before coding — it's a one-character difference with a wrong answer attached.

Sign In Pages — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Process ordered auth requests. Three operations, each returning success or failure: - `register(user, pass)` — **fails** if the user already exists. - `login(user, pass)` — **fails** if the user doesn't exist, the password is wrong, **or the user is already signed in**. - `logout(user)` — **fails** if the user is not signed in.

Case-sensitive; the store starts empty.

- The "already signed in" failure on `login` is the rule people miss — it means you must track session state, not just credentials.
- Case-sensitive means no normalisation: `"Alice"` and `"alice"` are different users. State that explicitly, since the opposite is the more common real-world choice.
- Edge cases: `logout` on a non-existent user (fails — not signed in); `register` then `login` with a wrong password; `login` twice in a row; `logout` twice in a row.
- Clarify: *"Does a failed `login` have any side effect — lockout counters, for instance? Should passwords be compared in constant time?"* Both are worth raising and neither is likely required.

BRUTE-FORCE APPROACH

Idea. Keep a list of `(user, password)` tuples and a list of signed-in usernames; scan the lists on every operation.

Data structures. Two lists.

Complexity. `O(n)` per operation → `O(n²)` overall for `n` requests.

Why it's not ideal. Every operation is a membership-and-lookup question, which is what a hash map answers in `O(1)`. With `n = 10⁵` requests the linear scans are `10¹⁰`. There is no algorithmic subtlety here — the entire test is whether you model the state correctly and enforce each rule in the right order.

OPTIMAL APPROACH

Idea (two hash structures, one per concern).

- `users: dict[username → password]` — the credential store.
- `signed_in: set[username]` — the session state.

Each operation is a short sequence of `O(1)` checks, and **the order of the checks is the specification**: - `register`: fail if `user in users`; else store, succeed. - `login`: fail if `user not in users`; fail if `users[user] != password`; fail if `user in signed_in`; else add to `signed_in`, succeed. - `logout`: fail if `user not in signed_in`; else discard, succeed.

Separating credentials from sessions is the modelling decision worth stating — conflating them (e.g. a single dict with a nullable session field) makes the rules harder to read and easier to get wrong.

Data structures. A dict and a set.

Complexity. `O(1)` per operation, `O(n)` total. Space `O(u)` for `u` distinct users.

PYTHON CODE

```python
from typing import Sequence


class AuthSystem:
    """
    Two structures, one per concern — keeping them separate is the modelling
    decision that makes each rule a one-line check:
      * users     : credential store   (username -> password)
      * signed_in : session state      (set of usernames)

    Case-SENSITIVE per the spec: no normalisation anywhere. Worth saying out loud,
    because case-insensitive is the more common real-world choice and an
    interviewer may be checking whether you noticed the difference.
    """

    def __init__(self):
        self.users: dict[str, str] = {}
        self.signed_in: set[str] = set()
```

```
    def register(self, user: str, password: str) -> bool:
        if user in self.users:
            return False                    # duplicate registration
        self.users[user] = password
        return True

    def login(self, user: str, password: str) -> bool:
        # THE ORDER OF THESE CHECKS IS THE SPECIFICATION. All three are failures,
        # and the third — "already signed in" — is the one people omit.
        if user not in self.users:
            return False                    # no such user
        if self.users[user] != password:
            return False                    # wrong password
        if user in self.signed_in:
            return False                    # already has an active session

        self.signed_in.add(user)
        return True

    def logout(self, user: str) -> bool:
        if user not in self.signed_in:
            return False                    # covers BOTH "no such user" and
                                            # "exists but has no active session"
        self.signed_in.discard(user)
        return True


def process_requests(requests: Sequence[Sequence[str]]) -> list[bool]:
    """requests: e.g. [("register","alice","pw"), ("login","alice","pw"), ...]"""
    auth = AuthSystem()
    out = []
    for req in requests:
        op = req[0]
        if op == "register":
            out.append(auth.register(req[1], req[2]))
        elif op == "login":
            out.append(auth.login(req[1], req[2]))
        elif op == "logout":
            out.append(auth.logout(req[1]))
        else:
            raise ValueError(f"unknown operation: {op}")
    return out
```

DRY RUN

```
register("alice", "pw1")
register("alice", "pw2")
login("alice", "wrong")
login("alice", "pw1")
login("alice", "pw1")
logout("alice")
logout("alice")
login("Alice", "pw1")
```

op	check that decides it	result	`users`	`signed_in`
`register("alice","pw1")`	not present → store	**True**	`{alice:pw1}`	`{}`
`register("alice","pw2")`	already present	**False** ✅	`{alice:pw1}` (unchanged — note the password was **not** overwritten)	`{}`
`login("alice","wrong")`	exists ✓, password `pw1 != wrong`	**False** ✅		`{}`
`login("alice","pw1")`	exists ✓, password ✓, not signed in ✓	**True**		`{alice}`
`login("alice","pw1")`	exists ✓, password ✓, **already signed in**	**False** ✅		`{alice}`
`logout("alice")`	signed in ✓	**True**		`{}`
`logout("alice")`	not signed in	**False** ✅		`{}`
`login("Alice","pw1")`	`"Alice" not in users` (case-sensitive)	**False** ✅		`{}`

Output: `[True, False, False, True, False, True, False, False]`

Three rules the trace demonstrates that a careless implementation would break: - **Row 2:** the failed re-registration left the stored password as `pw1`. An implementation that writes first and checks second would have silently changed the password to `pw2` — a security bug, and worth flagging out loud. - **Row 5:** the double `login` correctly failed. This is the rule most often missed, and it's the reason `signed_in` must exist as separate state. - **Row 8:** `"Alice"` is a different user from `"alice"`. If the interviewer expected case-insensitivity, this is where you'd find out — which is why you confirm it before coding rather than after.

Faulty Server Detection — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given chronological logs of the form `"<server> <error|success>"`: per server, an `error` increments a **consecutive**-failure counter and a `success` **resets** it to zero. On reaching **3 consecutive errors**, the server is replaced and **its counter resets**. Return the total number of replacements.

- "Consecutive" is per-server, interleaved across the global log — server A's errors are not broken by server B's entries.
- After a replacement the counter resets to 0, so **9 consecutive errors on one server means 3 replacements**, not 1.
- Edge cases: fewer than 3 errors anywhere → 0; a `success` between errors (resets, so `error, error, success, error` never triggers); a server appearing only once; exactly 3 errors.
- Clarify: *"Is the threshold always 3, or should it be a parameter? Are server names case-sensitive?"* Parameterising is free and reads well.

BRUTE-FORCE APPROACH

Idea. For each distinct server, filter the log down to that server's entries, then scan the filtered sequence counting runs.

Data structures. A per-server list of log entries.

Complexity. Time `O(S · n)` for `S` distinct servers and `n` log lines — a full pass per server. Space `O(n)`.

Why it's not ideal. It re-reads the entire log once per server. Since each log line concerns exactly one server, a **single pass** with a per-server counter dictionary does the same work in `O(n)`. There's no algorithmic depth here at all — this problem is testing whether you handle the counter-reset-on-replacement rule correctly, and whether you notice that repeated replacements are possible.

OPTIMAL APPROACH

Idea (one pass, a counter per server).

1. `consecutive: dict[server → current consecutive error count]`.
2. For each log line:
3. `success` → `consecutive[server] = 0`.
4. `error` → increment; if it reaches the threshold, `replacements += 1` and **reset the counter to 0**.
5. Return `replacements`.

That reset is the whole problem. Omitting it makes every subsequent error on that server trigger another replacement (off by a factor of 3); leaving the counter at the threshold makes the *next* error trigger immediately.

Data structures. One dict.

Complexity. Time `O(n)`. Space `O(S)`.

PYTHON CODE

```python
from collections import defaultdict
from typing import Sequence


def count_replacements(logs: Sequence[str], threshold: int = 3) -> int:
    """
    Logs are chronological, formatted "<server> <error|success>".
    Per server: `error` increments a CONSECUTIVE failure counter, `success`
    resets it. On reaching `threshold` consecutive errors the server is replaced
    and its counter resets — so 9 straight errors on one server is 3 replacements,
    not 1.

    `threshold` is parameterised rather than hard-coded to 3: it costs nothing
    and it's the obvious follow-up.
    """
    consecutive = defaultdict(int)
    replacements = 0

    for line in logs:
        # rsplit with maxsplit=1 so server names containing spaces still parse.
        server, status = line.rsplit(' ', 1)

        if status == 'success':
            consecutive[server] = 0        # a single success breaks the run
        elif status == 'error':
            consecutive[server] += 1

            if consecutive[server] == threshold:
                replacements += 1
                # RESET after replacement. This is the whole problem:
                #   * not resetting -> every later error triggers again (3x too many)
                #   * leaving it AT the threshold -> the next error fires immediately
                consecutive[server] = 0
        else:
            raise ValueError(f"unknown status: {status!r}")

    return replacements
```

DRY RUN

```
logs = [
  "s1 error",     # 1
  "s2 error",     # 2
  "s1 error",     # 3
  "s1 error",     # 4
  "s2 success",   # 5
  "s1 error",     # 6
  "s1 error",     # 7
  "s1 error",     # 8
  "s2 error",     # 9
]
```

#	line	action	consecutive after	replacements
1	`s1 error`	s1 → 1	`{s1:1}`	0
2	`s2 error`	s2 → 1	`{s1:1, s2:1}`	0
3	`s1 error`	s1 → 2	`{s1:2, s2:1}`	0
4	`s1 error`	s1 → **3 = threshold** → replace, **reset to 0**	`{s1:0, s2:1}`	**1**
5	`s2 success`	s2 → 0	`{s1:0, s2:0}`	1
6	`s1 error`	s1 → 1	`{s1:1}`	1
7	`s1 error`	s1 → 2	`{s1:2}`	1
8	`s1 error`	s1 → **3** → replace, reset	`{s1:0}`	**2**
9	`s2 error`	s2 → 1	`{s2:1}`	2

Answer: **2.** ✅

Three details the trace confirms: - **The reset works.** After the first replacement at line 4, `s1` needed three *fresh* errors (lines 6, 7, 8) to trigger again. Without the reset, line 6 alone would have pushed the counter to 4 — and depending on whether the check is `==` or `>=`, either nothing would ever fire again, or every single subsequent error would. - **Interleaving is handled.** `s1`'s errors at lines 1, 3, 4 are consecutive *for s1* despite `s2 error` sitting between them at line 2. A naive global counter would have broken the run. - `success` **genuinely resets.** `s2` accumulated one error, was reset by the success at line 5, and its final error at line 9 left it at 1 — nowhere near the threshold.

Using `==` rather than `>=` for the threshold check is deliberate and safe here, because the counter is reset to 0 the instant it hits the threshold and so can never exceed it. Say that; otherwise `==` looks like a latent bug.

Count Discount Pairs — ✅ Verified (InterviewDB)

UNDERSTAND THE PROBLEM

Given an array `prices` and an integer `x`, count the index pairs `i < j` with `(prices[i] + prices[j]) % x == 0`.

- Unordered pairs of **distinct indices** — `(i, j)` and `(j, i)` are the same pair and count once.
- **Insight:** bucket by remainder. Pair remainder `r` with remainder `x - r`; handle `r == 0` and `r == x/2` (when `x` is even) as **self-pairs**, since those pair with their own class.
- Edge cases: `x = 1` (every pair qualifies → `C(n,2)`); fewer than 2 prices → 0; all prices identical; negative prices (Python's `%` returns non-negative for positive `x`, which is what we want — in Java/C++ you'd normalise).

BRUTE-FORCE APPROACH

Idea. Double loop over all `i < j`, testing the modulus.

Complexity. Time `O(n²)`, space `O(1)`.

Why it's not ideal. `n = 10⁵` gives $5 \cdot 10^9$ pair tests. The observation that fixes it: whether a pair qualifies depends **only on the two remainders**, not the values. So group prices into `x` remainder buckets and count pairs *between buckets* combinatorially — one pass to bucket, then `O(x)` arithmetic.

OPTIMAL APPROACH

Idea (bucket by remainder, then count combinatorially).

1. `count[r]` = how many prices have `price % x == r`, for `r` in `0..x-1`. One `O(n)` pass.
2. `(a + b) % x == 0` iff `(a%x + b%x) % x == 0`, i.e. the remainders are `r` and `x - r`.
3. Count:
4. **`r == 0`:** these pair with each other → `C(count[0], 2) = count[0] · (count[0] - 1) / 2`.
5. **`r == x/2` when `x` is even:** same self-pairing → `C(count[x//2], 2)`.
6. **all other `r` with `0 < r < x/2`:** cross-pair → `count[r] · count[x - r]`.

Iterating `r` only up to `x // 2` avoids double-counting each cross pair.

Data structures. An integer array of length `x`.

Complexity. Time `O(n + x)`. Space `O(x)`.

PYTHON CODE

```python
from typing import Sequence


def count_discount_pairs(prices: Sequence[int], x: int) -> int:
    """
    Count index pairs i < j with (prices[i] + prices[j]) % x == 0.

    Whether a pair qualifies depends ONLY on the two remainders, never the
    values — so bucket by remainder and count combinatorially instead of
    testing O(n^2) pairs.
    """
    if x <= 0:
        raise ValueError("x must be positive")
    if len(prices) < 2:
        return 0

    # count[r] = how many prices are congruent to r mod x.
    # Python's % returns a value in [0, x-1] even for negative operands, so
    # negative prices need no special handling. In Java/C++ you'd normalise
    # with ((p % x) + x) % x — worth mentioning.
    count = [0] * x
    for p in prices:
        count[p % x] += 1

    total = 0

    # r == 0: these pair with EACH OTHER (0 + 0 = 0 mod x), so it's a
    # self-pairing -> C(n, 2), not a product.
    total += count[0] * (count[0] - 1) // 2

    # x even: r == x/2 is also self-pairing, since x/2 + x/2 = x = 0 mod x.
    if x % 2 == 0:
        half = x // 2
        total += count[half] * (count[half] - 1) // 2

    # Cross pairs: r with x-r. Iterate only up to x//2 (exclusive of the
    # self-pairing cases already handled) so each unordered pair of buckets is
    # counted exactly once.
    for r in range(1, (x + 1) // 2):
        total += count[r] * count[x - r]

    return total
```

DRY RUN

`prices = [1, 3, 2, 6, 1, 2]`, `x = 3`

Bucketing:

price	`price % 3`
1	1
3	0
2	2
6	0
1	1
2	2

`count = [2, 2, 2]` (two each in remainder classes 0, 1, 2).

Counting:

case	computation	contribution
`r == 0` self-pairs	`C(2,2) = 2·1/2 = 1`	**1**
`x` odd → no `x/2` case	—	0
cross `r = 1` with `x-r = 2`	range is `range(1, (3+1)//2) = range(1, 2)` → just `r = 1`: `count[1] · count[2] = 2 · 2`	**4**

Answer: `1 + 4 = 5`. ✅

Verify by listing all qualifying pairs (0-indexed): - `(1, 3)`: `3 + 6 = 9`, `9 % 3 = 0` ✓ — the remainder-0 self-pair. - `(0, 2)`: `1 + 2 = 3` ✓ - `(0, 5)`: `1 + 2 = 3` ✓ - `(4, 2)`: `1 + 2 = 3` ✓ - `(4, 5)`: `1 + 2 = 3` ✓

Five pairs ✓ — one from the zero class and four from the 1×2 cross product.

The even-`x` case, `prices = [2, 4, 6, 8]`, `x = 4`: remainders are `[2, 0, 2, 0]` → `count = [2, 0, 2, 0]`. - `r = 0` self-pairs: `C(2,2) = 1` (that's `4 + 8 = 12`). - `x` even → `half = 2`, self-pairs: `C(2,2) = 1` (that's `2 + 6 = 8`). - Cross loop: `range(1, 2)` → `r = 1`: `count[1] · count[3] = 0 · 0 = 0`.

Answer: **2** ✅. Without the `x/2` self-pair branch the answer would be 1, silently missing the `2 + 6` pair. Run an even-`x` case as your second dry run every time — it's the only way that branch gets exercised.

The rest of Day 12

The other Day 12 blocks produce no code, so there is nothing to answer in the five-part format. What to run:

The loop (3h 45m):

- **Round 1 (60 min) — Coding.** Unseen problems, recorded.
- **Round 2 (60 min) — LLD.** An unseen prompt: *design a vending machine* / *a library system* / *a food-delivery order flow*. Paper. Name the classes, the state machine, and the patterns.
- **Round 3 (60 min) — System design.** One of SD-1..SD-4. Paper. Defend two decisions under pushback.
- **Review all three recordings (45 min).**

Repair block (90 min): your three worst areas from the Aug 23 debrief, two fresh problems each, blank page. **This is the last day new problems help.**

Design block — final LLD review, all six, no notes. For each: name the classes, the pattern, and the one extension follow-up, out loud. The six briefs are reproduced in full in `Day12_Aug28_Fri.md`; the shape to rehearse is:

LLD	Core classes	Patterns	The one follow-up to have ready
1. Config Manager	`ConfigManager`, `ConfigSource`, `ConfigValue<T>`, `ConfigListener`	Strategy, Observer	add a new source without touching callers
2. Discount	`DiscountStrategy`, `EligibilityRule`, `DiscountEngine`, `Cart`	Strategy, Decorator, Chain of Responsibility, Factory	order of application changes the total
3. Query Builder	`QueryBuilder`, `Predicate` hierarchy, `Query`	Builder, Composite, Interpreter	nested grouping `(a OR b) AND c`
4. Shipping Cost	`ShippingCostCalculator`, `Carrier`, `RegionalPricing`	Strategy, Factory, Decorator	**dimensional weight** — raise it unprompted
5. Spell Checker	`Dictionary` (Trie), `SpellChecker`, `SuggestionStrategy`	Strategy	BK-tree for fuzzy matching
6. Validation Framework	`Validator<T>`, `ValidationRule<T>`, `CompositeValidator`	Chain of Responsibility, Composite, Specification	add a rule without modifying the engine

DAY 13 — Sat Aug 29 · ★★★ MOCK LOOP #3 — Dress Rehearsal

No new problems. Day 13 is the full four-round dress rehearsal plus consolidation and logistics. Nothing here takes the five-part format.

The loop — back to back, minimal breaks. Start at the same time of day as the real interview if you know it. - **Round 1 (60 min):** Coding + LP - **Round 2 (60 min):** Coding + LP - **Round 3 (60 min):** Design — have someone pick LLD or SD, or roll a die - **Break (20 min)** - **Round 4 (60 min):** Bar-Raiser style — heavy LP, one coding problem - **Review (60 min).** Last honest look.

Consolidation (90 min). The day file calls the Pattern Trigger tables in `01_REFERENCE...md` §2 the *highest-value review available to you*, because the phrasing→pattern reflex is the actual gap. Re-read them first. Then skim your solution book end to end, re-read the six LLD and four SD briefs (classes/API, one pattern, one failure mode, out loud, no notes), and recite the Round Script from memory.

A condensed trigger table, drawn from this file's 149 problems — use it as a self-test before opening the reference:

The prompt says...	Reach for
"pair summing to", "two numbers that"	hash map of complements (1)
"longest/shortest substring with..."	sliding window (3, 76, 209)
"count subarrays summing to k", **negatives allowed**	prefix sum + hashmap (560, 974, 525)
"next/previous greater/smaller"	monotonic stack (739, 84)
"maximum in every window of size k"	monotonic deque (239)
"kth largest"	**min**-heap of size k (215)
"k closest / kth smallest"	**max**-heap of size k (973)
"running median"	two heaps (295)
"can all be finished", "valid ordering", "prerequisites"	Kahn's topological sort (207, 210, 802, 310)
"shortest path, unweighted"	BFS (127, 752, 863)
"connected groups", "merge accounts"	Union-Find (547, 721, 684)
"shortest path with at most k stops"	Bellman-Ford — the cap breaks Dijkstra (787)
"distance to nearest X for every cell"	multi-source BFS (994, 542)
"minimise the maximum" / "maximise the minimum"	binary search on the answer (875, 1011, 410, 1552)
"merge overlapping"	sort by **start** (56, 57)
"most non-overlapping"	sort by **end** (435)
"how many rooms/resources at once"	heap or sweep line (253, 759)
"each item once", subset-sum	0/1 knapsack — inner loop **backwards** (416, 494)
"unlimited reuse", coin change	unbounded knapsack — inner loop **forwards** (322, 518)
"combinations vs permutations"	**loop order** decides it (518 vs 377)
"O(1) get and put with eviction"	hash map + doubly linked list (146)
"O(1) insert, delete, random"	array + index map, swap-with-last (380)
"prefix search", "wildcard search", "many patterns at once"	trie (211, 212, Block Words)

Logistics (45 min). Print 4 resumes; ID out; clothes laid out; bag packed. **Confirm the exact building and suite with your recruiter** — Amazon has Herndon, Reston and Arlington sites and people arrive at the wrong one every year. Have 2 questions per interviewer ready: *"What does on-call look like here?"* · *"What's the biggest technical debt you're carrying?"* · *"How does the team decide what to build next?"*

Stop by 8pm. Sleep 8 hours.

END — coverage summary

Day	Focus	Problems answered
1	Arrays, Hashing & Two Pointers	11
2	Sliding Window & Prefix Sum	13
3	Stacks, Monotonic Stacks & Linked Lists	16
4	Trees	16
5	Grids & Backtracking	13
6	Graphs	15
7	Mock Loop #1	protocol only
8	Heaps & Top-K	13
9	Binary Search	16
10	Intervals, Greedy & DP-1	17
11	DP-2 & Design-in-Code	19
12	Mock Loop #2 + repair	4
13	Mock Loop #3	protocol only
	Total	**153**

Every entry follows the same five parts: understand → brute force (idea · data structures · algorithm · complexity · why it's not ideal) → optimal (idea · data structures · algorithm · complexity) → full commented LeetCode-compatible Python → step-by-step dry run.

One last thing. The dry runs are the part to actually rehearse out loud. Every Amazon interviewer is assigned Leadership Principles to gather evidence against, and they can only score what you say. A narrated near-correct solution outscores a silent correct one — which is the single fact this entire file exists to serve.